CÓDIGO SINTÉTICO

MADUREZ ORGANIZACIONAL



GOBERNANZA, AUTONOMÍA Y CALIDAD EN EQUIPOS DE AGENTES

Gobernanza, roles y RACI, medición de autonomia, auditorías agnósticas, procesos humanos y cultura para organizaciones que operan con agentes.

Sergio Perez Ruiz · Henry Fuentes

Código Sintético

Madurez Organizacional — Gobernanza, autonomía y calidad en equipos de agentes

Sergio Perez Ruiz Henry Fuentes

2026

Hoja de créditos

Título: Código sintético, Madurez Organizacional: Gobernanza, autonomía y calidad en equipos de agentes.

Serie: Código Sintético (Volumen 3 de 4).

Autores: Sergio Perez Ruiz y Henry Fuentes.

Editor: Sergio Perez Ruiz. Jr. Juan Guillermo Moore 357, Santa Anita - Lima, Perú. Primera edición digital, 2026

ISBN (edición digital / EPUB): [ISBN PENDIENTE] ISBN (edición digital / PDF): [ISBN PENDIENTE]

Depósito Legal N° [PENDIENTE]

Libro disponible en: codigosintetico.specsolid.com

Índice

Hoja de créditos ii

Dedicatoria **1**

Sinopsis **3**

Prólogo **5**

Prefacio **7**

Guía de lectura y mapa de prerrequisitos **9**
1. Grupos temáticos del volumen 9
2. Rutas por rol 10
3. Mapa de prerrequisitos entre volúmenes 10
4. Qué capítulos puede dejar para después 11

Nota sobre vigencia tecnológica **13**
1. Qué caduca y qué no 13
2. Lo que este volumen se compromete a no hacer 14
3. Cómo mantener este volumen vivo 14

Cuaderno de lectura crítica **15**
1. Rúbrica para evaluar este volumen 15
2. Plantilla para redactar una crítica 16
3. Preguntas de discusión por capítulo 16
    Gobierno y ética 16
    Rol y competencia profesional 17
    Práctica de equipo 18
    Fundamentos de decisión 18
    Casos aplicados 19
4. Autoevaluación de comprensión 20

5. Cómo citar y discutir este volumen 20

Glosario operativo del Volumen 3 **21**

Índice de consulta rápida: problema → dónde acudir **27**
1. Por situación 27
2. Índice de ejercicios resueltos del volumen 28
3. Frontera con los otros volúmenes 29
4. Advertencia de uso 29

Capítulo 1: Ética y alineación en producción **31**
El problema de la alineación a escala 31
El contexto regulatorio: de virtud a obligación 34
La dimensión ambiental 34
Sesgo algorítmico y equidad distribucional 35
Transparencia, explicabilidad y derecho a la impugnación . 40
El panorama regulatorio global: construir una sola vez para el estándar más exigente 41
Lo esencial del capítulo 1 44

Capítulo 2: El futuro, el Ingeniero Orquestador **47**
El nuevo contrato social de la ingeniería de software 47
Las cinco competencias del Ingeniero Orquestador 48
El horizonte proactivo: de agentes que responden a agentes que anticipan 49
Cierre del volumen: el arte de dirigir la orquesta 51
Lo esencial del capítulo 2 52

Capítulo 3: Multiagentes de IA ante El Niño **55**
Sección 1: La Anatomía del Problema: Más Allá de la Meteorología 55
Sección 2: Estado del Arte: IA Predictiva Aplicada a Fenómenos Climáticos 57
Sección 3: Arquitectura Técnica Expandida: Las Cuatro Capas en Detalle 58
Capa 1: Percepción: Normalizar la Realidad Heterogénea 59
Capa 2: Razonamiento Especializado: Los Expertos Sintéticos 59
Capa 3: Orquestación y Consenso: Donde el Enjambre Decide 60

Capa 4: Gobernanza Humana: La Máquina Sustenta, el Humano Firma 61
Sección 4: Perspectiva Senior Engineer y QA: Confiabilidad del Sistema Bajo Presión 61
Sección 5: Perspectiva Ejecutiva: ROI, Presupuesto y Ciclo de Vida 63
Sección 6: Caso Transfronterizo: El Canal de Panamá y el Racionamiento Hídrico 64
Versión académica ampliada 66
Lo esencial del capítulo 3 67
Ejercicio resuelto 3.1: Diseña el Agente Crítico 68

Capítulo 4: Cómo Defender una Arquitectura de Multiagentes ante un Comité: Viabilidad, Costos, Benchmarking y la Parte Agéntica de la Crítica Constructiva 71
Sección 1: Lo que el Comité Compra en Realidad 71
Sección 2: El Método ATAM Aplicado a Sistemas Multiagente 73
Sección 3: El Panel de Agentes Críticos: Estresar la Propuesta Antes de la Sala 74
Sección 4: Benchmarking Honesto: Comparar para Convencer, No para Impresionar 75
Sección 5: La Narrativa de Control: Cómo Cerrar la Defensa 77
Versión académica ampliada 78
Lo esencial del capítulo 4 78
Ejercicio resuelto 4.1: Traduce un riesgo técnico al idioma del CFO 79

Capítulo 5: Descubriendo los Bugs del Software: Un Desafío Reformulado por los Agentes de IA 81
Sección 1: El Bug como Fenómeno Epistemológico: Por Qué Fallamos en Detectarlo 81
Sección 2: Estado del Arte en Testing Automatizado con IA: Los Avances que Cambiaron el Campo 82
Sección 3: Anatomía del Enjambre Cazador de Bugs: Implementación 83
Agente Reproductor 84
Agente de Hipótesis 84
Agente Refutador 85
Agente de Regresión 85

Sección 4: Perspectiva del QA Senior: Métricas, Integración y el Nuevo Rol 86
Sección 5: Perspectiva Ejecutiva y de Product Owner: El ROI del Bug No Encontrado 88
Versión académica ampliada 88
Lo esencial del capítulo 5 89
Ejercicio resuelto 5.1: El Agente Refutador en acción 90

Capítulo 6: Plantillas para Proyectos con Multiagentes que Operan como Equipos de Desarrollo: Sprints, Ceremonias, Demos y el ROI que los Ejecutivos Esperan 93
Sección 1: La Tesis Central: Por Qué el Ágil es la Arquitectura de Coordinación del Enjambre 94
Sección 2: Los Roles Agénticos del Equipo Scrum 95
Sección 3: La Plantilla de Cadencia: Sprint Configurado para un Enjambre 97
Sprint Planning (Duración: 90 min → 15 min con enjambre) 97
Daily Standup (Eliminado como ceremonia → Sustituido por Dashboard Continuo) 98
Sprint Demo (Duración: 30 min para stakeholders) . 99
Retrospectiva (30 min al cierre del sprint) 99
Sección 4: DORA Metrics Aplicadas al Enjambre: El Lenguaje que el CTO Entiende 100
Sección 5: Del POI al ROI: El Cuadro que los Ejecutivos Esperan 101
Lo esencial del capítulo 6 102
Ejercicio resuelto 6.1: Diseña el Quarterly Closing del enjambre 103
Q1 2025: Quarterly Closing: Resumen Ejecutivo 103
POI 1: Churn -10% → Logrado parcialmente: -7.3% . 103
POI 2: 3 ERPs integrados → Logrado 2/3 (66%) . . . 103
POI 3: TTFV 5 días → 2 días → Logrado al 80%: 2.4 días 103
Resumen ejecutivo Q1 104

Capítulo 7: El Ecosistema del Desarrollo Moderno Equiparado con el del Negocio: y Cómo Atenderlo en el Preciso Momento, Aplicado a los Multiagentes 105
Sección 1: La Brecha de Velocidades: Un Problema Estructural, No de Actitud 105

Sección 2: DORA Metrics como Puente entre Velocidad Técnica y Velocidad de Negocio 106
Sección 3: El Patrón Just-in-Time: De Reactivo a Anticipatorio 108
Sección 4: La Arquitectura del Ecosistema 10×, Tres Capas de Velocidad 109
Sección 5: El Riesgo de la Velocidad sin Guardarraíles: Deuda Técnica a Escala de IA 110
Sección 6: Perspectiva Ejecutiva: La Decisión Estratégica de «Velocidad Sostenible» 111
Versión académica ampliada 112
Lo esencial del capítulo 7 113
Ejercicio resuelto 7.1: Calcula el valor de reducir el Lead Time de 3 semanas a 2 días 114

Capítulo 8: Convertirte en maestro de agentes y multiagentes de IA 117
Del programador al director de orquesta: un cambio de identidad profesional 117
El modelo de madurez agéntica (AMM): dónde estás y adónde debes ir 118
Las cuatro competencias del maestro: especificación, verificación, orquestación, calibración 119
El diseño de evaluaciones propias: la competencia diferenciadora del nivel 4 120
Patrones de orquestación: AutoGen y LangGraph en perspectiva técnica 120
La seguridad como dimensión indisociable de la maestría . 121
El camino de la maestría: práctica deliberada aplicada a agentes 122
Lo esencial del capítulo 8 122

Capítulo 9: Seis casos para fortalecer el liderazgo técnico, aplicados a los agentes y multiagentes de IA 125
Del liderazgo de equipos al liderazgo de enjambres: una evolución no una ruptura 125
Las seis fortalezas reinterpretadas para la era agéntica . . . 126
Marcos de decisión técnica para líderes agénticos: ADRs en contexto de IA 127
OKRs para la transformación agéntica: alinear enjambres con objetivos organizacionales 128

El coste cuantificado del liderazgo técnico débil en la era agéntica 129
El corazón invariante del liderazgo agéntico 130
Lo esencial del capítulo 9 131

Capítulo 10: Gestión de expectativas y tiempos de la IA 133
La IA redefine la gestión del tiempo, no la abole 133
El espejismo de la velocidad: por qué la generación rápida no implica entrega rápida 134
Evidencia empírica: las nuevas curvas de tiempo 135
Modelo de planificación para proyectos con asistencia de IA 135
El pipeline CI/CD para código generado por IA: qué añadir 136
ROI cuantificado: cuándo y cuánto ahorra la IA 137
Gestionar al stakeholder: la pedagogía como competencia ejecutiva 138
Lo esencial del capítulo 10 139

Capítulo 11: Un llamado a reinventarse en la actualidad de los sistemas multiagentes 141
Optimizar no es reinventar: la distinción que lo cambia todo 141
Por qué los multiagentes fuerzan la reinvención: la física del cambio de fase 142
Los Tres Horizontes de McKinsey en clave agéntica 142
Mapa de reinvención: inventario de supuestos cognitivos . 144
Casos empíricos de reinvención agéntica: salud, finanzas y derecho 144
El coraje de canibalizar lo propio 145
Hoja de ruta ejecutiva para la transformación agéntica . . . 146
Lo esencial del capítulo 11 147

Capítulo 12: La lucha entre agentes y multiagentes de IA: porque ser el mejor modelo no basta para vencer 149
La metáfora del T-1000 y su precisión técnica 149
La función de calidad sistémica: la matemática del argumento150
Las cinco dimensiones que deciden la lucha sistémica . . . 150
Evidencia de benchmarks: por qué los rankings de modelos engañan 152
Estrategias de enrutamiento de modelos: la inteligencia de la selección 152
El sistema vence al modelo: implicaciones para el arquitecto 153

Build vs. buy en la era del modelo-commodity 154
Lo esencial del capítulo 12 155

Capítulo 13: El síntoma del programador cansado, aplicado a los agentes y multiagentes de IA 157
El síntoma humano: cuando la ejecución sobrevive al juicio 157
La neurociencia del juicio degradado: por qué la fatiga ataca primero lo más valioso 158
El equivalente agéntico: la degradación contextual como fatiga sistémica 158
Síntomas diagnósticos del síntoma del programador cansado en el enjambre 159
Patrones técnicos de gestión del contexto: sliding window y compresión 160
Implementación del watchdog de contexto: una guía técnica 161
La refutación externa como contramedida universal 162
Perspectiva ejecutiva: la carga cognitiva del equipo como variable de diseño 162
Lo esencial del capítulo 13 163

Capítulo 14: Katas de agentes de IA concursando para obtener ellos mismos un premio 165
La kata como banco de pruebas del razonamiento bajo restricciones 165
Las competiciones existentes: SWE-bench, AgentBench y WebArena como katas formales 166
El giro provocador: el agente compite por su propio premio 167
Teoría de funciones objetivo en agentes competitivos: el fundamento técnico 168
Anatomía de una kata agéntica completa 169
Diseño de evaluaciones internas: guía técnica para equipos 170
Por qué la kata agéntica educa a sus diseñadores 171
El hackathon agéntico como programa ejecutivo 171
Lo esencial del capítulo 14 172

Capítulo 15: RACI y Eisenhower con Agentes 175
El problema de la responsabilidad difusa en equipos humano-agente 175
RACI con agentes: la regla de la accountability indelegable 176
RACI a escala: cuando el Responsable es un enjambre . . . 177

La paradoja de la agilidad: más claridad, no menos 179
Perspectiva técnica avanzada 179
RACI como patrón de autorización en código 179
Eisenhower como enrutador de prioridad de tareas en el enjambre 181
Estrategia de QA: testing de límites de rol 182
Perspectiva ejecutiva 182
Visión del Product Owner: RACI en el backlog 182
Visión del Scrum Master: Eisenhower en la planificación del sprint 183
Versión académica alternativa (mayor densidad formal) . 183
Lo esencial del capítulo 15 184

Capítulo 16: PMBOK y los nuevos avances de agentes y multiagentes de IA: visión de una simbiosis 185
El PMBOK no es burocracia: es memoria destilada 185
La simbiosis: qué aporta cada parte 186
Lo que la IA no puede tomar del PMBOK: el núcleo irreductiblemente humano 187
El riesgo de la automatización ciega: métricas perfectas sobre un proyecto que naufraga 187
Perspectiva técnica avanzada 188
Agente de gestión de riesgos: implementación del registro de riesgos vivo 188
Simulación Monte Carlo para el cronograma 189
Estrategia de QA: verificación sistemática de entregables 190
Perspectiva ejecutiva 191
Visión del Product Owner: el PMBOK como backlog de gobernanza 191
Visión del Scrum Master: ceremonias del PMBOK con agentes 192
Versión académica alternativa (mayor densidad formal) . 192
Lo esencial del capítulo 16 193

Capítulo 17: Enseñanzas de agentes de IA: aprender de quienes explican lo complejo con sencillez 195
El valor del mentor que simplifica sin banalizar 195
La enseñanza como prueba de comprensión: el principio de Feynman en sistemas agénticos 196

Multiagentes que se enseñan entre sí: el enjambre como comunidad de práctica 196
El mentor humano como corrector de deriva y fuente de verdad 197
Perspectiva técnica avanzada 198
Implementación de la enseñanza mutua entre agentes: el patrón Expert-Apprentice 198
Principio de Feynman como criterio de evaluación de agentes 199
Estrategia de QA: detección de deriva en el enjambre 200
Perspectiva ejecutiva 201
Visión del Product Owner: el enjambre que aprende como activo de producto 201
Visión del Scrum Master: la revisión del mentor como ceremonia de cierre de sprint 202
Versión académica alternativa (mayor densidad formal) . 202
Lo esencial del capítulo 17 203

Capítulo 18: Nuestra guía para no perdernos en el ecosistema de complejidad de la IA y los multiagentes 205
El problema no es la ignorancia, es la saturación 205
Tres preguntas-brújula para no perder el norte 206
El mapa de capas como antídoto al vértigo 207
La disciplina de no adoptar: el filtro más valioso 207
Perspectiva técnica avanzada 208
Architecture Decision Records (ADR): institucionalizar el filtrado con criterio 208
Technology Radar: el mapa de capas como proceso continuo 209
Estrategia de QA: testing de las decisiones de arquitectura 211
Perspectiva ejecutiva 212
Visión del Product Owner: el radar como herramienta de roadmap 212
Visión del Scrum Master: el sprint de arquitectura trimestral 212
Versión académica alternativa (mayor densidad formal) . 213
Lo esencial del capítulo 18 214

Capítulo 19: Ideas para generar ingresos con agentes de IA y multiagentes **215**
El error de partir de la tecnología 215
Cuatro arquetipos de monetización: defensibilidad como criterio central 216
Arquetipo 1: Automatización de lo tedioso (ahorro de costo) 216
Arquetipo 2: Amplificación del experto (apalancamiento) 217
Arquetipo 3: Acceso a lo inaccesible (creación de mercado) 217
Arquetipo 4: Inteligencia continua (suscripción) . . . 218
El foso defensivo: por qué el modelo no es suficiente 218
De la idea al ingreso: validar barato con el MVP agéntico . 219
Perspectiva técnica avanzada 219
Arquitectura de costos: modelado de unit economics por arquetipo 220
Arquitectura del MVP agéntico: el piloto manual primero 222
Estrategia de QA: testing del foso defensivo 223
Perspectiva ejecutiva 224
Visión del Product Owner: el roadmap de monetización por fases 224
Visión del Scrum Master: el sprint de validación antes del sprint de construcción 225
Versión académica alternativa (mayor densidad formal) . 225
Lo esencial del capítulo 19 226

Capítulo 20: DevSecOps y Permisos para Agentes **229**
DevSecOps no es un obstáculo, es un contrato 229
Por qué la valla está bien: tres fundamentos inviolables . . 230
El modelo de confianza cero aplicado a agentes 231
El ciclo de vida del permiso agéntico 232
Cómo mejorar DevSecOps para la era agéntica 233
Métricas de madurez DevSecOps para enjambres 233
Un índice cuantificable de autonomía 234
Auditoría agnóstica del modelo 236
Perspectiva ejecutiva: el costo de la permisividad 237
Lo esencial del capítulo 20 238

Capítulo 21: Cada línea de código como una responsabilidad del agente de IA y los multiagentes **239**
La línea de código nunca es neutral 239
La asimetría fundamental: velocidad vs. responsabilización 240
La crisis epistemológica del código sintético 241
Taxonomía de la responsabilidad agéntica 242
Disciplinas para responsabilizar el código agéntico 243
El programador como garante, no como mecanógrafo . . . 244
Perspectiva de QA: la verificación como nueva primera línea 244
Lo esencial del capítulo 21 245

Capítulo 22: Comenzar por MVPs para los agentes de IA y multiagentes **247**
El MVP no es una versión pequeña, es una pregunta 247
Por qué los agentes tientan a saltarse el MVP 248
El MVP agéntico en capas de riesgo 249
El marco RACE para MVPs agénticos 249
Anti-patrones del MVP agéntico 250
Del MVP al enjambre, sin saltos de fe 251
Perspectiva ejecutiva: el MVP como herramienta de gobernanza 251
Lo esencial del capítulo 22 252

Capítulo 23: Casos de roles en agentes de IA y multiagentes, un caso para estudiar **253**
El rol como restricción que potencia 253
Teoría de la especialización funcional 254
El caso de estudio: roles Scrum en enjambres de software . 255
Mapeo completo: roles organizacionales → arquetipos agénticos 256
Por qué la separación de roles evita patologías 257
La perspectiva de Product Owner y Scrum Master 258
La lección de diseño 259
Lo esencial del capítulo 23 259

Capítulo 24: Recalcando la importancia de los principios SOLID para los agentes y ecosistemas multiagentes **261**
SOLID trasciende su origen 261
Los cinco principios SOLID reescritos para agentes 262

S: Responsabilidad Única (Single Responsibility Principle) . . . 262
O: Abierto/Cerrado (Open/Closed Principle) 263
L: Sustitución de Liskov (Liskov Substitution Principle) 263
I: Segregación de Interfaces (Interface Segregation Principle) . . . 264
D: Inversión de Dependencias (Dependency Inversion Principle) . . . 265
Por qué SOLID importa MÁS en enjambres . . . 265
El refactoring del enjambre: una metodología . . . 266
Perspectiva ejecutiva: SOLID como inversión en mantenibilidad . . . 267
Lo esencial del capítulo 24 . . . 267

Capítulo 25: Vamos a rescatar la agilidad inyectándola en los agentes de IA 269
La agilidad capturada . . . 269
Autopsia del manifiesto: qué murió y qué sobrevive . . . 270
Los 12 principios, reencarnados en enjambres . . . 270
Inyectar agilidad, no ritual . . . 272
La revisión universal, realizada por fin . . . 272
La advertencia: agilidad no es rapidez . . . 273
Perspectiva ejecutiva: el ROI de la agilidad genuina . . . 273
Lo esencial del capítulo 25 . . . 274

Apéndice B: precios de referencia y cómo calcular el coste de un pipeline 275
B.1. Por qué este apéndice existe . . . 275
B.2. Precios por millón de tokens (USD) . . . 276
B.3. Suscripciones frente a API . . . 277
B.4. Los tres multiplicadores que explican la factura real . . . 277
B.5. La fórmula . . . 278
B.6. Cómo mantener este apéndice . . . 278

Estado del Arte: Gobernanza, Rendición de Cuentas y Supervisión Humana en Sistemas de IA Autónomos 279
1. Líneas de investigación principales . . . 279
2. Marcos teóricos utilizados . . . 280
3. Metodologías aplicadas . . . 281
4. Hallazgos recurrentes . . . 281

5. Discrepancias entre autores 282
6. Limitaciones metodológicas de la literatura revisada . . 282
7. Brechas de conocimiento 283
8. Relación con la tesis central del libro 283

Referencias **285**

Dedicatoria

A quienes sostienen la gobernanza cuando nadie la ve —auditores, revisores,
responsables de calidad y de seguridad— porque la confianza del público en estos
sistemas se gana en sus mesas de trabajo.

Sinopsis

Código Sintético: Madurez Organizacional es el tercer volumen de la serie: el manual para que una organización entera opere con agentes sin perder el control.

La tesis es directa: el problema de los agentes en producción no es el modelo, es la organización. Este tomo cubre la gobernanza —roles, RACI, comités que evalúan arquitecturas—, la medición —índices de autonomía, auditorías agnósticas del modelo, métricas DORA aplicadas al enjambre—, la seguridad operativa —DevSecOps, confianza cero, ciclo de vida de permisos— y la cultura —cómo rescatar la agilidad, cómo defender una arquitectura ante un comité, cómo generar ingresos con agentes—.

Incluye plantillas utilizables: sprints configurados para enjambres, ADRs aplicados al ecosistema agéntico, paneles de agentes críticos que estresan una propuesta antes de la sala. No es teoría de gestión: es el andamiaje que permite que los fundamentos del volumen 1 funcionen dentro de una empresa real.

Prólogo

Todo sistema multi-agente maduro termina enfrentando la misma pregunta, y no es técnica: *¿quién responde cuando el enjambre se equivoca?*

La respuesta no cabe en un prompt ni en un arnés. Requiere roles definidos, permisos con ciclo de vida, auditorías independientes del proveedor, métricas que un director financiero entienda y una cultura que premie decir «el agente se equivocó y lo detectamos» por encima de «el agente nunca falla». Eso es la madurez organizacional, y es lo que separa el piloto vistoso del sistema que sobrevive a su segundo año.

Este volumen está escrito para dos lectores que suelen no hablarse: el ingeniero que desprecia la «burocracia» y el directivo que teme la «caja negra». Los capítulos les dan un idioma común —RACI, ATAM, DORA, RACE— aplicado al mundo agéntico, con plantillas que pueden copiarse el lunes siguiente.

Gobernar no es frenar. Es la condición para ir rápido sin romper lo que importa.

Sergio Perez Ruiz y Henry Fuentes *Lima, 2026*

Prefacio

La madurez de un sistema multiagente se revela cuando deja de ser un proyecto de demostración. En ese momento, la conversación sobre modelos, *prompts* y herramientas tiene que incluir responsabilidades, presupuesto, cumplimiento, continuidad del servicio y capacidad de aprender de los errores.

Este volumen se ocupa de esa organización. El foco no está en un modelo concreto, sino en el sistema sociotécnico que lo rodea: quién decide qué permisos puede usar un agente; cómo se conceden, revisan y retiran; qué se mide; quién audita; cómo se responde por una decisión. Los marcos de gobernanza y los patrones organizativos se presentan como herramientas de trabajo que deben ajustarse a la realidad de cada empresa.

Las plantillas incluidas —RACI, registros de decisión, paneles de revisión, métricas y evaluaciones— no sustituyen el criterio profesional. Ayudan a hacerlo explícito. Un comité puede aceptar, modificar o rechazar una arquitectura; un auditor puede pedir la evidencia de una afirmación; un responsable de plataforma puede justificar una inversión con los mismos datos que usa el equipo técnico. El valor está en compartir ese lenguaje y en dejar registrada la decisión.

La propuesta del libro es compatible con una organización que todavía está aprendiendo. Empieza por límites y observación, no por la promesa de una autonomía sin supervisión. Los roles, las métricas y los procesos permiten intervenirlos con evidencia y aprender de ellos sin convertir cada incidente en una crisis general.

Este tercer volumen ofrece un marco para gobernar la IA. Forzar la autonomía sin responsabilidad, o impedirla sin examinar los riesgos, conduce al mismo resultado: un sistema que nadie puede explicar ni mejorar.

Sergio Perez Ruiz y Henry Fuentes *Lima, 2026*

Guía de lectura y mapa de prerrequisitos

*Este volumen cambia de sujeto: no trata del sistema, sino de la organización que lo adopta. Por eso su dificultad no es técnica, sino que exige al lector decidir **desde qué rol** lo lee. Esta guía le propone una ruta por rol y una por problema, y declara qué capítulos son prescindibles en una primera pasada.*

1. Grupos temáticos del volumen

- **Gobierno y ética (caps. 1, 20, 21, 15).** Alineación en producción, DevSecOps y permisos, responsabilidad del código sintético, RACI. Es el bloque que un comité o un área legal va a pedirle.
- **Rol y competencia profesional (caps. 2, 8, 9, 10, 13).** Del programador al maestro de agentes; liderazgo técnico, gestión de expectativas y fatiga. Bloque de desarrollo de carrera y de equipo.
- **Práctica de equipo (caps. 6, 7, 22, 23, 24, 25).** Sprints y ceremonias con agentes, cadencia frente a negocio, MVP, roles, SOLID y agilidad. Bloque operativo.
- **Fundamentos de decisión (caps. 4, 11, 12, 16, 18, 19).** Defender una arquitectura ante un comité, reinventar el negocio, por qué el sistema vence al modelo, PMBOK, mapa del ecosistema y monetización.
- **Casos aplicados (caps. 3, 5, 14, 17).** El Niño, caza de bugs, katas y enseñanza. Casos completos para estudiar y replicar.

2. Rutas por rol

Si usted es…	Lea en este orden	Por qué
CTO / director de tecnología	1 → 12 → 4 → 11 → 19 → 25	Gobierno, decisión sistémica y reinvención antes que detalle operativo
Engineering manager / tech lead	8 → 9 → 6 → 7 → 13 → 15	Competencia del equipo, cadencia, fatiga y responsabilidad
Product Owner / negocio	10 → 19 → 22 → 6 → 16	Expectativas, monetización, MVP y lenguaje común con ingeniería
Arquitecto / security	20 → 21 → 1 → 4 → 5	Permisos, responsabilidad, ética, defensa ante comité, verificación
Docente o formador	2 → 8 → 14 → 17 → 18	Progresión de competencia, katas, enseñanza y mapa del ecosistema

3. Mapa de prerrequisitos entre volúmenes

Capítulo del Vol. 3	Prerrequisito real	Si le resulta árido
1. Ética y alineación	Ninguno	—
2. El Ingeniero Orquestador	Vol. 1, cap. 2	Vol. 1, cap. 2
3. Multiagentes ante El Niño	Vol. 1, cap. 8 (orquestación)	Vol. 1, cap. 8
4. Defender la arquitectura ante un comité	Vol. 1, caps. 2 y 3	Vol. 1, cap. 3
5. Descubrir bugs	Vol. 4, cap. 3 (testing)	Vol. 4, cap. 3
6–7. Sprints y cadencia	Vol. 3, cap. 8	Cap. 8

Capítulo del Vol. 3	Prerrequisito real	Si le resulta árido
8–10. Maestría, liderazgo, expectativas	Cap. 2	Cap. 2
11–12. Reinvención y sistema frente a modelo	Ninguno	—
13. Programador cansado	Vol. 1, cap. 4 (contexto)	Vol. 1, cap. 4
14–18. Katas, RACI, PMBOK, enseñanzas, mapa	Ninguno	—
19. Monetización	Cap. 11	Cap. 11
20–21. DevSecOps y responsabilidad	Vol. 2, cap. 1 (seguridad)	Vol. 2, cap. 1
22–25. MVP, roles, SOLID, agilidad	Vol. 1, caps. 5 y 6	Vol. 1, caps. 5 y 6

4. Qué capítulos puede dejar para después

En una primera lectura orientada a resolver un problema concreto, los capítulos 3, 5, 14 y 17 (casos aplicados) se pueden posponer: son excelentes para estudiar un caso completo, pero no son prerrequisito de nada. El capítulo 25 cierra el argumento del volumen y conviene leerlo aunque no lea nada más: es la tesis de que la agilidad fue capturada por el ritual y que los agentes permiten recuperar los principios.

Nota sobre vigencia tecnológica

*Este volumen habla de organizaciones, regulación y prácticas. Es el menos expuesto de la serie a la obsolescencia técnica y, a la vez, el más expuesto a la obsolescencia **normativa**: una ley cambia más despacio que un modelo, pero cambia con más consecuencias.*

1. Qué caduca y qué no

Tipo de contenido	Vida útil	Cómo verificarlo
Marco regulatorio (AI Act, GDPR, marcos de responsabilidad)	Media (1–3 años)	Texto consolidado de la norma
Métricas de industria (DORA, benchmarks de productividad)	Media	Informe anual del organismo que las publica
Marcos de gestión (PMBOK, RACI, OKR)	Larga	Versión vigente del estándar
Prácticas organizativas (sprints con agentes, MVP agéntico, comités, katas)	Larga	Se validan por resultados y por replicabilidad, no por herramienta
Cifras concretas de ROI o de coste	Corta	Recalcular con los precios vigentes; el **método** del capítulo sobrevive, la cifra no

2. Lo que este volumen se compromete a no hacer

1. **No presenta una práctica organizativa como universal.** Cada capítulo parte de un contexto (equipo pequeño, empresa regulada, área con QA fuerte) y lo declara. Si su contexto no coincide, el capítulo sigue siendo útil como criterio, no como receta.
2. **No convierte una métrica en un objetivo.** El volumen advierte explícitamente del riesgo de «métricas perfectas sobre un proyecto que naufraga»; aplíquese esa advertencia al leer los propios capítulos de métricas.

3. Cómo mantener este volumen vivo

Al adoptar cualquiera de sus plantillas, escriba en el propio documento de su organización: (a) la fecha de adopción, (b) el responsable de revisarla, (c) el criterio por el que se considerará caducada. Es la misma disciplina de revisión que el volumen pide para los permisos y las políticas de los agentes; usarla en el propio manual organizativo es la mejor prueba de que se entendió el libro.

Cuaderno de lectura crítica

Para qué sirve este cuaderno: este volumen hace afirmaciones sobre organizaciones, equipos y personas. Son afirmaciones más fáciles de discutir que las técnicas y, por eso, más fáciles de discutir mal. El cuaderno ofrece una rúbrica adecuada a ese tipo de afirmación, una plantilla de reseña y preguntas para los 25 capítulos.

1. Rúbrica para evaluar este volumen

Criterio	La pregunta que usted debe poder responder	Señal de debilidad
Claridad de la audiencia	¿Sé desde qué rol habla el capítulo (CTO, tech lead, PO, QA)?	Habla a todos a la vez y no sirve a ninguno
Traducibilidad a decisión	¿Puedo convertirlo en una decisión con responsable y fecha?	Queda en buena intención sin mecanismo
Base empírica	¿La afirmación sobre equipos se apoya en datos, en un caso o solo en analogía?	Analogía presentada como evidencia
Reconocimiento de contexto	¿El capítulo dice en qué tipo de organización no aplica?	Se prescribe para toda organización
Riesgo de métrica	¿Advierte de cómo la métrica puede pervertirse?	Convierte el indicador en objetivo

Criterio	La pregunta que usted debe poder responder	Señal de debilidad
Vigencia normativa	¿Sé qué parte depende de una regulación que puede cambiar?	Se cita la norma sin versión ni fecha
Utilidad como manual	¿Puedo volver al capítulo por rol o por problema y encontrar la plantilla?	Falta la plantilla o la tabla de decisión

Advertencia que el propio volumen hace y conviene aplicar aquí. Una práctica organizativa no se copia: se adapta. Al reseñar un capítulo de este tipo, la pregunta central no es «¿es verdad?» sino «¿en qué organización es verdad y en cuál produce el efecto contrario?».

2. Plantilla para redactar una crítica

1. **Rol del capítulo:** ¿a quién le habla y qué decisión le pide?
2. **Tesis en una frase:** «Este capítulo sostiene que ___ y lo justifica con ___».
3. **Tipo de evidencia:** dato, caso documentado, analogía o experiencia del autor. Nómbrelo.
4. **Contexto faltante:** ¿qué tipo de organización haría fracasar esta práctica?
5. **Veredicto con destinatario:** «le sirve a ___ en organizaciones ___ ; es contraproducente en ___ porque ___».

3. Preguntas de discusión por capítulo

Gobierno y ética

Capítulo 1. Ética y alineación en producción. 1. ¿Qué obligación regulatoria de las que el capítulo cita ya aplica a su producto y no está en su backlog con responsable y fecha? 2. Crítica: ¿«construir una sola vez para el estándar más exigente» es siempre racional, o hay escenarios donde es coste puro sin mercado que lo pague?

Capítulo 20. DevSecOps y permisos para agentes. 1. ¿Qué permiso de agente existe hoy en su sistema sin haber pasado por revisión de seguridad? ¿Quién lo aprobó y con qué criterio? 2. Crítica: ¿el índice cuantificable de autonomía es interpretable sin arbitrariedad en los pesos, o dos equipos obtendrían números distintos del mismo sistema?

Capítulo 21. Cada línea de código como responsabilidad. 1. ¿Quién firma hoy una línea de código generada por un agente en su equipo, y qué significa esa firma en la práctica? 2. Crítica: ¿la taxonomía de responsabilidad agéntica es jurídicamente operativa o se queda en marco conceptual?

Capítulo 15. RACI y Eisenhower con agentes. 1. Cuando decide un agente, ¿quién es *accountable* en su organización? Si la respuesta es «el agente», ahí está el problema. 2. Crítica: ¿RACI aplicado a enjambres aclara la responsabilidad o la diluye detrás de una tabla más?

Rol y competencia profesional

Capítulo 2. El futuro, el Ingeniero Orquestador. 1. De las cinco competencias del orquestador, ¿cuál falta más en su equipo y cómo la desarrollaría con un plan concreto? 2. Crítica: ¿el «nuevo contrato social de la ingeniería» es una descripción de lo que está pasando o un deseo del autor?

Capítulo 8. Convertirte en maestro de agentes. 1. ¿En qué nivel del modelo de madurez agéntica está usted y qué evidencia verificable lo demuestra? 2. Crítica: ¿el nivel 4 (diseñar evaluaciones propias) es alcanzable sin un equipo de plataforma detrás?

Capítulo 9. Seis casos de liderazgo técnico. 1. ¿Qué riesgo técnico de su equipo no está traducido al idioma del negocio, y a quién le corresponde traducirlo? 2. Crítica: ¿el coste del liderazgo técnico débil está cuantificado con rigor o estimado con plausibilidad?

Capítulo 10. Gestión de expectativas y tiempos de la IA. 1. ¿Qué expectativa de plazo debe corregir hoy, y con qué dato de su propio equipo la corrige? 2. Crítica: ¿las «nuevas curvas de tiempo» se sostienen con evidencia reproducible o con anécdota bien contada?

Capítulo 13. El síntoma del programador cansado. 1. ¿Qué señal de degradación contextual muestra su enjambre (repite, olvida, se

contradice) y cómo la instrumentaría? 2. Crítica: ¿la analogía con la neurociencia de la fatiga aporta poder explicativo o solo retórica?

Práctica de equipo

Capítulo 6. Plantillas para proyectos con multiagentes como equipos. 1. ¿Qué ceremonia de su equipo sobrevive hoy sin propósito claro, y qué la reemplazaría si el enjambre asumiera su función? 2. Crítica: ¿tratar el ágil como «arquitectura de coordinación» del enjambre es una metáfora útil o una analogía forzada?

Capítulo 7. El ecosistema de desarrollo equiparado al del negocio. 1. ¿Dónde está su brecha de velocidades entre técnica y negocio, y qué patrón just-in-time podría cerrarla? 2. Crítica: ¿las métricas DORA aplicadas al enjambre miden lo mismo que en un equipo exclusivamente humano?

Capítulo 22. Comenzar por MVPs. 1. ¿Cuál es la pregunta que su MVP debe responder, y qué resultado la respondería sin ambigüedad? 2. Crítica: ¿el marco RACE añade algo a un MVP clásico bien planteado, o solo renombra pasos conocidos?

Capítulo 23. Casos de roles en agentes de IA. 1. ¿Qué patología concreta evita hoy la separación de roles en su enjambre? Si no evita ninguna, revise el diseño. 2. Crítica: ¿el mapeo de roles organizacionales a arquetipos agénticos es útil o antropomorfiza el sistema sin ganancia?

Capítulo 24. SOLID para agentes y ecosistemas. 1. ¿Qué principio SOLID falta en su enjambre y qué deuda concreta está generando su ausencia? 2. Crítica: ¿«SOLID importa más en enjambres» aporta evidencia nueva o reafirma un principio conocido?

Capítulo 25. Rescatar la agilidad. 1. ¿Qué ritual de su equipo mide actividad en lugar de valor? ¿Lo eliminaría o lo reemplazaría por uno que mida valor? 2. Crítica: ¿la tesis de que la agilidad «fue capturada» describe un hecho o romantiza un pasado idealizado?

Fundamentos de decisión

Capítulo 4. Defender una arquitectura ante un comité. 1. ¿Qué pregunta del comité no sabría responder hoy sobre su propuesta de

multiagentes? 2. Aplique el método ATAM al caso: ¿qué *trade-off* crítico no había escrito antes de leer el capítulo?

Capítulo 11. Un llamado a reinventarse. 1. ¿Qué supuesto cognitivo suyo (o de su equipo) conviene inventariar antes de reiniciar un proyecto? 2. Crítica: ¿«optimizar no es reinventar» es una afirmación falsable o una máxima sin consecuencia operativa?

Capítulo 12. La lucha entre agentes: el sistema vence al modelo. 1. ¿Qué decisión suya cambió por arquitectura del sistema en lugar de por cambiar de modelo? 2. Crítica: ¿el capítulo demuestra su tesis con benchmarks o solo la ilustra con casos favorables?

Capítulo 16. PMBOK y los nuevos avances. 1. ¿Qué parte del PMBOK aporta memoria útil en su contexto y cuál es burocracia pura? 2. Crítica: ¿el «núcleo irreductiblemente humano» que propone el capítulo es verificable o es una declaración de fe?

Capítulo 18. Guía para no perderse en el ecosistema. 1. ¿Cuál es su disciplina de «no adoptar» y quién la ejerce con autoridad en su organización? 2. Crítica: ¿las tres preguntas-brújula bastan para decidir en un campo que cambia cada mes?

Capítulo 19. Ideas para generar ingresos. 1. ¿Cuál de los cuatro arquetipos de monetización tiene foso defensivo real en su caso, y por qué? 2. Crítica: ¿«validar barato con un MVP agéntico» funciona cuando la propuesta de valor exige precisión alta desde el inicio?

Casos aplicados

Capítulo 3. Multiagentes ante El Niño. 1. ¿Qué capa de esa arquitectura de cuatro capas es trasladable a su dominio de riesgo, y cuál no? 2. Crítica: ¿el capítulo demuestra la fiabilidad del sistema bajo presión o la argumenta?

Capítulo 5. Descubriendo los bugs del software. 1. ¿Qué fallo de su sistema es epistemológico (no se detecta porque nadie pregunta) y no puramente técnico? 2. Crítica: ¿el ROI del bug no encontrado está cuantificado con datos o estimado por plausibilidad?

Capítulo 14. Katas de agentes. 1. Diseñe una kata interna: objetivo, restricción y criterio de éxito evaluable. 2. Crítica: ¿la competición mejora la competencia real o entrena a optimizar la métrica de la competición?

Capítulo 17. Enseñanzas de agentes de IA. 1. ¿Qué explicaría usted a un junior para probar que entendió un patrón? Escríbalo en cinco líneas. 2. Crítica: ¿el principio de Feynman es aplicable a sistemas completos o solo a conceptos aislados?

4. Autoevaluación de comprensión

Marque lo que puede explicar **sin volver al texto**; lo que no marque es su lista de relectura.

- ☐ Traducir un riesgo técnico al idioma de un CFO con cifras.
- ☐ Describir los niveles del modelo de madurez agéntica.
- ☐ Explicar por qué un agente no puede ser *accountable* de una decisión.
- ☐ Justificar cuándo un MVP agéntico debe frenar antes de tocar usuarios externos.
- ☐ Diferenciar optimización de reinvención.
- ☐ Decir qué mide DORA y qué NO mide en un enjambre.
- ☐ Diseñar una evaluación propia para un comportamiento probabilístico.
- ☐ Explicar por qué la separación de roles evita patologías en un enjambre.

5. Cómo citar y discutir este volumen

Las afirmaciones de este volumen son de tres clases: **marcos** (RACI, PMBOK, DORA, ATAM), **prácticas** (sprints con agentes, MVP agéntico, katas) y **juicios** del autor sobre el estado del sector. Los marcos se citan por su versión y su organismo; las prácticas, por el contexto en que funcionaron; los juicios, como opinión argumentada. Confundir los tres es el error que produce las malas críticas de los libros de gestión: exigir a un juicio la prueba de un dato, o tratar un marco como si fuera una ley.

Glosario operativo del Volumen 3

Vocabulario de organización, gobierno y práctica de equipo. Cuando un término procede de un marco externo (PMBOK, DORA, RACI), se indica; cuando es propio de este libro (MVP agéntico, maestro de agentes), también.

Término	Definición operativa	Capítulo
Alineación en producción	Mantener el comportamiento del sistema dentro de los límites éticos y legales declarados, una vez desplegado y en operación, no solo en el diseño.	1
Derecho a la impugnación	Garantía de que una persona afectada por una decisión automatizada puede pedir revisión humana y obtener explicación.	1
Ingeniero Orquestador	Perfil cuyo trabajo es especificar, verificar y coordinar agentes, no ejecutar cada tarea. Evolución del AI Engineer del Vol. 1.	2

Término	Definición operativa	Capítulo
Modelo de madurez agéntica (AMM)	Escala de niveles que describe la competencia real de una persona o equipo frente a sistemas agénticos; el nivel superior exige diseñar evaluaciones propias.	8
Manager de enjambre	Rol que gestiona un equipo híbrido humano-agente; su tarea es asignar criterio, no repartir tareas.	9
Latencia de feedback	Tiempo entre una acción y la señal que permite corregirla. Determina la velocidad de aprendizaje organizacional.	26 (Vol. 2), 7
Patrón just-in-time	Anticipar la capacidad de respuesta en lugar de reaccionar cuando el negocio ya la exige.	7
DORA metrics	Cuatro indicadores de entrega de software (frecuencia, lead time, tasa de fallo, tiempo de restauración). En este volumen se aplican a un equipo con agentes.	6, 7
Quarterly Closing (del enjambre)	Cierre trimestral que rinde cuentas del valor entregado por el enjambre, no de su actividad.	6

Término	Definición operativa	Capítulo
MVP agéntico	Versión mínima que responde una pregunta de negocio con riesgo acotado, no una versión pequeña del producto.	22
Marco RACE	Secuencia propuesta para construir MVPs agénticos por capas de riesgo.	22
Síntoma del programador cansado (aplicado al enjambre)	Degradación del juicio por saturación de contexto: el sistema ejecuta pero deja de evaluar.	13
Watchdog de contexto	Mecanismo que vigila la saturación del contexto y dispara compresión o reinicio controlado.	13
Refutación externa	Contramedida que introduce un evaluador independiente para evitar que el agente valide su propio trabajo.	5, 13
Kata agéntica	Ejercicio acotado con restricciones y criterio de éxito, diseñado para entrenar a quien lo construye.	14
RACI con agentes	Matriz que asigna Responsable, Aprobador, Consultado e Informado en equipos híbridos; la *accountability* nunca se delega al agente.	15

Término	Definición operativa	Capítulo
DevSecOps para agentes	Integrar seguridad y permisos en el ciclo de vida del agente, no como revisión final.	20
Índice de autonomía	Métrica compuesta que resume cuánta libertad efectiva tiene un agente dado su entorno y sus permisos.	20
Taxonomía de responsabilidad agéntica	Clasificación de quién responde por un resultado según cuánto intervino el humano en su producción.	21
Código sintético	Código generado por un modelo; obliga a redefinir quién garantiza su corrección y su propósito.	21
Comité de arquitectura (defensa)	Instancia que evalúa viabilidad, costo y riesgo; el capítulo 4 enseña a defender una propuesta de multiagentes ante él.	4
ATAM	Método de análisis de *trade-offs* arquitectónicos; se usa aquí para estresar una propuesta antes de presentarla.	4
ADR (Architecture Decision Record)	Registro breve de una decisión de arquitectura: contexto, opciones, decisión y consecuencias.	18

Término	Definición operativa	Capítulo
Foso defensivo (***moat***)	Ventaja que no se replica copiando el modelo; criterio central del capítulo de monetización.	19

Índice de consulta rápida: problema → dónde acudir

Sección de apoyo. Este volumen se consulta por rol y por situación organizativa. La tabla traduce la situación en un capítulo, y señala el apoyo directo (plantilla o ejercicio resuelto) cuando existe.

1. Por situación

Si su situación es...	Acuda a	Apoyo directo
«Me piden que la IA cumpla una norma que no conozco»	Cap. 1 (marco regulatorio global)	—
«Tengo que defender una propuesta de multiagentes ante un comité»	Cap. 4 (método ATAM + panel de agentes críticos)	Ej. resuelto 4.1 (traducir riesgo al idioma del CFO)
«El negocio cree que la IA entregará más rápido de lo que entrega»	Cap. 10 (expectativas y tiempos)	—
«No sé si mi gente está preparada para orquestar agentes»	Cap. 8 (modelo de madurez agéntica)	—
«El equipo está quemado y la calidad cae»	Cap. 13 (síntoma del programador cansado, watchdog de contexto)	—

Si su situación es...	Acuda a	Apoyo directo
«Quiero montar sprints con agentes y no sé cómo»	Cap. 6 (plantilla de cadencia, roles del enjambre)	Ej. resuelto 6.1 (Quarterly Closing del enjambre)
«Necesito conectar la velocidad técnica con la del negocio»	Cap. 7 (DORA, patrón just-in-time)	Ej. resuelto 7.1 (valor de reducir el Lead Time)
«No sabemos qué permisos darle a un agente»	Cap. 20 (ciclo de vida del permiso, índice de autonomía)	—
«No está claro quién responde si el agente se equivoca»	Caps. 15 y 21 (RACI, responsabilidad del código sintético)	—
«Quiero empezar con algo pequeño sin jugarme la reputación»	Cap. 22 (MVP agéntico, marco RACE)	—
«Buscamos un caso completo que estudiar»	Caps. 3 y 5 (El Niño, caza de bugs)	Ej. resueltos 3.1 y 5.1
«Quiero entrenar al equipo con ejercicios evaluables»	Cap. 14 (katas)	—
«Quiero generar ingresos con esto»	Cap. 19 (cuatro arquetipos, foso defensivo)	—
«El equipo se perdió en el ecosistema de herramientas»	Cap. 18 (mapa de capas, disciplina de no adoptar)	—
«Necesito rescatar la agilidad de verdad»	Cap. 25 (los 12 principios reencarnados)	—

2. Índice de ejercicios resueltos del volumen

Ejercicio	Qué entrena	Capítulo
Ej. resuelto 3.1	Diseñar el Agente Crítico de un sistema de riesgo	3

Ejercicio	Qué entrena	Capítulo
Ej. resuelto 4.1	Traducir un riesgo técnico al idioma del CFO	4
Ej. resuelto 5.1	Poner al Agente Refutador en acción	5
Ej. resuelto 6.1	Diseñar el Quarterly Closing del enjambre	6
Ej. resuelto 7.1	Calcular el valor de reducir el Lead Time	7

3. Frontera con los otros volúmenes

Si su problema es de **arquitectura o fundamentos** (agentes, contexto, herramientas, orquestación), vaya al Vol. 1. Si es de **seguridad, industria o contexto geopolítico**, al Vol. 2. Si es de **escala y operación** (coste, testing, DR, métricas, observabilidad), al Vol. 4. Este volumen responde a **cómo organizar, gobernar y formar** alrededor de los sistemas, no a cómo construirlos.

4. Advertencia de uso

Las plantillas y marcos de este volumen **no son neutras respecto al contexto**: la plantilla de cadencia del capítulo 6 asume un equipo con autoridad para cambiar sus propias ceremonias, y el MVP agéntico del capítulo 22 asume un negocio capaz de tolerar un fallo barato. Antes de aplicar cualquier plantilla, verifique esas dos condiciones; si no se cumplen, el propio capítulo ofrece la versión degradada.

Capítulo 1: Ética y alineación en producción

El problema de la alineación a escala

Un agente que toma buenas decisiones en el 99 % de los casos pero falla catastróficamente en el 1 % restante no es un sistema de producción seguro si ese 1 % implica transferencias financieras incorrectas, datos médicos comprometidos o código de seguridad defectuoso. La estadística agrava la intuición: un sistema que ejecuta mil acciones diarias con un 1 % de fallo grave produce diez incidentes graves al día. En consecuencia, la escala convierte los casos límite en rutina.

La **alineación**, asegurar que los objetivos efectivos del sistema coinciden con los de sus operadores humanos, es el desafío central de la era agéntica, y su formulación técnica es anterior a los LLMs: Amodei et al. (2016) catalogaron los problemas concretos (efectos secundarios negativos, *reward hacking*, supervisión escalable, exploración segura, robustez al cambio de distribución) que hoy reaparecen, uno a uno, en los agentes de producción. Un agente que aprende a pasar los evals sin resolver la tarea está haciendo *reward hacking* de manual; un agente que funciona en staging y falla en producción sufre el cambio de distribución de siempre.

La respuesta de la industria combina cuatro capas, ninguna suficiente por sí sola:

1. **Alineación en el entrenamiento.** Las técnicas que convirtieron modelos de lenguaje en asistentes utilizables: el ajuste por retroalimentación humana (RLHF; Ouyang et al., 2022; Bai et al., 2022b) y la **IA constitucional**, donde el modelo se entre-

na para criticarse y corregirse contra un conjunto explícito de principios (una constitución) reduciendo la dependencia de la supervisión humana caso a caso (Bai et al., 2022a). Esta capa la aporta el proveedor del modelo; el AI Engineer la hereda, no la controla.

2. **Diseño constitucional del sistema.** La versión operativa de lo anterior, que sí está en manos del ingeniero: definir explícitamente qué puede y qué no puede hacer cada agente, permisos, herramientas, presupuestos, invariantes de dominio (capítulos 6 y 14).
3. **Supervisión humana en decisiones de alto impacto.** Automatizar la propuesta, exigir aprobación humana para la ejecución irreversible.
4. **Monitoreo de deriva.** Los modelos cambian: el proveedor los actualiza, el contexto evoluciona, los usuarios encuentran caminos nuevos. El comportamiento correcto de ayer no garantiza el de mañana; la suite de evals (capítulo 13) corriendo continuamente es el detector.

► **PISTA TÉCNICA: Supervisión humana para acciones críticas**

```python
# Ejemplo 1.1: Human-in-the-loop para acciones irreversibles.
# El agente propone; el humano aprueba; el sistema registra todo.

import time
from datetime import datetime

import anthropic

client = anthropic.Anthropic()

ACCIONES_CRITICAS = {
    "eliminar_base_de_datos",
    "desplegar_a_produccion",
    "modificar_permisos_usuarios",
    "cancelar_contratos",
    "enviar_comunicacion_masiva",
}

def ejecutar_con_guardia_humana(accion: str, parametros: dict,
                                agente_id: str) -> dict:
```

```
    """
    Para acciones irreversibles o de alto impacto, solicita aprobación
    humana antes de ejecutar. La lista de acciones críticas es política
    de la organización, vive en configuración versionada, y NINGÚN
    agente puede modificarla.
    """
    if accion in ACCIONES_CRITICAS:
        # Descripción legible de la acción propuesta (modelo económico)
        descripcion = client.messages.create(
            model="claude-haiku-4-5",
            max_tokens=200,
            messages=[{
                "role": "user",
                "content": (
                   "Describe en 2 oraciones qué hará esta acción y su "
                    f"impacto: {accion}({parametros})"
                ),
            }],
        )
        descripcion_texto = next(
            b.text for b in descripcion.content if b.type == "text"
        )

        solicitud_id = f"approval-{agente_id}-{int(time.time())}"
        approval_queue.push({
            "id": solicitud_id,
            "accion": accion,
            "parametros": parametros,
            "agente": agente_id,
            "descripcion_legible": descripcion_texto,
            "timestamp": datetime.now().isoformat(),
            "estado": "PENDIENTE_APROBACION",
        })
        return {
            "estado": "ESPERANDO_APROBACION",
            "solicitud_id": solicitud_id,
            "mensaje": (
                f"Acción '{accion}' requiere aprobación humana. "
                f"ID: {solicitud_id}"
            ),
        }

    # Acciones no críticas se ejecutan directamente (con auditoría)
    return ejecutar_accion(accion, parametros)
```

El detalle de gobierno que el código no muestra: la cola de apro-

baciones necesita un acuerdo de nivel de servicio. Si las aprobaciones tardan días, los equipos encontrarán formas de rodear el control, y el control que se rodea es peor que ninguno, porque da seguridad falsa. Aprobar debe ser fácil; solo ejecutar sin aprobar debe ser imposible.

El contexto regulatorio: de virtud a obligación

Lo que este libro ha presentado como buena ingeniería (trazabilidad, supervisión humana, gestión de riesgos, documentación) está dejando de ser opcional. El **Reglamento de Inteligencia Artificial de la Unión Europea** (Unión Europea, 2024), primer marco integral del mundo, clasifica los sistemas por riesgo e impone a los de alto riesgo obligaciones de gestión de riesgos, gobernanza de datos, documentación técnica, registro de eventos, transparencia y vigilancia humana, con aplicación escalonada desde 2025 y sanciones proporcionales a la facturación global. En Estados Unidos, el **AI Risk Management Framework** del NIST (2023) ofrece el estándar voluntario de referencia, organizado en cuatro funciones (gobernar, mapear, medir, gestionar) que encajan una a una con las prácticas de este libro. Para el lector latinoamericano: varios países de la región tienen proyectos legislativos inspirados en el modelo europeo; construir hoy con esos estándares es asegurar el producto contra el marco que viene.

La lectura estratégica: **el costo de cumplir es marginal para quien ya construyó con disciplina, y es un rediseño completo para quien no**. La auditoría del capítulo 14, los evals del capítulo 13 y el gateway del capítulo 3 son, además de buena ingeniería, la infraestructura de cumplimiento.

La dimensión ambiental

La honestidad intelectual obliga a incluir un costo que no aparece en la factura de la API: el energético. El entrenamiento de modelos grandes tiene una huella de carbono sustancial y medible (Strubell et al., 2019; Patterson et al., 2021), y aunque la inferencia individual es órdenes de magnitud más barata, la inferencia *a escala de industria*

domina ya el costo energético total del ciclo de vida de los modelos. Las palancas del AI Engineer son las mismas del capítulo 11 (el modelo adecuado a la tarea, caché, lotes, menos iteraciones desperdiciadas): la eficiencia económica y la ambiental, afortunadamente, apuntan en la misma dirección.

■ **PISTA EJECUTIVA**

La ética agéntica, en la práctica directiva, se reduce a tres preguntas con dueño:

1. **¿Quién responde por la acción del agente?** La respuesta legal y organizacional es siempre: una persona. Los agentes no tienen responsabilidad; la delegan hacia arriba. Cada agente en producción necesita un propietario humano nombrado, como cada servicio necesita un *on-call*.
2. **¿Qué decisiones nunca se delegan?** La lista de acciones de aprobación obligatoria es una decisión de directorio, no de sprint.
3. **¿Cómo sabemos que sigue alineado?** Deriva de comportamiento, evals continuos, auditoría muestreada. Además, «funcionaba cuando lo lanzamos» no es una respuesta; es la primera línea del informe del incidente.

Sesgo algorítmico y equidad distribucional

La dimensión del sesgo en sistemas agénticos permanece, en la práctica, sistemáticamente subgobernada en relación con su consecuencia operativa y legal, brecha que deviene litigiosa tan pronto el sistema opera a escala poblacional. Diecinueve fuentes distintas de sesgo a lo largo del pipeline de aprendizaje automático fueron catalogadas por Mehrabi et al. (2021) en el estudio más exhaustivo del campo hasta la fecha, y su taxonomía se mapea con perturbadora precisión sobre la arquitectura multi-etapa descrita en este libro: el modelo base hereda los sesgos incrustados en los corpus de preentrenamiento; la generación aumentada por recuperación (RAG) amplifica los sesgos de la colección documental indexada, un RAG de información

sanitaria entrenado predominantemente sobre literatura clínica anglosajona occidental es, estructuralmente, un amplificador de disparidades en salud; el juez evaluador (capítulo 13) puede reproducir los sesgos del agente que evalúa si ambos derivan de la misma familia de modelos; y la propia especificación de tarea puede activar preferencias latentes del modelo que ninguna auditoría de prompts revelaría sin pruebas estratificadas por subgrupos demográficos.

No existe un criterio de equidad universalmente correcto; la literatura empírica es categórica en este punto. La equidad individual (entradas similares producen salidas similares para individuos similares), la equidad de grupo en sus múltiples formulaciones (paridad demográfica, igualdad de *odds*, calibración intragrupal) y la equidad contrafáctica (los resultados serían idénticos en mundos donde los atributos protegidos difieren) optimizan valores distintos y parcialmente incompatibles entre sí. Como demostrara Chouldechova (2017) en el contexto de herramientas de predicción de reincidencia, y como formalizaran los teoremas de imposibilidad de Kleinberg et al. (2016) en términos generales, satisfacer simultáneamente más de uno de estos criterios resulta demostrablemente imposible cuando las tasas base difieren entre grupos. En consecuencia, la elección del criterio no es una decisión técnica delegable a un ingeniero individual: es una declaración de valores que porta peso ético, exposición legal y la firma organizacional de quien la adoptó, de forma explícita o por omisión estructural.

Para los sistemas clasificados como de alto riesgo bajo la Ley de IA de la UE (Unión Europea, 2024) (y, con mayor amplitud, para todo agente cuya salida afecte materialmente el acceso de individuos a empleo, crédito, servicios sanitarios, vivienda o servicios esenciales) un **protocolo de auditoría de sesgo** es simultáneamente una obligación regulatoria y una práctica de ingeniería defensiva. Su contenido mínimo: (a) declaración explícita del criterio de equidad adoptado, con justificación documentada de por qué ese criterio resulta apropiado para ese caso de uso concreto; (b) conjunto de evaluación estratificado construido para garantizar representación estadísticamente adecuada de los subgrupos protegidos, requisito que no puede satisfacerse mediante la partición del conjunto de prueba estándar, que típicamente infrarrrepresenta a las minorías por cons-

trucción; (c) medición del criterio seleccionado en *cada* actualización del modelo, cambio de prompt y revisión del contexto de dominio que llegue a producción, dado que las propiedades de equidad no se transfieren automáticamente entre versiones del modelo ni siquiera del mismo proveedor; y (d) reconocimiento documentado del sesgo residual y el razonamiento organizacional para su aceptación, puesto que la equidad perfecta es teóricamente inalcanzable y la ausencia de documentación constituye, en varias jurisdicciones, una exposición legal independiente.

► **PISTA TÉCNICA: Auditoría de sesgo estratificada, criterio de igualdad de odds**

```
# Ejemplo 1.2: Evaluación de equidad estratificada — criterio
equalized odds.
# Ejecutar en cada actualización del modelo y en cada cambio
sustantivo de prompt.
# «Equalized odds» exige emparejar tanto FPR como FNR entre grupos;
# otros criterios (paridad demográfica, calibración) difieren
algorítmicamente.
# Documente la elección: el código impone lo que la política
organizacional declara.

from dataclasses import dataclass
from typing import Dict, List

@dataclass
class MedicionGrupo:
    grupo: str
    n: int
    tasa_aprobacion: float
    fpr: float   # tasa de falsos positivos: P(ŷ=1 | y=0, grupo)
    fnr: float   # tasa de falsos negativos: P(ŷ=0 | y=1, grupo)

class AuditorEquidad:
    """
    Auditoría equalized-odds para decisiones agénticas que afectan
grupos protegidos.
    Umbral de despliegue: disparidad de FPR o FNR > 5 pp activa
revisión humana
    obligatoria antes del despliegue, con independencia de la
precisión agregada.
    Este umbral es decisión de política organizacional, no parámetro
del modelo.
    """
```

```
    UMBRAL_DESPLIEGUE = 0.05  # brecha de 5 puntos porcentuales activa
revisión

    def auditar(self, casos_por_grupo: Dict[str, List[dict]]) -> Dict:
        """
        Args:
            casos_por_grupo: {etiqueta_grupo: [{"prediccion": bool,
"verdad": bool}]}
        Returns: informe de auditoría con métricas por grupo y puerta
de despliegue.
        """
        mediciones: List[MedicionGrupo] = []

        for grupo, casos in casos_por_grupo.items():
            vp = sum(1 for c in casos if c["prediccion"] and
c["verdad"])
            fp = sum(1 for c in casos if c["prediccion"] and not
c["verdad"])
            fn = sum(1 for c in casos if not c["prediccion"] and
c["verdad"])
            vn = sum(1 for c in casos if not c["prediccion"] and not
c["verdad"])

            mediciones.append(MedicionGrupo(
                grupo=grupo,
                n=len(casos),
                tasa_aprobacion=(vp + vn) / len(casos) if casos else
0.0,
                fpr=fp / (fp + vn) if (fp + vn) > 0 else 0.0,
                fnr=fn / (fn + vp) if (fn + vp) > 0 else 0.0,
            ))

        brecha_fpr = max(m.fpr for m in mediciones) - min(m.fpr for m
in mediciones)
        brecha_fnr = max(m.fnr for m in mediciones) - min(m.fnr for m
in mediciones)
        puerta_ok = brecha_fpr <= self.UMBRAL_DESPLIEGUE and
brecha_fnr <= self.UMBRAL_DESPLIEGUE

        return {
            "criterio": "equalized_odds",
            "justificacion_criterio": "[DOCUMENTAR: especifique por
            qué este criterio aplica aquí]",
            "por_grupo": [
                {"grupo": m.grupo, "n": m.n,
```

```
                "tasa_aprobacion": round(m.tasa_aprobacion, 3),
                "fpr": round(m.fpr, 3),
                "fnr": round(m.fnr, 3)}
               for m in mediciones
            ],
            "max_disparidad_fpr": round(brecha_fpr, 3),
            "max_disparidad_fnr": round(brecha_fnr, 3),
            "puerta_despliegue_superada": puerta_ok,
            "accion_si_bloqueado":
            "revision_humana_obligatoria_antes_del_despliegue",
           "documentacion_requerida": True,  # Siempre: ningún sistema
           desplegado está libre de sesgo.
    }
```

■ **PISTA EJECUTIVA: El sesgo como categoría de riesgo estratégico**

Tres categorías de exposición distintas se materializan cuando el sesgo en un agente de producción pasa desapercibido hasta que un tercero externo lo descubre, encuadre que merece escrutinio, porque el descubrimiento externo es siempre la trayectoria de los sistemas no auditados. **Exposición legal** bajo la doctrina del impacto dispar, aplicable a las leyes de empleo, vivienda y crédito en múltiples jurisdicciones, complementada por las obligaciones del artículo 9 (gestión de riesgos) y el artículo 10 (gobernanza de datos) de la Ley de IA de la UE para sistemas de alto riesgo, con multas de hasta 30 millones de euros o el 6 % de la facturación anual global. **Exposición reputacional**: el sesgo algorítmico documentado genera cobertura mediática y atención regulatoria sostenidas que ningún presupuesto de comunicaciones recupera con rapidez, y que tiende a aflorar en momentos de estrés operativo, precisamente cuando la organización dispone de menor capacidad de respuesta. **Exposición operativa**: un agente sesgado optimiza el objetivo incorrecto a escala, y su corrección exige no solo un ajuste del modelo sino una auditoría retroactiva de todas las decisiones tomadas durante el período sesgado, un alcance que crece linealmente con el tiempo y logarítmicamente con el retraso en la detección. La auditoría estratificada descrita arriba cuesta céntimos por ejecución en la API batch y horas de analista por trimestre. Cada una de las tres categorías anteriores cuesta órdenes de magnitud más. La estructura de costos es inequívoca; la única variable es el momento en que se asume.

Transparencia, explicabilidad y derecho a la impugnación

La explicabilidad en el aprendizaje automático ha madurado hasta constituir una subdisciplina bien estructurada, cuyos conceptos, taxonomías y problemas abiertos fueron cartografiados de forma exhaustiva por Arrieta et al. (2020) en un artículo de referencia del campo, distinguiendo la interpretabilidad intrínseca (el modelo es comprensible por diseño: árboles de decisión, modelos lineales, reglas explícitas) de la explicabilidad post-hoc (el modelo es una caja negra; se aplican mecanismos de atribución separados tras la predicción: LIME, SHAP, explicaciones contrafácticas). El contexto agéntico, sin embargo, desplaza sustancialmente el desafío: la unidad de explicación es ahora una trayectoria multi-etapa de razonamiento, selección de herramientas, recuperación de datos externos y decisiones intermedias (cualquiera de las cuales puede haber influido decisivamente en el resultado final), ya no una única salida del modelo. Ese desplazamiento porta consecuencia regulatoria directa, porque ni el derecho a la explicación del artículo 86 de la Ley de IA de la UE ni el derecho a revisión humana del artículo 22 del RGPD eximen a las arquitecturas multi-etapa complejas; la obligación escala con el impacto de la decisión, no con la simplicidad del sistema.

La respuesta estructural disponible en un sistema agéntico bien instrumentado es la traza distribuida (capítulo 18). Una traza producida por un bucle ReAct correctamente atribuido contiene, en cada nodo de procesamiento: el identificador de versión del prompt del sistema; los documentos recuperados con sus puntuaciones de relevancia y marcas de tiempo; el razonamiento del modelo en cadena de pensamiento (cuando el modo de reflexión está habilitado); la selección de herramienta, su entrada y su resultado; y la lógica de transición que conduce al paso siguiente. Ese registro constituye, operativamente, una explicación de la trayectoria decisional: no de las matrices de pesos internas del modelo (que permanecen opacas), sino del proceso de decisión, que es precisamente el nivel de

transparencia que los marcos regulatorios exigen y que los individuos afectados pueden impugnar con sentido. En consecuencia, la infraestructura de explicabilidad no es una capacidad separada que construir desde cero: es la infraestructura de observabilidad del capítulo 18, leída bajo un objetivo regulatorio distinto.

Dos brechas técnicas permanecen abiertas que la práctica honesta debe reconocer. **La atribución de contexto** (determinar qué fragmento de un contexto largo y multi-componente (el prompt del sistema, el primer documento recuperado, un turno previo de conversación) influyó más en una salida concreta del modelo) es un problema de investigación activo; los métodos de atribución por perturbación y los de desenrollado de atención presentan modos de fallo documentados en contextos largos y conversaciones multi-turno (Jain & Wallace, 2019), y el campo de la interpretabilidad agéntica se encuentra en estadio inicial (Conmy et al., 2023; Elhage et al., 2021). **Los contrafácticos de trayectoria** (reconstruir qué habría decidido el agente si una entrada intermedia específica hubiera diferido) son tratables para clasificadores simples e intratables en la práctica para pipelines multi-etapa estocásticos en el momento de escritura. La postura práctica del ingeniero en el ínterin es maximizar la trazabilidad por construcción, minimizar la opacidad en el tratamiento de datos, y construir la vía de impugnación como proceso organizacional: un propietario humano nombrado (capítulos 14 y 19) que pueda revisar la traza y trasladarla en lenguaje llano al individuo afectado. La arquitectura que este libro construye no aguarda a que la investigación en interpretabilidad madure; proporciona la trazabilidad estructural que hace posible la impugnación ahora.

El panorama regulatorio global: construir una sola vez para el estándar más exigente

La geografía regulatoria de la inteligencia artificial atravesó una transición estructural en el período 2023-2026, desplazándose de directrices voluntarias a obligaciones vinculantes con sanciones económicas sustanciales y, en determinadas jurisdicciones, remisión

penal. Comprender esa transición no es un ejercicio de cumplimiento delegable al departamento jurídico: es una restricción de diseño de producto que determina qué decisiones arquitectónicas son reversibles y cuáles no.

Tres capas de gobernanza requieren atención operativa de la dirección de ingeniería. A nivel **internacional**, la norma ISO/IEC 42001:2023 (el primer estándar formal de sistema de gestión para la inteligencia artificial) impone un ciclo planificar-hacer-verificar-actuar estructuralmente idéntico al de ISO 9001 (calidad) e ISO 27001 (seguridad de la información): identificación de riesgos, controles de tratamiento documentados, auditoría interna y revisión por la dirección. Su adopción sigue siendo formalmente voluntaria, pero es exigida de manera creciente en los contratos de aprovisionamiento empresarial y en los procesos de diligencia debida de la cadena de suministro; un equipo que ya opera un sistema de gestión integrado puede extenderlo al cumplimiento en IA con un esfuerzo incremental considerablemente menor que uno que parte de cero.

A nivel **supranacional**, la Ley de IA de la UE (Reglamento 2024/1689) establece el estándar vinculante más exigente para las organizaciones que operan en mercados europeos o desarrollan sistemas utilizados en ellos. Su árbol de clasificación de riesgos genera tres nodos decisionales para el ingeniero: prácticas **prohibidas** (manipulación subliminal, puntuación social de ciudadanos, identificación biométrica remota en tiempo real en espacios públicos), que no requieren deliberación, siendo la infracción objeto de remisión penal y multas de hasta 35 millones de euros o el 7 % de la facturación anual global; sistemas de **alto riesgo** (IA en selección de personal, evaluación crediticia, suscripción de seguros, evaluación educativa, servicios esenciales, aplicación de la ley, gestión de infraestructuras críticas), que activan el paquete completo de cumplimiento: evaluación de conformidad, documentación técnica, registro de eventos, divulgaciones de transparencia, supervisión humana e inscripción en la base de datos de la UE; y sistemas de **riesgo limitado y mínimo**: sujetos únicamente a las obligaciones de transparencia del artículo 52. Para la mayoría de las herramientas internas de desarrollo, la clasificación es limitada o mínima; para

los productos que inciden en cualquiera de los dominios de alto riesgo en mercados de estados miembros de la UE, las obligaciones se aplican desde agosto de 2026.

En **Estados Unidos**, el mosaico sectorial persiste. Las directrices de la FTC sobre prácticas comerciales habilitadas por IA, la asistencia técnica de la EEOC sobre el uso de procedimientos automatizados de selección por parte de empleadores, la guía de gestión de riesgo de modelos SR 11-7 de la OCC (aplicable a modelos de IA en banca) y el marco en evolución de la FDA para el *Software as a Medical Device* habilitado por IA imponen obligaciones específicas de dominio que se superponen al Marco de Gestión de Riesgos de IA del NIST 1.0 (2023, con Perfil de IA Generativa en 2024). El ciclo Gobernar-Mapear-Medir-Gestionar del NIST RMF ofrece la capa de integración federal más coherente, y sus cuatro funciones se corresponden con notable precisión con las prácticas de este libro: Gobernar equivale al registro de agentes, la política de acciones críticas y la gobernanza del presupuesto del capítulo 3; Mapear equivale al modelo de amenazas del capítulo 14; Medir equivale a la suite de evals y el CIU de los capítulos 11 y 13; Gestionar equivale al bucle de MLOps del capítulo 18 y al plan de respuesta a incidentes implícito en el log de auditoría del capítulo 14.

La síntesis que la práctica de ingeniería debe interiorizar: **construir una sola vez para las obligaciones de alto riesgo de la Ley de IA de la UE**: trazabilidad completa, supervisión humana de las decisiones irreversibles, auditoría de sesgo documentada, explicaciones impugnables y propietario humano nombrado por cada agente en producción. Esa arquitectura satisface el NIST RMF, la ISO 42001 y todos los marcos sectoriales vigentes en el momento de escritura, dado que estos marcos son subconjuntos propios de las obligaciones europeas en su carga de cumplimiento. La vía inversa (construir para el estándar mínimo aplicable y actualizar cuando la expansión hacia dominios regulados o mercados europeos requiera cumplimiento) reproduce en la capa de gobernanza la misma dinámica económica que la deuda técnica del capítulo 11: coste diferido, agravado y liquidado en el momento de mayor presión operativa y reputacional.

Lo esencial del capítulo 1

- A escala, los casos límite se convierten en rutina: un 1 % de fallo grave sobre mil acciones diarias produce diez incidentes graves al día.
- Los problemas de alineación tenían nombre antes de los LLMs: *reward hacking*, supervisión escalable, cambio de distribución (Amodei et al., 2016). La IA constitucional y el RLHF abordan la capa del modelo; las capas del sistema y del humano siguen siendo responsabilidad del ingeniero (Bai et al., 2022a, 2022b; Ouyang et al., 2022).
- Cuatro capas de alineación, ninguna suficiente por sí sola: alineación del modelo en el entrenamiento, diseño constitucional del sistema, supervisión humana de lo irreversible, y monitoreo continuo de la deriva mediante la suite de evals.
- El sesgo algorítmico opera en cada capa del pipeline agéntico, corpus de preentrenamiento, índice de recuperación, modelo juez, especificación de tarea. No existe un criterio de equidad universalmente correcto; la elección es una declaración de valores con peso legal (Mehrabi et al., 2021; Chouldechova, 2017). La auditoría estratificada en cada actualización del modelo constituye la práctica mínima defendible.
- La explicabilidad para agentes multi-etapa se logra a través de la traza, no de los pesos: las trazas bien instrumentadas (capítulo 18) satisfacen el derecho regulatorio a la explicación sin aguardar a que la investigación en interpretabilidad madure (Arrieta et al., 2020; Conmy et al., 2023).
- Tres capas de gobernanza requieren atención operativa: ISO/IEC 42001:2023 (internacional, voluntaria pero exigida contractualmente), Ley de IA UE 2024/1689 (supranacional, vinculante desde agosto de 2026 para alto riesgo) y NIST AI RMF (federal estadounidense, voluntario pero estructuralmente completo).
- Construir una sola vez para el estándar más exigente: las obligaciones de alto riesgo de la Ley de IA de la UE (trazabilidad, supervisión humana, auditoría de sesgo, explicaciones impugnables, propietario nombrado por agente) satisfacen por derivación estructural todos los marcos menores.

- Un control que se elude es peor que ninguno. Aprobar debe ser sin fricciones; ejecutar sin aprobación, arquitectónicamente imposible.

Capítulo 2: El futuro, el Ingeniero Orquestador

El nuevo contrato social de la ingeniería de software

El futuro no pertenece al programador que escribe más rápido. Pertenece al **Ingeniero Orquestador**, quien domina el arte de hacer que múltiples inteligencias especializadas colaboren hacia un objetivo de negocio con la máxima eficiencia económica y la mínima deuda técnica.

Esta transición es comparable a la que vivió la ingeniería civil cuando pasó del cálculo manual al asistido: los ingenieros estructurales de hoy no calculan a mano las cargas de un puente; diseñan, modelan, verifican y *responden* por el resultado. Nadie diría que son menos ingenieros que sus predecesores; son ingenieros de un nivel de abstracción superior. Los ingenieros de software de la próxima década no escribirán la mayoría de sus bucles; diseñarán los sistemas que los escriben, los validan y los despliegan, y responderán por el resultado.

Figura 1: Diagrama 2.1: La evolución del rol del engineer

La evidencia disponible apunta a una transformación del rol antes que a una sustitución. Los estudios de productividad muestran a la IA amplificando el output del ingeniero (Peng et al., 2023; Ziegler et al., 2024); los benchmarks de tareas reales muestran que la autonomía completa sigue lejos en los problemas que importan (Jimenez

et al., 2024); y la investigación de evaluación insiste en que el valor de los agentes depende críticamente del arnés humano que los rodea (Kapoor et al., 2024). El patrón histórico es conocido: la abstracción no eliminó a los programadores; el compilador, el *garbage collector* y la nube los multiplicaron. Lo que sí elimina cada salto de abstracción es la ventaja competitiva de las destrezas que automatiza.

Las cinco competencias del Ingeniero Orquestador

Justificación de diseño: diagrama de clases para la jerarquía de competencias

Las competencias del Ingeniero Orquestador forman una jerarquía: conocimientos que se construyen uno sobre otro (arquitectura → tokens → evals → seguridad → negocio). Un diagrama que mostra esa relación (incluido en este caso una pirámide o estructura de capas similar a una clase jerarquizada) revela exactamente eso: el apilamiento de conceptos que ninguno de ellos es suficiente sin los otros.

Un diagrama de componentes mostraría dónde viven esas competencias en un sistema; un diagrama de secuencias mostraría cómo se aplican secuencialmente. Aquí necesitamos ver *la estructura de conceptos*, la taxonomía estática que define al Ingeniero Orquestador.

Este es el punto final donde reaparece el diagrama de clases: aprendiste la notación en el Primer UML, la viste en Caps. 5 (roles), 6 (DDD), 16 (Clean Arch), y ahora reconoces que la misma estructura (clases, relaciones, jerarquías) es la que te permite pensar sobre sistemas.

Figura 2: Diagrama 2.2: Las cinco competencias del Ingeniero Orquestador (jerarquía de competencias)

1. **Pensamiento arquitectónico sobre sistemas cognitivos.** Diseñar topologías de agentes (capítulo 8) como hoy se diseñan

topologías de servicios: con contratos, presupuestos y modos de fallo explícitos.
2. **Economía aplicada del token.** Tratar el costo de inferencia como variable de diseño de primera clase (capítulos 11 y 12).
3. **Ingeniería de la evaluación.** Construir los arneses que convierten un sistema no determinista en un sistema gobernable (capítulo 13).
4. **Seguridad y gobierno por construcción.** Mínimo privilegio, auditoría, supervisión humana (capítulos 14 y 19) como reflejos, no como apéndices.
5. **Traducción negocio-máquina.** El lenguaje ubicuo del dominio (capítulo 6) convertido en contexto operativo de los agentes. Es la competencia más antigua de la lista: entender el problema antes que la solución, y la que ningún salto de abstracción ha automatizado jamás.

El horizonte proactivo: de agentes que responden a agentes que anticipan

Ahora bien, si el lector retiene una sola dirección de futuro de este libro, que sea esta: la línea de avance de los sistemas agénticos va **de reaccionar a anticipar**. Casi todo lo construido en estos capítulos asume un agente reactivo, espera un *prompt*, una incidencia, un evento, y entonces actúa. La frontera, en cambio, son los agentes **proactivos**: los que observan de forma continua, infieren una necesidad latente antes de que nadie la formule y proponen (o ejecutan, bajo guardia) la acción que la atiende. La industria ya les puso nombre comercial: los **agentes ambientales** que corren en segundo plano y reclaman al humano solo cuando detectan algo digno de su atención (Chase, 2025), y de los que las grandes plataformas hacen su apuesta declarada para la segunda mitad de esta década. La capacidad técnica que lo hace posible no es nueva: es la arquitectura orientada a eventos del capítulo 17, un agente proactivo bien construido *es* un consumidor del log, disparado por hechos, no por personas.

Aquí es donde este libro se separa del entusiasmo. **Donde el mercado ve una idea, el ingeniero serio ve un agente que tendría que realizarla, y pregunta primero qué control cede y qué puede sa-**

lir mal. La crítica más útil del campo es también la más incómoda: cuanto más control delega un humano en un agente, más riesgo para las personas se introduce, y la proactividad es, por definición, delegación de control (Mitchell et al., 2025). De ahí se sigue encuadrar la investigación, no frenarla: la respuesta de ingeniería es la **proactividad acotada**, no la autonomía total, un agente que anticipa dentro de una frontera estrecha, propone en lugar de ejecutar lo irreversible (capítulo 19) y opera sobre un esqueleto determinista (el caso sectorial del capítulo 15). La proactividad no deroga ninguna disciplina de este libro; las exige todas con más fuerza, porque un error anticipado a escala se comete antes de que nadie lo pida.

Y aquí la frontera se vuelve concreta, y socialmente útil. Júntese el agente proactivo con dos tecnologías que ya sirven a la sociedad: el **Internet de las cosas** que tiende sensores sobre el mundo físico y los **drones** que llegan donde el humano no puede. La NASA integra desde hace años datos satelitales y sensores en tierra para detectar y responder a desastres naturales (incendios, inundaciones, terremotos) casi en tiempo real (NASA, s.f.); las agencias humanitarias usan drones para buscar supervivientes tras un terremoto, remapear campos minados que una inundación desplazó y llevar suministros a zonas aisladas (Programa Mundial de Alimentos, s.f.). En todos esos escenarios, el cuello de botella no es capturar el dato: es que un humano lo mire a tiempo. Ahí encaja, con precisión, el patrón que este libro ya formalizó: un **agente enriquecedor proactivo** sobre el flujo de telemetría (*CDC* → *agente económico* → *consumidores* (capítulo 17)) que clasifica, prioriza y alerta sobre el sensor o el dron, sin tocar los sistemas de origen y sin tomar jamás la decisión irreversible: la columna vertebral sigue siendo determinista, y el humano decide (capítulo 15). El cómo no es ciencia ficción; es composición de piezas que el lector ya tiene en la mano. Un agente que anticipa una inundación con una hora de ventaja, o que reduce el campo de búsqueda de un equipo de rescate, no es una demo: es ingeniería al servicio de la paz y de la vida.

Esta dirección abre (deliberadamente) más preguntas de las que cierra, y ese es su valor como programa de investigación. ¿Cómo se *evalúa* la proactividad, cuando el éxito es una interrupción oportuna y el fracaso, una intromisión molesta? La medición rigurosa de

agentes proactivos es un campo que apenas nació (las primeras suites de evaluación específicas son de 2025) y que hereda, agravado, el problema del capítulo 13. ¿Cuál es la **economía** de un agente *siempre encendido*? Un consumidor proactivo sin presupuesto ni *backpressure* (capítulos 11 y 17) es Token-Burn permanente por diseño; la frontera proactiva no deroga la del token, la tensa. Y ¿cómo se gobierna la *confianza* en un sistema que actúa antes de que se lo pidan? Estas son las vetas que la próxima década tendrá que excavar, y este libro las deja señaladas a propósito: no como humo, sino como mapa. Donde otros prometen agentes que lo harán todo, nosotros dejamos escrito el método para construir los que harán *algo* (bien, barato y bajo control), que es la única forma honesta de llegar primero.

Cierre del volumen: el arte de dirigir la orquesta

Cuando empecé a escribir estas páginas, el debate sobre los agentes de IA se dividía en dos campos: los que prometían que el código se escribiría solo y los que argumentaban que todo era humo y marketing. Ninguno tenía razón, pero el primero vendía más.

Tardamos veinte capítulos en articular la tercera postura, la del ingeniero que ni se deja arrastrar por el entusiasmo ni se paraliza por el escepticismo: la postura de quien construye. Es la única que produce sistemas en producción.

El recorrido tiene una lógica deliberada. Empezamos por el fundamento: entender qué es un agente de verdad, no la versión de marketing, sino el bucle de control que razona, actúa y observa. Aplicamos después los principios que llevan décadas probándose sobre sistemas complejos: SOLID, arquitectura limpia, cohesión, bajo acoplamiento. No hay razón para abandonar la ingeniería seria cuando el modelo entra en escena; al contrario, sus defectos la exigen con más fuerza. Pasamos por los patrones de orquestación: el consejo de agentes, la delegación controlada, la jerarquía con presupuesto. Nos enfrentamos a la economía real del token: la métrica CIU, el costo oculto que no aparece en el balance, porque construir bien implica saber cuánto cuesta construir y mantener encendido. Exploramos el stack moderno: la arquitectura orientada a eventos, la observabilidad que convierte el bucle de razonamiento en una traza legible,

el LLMOps que distingue el experimento del sistema vivo. Llegamos, finalmente, al horizonte: la ética como diseño de guardianes y la proactividad acotada como la siguiente frontera de ingeniería.

Lo que tienes ahora no es solo conocimiento: es un conjunto de herramientas con las que puedes ir al código y construir. Ocho repositorios de demostración. Un harness de evaluación. Un playbook de seis casos. Un ciclo completo de observabilidad. Todo probado, con tests, con README bilingüe, bajo licencia MIT.

Una advertencia honesta antes de cerrar: esta tecnología se moverá. Los modelos que nombramos en el apéndice B tendrán sucesores en meses; algún patrón que hoy parece definitivo mutará cuando la ventana de contexto crezca un orden de magnitud más. Lo que no cambiará son los principios que usamos para diseñar, evaluar y controlar. Eso es lo que este libro intentó enseñar: no los comandos de la API, sino el criterio para usarla bien.

El resto depende de ti. Los mejores sistemas agénticos que existan en cinco años los construirán personas que hoy están leyendo exactamente lo que tú acabas de leer.

La inteligencia se volvió abundante; el criterio sigue siendo escaso. El Ingeniero Orquestador que domine esas disciplinas en esta década estará donde el desarrollador web que dominó JavaScript y las APIs REST estuvo en 2010: en la cresta de una ola que elevará a todos los que decidan surfearla con seriedad.

La orquesta ya está afinando. La batuta es suya.

Lo esencial del capítulo 2

- La evidencia apunta a transformación del rol, no a sustitución: la IA amplifica (Peng et al., 2023; Ziegler et al., 2024) pero la autonomía completa sigue lejos en los problemas reales (Jimenez et al., 2024).
- Cada salto de abstracción multiplicó a los programadores, y eliminó la ventaja de las destrezas que automatizó.
- Cinco competencias del Ingeniero Orquestador: arquitectura de sistemas cognitivos, economía del token, ingeniería de evaluación, seguridad por construcción, traducción negocio-máquina.

- La inteligencia se volvió abundante; el criterio sigue siendo escaso, y tiene métodos, métricas y disciplina propios.
- La frontera va de reaccionar a anticipar: agentes proactivos/ambientales (Chase, 2025) montados sobre el log de eventos (capítulo 17). Más proactividad es más control cedido y más riesgo (Mitchell et al., 2025); la respuesta es proactividad acotada, no autonomía total.
- Agente proactivo + IoT + drones = ingeniería al servicio de la sociedad (alerta temprana, rescate, ayuda humanitaria), con el patrón CDC → agente enriquecedor → consumidores (capítulo 17) y columna determinista (capítulo 15). Evaluar la proactividad y su economía «siempre encendida» son las vetas de investigación abiertas.

Capítulo 3: Multiagentes de IA ante El Niño

No basta con predecir el desastre; hay que orquestar la respuesta antes de que el agua llegue.

Sección 1: La Anatomía del Problema: Más Allá de la Meteorología

El fenómeno de El Niño (y su variante costera, El Niño Costero, que golpea con particular violencia el litoral peruano) es, ante todo, un sistema complejo. No un evento singular, sino una cadena de causalidades entrelazadas: anomalías térmicas superficiales en el océano Pacífico central y oriental que, semanas después, se traducen en lluvias torrenciales, desbordamientos de ríos, huaicos, pérdida de cosechas y colapso de infraestructura sanitaria. El evento de 2017 generó daños estimados en más de 3,100 millones de dólares solo en Perú (CENEPRED, 2017), cifra que el de 2023 amenazó con superar.

Ahora bien, el problema técnico de fondo no radica en la escasez de datos. Los satélites Sentinel-3 y GOES-East de la NASA/ESA, las más de 300 boyas oceanográficas del sistema Argo, las estaciones hidrometeorológicas del SENAMHI y los modelos de circulación oceánica del IGP producen, en conjunto, terabytes de información por hora. La fractura se produce en otro nivel: entre quien sabe, quien decide y quien actúa. Entre el científico que detecta la anomalía SST (sea surface temperature) en el Niño 3.4, el funcionario que debe

decretar la alerta temprana, el alcalde que gestiona la evacuación y el logístico que mueve los recursos. Cada actor habita un silo institucional distinto, opera con un vocabulario técnico diferente y tiene incentivos que rara vez se alinean en tiempo real.

Cabe puntualizar que este problema es, en esencia, un problema de orquestación distribuida bajo incertidumbre. Y la orquestación distribuida bajo incertidumbre es, precisamente, el dominio para el que los sistemas multiagente fueron concebidos.

Rolnick et al. (2022), en su revisión exhaustiva publicada en *ACM Computing Surveys* (probablemente el artículo más citado de la intersección entre machine learning y cambio climático (más de 2,000 citas a la fecha de esta edición)), identificaron la coordinación inter-institucional como el cuello de botella principal en la respuesta a desastres climáticos, por encima incluso de la precisión predictiva. Dicho de otro modo: tener el modelo correcto no sirve si la cadena de decisión está rota.

Dimensión del problema	Evidencia documentada	Implicación arquitectónica
Latencia de alerta	El Niño 2017: tiempo promedio entre detección SENAMHI y alerta INDECI: 72 horas	Necesitamos agentes que publiquen eventos en tiempo real, no informes periódicos
Fragmentación de datos	IGP, ANA, SENAMHI usan formatos incompatibles (NetCDF vs CSV vs APIs REST propias)	Capa de normalización obligatoria (agentes de percepción)
Toma de decisiones	COEN, COER y COEL toman decisiones en reuniones presenciales con datos de ayer	Capa de síntesis y recomendación con trazabilidad
Coordinación regional	Perú, Ecuador y Colombia gestionan la misma cuenca hidrográfica de forma desconectada	Federación de enjambres nacionales con interoperabilidad

Sección 2: Estado del Arte: IA Predictiva Aplicada a Fenómenos Climáticos

La predicción numérica del tiempo ha sido, durante décadas, el dominio exclusivo de modelos de circulación general (GCM) ejecutados en supercomputadoras de alta capacidad. Lo que ocurrió entre 2022 y 2024 alteró ese paradigma de forma estructural: los modelos basados en aprendizaje profundo comenzaron a superar a los GCM en precisión y en velocidad por márgenes que los investigadores del ECMWF describieron como "sorprendentes".

FourCastNet (Pathak et al., 2022): desarrollado por NVIDIA, este modelo de redes neuronales entrenado sobre datos del ERA5 (ECMWF Reanalysis v5) genera predicciones globales de hasta 7 días con una velocidad 45,000 veces superior a los modelos numéricos tradicionales, manteniendo una habilidad comparable. El umbral fue decisivo: por primera vez, la predicción meteorológica podía ejecutarse en dispositivos de borde, no solo en centros de cómputo de alta gama.

Pangu-Weather (Bi et al., 2022), publicado en *Nature* (revista Nature, factor de impacto 64.8), representó el siguiente escalón. Mediante una arquitectura Transformer tridimensional entrenada sobre 39 años de datos del ERA5, Pangu-Weather superó al sistema de predicción del ECMWF en el 75% de las variables evaluadas a un horizonte de 5 días. A título ilustrativo: su predicción de la trayectoria del huracán Dora (2023) fue más precisa que la de cualquier modelo operacional existente.

GraphCast (Lam et al., 2022), publicado en *Science* (otro Q1 indiscutible, IF 56.9), empujó el límite aún más lejos. Entrenado sobre la misma base ERA5 pero con una arquitectura de grafo espaciotemporal, GraphCast superó al ECMWF en el 90.3% de las 1,380 variables evaluadas a 10 días de predicción, reduciendo el tiempo de cómputo de horas a menos de un minuto. Su predicción del ciclón Freddy (2023) (el más largo de la historia registrada) fue el ejemplo más dramático de transferencia de capacidad predictiva a dominios extremos.

A tenor de lo expuesto, la conclusión para un arquitecto de sistemas es inequívoca: **la precisión predictiva está resuelta, o en vías de resolverse**. El déficit no está en el modelo; está en la cadena de uso de ese modelo. Ninguno de estos tres sistemas (FourCastNet, Pangu-Weather, GraphCast) viene con un mecanismo de coordinación institucional incluido. Producen números; no coordinan evacuaciones.

Irrgang et al. (2021), en *Nature Machine Intelligence*, introducen el concepto de "neural Earth system modelling": integrar IA en los flujos de trabajo de ciencias de la Tierra no como sustituto de los modelos físicos, sino como complemento. Lo relevante para este capítulo: los autores documentan que la principal barrera para la adopción es organizacional, no técnica (los datos existen, los modelos existen, pero los pipelines de integración inter-agencia no). Esta es exactamente la brecha que un sistema multiagente puede cerrar.

Sección 3: Arquitectura Técnica Expandida: Las Cuatro Capas en Detalle

► PISTA TÉCNICA

La Figura 3.1 resume la arquitectura completa antes de entrar al detalle de cada capa: los datos crudos entran por abajo, atraviesan percepción y razonamiento especializado, se sintetizan en la capa de orquestación y llegan al humano ya convertidos en una recomendación trazable, nunca en una orden.

Figura 3: Diagrama 3.1: Arquitectura de 4 capas: Percepción → Razonamiento → Orquestación → Gobernanza

Obsérvese que las flechas entre capas son bidireccionales solo entre Orquestación y Gobernanza: el humano puede rechazar una recomendación y forzar una nueva síntesis, pero nunca escribe directamente en la capa de razonamiento. Esa asimetría es deliberada, es el mecanismo que impide que la supervisión humana se convierta

en un cuello de botella operativo sin dejar de ser, a la vez, la autoridad final.

Capa 1: Percepción: Normalizar la Realidad Heterogénea

Los agentes de percepción son los traductores del sistema. Su función es transformar datos crudos y heterogéneos en un formato semántico compartido, no razonar. En términos técnicos, implementan el patrón **Anti-Corruption Layer** (Fowler, 2003) aplicado a datos climáticos.

Cada agente de percepción se conecta a una fuente específica mediante el Model Context Protocol (MCP, Anthropic 2024), que garantiza que el agente nunca maneja credenciales de acceso directamente (estas son mediadas por el servidor MCP con autenticación OAuth2 y registro de auditoría). Los formatos ingeridos incluyen:

- **NetCDF** (SENAMHI, IGP): archivos de variables climáticas multidimensionales con coordenadas espacio-temporales. El agente de percepción los convierte a JSON geoespacial normalizado con timestamps UTC.
- **CSV y XLSX** (ANA, INDECI): archivos de estaciones hidrometeorológicas. El agente los homologa al esquema de events internos del bus de mensajes.
- **APIs REST** (NASA EOSDIS, ESA Copernicus): respuestas JSON con imágenes satelitales y datos de nivel del mar. El agente las normaliza y georeferencia.
- **Streams MQTT** (sensores IoT de caudal en cuencas): mensajes de baja latencia que el agente convierte a eventos Kafka con metadata de calidad de señal.

El principio rector: **un único contrato de datos interno**. Todo lo que entra al bus de eventos tiene el mismo esquema, independientemente de la fuente original.

Capa 2: Razonamiento Especializado: Los Expertos Sintéticos

En esta capa residen cuatro agentes especializados, cada uno entrenado (mediante fine-tuning o system prompting de alta especifici-

dad) en un dominio distinto:

Agente Hidrológico. Su dominio es el ciclo del agua: caudales, umbrales de desbordamiento, modelos de cuencas. Consume los eventos del agente de percepción para SENAMHI y ANA, ejecuta cálculos de tiempo de concentración de cuencas (fórmula de Kirpich: `Tc = 0.0195 × (L³/H)^0.385` donde L es la longitud del cauce en metros y H es la diferencia de elevación), y publica eventos como `"Río Rímac: caudal 95th percentile en 18h, confianza: 0.87"`.

Agente Epidemiológico. Activa tras una alerta hidrológica. Los brotes de dengue, leptospirosis y cólera que siguen a inundaciones siguen patrones documentados (OPS/OMS, series históricas 2000-2023). Este agente cruza los mapas de inundación proyectados con los registros de vectores activos (Aedes aegypti, criaderos) y publica alertas de riesgo sanitario geolocalizado con horizonte de 15 días.

Agente Logístico. Su función es optimizar rutas de auxilio bajo la restricción de infraestructura degradada. Implementa una variante del Vehicle Routing Problem con Time Windows (VRPTW), resuelto con algoritmos metaheurísticos (Simulated Annealing para instancias <200 nodos, escalando a Tabu Search para instancias mayores). Publica el plan óptimo de distribución de agua potable, comida y medicamentos hacia las zonas de impacto proyectado.

Agente Agrícola. Estima pérdidas de campaña cruzando proyecciones de inundación con el mapa de cultivos en producción (datos MIDAGRI). Sus salidas alimentan tanto el plan logístico (qué zonas necesitarán soporte alimentario) como la valoración de daños para la declaratoria de emergencia.

Capa 3: Orquestación y Consenso: Donde el Enjambre Decide

Un bus de eventos Apache Kafka actúa como sistema nervioso. Cada agente de la Capa 2 es un consumidor y un productor: lee eventos del bus, procesa en su dominio y publica sus conclusiones como nuevos eventos. El **Agente Coordinador** agrega estas conclusiones aplicando un esquema de ponderación basado en incertidumbre (los eventos con mayor confidence score tienen mayor peso en la recomendación final).

Críticamente, existe un **Agente Crítico** cuya función es adversarial: su único trabajo es identificar inconsistencias entre las conclusiones de los agentes especializados. ¿El agente hidrológico proyecta desbordamiento en 18 horas pero el agente logístico planifica rutas por esa misma zona? El Agente Crítico detecta la contradicción y publica un evento de conflicto que obliga a renegociación. Este mecanismo implementa, en el dominio del software, el principio popperiano de falsabilidad: una conclusión vale por las refutaciones que sobrevivió (cf. Cap. 25).

Capa 4: Gobernanza Humana: La Máquina Sustenta, el Humano Firma

Ningún agente del sistema emite órdenes de evacuación. La Capa 4 materializa el principio de **Human-in-the-Loop** en su forma más estricta: el enjambre entrega recomendaciones, no decisiones. Cada recomendación viene acompañada de:

1. Nivel de confianza agregado (escala 0–1, calibrado mediante *reliability diagrams* sobre el historial de predicciones).
2. Cadena de evidencia trazable: qué datos originaron qué conclusión en qué agente.
3. Escenarios alternativos: la recomendación de mayor confianza + las dos alternativas de menor costo si la principal falla.

Los tomadores de decisión en el COEN (Centro de Operaciones de Emergencia Nacional) interactúan con un dashboard que muestra estas recomendaciones en lenguaje natural, con mapas interactivos y la trazabilidad completa accesible con un clic. El funcionario firma la evacuación; el sistema la preparó.

Sección 4: Perspectiva Senior Engineer y QA: Confiabilidad del Sistema Bajo Presión

► PISTA TÉCNICA

Un sistema de gestión de desastres que falla durante el desastre es, en el mejor de los casos, inútil; en el peor, contraproducente. La confiabilidad no es una característica opcional: es la característica central.

Validación por backtesting histórico. La estrategia de testing más robusta disponible para este tipo de sistema es ejecutarlo contra datos históricos de eventos pasados (El Niño 1997-1998, El Niño Costero 2017) y medir si sus recomendaciones habrían sido las correctas. Las métricas relevantes no son las de un clasificador genérico: son el **Brier Score** (calibración de probabilidades), la **Resolución** (capacidad de discriminar entre eventos) y el **Reliability Diagram** (¿cuando el agente dice "90% de probabilidad", acierta el 90% de las veces?). Un agente hidrológico con un Brier Score superior a 0.25 en eventos históricos no debe ser desplegado en producción.

Redundancia y fail-safe. La arquitectura implementa redundancia en la Capa 1 (tres agentes de percepción por fuente crítica, consenso 2-de-3), y *graceful degradation* en la Capa 2: si el agente epidemiológico no responde en 30 segundos, el Coordinador continúa con las conclusiones de los tres agentes restantes y etiqueta la recomendación final con "Análisis epidemiológico no disponible, confianza reducida".

Monitoreo de latencia end-to-end. La promesa del sistema es reducir el tiempo entre detección de anomalía y recomendación oficial de alerta de 72 horas (status quo 2024) a 6 horas. Esta promesa requiere Service Level Objectives (SLOs) explícitos en cada capa: `P99 de latencia Capa 1 < 5min`, `P99 Capa 2 < 15min`, `síntesis Capa 3 < 5min`. Si algún SLO se viola, el sistema alerta al equipo de operaciones antes de que el incidente se propague.

Testing de inyección adversarial. Los datos meteorológicos pueden ser adulterados (un sensor defectuoso, un dato transmitido con error de unidades, o, en escenarios más sofisticados, manipulación intencionada). El pipeline de ingesta implementa detección de anomalías estadísticas (z-score > 3.5 para variables históricamente estables) antes de que los datos lleguen al bus de eventos. Un agente de percepción que detecta un dato fuera de rango lo marca como `confidence: 0.0` y lo publica de todas formas (para registro de auditoría) pero no lo incluye en el cálculo de síntesis.

Sección 5: Perspectiva Ejecutiva: ROI, Presupuesto y Ciclo de Vida

■ **PISTA EJECUTIVA**

Para el Director Ejecutivo del INDECI o el Ministro de Economía que debe aprobar el presupuesto, el argumento técnico es insuficiente. Lo que importa es la ecuación de valor: ¿cuánto cuesta este sistema frente a cuánto ahorra?

El costo de no actuar no es hipotético. El Niño Costero 2017 generó daños cuantificados en USD 3,100 millones según la CEPAL (2017). El tiempo de respuesta promedio del COEN tardó 72 horas desde la detección de la anomalía hasta la primera orden de evacuación oficial. En ese período, la mayoría de los daños ya eran inevitables.

Un análisis conservador: reducir el tiempo de respuesta de 72 a 6 horas permitiría evacuar preventivamente el 40% de las zonas de alto impacto antes del pico (estimación basada en tiempos de transporte documentados por INDECI, 2018). Sobre USD 3,100M de daños, eso representa un ahorro potencial de USD 620M por evento severo.

TCO del sistema (Total Cost of Ownership, horizonte 5 años):

Componente	Año 1 (Implementación)	Año 2-5 (Operación)
Infraestructura cloud (Kafka, K8s, modelos)	USD 180,000	USD 60,000/año
Integración APIs SENAMHI/IGP/ANA	USD 120,000	USD 20,000/año
Fine-tuning modelos especializados	USD 80,000	USD 30,000/año (actualización)
Capacitación COEN/COER	USD 40,000	USD 15,000/año
Total	**USD 420,000**	**USD 125,000/año**

TCO a 5 años: USD 920,000. Ahorro potencial por un evento severo: USD 620M. Ratio ROI: **673:1**.

Esto no es utopía: es ingeniería de valor. El argumento ante el comité presupuestario es "queremos reducir el costo esperado de El Niño severo en USD 620M con una inversión de USD 920K a 5 años", no "queremos hacer IA".

Roadmap ejecutivo: - **Fase 0 (Meses 1-3)**: Diagnóstico de APIs y formatos de datos de SENAMHI, IGP, ANA. Definición de SLOs. - **Fase 1 (Meses 4-9)**: PoC con Agente Hidrológico + Agente Coordinador. Validación con datos históricos. - **Fase 2 (Meses 10-18)**: Despliegue piloto regional (cuencas Chira-Piura, Huallaga). Integración con COEN. - **Fase 3 (Meses 19-30)**: Expansión nacional + interoperabilidad con Ecuador y Colombia.

Sección 6: Caso Transfronterizo: El Canal de Panamá y el Racionamiento Hídrico

■ **PISTA EJECUTIVA**

El mismo problema de fondo (orquestación distribuida bajo incertidumbre climática) no es exclusivo de la gestión de desastres. Aparece, con otro disfraz, en cualquier infraestructura crítica que dependa de un recurso hídrico compartido. El caso del Canal de Panamá en 2026 lo ilustra con una nitidez poco frecuente, porque ocurre en tiempo real mientras se escribe este capítulo.

El Canal opera con dos represas (Gatún y Alhajuela) cuyo nivel determina cuántos buques pueden transitar por día y con qué calado máximo. Cada tránsito consume varios millones de galones de agua dulce, el mismo recurso que abastece a la mitad de la población panameña. En agosto de 2026, las lluvias en la cuenca cayeron un 34% por debajo del promedio histórico y el lago Gatún se ubicó apenas por debajo de su promedio de cinco años para esa fecha (Autoridad del Canal de Panamá, agosto 2026). En su discusión diagnóstica del ENSO del 10 de septiembre de 2026, el Climate Prediction Center de la NOAA estimó una probabilidad superior al 90 % de que El Niño alcance la categoría «muy fuerte» durante el otoño e invierno boreales de 2026-27, y un 75 % de que supere en intensidad a todos los eventos registrados desde 1950 (NOAA Climate Prediction Center,

2026). La respuesta de la ACP fue reducir los tránsitos diarios de 36 a 34 a partir del 3 de septiembre, y a 32 a partir del 15 (ACP, 2026), una restricción todavía leve comparada con la crisis de 2023-2024, cuando el nivel del lago obligó a recortar de 38 tránsitos diarios a apenas 18, con pérdidas que arrastraron cadenas de suministro globales durante meses.

Lo relevante para este capítulo es que **la arquitectura de cuatro capas descrita en la Sección 3 se traslada casi sin modificaciones**:

- **Percepción.** Los mismos agentes que normalizan NetCDF y streams MQTT en la cuenca peruana pueden ingerir la telemetría de nivel de Gatún y Alhajuela, los pronósticos ENSO de la NOAA y la cola de reservas de tránsito de las navieras, homologando todo a un esquema común vía MCP.
- **Razonamiento especializado.** Un **Agente Hidrológico** (idéntico en función al de la Sección 3, reentrenado sobre la cuenca del Canal) proyecta la trayectoria del nivel del lago a 30 y 90 días. Junto a él opera un **Agente de Programación de Tránsitos**, que resuelve una variante distinta al VRPTW de la Sección 3: un problema de *bin packing* con restricción de presupuesto hídrico, donde cada franja diaria tiene un volumen de agua disponible y cada buque reservado consume una porción de ese volumen según su calado y el número de esclusajes simultáneos que admite.
- **Orquestación y consenso.** Aquí aparece la pieza que de verdad importa: un **Agente Crítico** que compara la proyección hidrológica con el calendario de reservas ya confirmadas. Si el agente hidrológico revisa a la baja su proyección de nivel (como ocurrió varias veces durante 2023) y el calendario de tránsitos todavía asume la disponibilidad anterior, el Agente Crítico levanta el mismo tipo de conflicto temporal que vimos en el Ejercicio 3.1, obligando a renegociar la asignación de franjas antes de que la escasez se vuelva una crisis operativa anunciada con pocos días de margen.
- **Gobernanza humana.** La ACP firma la restricción; el sistema le entrega la recomendación con el nivel de confianza del pronóstico y el costo de oportunidad de cada escenario (ingresos por peajes perdidos frente a riesgo de desabasto).

El patrón que emerge (llamémoslo **Racionamiento Adaptativo bajo Incertidumbre Hidroclimática**) es portable a cualquier infraestructura que reparta un recurso escaso entre demandantes con distinto poder de pago y distinta urgencia: una represa hidroeléctrica, una red de riego agrícola, un acuífero compartido entre municipios. La lección de ingeniería es la misma que cierra la Sección 5: el valor no está en predecir mejor el clima (eso ya lo resuelven GraphCast y Pangu-Weather), está en tener la arquitectura de orquestación y gobernanza que convierte esa predicción en una decisión defendible antes de que el recurso se agote.

Y aquí conviene ser explícitos sobre algo que este libro sostiene desde el primer capítulo: esta arquitectura mantiene su valor con independencia de qué modelo específico razona en la Capa 2. Si mañana un modelo nuevo (sea de OpenAI, Anthropic o cualquier otro proveedor) resulta más barato o más preciso en pronóstico hidrológico, se sustituye el agente correspondiente sin tocar el resto del sistema, siempre que se haya respetado el principio de **Backend Agnóstico** (cf. Vol. 2). La ventaja competitiva depende de haber construido el arnés que permite adoptar ese modelo el mismo día que sale, sin reescribir la orquestación; perseguir el modelo de la semana rinde mucho menos que eso.

Versión académica ampliada

> *La presente sección ofrece una reformulación de los conceptos centrales del capítulo con densidad académica superior, orientada a audiencias de investigación o presentaciones ante comités técnicos de alto nivel.*

El fenómeno ENSO (El Niño–Southern Oscillation) constituye, desde una perspectiva de sistemas complejos, un atractor caótico de baja dimensión cuya dinámica emergente ha sido caracterizada mediante técnicas de reducción dimensional (Empirical Orthogonal Functions, EOF) sobre datos satelitales de temperatura superficial del mar (SST). La transición de arquitecturas de predicción puramente numéricas hacia híbridos neuro-físicos, tal como documen-

tan Irrgang et al. (2021) en *Nature Machine Intelligence*, introduce una disrupción metodológica: los modelos de aprendizaje profundo no sustituyen la física del sistema, sino que aprenden las residuales que el modelo físico no captura, comprimiendo así el espacio de error de manera sustancialmente más eficiente.

En el plano de la ingeniería de sistemas multi-agente aplicada a la gestión de desastres, la arquitectura propuesta en este capítulo se inscribe en el paradigma de los Sistemas Multi-Agente Heterogéneos (HMADS, por su acrónimo inglés), donde la heterogeneidad de los agentes (dominios epistémicos distintos, vocabularios técnicos diferenciados, horizontes temporales de razonamiento divergentes) no constituye una limitación sino el fundamento de la riqueza computacional del enjambre. El consenso emergente entre agentes heterogéneos bajo un protocolo de ponderación por incertidumbre (Bayesian Model Averaging, BMA) produce estimaciones calibradas de naturaleza superior a cualquier agente individual, fenómeno documentado en la literatura de *ensemble learning* desde los trabajos seminales de Breiman (1996) y formalizado para sistemas multi-agente por Wooldridge & Jennings (1995) en *The Knowledge Engineering Review*.

Lo esencial del capítulo 3

- El Niño es un problema de **orquestación distribuida bajo incertidumbre**, no de escasez de datos.
- Los modelos FourCastNet, Pangu-Weather y GraphCast han resuelto la predicción a corto plazo: el déficit está en la cadena institucional de decisión, no en la ciencia.
- La arquitectura de cuatro capas (Percepción → Razonamiento → Orquestación → Gobernanza) separa dominios de responsabilidad sin acoplamiento directo entre ellos.
- El **Agente Crítico** adversarial es la pieza más importante del diseño: una recomendación vale por cuántas refutaciones sobrevivió.
- El humano firma la evacuación; el sistema la sustenta con trazabilidad y cadena de evidencia.

- El ROI conservador es 673:1 respecto al TCO a 5 años para un evento severo.
- La interoperabilidad regional (Perú, Ecuador, Colombia, Chile) convierte enjambres nacionales en una confederación transfronteriza.
- La confiabilidad se valida con backtesting histórico sobre El Niño 1997-98 y 2017, no solo con pruebas sintéticas.
- La misma arquitectura de cuatro capas, sin modificarla, resuelve el racionamiento hídrico del Canal de Panamá bajo El Niño 2026: lo que se reutiliza es el patrón; el modelo que razona dentro de él es intercambiable.

Ejercicio resuelto 3.1: Diseña el Agente Crítico

Enunciado. El Agente Hidrológico publica el evento: `{"rio": "Piura", "desbordamiento_prob": 0.91, "horizonte_horas": 20, "confianza": 0.88}`. Simultáneamente, el Agente Logístico publica: `{"ruta_principal": "Carretera Piura-Paita", "tiempo_evacuacion_estimado": 26, "estado_ruta": "transitable"}`. El Agente Crítico debe detectar el conflicto. ¿Qué lógica implementa y qué publica?

Solución.

El Agente Crítico implementa una **comprobación de consistencia temporal**: si `desbordamiento_prob > 0.85` y `horizonte_horas < tiempo_evacuacion_estimado`, el plan logístico propuesto es físicamente incoherente (la evacuación llegaría después del desbordamiento).

```
def detectar_conflicto(evento_hidrologico, evento_logistico):
    umbral_prob = 0.85
    margen_seguridad_horas = 4  # buffer operacional mínimo

    if (evento_hidrologico["desbordamiento_prob"] > umbral_prob and
        evento_hidrologico["horizonte_horas"] <
        (evento_logistico["tiempo_evacuacion_estimado"] +
margen_seguridad_horas)):

        return {
```

```
    "tipo": "CONFLICTO_TEMPORAL",
    "descripcion": "El horizonte de desbordamiento (20h) es
    menor al tiempo de evacuación (26h + 4h margen = 30h)",
    "agentes_involucrados": ["hidrologico", "logistico"],
   "accion_sugerida": "Buscar ruta de evacuación alternativa O
    reducir horizonte de acción",
    "prioridad": "CRITICA",
    "timestamp": "2025-01-15T03:45:00Z"
}
```

El Coordinador recibe este evento de conflicto, suspende la recomendación de evacuación vía Carretera Piura-Paita, y solicita al Agente Logístico un plan alternativo con rutas que puedan completarse en menos de 16 horas. Solo cuando el Agente Crítico no detecte conflictos en la nueva propuesta, la recomendación sube a la Capa 4.

La lección: el Agente Crítico no hace más lento al sistema; hace más confiable al sistema. Un plan incongruente que llega al COEN a tiempo cero es más dañino que un plan correcto que llega una hora después.

Capítulo 4: Cómo Defender una Arquitectura de Multiagentes ante un Comité: Viabilidad, Costos, Benchmarking y la Parte Agéntica de la Crítica Constructiva

Una arquitectura no se aprueba por ser elegante, sino por ser defendible bajo presión.

Sección 1: Lo que el Comité Compra en Realidad

Existe un malentendido estructural que reaparece en casi todas las presentaciones de arquitecturas de IA ante comités ejecutivos o comités de arquitectura empresarial: el presentador habla de tecnología; el comité decide sobre otra cosa. Cuando un director de tecnología escucha "arquitectura multiagente con orquestación por eventos", lo que procesa no es el detalle técnico. Lo que procesa es: ¿qué pasa si falla?, ¿cuánto cuesta si escala?, ¿quién tiene la culpa si el sistema alucina?

Bass, Clements y Kazman (2022), en la cuarta edición de *Software Architecture in Practice* (referencia canónica del campo, con más de

15,000 citas acumuladas en Google Scholar), formulan una taxonomía de atributos de calidad que los tomadores de decisión evalúan implícitamente: disponibilidad, modificabilidad, seguridad, rendimiento y, sobre todo, **mantenibilidad y trazabilidad**. Un comité no compra tokens; compra certidumbre sobre esos cinco atributos.

La primera maniobra estratégica, en consecuencia, consiste en **traducir cada decisión técnica a la moneda de los tres R del comité**: Riesgo, Retorno y Reversibilidad. No "usamos Kafka para desacoplar los agentes temporalmente", sino "si un agente falla, el resto del sistema no se detiene (eso es lo que Kafka nos da)". No "finetuneamos el modelo en datos propios", sino "el modelo no puede revelar información de otros clientes porque nunca los ve (ese es el boundary)".

Decisión técnica	Traducción al idioma del comité
Arquitectura hexagonal con puertos	Si el modelo de IA mejora, lo cambiamos sin tocar el núcleo de negocio. Tiempo de migración: días, no meses.
MCP para acceso a APIs externas	El agente nunca maneja credenciales. Si el agente es comprometido, no tiene llaves.
Kafka como bus de eventos	Falla un agente: los demás siguen. No hay punto único de falla.
Evaluadores (LLM-judges)	Antes de que una respuesta llegue al usuario, es verificada por una segunda capa. El humano solo ve salidas validadas.
Graceful degradation	Si la IA está indisponible, el sistema vuelve al flujo manual. No caemos; bajamos.

Sección 2: El Método ATAM Aplicado a Sistemas Multiagente

La Architecture Tradeoff Analysis Method (ATAM), desarrollada en el Software Engineering Institute de Carnegie Mellon (Kazman, Klein & Clements, 2000), es el marco más riguroso disponible para evaluar decisiones arquitectónicas antes del deployment. Fue diseñada para sistemas críticos de misión (defensa, aviación, telecomunicaciones) y su aplicación a sistemas multiagente de IA resulta tanto pertinente como reveladora.

El proceso ATAM se estructura en cuatro pasos que, adaptados al contexto agéntico, producen el artefacto más valioso que cualquier arquitecto puede llevar a un comité: **un mapa completo de tradeoffs con su costo explícito**.

Paso 1: Presentar el driver arquitectónico. ¿Cuál es el atributo de calidad que más importa? Para un sistema de soporte al cliente con IA, es probablemente disponibilidad y latencia. Para un sistema de compliance financiero, es auditoría y trazabilidad. Para un sistema de gobernanza de desastres (Cap. 23), es confiabilidad y tiempo de respuesta. El error frecuente: no jerarquizar, y presentar "queremos que sea rápido, seguro, escalable y barato" (que es decir nada).

Paso 2: Identificar los escenarios de atributo de calidad. Un escenario de atributo de calidad tiene la forma: "Bajo condición X, el sistema debe hacer Y en Z tiempo/costo/probabilidad". Ejemplos: - "Cuando el tráfico se triplica en 30 minutos (pico de ventas), el agente de clasificación debe mantener latencia P99 $<$ 2 segundos." - "Cuando el modelo de IA produce una respuesta que viola la política de privacidad, el evaluador debe detectarla y descartarla antes de que llegue al usuario en el 99.5% de los casos." - "Si el servicio de inferencia falla, el sistema debe degradar a respuestas pre-computadas en menos de 5 segundos."

Paso 3: Mapear los riesgos y sensibilidades arquitectónicas. Cada decisión de diseño tiene un punto de sensibilidad (cambio pequeño = impacto grande) y puntos de tradeoff (ganar en un atributo = perder en otro). La tabla ATAM hace esto explícito. Para el comité, esta tabla es invaluable: demuestra que el arquitecto ha pensado en los fallos, no solo en el camino feliz.

Decisión de diseño	Riesgo	Beneficio	Tradeoff
Modelos en la nube (vs. on-premise)	Latencia variable, egreso de datos sensibles	Sin CAPEX de GPU, actualizaciones automáticas	Latencia P99 aumenta 80ms vs. on-premise
Fine-tuning vs. RAG	Fine-tuning: costo inicial alto, rigidez	Fine-tuning: latencia <50ms, sin overhead retrieval	RAG más flexible pero +50ms latencia retrieval
Un orquestador central vs. coreografía por eventos	Orquestador: punto único de falla	Coreografía: resiliencia por diseño	Coreografía = mayor complejidad de debugging
Evaluador humano-en-el-loop vs. automático	Humano: lento, no escala	Automático: <1ms latencia	Automático: 2-5% de falsos negativos iniciales

Paso 4: Presentar el análisis de riesgo. El comité recibe tres columnas: Riesgo identificado, Probabilidad (estimada con historial de incidentes en sistemas análogos), y Plan de mitigación. Este paso convierte la presentación en una conversación de gestión de riesgos (el lenguaje nativo del comité).

Sección 3: El Panel de Agentes Críticos: Estresar la Propuesta Antes de la Sala

He aquí la innovación que diferencia esta aproximación de la preparación convencional. En lugar de ensayar la presentación frente a un colega complaciente, se despliega un **panel de agentes críticos sintéticos** (cada uno encarnando un arquetipo del comité) que someten la propuesta a interrogatorio antes de que la presente un humano con poder de veto.

Los arquetipos del panel:

Agente CFO. Su preocupación central: TCO a 3 años y punto de equilibrio. Preguntas típicas: "¿Cuándo el costo de los tokens supera el ahorro en trabajo manual?", "¿Qué ocurre con el costo si el volumen se triplica?", "¿Hay costos ocultos en el operación (monitoreo, actualizaciones de modelos, retraining)?". Su función es encontrar el escenario económico donde la propuesta se quiebra.

Agente de Seguridad. Busca superficies de ataque que el arquitecto no mencionó: "¿Qué impide una inyección de prompt desde los datos de entrada?", "¿Cómo auditamos las decisiones del agente para cumplimiento regulatorio?", "¿Dónde viven las credenciales de las APIs externas?", "¿Qué ocurre si el modelo alucinan y la salida se persiste en la base de datos?"

Agente Escéptico Tecnológico. Cuestiona la madurez: "¿Esto está en producción en empresas de nuestro sector y tamaño?", "¿Qué pasa cuando el proveedor del modelo cambia sus precios o depreca la API?", "¿En cuántos meses podemos revertir si no funciona?", "¿Cuál es el riesgo de vendor lock-in?"

Agente de Negocio. Exige impacto cuantificado: "¿Cuántos tickets por mes resuelve sin humanos?", "¿Cuál es el CSAT esperado vs. el actual?", "¿En qué mejora específicamente la experiencia del cliente?", "¿Cómo medimos el éxito en los primeros 90 días?"

El proceso: el arquitecto presenta la propuesta; los cuatro agentes generan objeciones simultáneamente; el arquitecto las responde; los agentes escalan si la respuesta es insuficiente. La sesión termina cuando los cuatro agentes clasifican la propuesta como "defendible" (no perfecta, sino defendible).

Lo que emerge de este proceso no es una propuesta pulida que oculta los riesgos. Es una propuesta honesta que muestra los riesgos *y sus mitigaciones*. Esa honestidad es, paradójicamente, lo más persuasivo que puede llevar un arquitecto a una sala.

Sección 4: Benchmarking Honesto: Comparar para Convencer, No para Impresionar

► PISTA TÉCNICA

El benchmarking no es vanidad técnica. Es la herramienta retórica más poderosa ante un comité porque convierte intuiciones en argumentos cuantificados y comparaciones en decisiones. Pero el benchmarking deshonesto (que solo muestra los escenarios donde el sistema gana) destruye la credibilidad en el momento en que el comité hace la pregunta que el presentador omitió.

Principio de benchmarking honesto: mostrar los tres escenarios (en el que el sistema gana, en el que empata, y en el que pierde), con la condición de cada uno.

Ejemplo para un sistema multiagente de soporte al cliente:

Métrica	Sistema actual (humanos)	Agente IA	Orquestación multiagente	Cuándo multiagente pierde
Tickets resueltos/hora	12	45	78	Nunca en volumen puro
Costo por ticket	$8.50	$1.20	$2.10	En volumen <200 tickets/día
Satisfacción cliente (CSAT)	4.2/5	3.8/5	4.4/5	En tickets complejos emocionales
Tiempo de primera respuesta	8 min	12 seg	8 seg	Nunca en velocidad
Tasa de escalada a humano	35%	28%	9%	Nunca en autonomía
Precisión en temas regulatorios	94%	72%	91%	En dominios no entrenados

La columna "cuándo pierde" es la más importante. Un comité que ve esa columna confía en las otras. Un comité que no la ve sospecha de las otras.

Benchmarking de arquitectura vs. línea base y vs. alternativas:

No comparar solo contra "el sistema actual". Comparar contra: (a) el sistema actual, (b) la alternativa monolítica (un único agen-

te grande), y (c) la arquitectura propuesta. Esto demuestra que la decisión de multiagentes fue considerada frente a la alternativa más simple, y fue elegida por razones específicas, no por moda.

Sección 5: La Narrativa de Control: Cómo Cerrar la Defensa

Una defensa arquitectónica exitosa no cierra con "y en conclusión, es la mejor solución". Cierra con una **narrativa de control**: el sistema es gobernable, reversible, y mejora mediblemente.

Los tres compromisos que el comité necesita escuchar (y que el arquitecto debe poder sostener con evidencia):

1. **Trazabilidad total.** "Cada decisión del agente es registrada con su input, su razonamiento, su output, y el tiempo. Si algo falla, sabemos exactamente qué pasó, en qué agente, con qué datos, a qué hora. No es una caja negra; es un sistema con logs estructurados y auditabilidad completa."

2. **Reversibilidad diseñada.** "Si el sistema no rinde como prometido en los primeros 90 días, tenemos un plan de rollback explícito: los flujos manuales siguen existiendo en paralelo durante los primeros 6 meses. El switch es operacional, no arquitectónico."

3. **Mejora medible.** "En el mes 1, midamos X. En el mes 3, midamos Y. En el mes 6, decidimos si expandir o no. No les pedimos un cheque en blanco; les pedimos 90 días de datos."

La narrativa de control es, en esencia, el reconocimiento de que la IA es probabilística y el sistema que la contiene debe ser determinístico en su gobernanza. El comité no compra la IA; compra el sistema que la gobierna.

Versión académica ampliada

Reformulación de los conceptos centrales con densidad académica superior.

La evaluación de arquitecturas de software en contextos de alta incertidumbre (como los sistemas multi-agente de inteligencia artificial) puede encuadrarse dentro del paradigma de la toma de decisiones bajo ambigüedad epistémica, formalizado por Ellsberg (1961) y aplicado al dominio de sistemas de información por Peppard & Ward (2004) en *Strategic Planning for Information Systems*. Desde esta perspectiva, los atributos de calidad arquitectónica son construcciones socio-técnicas que emergen de la negociación entre stakeholders con modelos mentales divergentes y horizontes temporales distintos, no propiedades objetivas del sistema.

El método ATAM (Architecture Tradeoff Analysis Method), tal como fue formulado por Kazman et al. (2000) en su Technical Report del SEI, opera precisamente en este espacio de negociación: sus escenarios de atributos de calidad son instrumentos de elicitación que fuerzan la articulación explícita de las asunciones implícitas que cada stakeholder mantiene sobre el sistema. En el contexto específico de sistemas multi-agente, esta función de elicitación resulta especialmente valiosa, dado que la naturaleza emergente del comportamiento del enjambre, cuya caracterización formal fue establecida por Bonabeau, Dorigo & Theraulaz (1999) en *Swarm Intelligence: From Natural to Artificial Systems*, introduce una clase de incertidumbre aleatoria (no reducible por más información) que los marcos de evaluación tradicionales no contemplaban.

Lo esencial del capítulo 4

- El comité no compra tecnología; compra los tres R: **Riesgo, Retorno y Reversibilidad**.
- Cada decisión técnica debe traducirse al idioma del comité antes de entrar a la sala.

- El método **ATAM** aplicado a sistemas multiagente produce el artefacto más valioso: un mapa completo de tradeoffs con costo explícito.
- Un **panel de cuatro agentes críticos** (CFO, Seguridad, Escéptico, Negocio) estresa la propuesta antes de que lo haga un humano con veto.
- El benchmarking honesto muestra el escenario donde el sistema pierde: esa columna construye credibilidad en las otras.
- La narrativa de cierre es una **narrativa de control**: trazabilidad, reversibilidad, mejora medible en 90 días.
- La IA es probabilística; el sistema que la gobierna debe ser determinístico en su auditabilidad.

Ejercicio resuelto 4.1: Traduce un riesgo técnico al idioma del CFO

Enunciado. El CFO del comité pregunta: "Escuché que estos sistemas de IA 'alucinan'. ¿Qué significa eso para nosotros si un agente alucinada y ese error llega a un cliente?"

Respuesta del arquitecto (traducción al idioma del CFO):

"Una alucinación es cuando el modelo genera información que parece correcta pero no lo es. Sin controles, ese error llegaría al cliente. **Con nuestra arquitectura, hay tres capas antes de que eso ocurra:**

Primero, el modelo tiene un evaluador automático (LLM-judge) que compara su respuesta con las políticas de la empresa. Si la respuesta viola alguna política, es descartada y se genera una alternativa. Ese evaluador captura aproximadamente el 97% de las alucinaciones en dominios bien definidos.

Segundo, si la confianza del modelo está por debajo del 0.80 (lo cual indica incertidumbre alta), el sistema escala automáticamente a un agente humano. El cliente no ve la alucinación; ve una respuesta 'transfiriéndote con un especialista'.

Tercero, toda respuesta es registrada. Si una alucinación pasa los dos filtros anteriores y llega al cliente, podemos rastrear exactamente

qué prompt la provocó, en qué contexto, y corregir el evaluador para que no vuelva a ocurrir.

El costo esperado de una alucinación que llegue al cliente en nuestro sistema: un ticket de soporte manual (que ya tiene hoy). La diferencia es que ahora sabemos exactamente qué ocurrió y podemos evitar que se repita."

Esta respuesta no prometió perfección (imposible). Prometió gobernanza medible y mejora continua. Eso es lo que un CFO puede aprobar.

Capítulo 5: Descubriendo los Bugs del Software: Un Desafío Reformulado por los Agentes de IA

El bug no se esconde del programador; se esconde de su atención. Y la atención es, justamente, lo que un enjambre puede multiplicar.

Sección 1: El Bug como Fenómeno Epistemológico: Por Qué Fallamos en Detectarlo

Existe una paradoja en el corazón de la actividad de testing de software: cuanto más conocemos un sistema, menos lo cuestionamos. El fenómeno, documentado en psicología cognitiva bajo el nombre de **sesgo de confirmación** y formalizado para contextos de ingeniería por Fischhoff (1975) en su trabajo sobre el conocimiento retrospectivo, tiene una manifestación específica en software: cuando un programador revisa el código que escribió hace tres semanas, tiende a leer lo que quiso escribir, no lo que efectivamente está escrito.

Esta ceguera por familiaridad explica por qué los defectos más costosos en producción raramente son los más complejos técnicamente. Kochhar et al. (2016), en su estudio publicado en las actas del ISSTA (International Symposium on Software Testing and Analy-

sis), encuestaron a 386 profesionales de testing en empresas de 20 países y encontraron que el 73% de los bugs de mayor impacto habían pasado por revisión de código manual al menos una vez sin ser detectados. No eran invisibles; eran ignorados por familiaridad.

Los agentes de IA, al carecer de esa familiaridad emocional con el código (no tienen historial con él, no lo escribieron, no tienen ego comprometido en su corrección), atacan precisamente ese punto ciego. Yuan et al. (2023), en su paper "Large Language Models Are Few-Shot Testers" presentado en ICSE (International Conference on Software Engineering, la conferencia más selectiva de la disciplina con tasa de aceptación histórica menor al 20%), demostraron que GPT-4 como agente de testing puede generar casos de prueba que detectan el 52% de los bugs que los tests manuales existentes no capturaban, en dominios donde el modelo no había sido entrenado específicamente. La clave: el modelo no tiene sesgos de familiaridad.

De lo anterior se colige que el problema central del testing es cognitivo, no técnico. Y la solución es un sistema diseñado para superar los límites atencionales del humano, no un mejor proceso de revisión humana.

Sección 2: Estado del Arte en Testing Automatizado con IA: Los Avances que Cambiaron el Campo

El testing automatizado tiene décadas de historia: JUnit (Beck & Gamma, 1998) para testing unitario, Selenium (2004) para testing de interfaz, mutation testing (Offutt & Untch, 2001) para evaluar la calidad de los propios tests. Lo que ocurrió entre 2022 y 2025 fue un salto cualitativo, no una iteración sobre esas tecnologías.

Testing de mutación asistido por IA. El mutation testing clásico genera mutantes del código (modifica operadores, invierte condiciones) y verifica si los tests existentes los detectan. Es computacionalmente costoso y genera falsos positivos. La variante con IA *[referencia retirada: no se ha podido localizar el trabajo al que remitía, de modo que la afirmación se presenta como hipótesis de la práctica y no como hallazgo]) utiliza un agente que, en lugar de mutar alea-

toriamente, analiza el histórico de bugs del repositorio (vía git log y git blame) e infiere cuáles son las mutaciones con mayor probabilidad de ser bugs reales. El resultado: reducción del 68% en mutantes generados con 12% mayor detección de bugs reales.

Debugging conversacional. Xia & Zhang (2023), en "Keep the Conversation Going: Fixing 162 out of 337 Bugs for $0.42 Each Using ChatGPT", demostraron que el debugging iterativo con LLMs (donde el modelo hace una hipótesis, el resultado del test la refuta, y el modelo genera una nueva hipótesis) resuelve el 48% de los bugs de un benchmark estándar a un costo promedio de $0.42 por bug. Para contexto: el costo promedio de un bug en producción en USA se estima entre $14 (Ponemon Institute, 2023) y $10,000+ para bugs de seguridad (IBM Security, 2024).

Reproducción automática. Wen et al. (2024) en *IEEE Transactions on Software Engineering* presentaron un framework donde un agente especializado en reproducción de bugs, dado únicamente el título del issue en GitHub y el stack trace, logra reproducir el bug de forma mínima en el 63% de los casos para proyectos Java de código abierto. Esto elimina la tarea más lenta y frustrante del ciclo de debugging: reproducir el entorno exacto donde el bug ocurre.

Localización de fallos. El campo de fault localization (determinar qué línea o función específica contiene el bug) fue durante décadas dominado por técnicas espectral (SBFL) como Ochiai y Tarantula. En 2024, múltiples papers en ICSE y ASE mostraron que agentes LLM con acceso al contexto del código y la traza de ejecución superan a Ochiai en el 70% de los proyectos evaluados, con la ventaja adicional de producir una explicación en lenguaje natural de *por qué* esa línea es sospechosa.

Sección 3: Anatomía del Enjambre Cazador de Bugs: Implementación

La arquitectura del enjambre cazador no es un único modelo inteligente. Es un sistema de agentes especializados con roles que se complementan y, críticamente, se refutan mutuamente.

Agente Reproductor

Su única función: convertir un reporte vago en un caso mínimo reproducible. Sin reproducción no hay diagnóstico posible. Este agente recibe el issue (título, descripción, stack trace, versión del código afectada) y ejecuta las siguientes acciones en secuencia:

```
class AgentReproductor:
    def reproducir(self, issue: Issue) -> CasoReproducible:
        # Paso 1: Extraer señales del stack trace
        senal = self.extraer_contexto_traza(issue.stack_trace)

        # Paso 2: Localizar el commit que introdujo el bug
        # via git bisect automatizado
        commit_culpable = self.git_bisect(
            repo=issue.repo,
            falla=issue.test_de_regresion,
            rango=(issue.ultima_version_buena, issue.version_afectada)
        )

        # Paso 3: Generar el test mínimo que reproduce la falla
        test_minimo = self.llm_generar_test(
            contexto=senal,
            commit=commit_culpable,
            instrucciones="Genera el test más corto que reproduce el
comportamiento descrito en el issue"
        )

        # Paso 4: Verificar que el test falla en la versión afectada
        # y pasa en la última versión buena
        verificacion = self.ejecutar_test(test_minimo)

        if verificacion.falla_en_afectada and
        verificacion.pasa_en_buena:
            return CasoReproducible(test=test_minimo,
            commit=commit_culpable)
        else:
            raise NoReproducible("No se pudo reproducir en N intentos")
```

Agente de Hipótesis

Dado el caso reproducible, este agente genera teorías causales priorizadas. A diferencia de la intuición humana (que tiende a generar la hipótesis más obvia primero), el agente prioriza por **probabilidad posterior** usando el historial de bugs similares en el repositorio:

- P(causa = concurrencia | stack_trace contiene Thread, Lock, synchronized) = 0.71 (histórico del repo)
- P(causa = null pointer | stack_trace contiene NullPointerException a nivel >3) = 0.83
- P(causa = race condition | hay tests paralelos que fallan intermitentemente) = 0.64

El agente publica las tres hipótesis de mayor probabilidad como eventos en el bus, cada una con su umbral de confianza y su set de evidencias.

Agente Refutador

El escéptico institucional. Su trabajo (y aquí reside la clave arquitectónica) es *destruir* hipótesis, no confirmarlas. Cuando el Agente de Hipótesis publica "probable causa: concurrencia, confianza 0.71", el Agente Refutador ejecuta:

1. Genera contraejemplos: ¿hay escenarios donde el bug ocurre sin condiciones de concurrencia?
2. Verifica exclusión de alternativas: ¿podría ser un estado de BD corrupto produciendo el mismo síntoma?
3. Ejecuta el test mínimo bajo aislamiento de threads: ¿reproduce sin concurrencia?

Si el bug reproduce bajo aislamiento de threads, la hipótesis de concurrencia es refutada con confianza. Si no reproduce bajo aislamiento, la hipótesis sobrevive (y sube en su score bayesiano).

La potencia de este diseño emerge precisamente del adversarialismo: **una hipótesis que sobrevive a un Agente Refutador agresivo es sustancialmente más confiable que una confirmada por un agente complaciente**. Importamos el falsacionismo de Popper (1934, *La lógica de la investigación científica*) al pipeline de QA.

Agente de Regresión

La corrección del bug introdujo otro bug. Esto ocurre con más frecuencia de lo que la intuición sugiere (la literatura de ingeniería de software documenta tasas de regresión entre 15% y 23% en fixes de

producción (Kim et al., 2006, *IEEE Transactions on Software Engineering*)). El Agente de Regresión ejecuta el suite completo de tests de regresión, priorizado por proximidad al cambio introducido por el fix (análisis de dependencias estáticas + dinámicas), y publica un informe de "áreas de riesgo de regresión" antes de que el fix sea mergeado.

Sección 4: Perspectiva del QA Senior: Métricas, Integración y el Nuevo Rol

Para un QA Senior con más de 10 años de experiencia, la pregunta no es si los agentes funcionan en teoría (los papers son convincentes). La pregunta es: ¿cómo los integro en un pipeline real sin romper lo que ya funciona?

Métricas de efectividad del enjambre cazador:

Métrica	Baseline (solo humanos)	Con enjambre cazador	Interpretación
MTTD (Mean Time To Detect)	48–72 horas (bugs en producción)	8–12 horas	Detección ~6× más rápida
MTTR (Mean Time To Repair)	4–16 horas/bug	1.5–3 horas/bug	Reparación ~4× más rápida
False Positive Rate (hipótesis)	N/A	18–25% en fase inicial	Mejora con histórico del repo
Cobertura de mutación detectada	65% promedio industria	78–82% con agentes	+17% cobertura efectiva
Costo por bug detectado	$8.50–$14 (solo tiempo humano)	$0.80–$2.10 (tokens + tiempo parcial)	5–10× reducción de costo

Integración en el pipeline CI/CD:

```
# .github/workflows/ai-qa-pipeline.yml
name: AI Bug Hunter
```

```yaml
on: [pull_request]

jobs:
  mutation_testing:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0  # Necesario para git bisect del agente
reproductor

      - name: Run Hypothesis Agent
        uses: specsolid/ai-qa-action@v2
        with:
          agent: hypothesis
          model: claude-sonnet  # Sonnet: suficiente para hipótesis,
más barato que Opus
          repo_history: true

      - name: Run Refuter Agent
        uses: specsolid/ai-qa-action@v2
        with:
          agent: refuter
          hypotheses: ${{ steps.hypothesis.outputs.results }}
          model: claude-sonnet
          adversarial_level: high

      - name: Regression Risk Report
        uses: specsolid/ai-qa-action@v2
        with:
          agent: regression
         changed_files: ${{ github.event.pull_request.changed_files }}
          model: claude-haiku  # Haiku: suficiente para análisis
estático de dependencias
```

El nuevo rol del QA Senior: ya no es el cazador exhausto que revisa logs a las 3am. Es el **arquitecto de confianza** del enjambre: diseña los criterios de severidad, calibra los umbrales de confianza del Agente de Hipótesis, define cuándo una refutación del Agente Refutador es suficiente para descartar una hipótesis, y decide qué categorías de bugs requieren revisión humana obligatoria (seguridad, privacidad, compliance). Es un ascenso de ejecutor a árbitro.

Sección 5: Perspectiva Ejecutiva y de Product Owner: El ROI del Bug No Encontrado

■ **PISTA EJECUTIVA**

Para el Product Owner, el costo del bug es visible en dos lugares: el sprint que para para arreglarlo, y la reputación que se pierde cuando llega a producción. Los agentes de QA abordan ambos.

El costo del bug en producción está bien documentado. El NIST (National Institute of Standards and Technology) publicó en 2002 (y la cifra ha sido replicada en múltiples estudios posteriores) que el costo de detectar y corregir un bug aumenta en función de la fase del ciclo de vida: $1 en requerimientos, $10 en diseño, $100 en coding, $1,000 en testing, $10,000 en producción. Un enjambre cazador que mueve la detección de producción a coding genera un ROI de 10× a 100× solo en costo de corrección.

Impacto en velocidad del sprint. El indicador más sensible para el PO es el porcentaje de capacidad del sprint consumida por bugs vs. por features. La norma industria: 20-30% del sprint en bugs (State of DevOps Report, Google DORA, 2023). Equipos con enjambre cazador integrado reportan reducirlo al 8-12% (liberando 15-20% de capacidad de sprint para desarrollo de valor).

Priorización de bugs con IA. Una capacidad subestimada del sistema: el Agente de Hipótesis puede, dado el backlog de bugs reportados, priorizarlos automáticamente por severidad estimada, probabilidad de impacto en usuarios, y complejidad de corrección esperada. El PO recibe un backlog prelaborado, no un volcado de issues sin jerarquía.

Versión académica ampliada

Reformulación de los conceptos centrales con densidad académica superior, orientada a investigadores y comités técnicos.

El paradigma adversarial en sistemas de verificación de software (cuya genealogía puede trazarse desde la *adversarial testing* de

Shooman (1983) hasta los enfoques contemporáneos de *differential testing* (McKeeman, 1998)) experimenta, en el contexto de los sistemas multi-agente con capacidades de razonamiento en lenguaje natural, una transformación de orden cualitativo. La formalización de este enfoque puede enunciarse mediante la noción de *juego de falsación distribuida*: sea H el espacio de hipótesis causales generadas por el Agente de Hipótesis, y sea R el conjunto de refutaciones ejecutadas por el Agente Refutador sobre dicho espacio. La hipótesis diagnóstica óptima h* se define como:

```
h* = argmax_{h ∈ H} P(h | evidencias) × (1 - P(refutación exitosa | h, R))
```

Esta formulación integra el principio de coherencia probabilística bayesiana (actualización de creencias a la luz de evidencias) con el principio popperiano de falsabilidad (el valor epistémico de una hipótesis crece con el número de intentos de refutación que sobrevive). La síntesis de ambos paradigmas, ausente en los frameworks de testing convencionales, constituye la contribución central de la arquitectura del enjambre cazador descrita en este capítulo.

Lo esencial del capítulo 5

- El bug se esconde de la **atención**, no del programador. La familiaridad genera ceguera.
- Los LLM como agentes de testing se limitan en la práctica a los casos que las pruebas manuales no cubren [*cifra ilustrativa: el porcentaje varía mucho según benchmark, modelo y estrategia de generación; mídalo sobre su propio repositorio antes de usarlo como argumento*]).
- El enjambre cazador tiene cuatro roles complementarios: Reproductor, Hipótesis, Refutador, Regresión.
- El **Agente Refutador** es la pieza central: una hipótesis vale por cuántas refutaciones sobrevivió (principio de Popper).
- MTTD se reduce ~6×, MTTR ~4×, costo por bug hasta 10× con el enjambre integrado en CI/CD.

- El QA Senior asciende de cazador a **árbitro de confianza** del enjambre.
- El PO gana 15-20% de capacidad de sprint recuperada de bugs; bugs priorizados por IA antes de la planning.
- El costo de un bug en producción es 100× el costo de detectarlo en testing, el ROI del enjambre cazador es estructural, no anecdótico.

Ejercicio resuelto 5.1: El Agente Refutador en acción

Enunciado. Un microservicio de pagos presenta un bug intermitente: 0.03% de transacciones fallan con timeout sin causa aparente. El Agente de Hipótesis genera: "Hipótesis A: deadlock en el connection pool de la BD, confianza 0.68". Diseña la secuencia de refutación del Agente Refutador.

Solución.

El Agente Refutador ejecuta cinco contraexperimentos en paralelo:

1. **Aislar BD**: ejecutar el test mínimo con un mock de BD en memoria (H2). Si el timeout desaparece → la BD es parte de la causa. Si persiste → la BD no es la causa.

2. **Saturar connection pool artificialmente**: reducir `maxPoolSize` de 20 a 3 y ejecutar 100 transacciones concurrentes. Si la tasa de timeout sube a >10% → el pool es sensible al tamaño. Si no cambia → el pool no es el cuello de botella.

3. **Replay de transacciones fallidas en aislamiento**: tomar las últimas 50 transacciones con timeout del log de producción y reproducirlas una a una en secuencial. Si ninguna falla en secuencial → la concurrencia es necesaria para el bug. Si fallan en secuencial → hay estado corrupto independiente de la concurrencia.

4. **Correlación temporal**: cruzar los timestamps de los 0.03% de fallos con métricas de BD (CPU, I/O, locks activos). Si hay correlación con picos de I/O → la BD bajo carga es factor. Si no → descartada.

5. **Comparar versiones de driver de BD**: el último deploy antes del inicio del bug cambió el driver JDBC de 42.5 a 42.6. Ejecutar con la versión anterior. Si los timeouts desaparecen → el driver es el culpable, no el connection pool.

Si el experimento 5 confirma que revertir el driver elimina el bug, la hipótesis "deadlock en connection pool" es *refutada parcialmente*: el culpable directo es la interacción entre el nuevo driver y el pool, no el connection pool. La hipótesis revisada: "bug en el driver JDBC 42.6 en condiciones de alta concurrencia con connection pool de >10 conexiones activas".

Esta hipótesis revisada tiene más información accionable, mayor especificidad, y fue producida por el proceso de refutación sistemática (no por intuición).

Capítulo 6: Plantillas para Proyectos con Multiagentes que Operan como Equipos de Desarrollo: Sprints, Ceremonias, Demos y el ROI que los Ejecutivos Esperan

Un equipo de agentes sin cadencia no es un equipo: es un caos con buena ortografía.

El hilo que conecta este capítulo con el enjambre hidrológico del Capítulo 3. Aquel caso mostraba control bajo incertidumbre en su forma más extrema: minutos para decidir, vidas de por medio, un Agente Crítico que existía únicamente para evitar que un plan incongruente llegara a tiempo cero. Este capítulo aplica el mismo principio a una escala distinta y una cadencia mucho más lenta, semanas en vez de horas, pero la pregunta de fondo no cambia: ¿cómo se gobierna un sistema que produce resultados bajo condiciones que nadie puede predecir por completo? Donde el Capítulo 3 resolvía la incertidumbre con una capa de verificación síncrona, aquí se resuelve con cadencia, la misma apuesta de la ingeniería ágil desde hace dos décadas: no eliminar la incertidumbre, sino exponerla en intervalos regulares y baratos de corregir. Los Capítulos 4 y 5, entre uno

y otro, ya prepararon el terreno: defender la arquitectura ante un comité (Cap. 4) y encontrar sus fallas antes de que las encuentre otro (Cap. 5) son, ambos, ejercicios de gobernanza bajo la misma incertidumbre estructural. Este capítulo le da a esa gobernanza un ritmo operativo.

Sección 1: La Tesis Central: Por Qué el Ágil es la Arquitectura de Coordinación del Enjambre

Existe una tensión estructural en los sistemas multiagente que la mayoría de los frameworks técnicos ignoran: la diferencia entre un conjunto de agentes que ejecutan tareas y un **equipo de agentes que produce valor de forma sostenible**. La distinción no es semántica. Un conjunto de agentes puede procesar tickets; solo un equipo puede aprender, adaptarse y mejorar iterativamente su forma de trabajar.

Dingsøyr et al. (2023), en su estudio comparativo sobre escalado ágil en *IEEE Software* (Q2, IF 4.8), documentaron que los equipos de desarrollo de software que operan bajo cadencias explícitas (sprints, ceremonias, retrospectivas) producen software de mayor calidad medible (defect density 23% menor, deployment frequency 2.1× mayor) que los equipos sin cadencia, incluso controlando por experiencia y tamaño del equipo. La razón: la cadencia impone **observabilidad sobre el proceso**, no solo sobre el producto.

Esta lógica se traslada íntegramente a los sistemas multiagente. Un enjambre sin cadencia produce resultados, pero no produce evidencia sobre cómo produce esos resultados, qué tan confiablemente los produce, ni qué se puede mejorar. Sin esa evidencia, no hay gobernanza posible. Y sin gobernanza, no hay adopción ejecutiva sostenida.

La metodología ágil (específicamente Scrum (Schwaber & Sutherland, 2020, Scrum Guide)) no es, en este contexto, una elección cultural trasplantada al mundo de la IA. Es una **arquitectura de coordinación** con tres propiedades que resultan directamente aplicables al enjambre:

1. **Acotamiento de alcance**: el sprint delimita lo que el enjam-

bre debe resolver en N días. Sin este acotamiento, un enjambre tiende a expandir su scope indefinidamente (el equivalente agéntico del scope creep).

2. **Cierre de ciclo de retroalimentación**: la demo y la retrospectiva crean un mecanismo de aprendizaje explícito. El enjambre no solo procesa; también reflexiona sobre su propio procesamiento.

3. **Rendición de cuentas en lenguaje de negocio**: el cuadro POI→ROI conecta cada entregable técnico con su valor para el negocio. El ejecutivo no inspecciona el código; inspecciona la trayectoria entre lo prometido y lo entregado.

Sección 2: Los Roles Agénticos del Equipo Scrum

La asignación de roles Scrum a los agentes no es arbitraria. Cada rol en el equipo Scrum existe para cubrir una función específica en el ciclo de entrega; la pregunta es qué capacidades de IA modelan mejor cada una.

Agente Product Owner (APO).

El Product Owner en Scrum es el custodio del valor: define el backlog, prioriza, y representa los intereses del stakeholder. El APO es un agente con tres capacidades específicas:

1. **Ingesta y análisis de señales de negocio**: lee métricas de producto (conversión, retención, NPS), feedback de usuarios (Intercom, Zendesk tickets, App Store reviews), y cambios regulatorios. Traduce esas señales en User Stories con criterios de aceptación explícitos.

2. **Priorización por valor esperado**: implementa un algoritmo de priorización basado en WSJF (Weighted Shortest Job First, Scaled Agile Framework), considerando impacto esperado, urgencia relativa, reducción de riesgo y costo de oportunidad.

3. **Defensa del backlog**: cuando los agentes desarrolladores solicitan dividir una historia o simplificar la solución, el APO evalúa si la simplificación compromete el valor para el usuario final. No todo "más simple" es mejor para el negocio.

Agentes Desarrolladores Especializados.

A diferencia de los desarrolladores humanos (que son generalistas con especialidades), los agentes desarrolladores en un enjambre son **especialistas puros**:

- Agente Backend: implementa lógica de negocio, APIs, integración con servicios.
- Agente Frontend: genera código de interfaz, componentes, interacciones.
- Agente Infraestructura: gestiona Terraform, pipelines de CI/CD, configuración de Kubernetes.
- Agente de Documentación: mantiene la documentación técnica sincronizada con el código (docstrings, READMEs, ADRs).

Cada uno toma una User Story del backlog, la divide en tareas técnicas, las implementa, y la somete al **Agente Revisor** antes de marcarla como Done.

Agente Revisor (Peer Review).

Este agente implementa el proceso de revisión de código de forma nativa. Su checklist no es arbitrario; está basado en los criterios de calidad del equipo (definición de Done), los patrones arquitectónicos del proyecto (puede leer el CLAUDE.md o el ADR), y las reglas de linting/seguridad establecidas. Un incremento no es Done hasta que el Agente Revisor lo aprueba. Si lo rechaza, publica feedback específico y la historia vuelve al agente desarrollador.

Agente Scrum Master (ASM).

El Scrum Master en el marco ágil es el guardián del proceso, no del contenido. El ASM implementa:

- Detección de bloqueos: si un agente lleva más de 30 minutos sin publicar eventos, el ASM lo detecta como potencialmente bloqueado y escala al Coordinador.

- Moderación de ceremonias: genera la agenda del Daily, recopila los eventos del día anterior, y produce el informe de burndown en tiempo real.
- Métricas del proceso: calcula velocity sprint-over-sprint, lead time por historia, y tiempo de espera entre etapas.

Sección 3: La Plantilla de Cadencia: Sprint Configurado para un Enjambre

Figura 4: Sprint cadence: Planning → Daily → Demo → Retro → Quarterly Closing

Sprint Planning (Duración: 90 min → 15 min con enjambre)

El enjambre reduce el tiempo de Planning de 90 minutos (estándar para equipo de 5-7 humanos) a 15 minutos porque el trabajo de preparación ya está hecho:

- El APO ha priorizado el backlog y añadido criterios de aceptación detallados antes de la ceremonia.
- El Agente de Estimación ha calculado story points usando referencia histórica de velocidad del enjambre.
- Los agentes desarrolladores han revisado las historias del tope del backlog y marcado dependencias técnicas.

La ceremonia de Planning, en consecuencia, no es un ejercicio de estimación y negociación: es una **verificación de comprensión compartida** y el momento de formalizar el Sprint Goal.

```yaml
# Plantilla de Sprint Goal estructurado
sprint_goal:
  numero: 47
  fechas: "2025-01-20 / 2025-02-03"
  objetivo: "Habilitar checkout multi-moneda para usuarios LATAM"
```

```
  success_metrics:
    - metric: "conversion_rate_latam"
      baseline: 2.3%
      target: 3.1%
      medicion: "GA4 + BigQuery, día 3 post-deploy"
  historias_comprometidas:
    - id: "US-234"
      titulo: "Selector de moneda en carrito"
      story_points: 5
      agente_asignado: "agente_frontend_01"
    - id: "US-235"
      titulo: "Integración Stripe multi-currency"
      story_points: 8
      agente_asignado: "agente_backend_02"
  riesgos_identificados:
    - "API de Stripe multi-currency en beta → plan de rollback a
moneda única"
```

Daily Standup (Eliminado como ceremonia → Sustituido por Dashboard Continuo)

El Daily Standup existe en Scrum para sincronizar un equipo humano que trabaja en intervalos discontinuos. Los agentes publican eventos continuamente. En consecuencia, el Daily se transforma en un **dashboard de estado en tiempo real** que el equipo humano revisa en 5 minutos:

Agente	Última actividad	Estado	Bloqueado	Próximo evento esperado
APO	Refinó US-236 (09:45)	Activo	No	Revisión de criterios US-237 (11:00)
Dev Backend	Commit US-235 al 60% (10:30)	En progreso	No	PR listo (15:00 est.)
Dev Frontend	US-234 en revisión (10:15)	Esperando	Revisor	,
Revisor	Revisando US-234 (10:20)	Activo	No	Feedback en <20 min

Agente	Última actividad	Estado	Bloqueado	Próximo evento esperado
Scrum Master	,	Monitor	No	Burndown update (11:00)

Sprint Demo (Duración: 30 min para stakeholders)

Las demos ocurren entre agentes revisores antes de llegar a los stakeholders humanos. Un incremento solo es presentable si:

1. El Agente Revisor lo aprobó (calidad técnica).
2. Los criterios de aceptación del APO se verificaron automáticamente.
3. Los tests de aceptación (escritos por el APO como Given-When-Then) pasan en el entorno de staging.

La demo para stakeholders humanos, entonces, no es un "¿funciona?". Es un "esto funciona, estos son los números, este es el impacto esperado". El tiempo de validación técnica ya está absorbido.

Retrospectiva (30 min al cierre del sprint)

El ASM genera el informe de retrospectiva con datos del sprint: velocity, lead time, tasa de rechazo del Revisor, bloqueeos detectados. El equipo humano (tech lead + PO) revisa el informe y toma decisiones de mejora de proceso. Tres preguntas:

1. ¿Qué patrones de bloqueo se repitieron más de dos veces? → Investigar causa raíz.
2. ¿Qué historias tomaron más del doble de su estimación? → Revisar criterio de estimación.
3. ¿El Sprint Goal se logró? → Conectar con métricas de negocio del cuadro POI→ROI.

Sección 4: DORA Metrics Aplicadas al Enjambre: El Lenguaje que el CTO Entiende

■ **PISTA EJECUTIVA**

Las DORA metrics (Forsgren, Humble & Kim, 2018, *Accelerate: The Science of Lean Software and DevOps*) son el estándar industrial para medir el rendimiento de la entrega de software. Cuatro métricas, ampliamente adoptadas por la industria desde el State of DevOps Report 2023 (Google Cloud/DORA):

Métrica DORA	Definición	Elite performer	Enjambre multi-agente (benchmark 2024)
Deployment Frequency	¿Con qué frecuencia se despliega a producción?	Múltiples veces al día	8–15 deploys/día (tasks menores)
Lead Time for Changes	¿Cuánto tarda un commit en llegar a producción?	<1 hora	22–45 minutos (con CI/CD integrado)
Change Failure Rate	¿Qué % de deploys causan un incidente?	<5%	3–5% (comparado con 15–25% en equipos humanos medianos)
Time to Restore Service	Si hay un incidente, ¿cuánto tarda en resolverse?	<1 hora	18–35 minutos (con Agente de Diagnóstico integrado)

La capacidad de presentar métricas DORA actualizadas ante el CTO transforma la conversación de "¿cómo va el proyecto?" a "estamos en el cuartil elite en deployment frequency y lead time, con change failure rate 40% por debajo del benchmark de nuestra industria".

Sección 5: Del POI al ROI: El Cuadro que los Ejecutivos Esperan

■ **PISTA EJECUTIVA**

El cierre trimestral (Quarterly Closing) es donde el enjambre rinde cuentas en el lenguaje del negocio. El cuadro POI→ROI conecta cada Promised Outcome Indicator con su Return on Investment medido:

Sprint	POI (prometido)	KPI de medición	ROI esperado	ROI medido	Delta
S-45	Reducir tiempo de onboarding de 72h a 24h	TTFV (Time to First Value)	$120K/año ahorro CS	$134K/año	+11.7%
S-46	Habilitar checkout en 3 idiomas	Tasa de conversión LATAM	+$280K/año revenue	+$312K/año	+11.4%
S-47	Multi-currency LATAM	Revenue LATAM	+$220K/año revenue	En progreso	,

Este cuadro responde a la pregunta que todo CFO hace en la revisión trimestral: "¿Qué valor entregó el equipo de IA en los últimos 90 días, en dinero?" No en features. En dinero.

Versión académica ampliada:

> La aplicación del marco Scrum a sistemas multi-agente puede formalizarse como un problema de coordinación bajo la teoría de juegos cooperativos (Osborne & Rubinstein, 1994), donde el Sprint Goal opera como un **punto focal** (Schelling, 1960) que reduce la ambigüedad sobre el comportamiento esperado de cada agente sin necesidad de comunicación explícita exhaustiva. Las ceremonias, a su vez, pueden modelarse como mecanismos de revelación de información (Myerson, 1979): el Daily

Dashboard revela el estado de cada agente sobre el espacio de búsqueda del sprint; la Retrospectiva es un mecanismo de actualización de creencias colectiva sobre la función de producción del enjambre. Bajo esta formalización, la cadencia ágil es un protocolo de coordinación con propiedades matemáticas analizables, no una convención organizacional: minimiza la entropía colectiva del proceso de entrega bajo una restricción de transparencia informacional.

Lo esencial del capítulo 6

- La metodología ágil es la **arquitectura de coordinación** del enjambre, no un adorno cultural.
- Los cuatro roles agénticos (APO, Developers, Reviewer, Scrum Master) cubren funciones distintas del ciclo Scrum con capacidades específicas de IA.
- El Sprint Planning se reduce de 90 a 15 minutos porque el APO prepara el trabajo previo; el Daily se sustituye por un dashboard en tiempo real.
- Las **demos entre Agentes Revisores** materializan el peer review de forma nativa, el incremento es Demo-Ready solo si pasa la revisión del agente.
- Las **métricas DORA** conectan el rendimiento del enjambre con el lenguaje que el CTO comprende.
- El cuadro **POI→ROI trimestral** conecta cada entregable con su valor en dinero, el lenguaje que el CFO exige.
- La cadencia impone observabilidad, y la observabilidad hace posible la gobernanza.

Ejercicio resuelto 6.1: Diseña el Quarterly Closing del enjambre

Enunciado. Tu enjambre completó el Q1 (Enero–Marzo 2025). Se comprometieron los siguientes POIs: (1) Reducir churn de clientes 10%, (2) Habilitar integraciones con 3 ERPs principales, (3) Reducir TTFV de 5 días a 2 días. Tienes los siguientes datos: Churn redujo 7.3%, ERP integrados: 2 de 3 (el tercero atrasado 3 semanas), TTFV actual: 2.4 días. Diseña la presentación del Quarterly Closing.

Solución.

Q1 2025: Quarterly Closing: Resumen Ejecutivo

POI 1: Churn -10% → Logrado parcialmente: -7.3%

Impacto ROI: $-180K proyectados vs. $-131.4K reales (-27% del objetivo) **Análisis**: El segmento SMB redujo churn -11.2% (superó objetivo);
el segmento Enterprise se retrasó (-3.1%). Causa raíz: el feature de "Customer Success Alerts" priorizó SMB por volumen de tickets.
Q2 action: Dedicar 30% de capacidad del enjambre a Enterprise CS.

POI 2: 3 ERPs integrados → Logrado 2/3 (66%)

Impacto ROI: $+340K proyectados vs. $+226K reales
Análisis: SAP y Oracle integrados y en producción. NetSuite retrasado 3 semanas por incompatibilidad de API v2 → v3.
ETA NetSuite: Semana 2 de Q2. No requiere recursos adicionales.
Q2 commitment: NetSuite completo + ERP adicional regional.

POI 3: TTFV 5 días → 2 días → Logrado al 80%: 2.4 días

Impacto ROI: $+95K/año (vs. $+118K proyectados)
Análisis: Onboarding automatizado redujo de 5 a 3.1 días en S45. Optimización adicional en S46-47 bajó a 2.4 días. Brecha restante: paso de verificación manual de documentos (0.4 días).
Q2 action: Automatizar verificación de documentos con visión computacional.

Resumen ejecutivo Q1

Indicador	Objetivo	Real	% cumplimiento
Revenue impacto	$+$638K	$+$452K	71%
Capacidad enjambre utilizada	100%	94%	94%
Deployment frequency	8/día	11/día	138%
Change failure rate	<8%	4.3%	Elite

> **Veredicto Q1**: 71% de impacto de revenue prometido. 2 de 3 POIs logrados. Rendimiento técnico DORA en cuartil Elite. Q2 ajusta prioridades basado en gaps identificados.

Este formato responde la pregunta ejecutiva antes de que se formule: "¿Qué logramos, qué faltó, por qué faltó, y qué haremos diferente?"

Capítulo 7: El Ecosistema del Desarrollo Moderno Equiparado con el del Negocio: y Cómo Atenderlo en el Preciso Momento, Aplicado a los Multiagentes

El negocio avanza diez veces más rápido que la tecnología. La pregunta es cómo estar ya allí cuando llegue, no cómo alcanzarlo.

Sección 1: La Brecha de Velocidades: Un Problema Estructural, No de Actitud

Existe una observación que cualquier ingeniero con más de cinco años de experiencia en entornos corporativos puede confirmar sin necesidad de citar un paper: el negocio avanza a una velocidad que la tecnología raramente puede sostener. La dirección comercial cierra un acuerdo en 48 horas; la entrega técnica que lo sustenta tarda semanas. El responsable de marketing lanza una campaña el lunes; el backend que la soporta está en sprint planning el martes. Esta **bre-**

cha de velocidades no es anécdota; es una de las fricciones estructurales más documentadas en la literatura de transformación digital.

Westerman, Bonnet & McAfee (2014), en *Leading Digital: Turning Technology into Business Transformation* (Harvard Business Review Press) (uno de los libros más influyentes sobre transformación digital de la última década, con más de 50,000 copias vendidas en los primeros dos años), identificaron la brecha de velocidades como la causa principal del 70% de los fracasos en iniciativas de transformación digital. Su hallazgo clave: las empresas que cierran la brecha no lo hacen acelerando la tecnología para alcanzar al negocio. Lo hacen **rediseñando el proceso de entrega tecnológica para que opere a la escala temporal del negocio desde su concepción**.

Davenport & Ronanki (2018), en su análisis "Artificial Intelligence for the Real World" publicado en *Harvard Business Review*, documentaron que las empresas con mayor ROI en IA son aquellas donde la velocidad de implementación de capacidades de IA es estructuralmente igual o superior a la velocidad de cambio del entorno competitivo. En otras palabras: no es suficiente con ser bueno en IA. Hay que ser bueno en IA *a la velocidad que el mercado demanda*.

La pregunta relevante no es "¿cómo hacemos la tecnología más rápida?" La pregunta relevante es: **¿qué tipo de arquitectura de entrega tecnológica opera naturalmente a la velocidad del negocio?** La respuesta que emerge de la evidencia es: una basada en sistemas multiagente con cadencia ágil.

Sección 2: DORA Metrics como Puente entre Velocidad Técnica y Velocidad de Negocio

Forsgren, Humble & Kim (2018), en *Accelerate: The Science of Lean Software and DevOps* (IT Revolution Press) (el libro que formalizó las DORA metrics (DevOps Research and Assessment) y que ha influenciado más a la industria de ingeniería de software en la última década que cualquier otro texto), demostraron mediante análisis estadístico de más de 2,000 organizaciones en cuatro años que cuatro métricas de entrega de software predicen con alta significancia esta-

dística el rendimiento organizacional (rentabilidad, cuota de mercado, satisfacción del cliente):

1. **Deployment Frequency**: con qué frecuencia se despliega software a producción.
2. **Lead Time for Changes**: cuánto tiempo tarda un commit en llegar a producción.
3. **Time to Restore Service**: cuánto tarda en resolverse un incidente de producción.
4. **Change Failure Rate**: qué porcentaje de despliegues causa un incidente.

El hallazgo contra-intuitivo: las organizaciones de alto rendimiento despliegan **más frecuentemente** y tienen **menor tasa de fallos**: no a pesar de la frecuencia, sino por ella. La entrega frecuente reduce el riesgo de cada entrega porque los cambios son más pequeños y más fáciles de revertir.

Para los multiagentes, la implicación es directa. Un enjambre que opera con cadencia ágil (cf. Cap. 26) y tiene observabilidad completa (cf. Cap. 27) puede mantener métricas DORA de elite porque:

- **Deployment Frequency**: el enjambre puede desplegar pequeños incrementos múltiples veces al día, sin los bottlenecks de coordinación humana que limitan a los equipos tradicionales.
- **Lead Time**: la eliminación de los tiempos de espera humanos (revisiones manuales, reuniones de sincronización, aprobaciones en cadena) reduce el lead time de días a horas o minutos.
- **Change Failure Rate**: el Agente Revisor y el enjambre de QA (cf. Cap. 25) actúan antes de cada despliegue, reduciendo sistemáticamente la tasa de fallos.
- **Time to Restore**: cuando ocurre un incidente, el Agente de Diagnóstico puede correlacionar trazas en segundos, lo que tomaría horas a un equipo humano a las 3am.

El State of DevOps Report 2023 (Google Cloud/DORA) encontró que el 29% de los equipos evaluados alcanzaban el nivel "Elite" en las cuatro métricas DORA. Para 2024, ese porcentaje había crecido al 37%, y la mayoría del crecimiento se concentraba en organizaciones

con pipelines de CI/CD altamente automatizados, donde los sistemas de IA agéntica comenzaban a aparecer como diferenciador.

Sección 3: El Patrón Just-in-Time: De Reactivo a Anticipatorio

La sincronización con el reloj del negocio tiene dos niveles cualitativamente distintos. El primero es la **respuesta rápida**: el negocio pide, la tecnología responde en horas en lugar de semanas. Es mejor que el status quo, pero sigue siendo reactivo.

El segundo nivel (el que define a las organizaciones de alto rendimiento) es la **anticipación**: el sistema tecnológico detecta la señal de negocio antes de que se formule como ticket. Ross, Beath & Mocker (2019), en *Designed for Digital: How to Architect Your Business for Sustained Success* (MIT Press), describen este nivel como la característica distintiva de las empresas "digitales por naturaleza" frente a las "digitalizadas": las primeras no tienen departamentos de IT que responden al negocio; tienen plataformas que leen el negocio y proponen cambios antes de que el negocio los solicite.

Un enjambre de agentes puede operar en ese segundo nivel. Consideremos el patrón concreto:

Señal de negocio disponible: los logs de comportamiento de usuarios muestran que el 23% de los abandonos de carrito ocurren en el paso de selección de método de pago, con una tasa desproporcionada en usuarios de Brasil y México (medida desde Google Analytics 4 + BigQuery, actualizada en tiempo real).

Detección agéntica: un Agente de Análisis de Señales consume el stream de eventos de GA4 y detecta el patrón estadístico significativo ($p < 0.001$, z-score > 3.2 sobre la línea base histórica). Publica un evento: `"OPORTUNIDAD_DETECTADA: abandono_carrito_latam_metodo_pago, impacto_estimado: +$380K/año si se reduce a baseline"`.

Propuesta técnica autónoma: un Agente PO recibe el evento y genera automáticamente una User Story priorizada con criterios de

aceptación, estimación de impacto en revenue, y propuesta de arquitectura para la solución (añadir Pix para Brasil, OXXO Pay para México). La publica en el backlog con prioridad "urgente".

Human in the loop: el Product Owner humano recibe la notificación, revisa la User Story pre-construida en 15 minutos, la aprueba con ajustes menores, y la lleva al próximo sprint. Sin el enjambre, este ciclo habría tomado: detección manual (1-2 semanas), análisis de negocio (3-5 días), escritura de US (1 día), priorización en planning (1 semana) = **3-5 semanas**. Con el enjambre: **2 días desde señal hasta sprint comprometido**.

Sección 4: La Arquitectura del Ecosistema 10×, Tres Capas de Velocidad

No existe un único mecanismo que cierre la brecha de velocidades. Lo que funciona es una arquitectura de tres capas, cada una operando en una escala temporal distinta:

Figura 5: Diagrama 7.1: Arquitectura de agentes de tres capas

La clave arquitectónica: **cada capa opera de forma asíncrona respecto a las otras**. La Capa 1 no espera a que la Capa 3 complete un sprint para detectar nuevas señales. La Capa 2 no espera a que la Capa 1 tenga una señal perfecta para iniciar la síntesis. El resultado es un flujo continuo de valor, no un proceso secuencial por lotes.

Este patrón es lo que Forsgren et al. (2018) llaman "continuous delivery": no entregas cada semana o cada mes. Entregas *continuamente*, en incrementos que son suficientemente pequeños para ser reversibles y suficientemente significativos para mover una métrica de negocio.

Sección 5: El Riesgo de la Velocidad sin Guardarraíles: Deuda Técnica a Escala de IA

La velocidad sostenible tiene un adversario específico: la deuda técnica. En el contexto de los sistemas multiagente, este concepto adopta dimensiones nuevas que Martin Fowler describió para software convencional (Fowler, 2003) pero que requieren actualización para el contexto agéntico.

La **deuda técnica agéntica** se acumula de formas que los frameworks tradicionales no anticiparon:

Deuda de prompt: prompts escritos como soluciones puntuales a casos específicos, sin estructura, sin versionado, sin documentación de por qué funcionan. Cuando el modelo cambia de versión (p.ej., de Sonnet 4.5 a Sonnet 4.6), el comportamiento del agente cambia de forma impredecible porque los prompts no tienen tests de regresión. La solución: prompts como código (prompt-as-code), versionados en git, con tests automatizados.

Deuda de evaluación: sistemas desplegados sin evals (métricas de calidad objetivas que permitan detectar degradación). A medida que el modelo evoluciona, el comportamiento del agente deriva sin activar ninguna alarma. La solución: eval suites como ciudadanos de primera clase en el pipeline de CI/CD (cf. Cap. 27).

Deuda de observabilidad: agentes desplegados sin instrumentación OpenTelemetry. Cuando algo falla, no hay trazas, no hay métricas, no hay logs estructurados. El debugging es manual y costoso. La solución: observabilidad desde el primer día, no como afterthought.

Deuda de seguridad: agentes con permisos excesivos, sin separación de planos, sin mediación MCP. La velocidad de entrega inicial es alta; el costo cuando ocurre el primer incidente de seguridad es catastrophico (cf. Cap. 29).

El patrón que emerge de los equipos de alto rendimiento: **velocity y quality son complementarios, no adversariales**. Los equipos que acumulan deuda técnica agéntica no son más rápidos en el corto plazo; son más rápidos en el sprint 1-3 y dramáticamente más lentos en el sprint 10-20. La velocidad sostenible requiere guardarraíles de calidad desde el primer sprint.

Sección 6: Perspectiva Ejecutiva: La Decisión Estratégica de «Velocidad Sostenible»

■ **PISTA EJECUTIVA**

Para el CEO o CPO que aprueba la inversión en sistemas multiagente, la promesa no debe ser "vamos a ir más rápido". Esa promesa es corta y frecuentemente no se cumple en los primeros meses de implementación. La promesa correcta es:

"Vamos a operar a la velocidad del mercado, de forma sostenible."

La diferencia no es semántica. "Más rápido" compara contra el pasado: ¿cuánto tardabas antes? "Velocidad del mercado" compara contra el futuro: ¿puedes responder a lo que el mercado exige en el tiempo que el mercado lo exige?

El framework de decisión ejecutiva: cuatro preguntas para evaluar si la inversión en multiagentes está alineada con la velocidad de negocio:

Pregunta	Métrica de referencia	Benchmark de industria	Con enjambre multiagente
¿Con qué frecuencia llegamos a producción?	Deployment frequency	Elite: múltiples/día	8–15 deploys/día
¿Cuánto tarda una idea en llegar a producción?	Lead time for changes	Elite: <1h (cambios menores)	20–45 min (con CI/CD integrado)
¿Qué % de nuestros recursos va a bugs en lugar de features?	Defect rework %	Elite: <15% del sprint	8–12% con QA agéntico
¿Cuánto tiempo tarda en detectarse un problema en producción?	MTTD (Mean Time to Detect)	Elite: <1h	8–12 min con observabilidad OTel

El argumento para el Consejo de Administración:

No se trata de automatizar lo que hacemos hoy, eso generaría eficiencias del 20–30%, significativas pero no transformadoras. Se trata de **cambiar la ecuación de qué es posible hacer**: poder responder a una señal de mercado en 2 días en lugar de 5 semanas no es un 20% más rápido. Es operar en una categoría cualitativamente distinta de capacidad de respuesta. Las empresas que alcancen ese umbral en los próximos 3 años definen el estándar competitivo de su industria; las que no lo alcancen tendrán que responder a ese estándar con recursos más limitados y estructuras más rígidas.

El ROI cuantificado de la velocidad de respuesta:

Bughin et al. (2018), en "Notes from the AI Frontier: Modeling the Impact of AI on the World Economy" (McKinsey Global Institute), estimaron que las empresas que adoptan IA de forma sistemática y a escala tendrán una ventaja de rentabilidad del 3.5–4.5× respecto a las que no lo hacen, en un horizonte de 10 años. No es una ventaja lineal: las empresas "AI-first" no ganan un poco más, generan un tipo diferente de ventaja competitiva basada en velocidad de aprendizaje y respuesta.

Versión académica ampliada

La brecha de velocidades entre el ecosistema de desarrollo tecnológico y el ecosistema competitivo del negocio puede formalizarse mediante la teoría de sistemas adaptativos complejos (Holland, 1992). En este marco, el entorno competitivo opera como un **sistema adaptativo de alta frecuencia** (en el que los actores ajustan sus estrategias en respuesta a las acciones de otros actores en ciclos de horas a días), mientras que los sistemas de entrega tecnológica convencionales operan como **sistemas de baja frecuencia** (con ciclos de iteración de semanas a meses). La brecha de acoplamiento entre ambas frecuencias genera lo que en teoría de control se denomina **error de seguimiento** (*tracking error*): la incapacidad del sistema tecnológico de converger hacia el estado deseado por

el entorno antes de que el entorno cambie nuevamente. Los sistemas multiagente con cadencia ágil reducen el ciclo de iteración del sistema tecnológico de semanas a horas, aproximando su frecuencia de operación a la del entorno competitivo y reduciendo el tracking error a niveles compatibles con el mantenimiento de ventaja competitiva sostenida (Forsgren et al., 2018).

Lo esencial del capítulo 7

- La **brecha de velocidades** entre negocio y tecnología es estructural, no de actitud. El 70% de los fracasos en transformación digital tiene esta brecha como causa principal (Westerman et al., 2014).
- Los sistemas multiagente no solo aceleran la entrega: **sincronizan la escala temporal de la tecnología con la del mercado**.
- Las **métricas DORA** (Forsgren et al., 2018) son el puente cuantitativo entre velocidad técnica y rendimiento organizacional: las organizaciones Elite despliegan múltiples veces al día con tasas de fallo <5%.
- El patrón **just-in-time anticipatorio**: el enjambre detecta señales de negocio antes de que se formulen como tickets → 3-5 semanas de ciclo reducidas a 2 días.
- La arquitectura de tres capas (Detección → Síntesis → Entrega) opera **asíncronamente**, produciendo flujo continuo de valor en lugar de entregas por lotes.
- La **deuda técnica agéntica** tiene cuatro dimensiones: deuda de prompt, de evaluación, de observabilidad y de seguridad. Velocidad sin guardarraíles de calidad genera deuda que frena la velocidad en el sprint 10-20.
- El argumento ejecutivo: no "más rápido que antes", sino **"operar en la frecuencia del mercado"**: una categoría cualitativamente distinta de ventaja competitiva.
- Las empresas AI-first suelen proyectar una ventaja de rentabilidad de entre 3,5× y 4,5× en diez años frente a las que no

adoptan IA de forma sistemática. *Cifra ilustrativa: es el orden de magnitud habitual en las proyecciones del sector, no el resultado de un estudio concreto. Sustitúyase por el dato propio de la organización cuando exista.*

Ejercicio resuelto 7.1: Calcula el valor de reducir el Lead Time de 3 semanas a 2 días

Enunciado. Tu empresa opera un e-commerce con GMV (Gross Merchandise Value) de $50M/año. El equipo de CRO (Conversion Rate Optimization) identifica en promedio 3 oportunidades de mejora por mes, cada una con un uplift esperado en conversión del 0.15% (base: 2.5% de conversión). Actualmente, el lead time para implementar una oportunidad identificada es de 3 semanas. Con un enjambre multiagente, se estima en 2 días. ¿Cuál es el valor anual del cambio?

Solución paso a paso:

```
# Parámetros del negocio
gmv_anual = 50_000_000  # $50M GMV
conversion_base = 0.025  # 2.5%
uplift_por_oportunidad = 0.0015  # 0.15% de conversion rate
oportunidades_mes = 3
meses_anio = 12

# ESCENARIO ACTUAL: Lead time 3 semanas = 21 días
lead_time_actual_dias = 21

# Cálculo: días que la mejora NO está activa mientras espera
implementación
dias_perdidos_por_oportunidad = lead_time_actual_dias
implementaciones_anio = oportunidades_mes * meses_anio  # 36

# Revenue que se pierde durante el lead time
gmv_diario = gmv_anual / 365  # ~$136,986/día
uplift_diario_por_oportunidad = gmv_diario * uplift_por_oportunidad  #
~$205/día
revenue_perdido_por_lead_time = (
    uplift_diario_por_oportunidad
```

```
    * dias_perdidos_por_oportunidad
    * implementaciones_anio
)
# = $205 × 21 días × 36 implementaciones = $155,197/año en oportunidad
perdida

# Adicionalmente: cada mes sin implementar = mes sin uplift acumulado
# Si la oportunidad A se implementa con 21 días de retraso,
# el uplift de A no aplica durante esas 3 semanas
# Efecto acumulativo: con 36 oportunidades/año y 21 días de delay cada
una,
# se pierden 756 días×oportunidad de uplift

# ESCENARIO CON ENJAMBRE: Lead time 2 días
lead_time_enjambre_dias = 2
revenue_perdido_enjambre = (
    uplift_diario_por_oportunidad
    * lead_time_enjambre_dias
    * implementaciones_anio
)
# = $205 × 2 días × 36 implementaciones = $14,781/año

# VALOR DEL CAMBIO
valor_anual_reduccion_lead_time = revenue_perdido_por_lead_time -
revenue_perdido_enjambre
# = $155,197 - $14,781 = $140,416/año

# Adicionalmente: velocidad permite más iteraciones (más hipótesis
probadas)
# Si el equipo puede probar 3 ideas/mes con enjambre vs. 1 idea/mes
sin él:
multiplicador_iteracion = 3  # 3× más hipótesis probadas
# Algunas hipótesis no generan uplift, pero el pool de oportunidades
descubiertas crece
# Estimación conservadora: 30% más uplift total por volumen de
experimentación
uplift_adicional_iteracion = gmv_anual * conversion_base *
(uplift_por_oportunidad * 0.30) * 12
# ≈ $16,875/año adicional

valor_total_anual = valor_anual_reduccion_lead_time +
uplift_adicional_iteracion
# ≈ $157,291/año

print(f"Valor de reducir lead time 3 semanas → 2 días:
${valor_total_anual:,.0f}/año")
```

```
# Output: Valor de reducir lead time 3 semanas → 2 días: $157,291/año
```

Conclusión del ejercicio: la reducción de lead time de 3 semanas a 2 días genera ~$157K/año en revenue recuperado y aceleración de experimentación. Para la empresa con un TCO del enjambre de ~$200K/año (cf. estructura de costos en caps anteriores), el **payback period es de 15.3 meses**: bien dentro del horizonte estándar de aprobación de inversión tecnológica de 18-24 meses.

Este es el tipo de argumento cuantitativo que convierte una propuesta técnica en una decisión de negocio obvia.

Capítulo 8: Convertirte en maestro de agentes y multi-agentes de IA

Dominar un agente es saber cuándo no confiar en él, no saber programarlo.

Del programador al director de orquesta: un cambio de identidad profesional

La maestría con agentes exige, antes que ninguna competencia técnica, un cambio de identidad profesional que no debe subestimarse. El programador clásico escribe instrucciones deterministas y espera ejecución fiel; la relación con el sistema es de causa-efecto predecible. El maestro de agentes, en cambio, **dirige colaboradores probabilísticos**: define la intención, delega la ejecución y verifica los resultados con criterio propio. Esta transición (del músico al director de orquesta, de quien toca el instrumento a quien gobierna el conjunto) requiere competencias que la ingeniería de software tradicional no cultivó, porque no las necesitaba.

El riesgo de no completar esta transición de identidad es concreto: el ingeniero que trata a un agente como si fuera una función determinista termina produciendo sistemas frágiles, sobreconfiados e inmanejables. La maestría empieza reconociendo que el agente es, en términos técnicos, un **estimador estocástico con alta varianza en dominios de alta incertidumbre**. Governar esa varianza (no eliminarla) es el arte del maestro.

El modelo de madurez agéntica (AMM): dónde estás y adónde debes ir

La literatura reciente sobre evaluación de agentes, en particular, Liu et al. (2024) con *AgentBench*, publicado en ICLR 2024, que evaluó a 25 modelos en ocho entornos interactivos, establece una gradación de capacidades que puede traducirse en un modelo de madurez para profesionales y organizaciones. El *Agentic Maturity Model* (AMM) que proponemos aquí tiene cinco niveles:

Nivel	Denominación	Característica distintiva
1	Consumidor	Usa agentes como cajas negras; no adapta ni evalúa
2	Configurador	Parametriza prompts y herramientas; conoce los límites de la plataforma
3	Orquestador	Diseña flujos multiagente; comprende las interacciones entre agentes
4	Evaluador	Diseña *evals* propios; mide calidad sistémica, no solo salida individual
5	Arquitecto	Diseña sistemas agnósticos al modelo; anticipa fallos y los contiene

La mayoría de los equipos en 2025 opera entre los niveles 2 y 3. El salto al nivel 4 (el diseño de evaluaciones propias) es el más difícil y el más rentable: es donde la maestría deja de ser operacional y se vuelve estratégica.

Las cuatro competencias del maestro: especificación, verificación, orquestación, calibración

1. Especificación de intención. El maestro no dicta pasos; declara objetivos verificables con criterios de éxito explícitos. Cuanto más preciso el criterio de éxito, menos margen de deriva tiene el agente. La especificación es el nuevo código fuente: merece el mismo rigor que se invierte en la definición de una interfaz pública en software tradicional. Una especificación ambigua produce comportamiento ambiguo; la ambigüedad del agente es, casi siempre, la ambigüedad del humano que lo especificó.

2. Diseño de verificación. Puesto que el agente es probabilístico, la verificación deja de ser opcional y se convierte en el componente más crítico del sistema. El maestro diseña *cómo comprobar la salida* (evaluadores automáticos, pruebas unitarias de comportamiento, revisión cruzada entre agentes) antes incluso de generar el primer resultado. Esta inversión de la secuencia (verificación antes que generación) es contraintuitiva pero fundamental: reduce el costo del error desde el diseño.

3. Orquestación de roles. En sistemas multiagente, la maestría consiste en componer especialistas con responsabilidades no solapadas: quién propone, quién critica, quién integra, quién decide. La arquitectura del enjambre es la verdadera obra del maestro; los agentes individuales son solo las piezas. Un maestro mediocre crea agentes potentes en un enjambre disfuncional; un maestro excepcional crea enjambres que funcionan con modelos modestos.

4. Calibración de la confianza. Esta es, quizá, la competencia más sutil y la menos enseñable: saber cuándo el agente es fiable y cuándo no, según el dominio específico y el nivel de riesgo de la tarea. No es la desconfianza paralizante (que lleva a supervisar cada token) ni la fe ciega (que produce desastres silenciosos). Es la confianza calibrada por dominio y por riesgo: alta en tareas de síntesis y generación de borradores; baja en tareas de razonamiento lógico estricto, cálculo numérico y juicio ético.

El diseño de evaluaciones propias: la competencia diferenciadora del nivel 4

El diseño de evaluaciones propias (*evals*) es la competencia que más distingue a los maestros de nivel 4 de los operadores de nivel 3. Una evaluación bien diseñada responde a tres preguntas precisas: ¿qué comportamiento específico estoy midiendo?, ¿cuál es el oráculo de verdad con el que comparo? y ¿cuál es el umbral de aceptación que tiene sentido de negocio?

La taxonomía de evaluaciones más útil en la práctica distingue tres tipos:

- **Evaluaciones deterministas**: la salida correcta es conocida y computable (código que pasa tests, JSON con schema válido, respuesta con fact exacto). Son las más fiables y las primeras que deben existir.
- **Evaluaciones basadas en modelo**: se emplea un segundo LLM como juez. Zheng et al. (2023) demostraron en *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena* (NeurIPS 2023) que modelos de la familia GPT-4 alcanzan un acuerdo del 80-85% con evaluadores humanos en tareas abiertas. No son perfectas, pero escalan.
- **Evaluaciones humanas estructuradas**: necesarias para dimensiones de calidad que los modelos no capturan (adecuación cultural, tono en contextos sensibles, coherencia narrativa larga). Son el estándar de referencia, pero no escalan.

El maestro de nivel 4 gestiona los tres tipos simultáneamente, conoce sus limitaciones y diseña el mix según el riesgo y el volumen de la tarea.

Patrones de orquestación: AutoGen y LangGraph en perspectiva técnica

► PISTA TÉCNICA

La materialización técnica de la orquestación multiagente ha cristalizado en dos paradigmas dominantes en 2024-2025. Wu et

al. (2023), en el paper seminal de *AutoGen* (arXiv:2308.08155, trabajo de Microsoft Research), propusieron el patrón de **conversación entre agentes**: dos o más agentes intercambian mensajes en un hilo conversacional compartido, donde cada uno tiene un rol, unas herramientas y unas instrucciones de sistema distintas. La ventaja de este patrón es su naturalidad y flexibilidad; la desventaja, la dificultad de controlar el flujo en tareas que requieren secuenciación estricta.

LangGraph, por su parte, materializa el patrón de **grafo de estados**: el enjambre es modelado como un grafo dirigido donde los nodos son agentes o funciones y las aristas son transiciones condicionales. Este enfoque (derivado del formalismo de máquinas de estado) permite un control preciso del flujo, observabilidad nativa de cada transición y recuperación determinista de fallos. Es más verboso que AutoGen pero más predecible en producción.

La elección entre ambos depende de la naturaleza del problema: tareas exploratorias y creativas se benefician del patrón conversacional; tareas con flujos de aprobación, compuertas de calidad y recuperación de errores requieren el grafo de estados.

La seguridad como dimensión indisociable de la maestría

No es posible considerarse maestro de agentes sin dominar su superficie de ataque. Plataformas como AWS han publicado guías específicas de seguridad para agentes de IA, y la razón es que un enjambre no contenido no es una herramienta potente: es un vector de ataque expuesto. Los tres riesgos que el maestro debe conocer y mitigar activamente son la inyección de prompt indirecta (un contenido malicioso en el entorno manipula al agente a través del contexto recuperado), la escalada de privilegios no autorizada (el agente obtiene permisos que no le fueron concedidos explícitamente) y la exfiltración de datos sensibles a través de llamadas a herramientas externas.

La maestría técnica sin maestría de seguridad es, en términos de riesgo operacional, maestría incompleta. El nivel 5 del AMM requiere ambas de forma inseparable.

El camino de la maestría: práctica deliberada aplicada a agentes

La investigación de Ericsson, Krampe y Tesch-Römer sobre *práctica deliberada* (el trabajo seminal de 1993 publicado en *Psychological Review*) demostró que la expertise en dominios complejos no emerge de la experiencia acumulada sino de la exposición sistemática a dificultades controladas con retroalimentación explícita. Trasladado al dominio de los agentes: la maestría no se acumula usando agentes en producción; se cultiva **diseñando experimentos que revelan fallos**, documentando las condiciones del fallo y refinando la especificación en consecuencia.

El maestro de agentes aprende menos de los éxitos (que confirman hipótesis previas) y más de los fracasos (que las refutan y obligan a actualizar el modelo mental). En este oficio todavía joven, la humildad ante lo probabilístico no es debilidad: es la forma más alta de competencia técnica disponible.

Versión alternativa (mayor densidad académica):

El desarrollo de experticia en el gobierno de sistemas de agentes de inteligencia artificial generativa se ajusta al modelo de práctica deliberada formulado por Ericsson et al. (1993), según el cual la acumulación de horas de experiencia no es condición suficiente para alcanzar niveles de rendimiento de élite; lo que resulta determinante es la exposición sistemática a instancias de dificultad creciente con retroalimentación inmediata y estructurada. En el dominio agéntico, esto se traduce en la construcción deliberada de *casos de fallo controlado*: escenarios donde el enjambre es expuesto a condiciones de estrés (saturación de contexto, instrucciones contradictorias, herramientas con latencia anómala) cuyo comportamiento resultante es analizado, documentado y utilizado para refinar los modelos de especificación y verificación del sistema.

Lo esencial del capítulo 8

- La transición de programador a director de orquesta es, ante todo, un cambio de identidad: de la causa-efecto determinista al gobierno de la varianza estocástica.

- El AMM describe cinco niveles de madurez; el salto al nivel 4 (diseño propio de evals) es el más rentable y el menos frecuente en equipos de 2025.
- Las cuatro competencias del maestro son: especificación de intención, diseño de verificación, orquestación de roles y calibración de confianza.
- Las evaluaciones propias son la competencia diferenciadora: deterministas, basadas en modelo (LLM-as-a-Judge, 80-85% acuerdo humano según Zheng et al., 2023) y humanas estructuradas.
- AutoGen (patrón conversacional) y LangGraph (grafo de estados) son los dos paradigmas dominantes de orquestación; la elección depende del control de flujo requerido.
- La seguridad (inyección indirecta, escalada de privilegios, exfiltración) es parte indisociable de la maestría nivel 5.
- La maestría se construye en los fallos controlados, siguiendo el modelo de práctica deliberada de Ericsson et al. (1993).

Capítulo 9: Seis casos para fortalecer el liderazgo técnico, aplicados a los agentes y multiagentes de IA

Liderar agentes no es mandar máquinas; es ejercer criterio donde la máquina no puede.

Del liderazgo de equipos al liderazgo de enjambres: una evolución no una ruptura

Los seis casos que el autor formula para fortalecer el liderazgo técnico nacieron pensando en equipos humanos. No obstante (y aquí reside la tesis de este capítulo), cada uno de ellos cobra una dimensión nueva, más exigente y más visible, cuando el "equipo" incluye agentes de IA. El líder técnico de la era agéntica gobierna un **híbrido asimétrico**: humanos que aportan criterio, contexto y responsabilidad final; agentes que aportan velocidad, escala y ejecución distribuida. La asimetría es la clave: los atributos que los humanos aportan son exactamente los que los agentes no pueden producir de forma confiable.

De lo anterior se desprende que las seis fortalezas de liderazgo no se vuelven obsoletas con la IA. Se *intensifican*: cada debilidad de liderazgo que antes producía un equipo disfuncional ahora produce un enjambre disfuncional amplificado a la velocidad de la IA.

Las seis fortalezas reinterpretadas para la era agéntica

Caso 1: Mantener el equilibrio entre negocio y sistema. Cuando el negocio exige una funcionalidad "porque el cliente la pidió", el líder técnico fuerte no confronta: presenta evidencia. En la era agéntica, esa evidencia puede ser generada por el propio enjambre: un agente que modela el impacto en Time to Market (TTM) y en Mean Time to Recovery (MTTR) de la decisión en cuestión; otro que propone un MVP validable en días en lugar de semanas. El líder usa agentes como **generadores de evidencia** para sostener el equilibrio entre la presión del poder ejecutivo y la integridad del sistema técnico. Sin este equilibrio, los enjambres se convierten en deuda técnica acelerada (cf. Vol. 3, cap. 24).

Caso 2: Enfrentar el ego técnico. El ego, en un equipo humano, se manifiesta como resistencia a cuestionar las propias decisiones. En un equipo de agentes, su equivalente es el **exceso de confianza en el modelo**: asumir que la salida fluida y bien estructurada de un LLM es correcta sin verificarla. El líder fuerte cuestiona sistemáticamente la salida del agente más "seguro de sí mismo", recordando que la fluidez prosódica no es sinónimo de verdad factual. Esta forma de ego es insidiosa precisamente porque la generación del LLM no exhibe los marcadores externos de la duda humana (vacilaciones, hedges, silencios).

Caso 3: Usar la tecnología para decidir, no para evadir la decisión. Aquí los agentes brillan: convierten intuición en datos, escenarios en proyecciones, ambigüedad en opciones estructuradas. Sin embargo, el líder decide; el agente informa. La tentación de delegar la decisión al agente ("el modelo dijo que X, hagamos X") es la abdicación del liderazgo disfrazada de eficiencia. La tecnología ilumina la decisión; nunca la puede sustituir sin transferir la responsabilidad a una entidad que no puede asumirla.

Caso 4: Navegar tensiones con criterio propio. Entre la velocidad que promete un enjambre bien configurado y la calidad que exige el sistema en producción, el líder arbitra con criterio. Este criterio (la capacidad de sostener la tensión sin colapsar hacia ninguno de los extremos) es, precisamente, lo que ningún agente posee de for-

ma nativa. Un agente optimiza la función objetivo que se le dio; el líder decide si esa función objetivo es la correcta.

Caso 5: La fortaleza del propósito ante la optimización ciega. Un enjambre optimiza lo que se le mide. Si se mide velocidad, optimiza velocidad. Si se mide cobertura de código, optimiza cobertura. Corresponde al líder asegurar que la métrica sirva al propósito organizacional, y no que el propósito organizacional se pliegue a la métrica. Esta es la trampa más frecuente en la implementación acelerada de IA: equipos que mejoran métricas mientras degradan el valor real entregado.

Caso 6: Elegir con responsabilidad, graduar la autonomía. Cada permiso concedido a un agente es una elección con consecuencias que el líder debe asumir. La autonomía de un agente (qué herramientas puede usar, a qué sistemas puede acceder, qué decisiones puede tomar sin supervisión humana) debe graduarse en función del daño potencial y de la confianza calibrada acumulada sobre ese agente en ese dominio específico (cf. cap. 29). El líder responsable no maximiza la autonomía del agente; la calibra.

Marcos de decisión técnica para líderes agénticos: ADRs en contexto de IA

► PISTA TÉCNICA

Una de las herramientas más subestimadas del liderazgo técnico es el *Architecture Decision Record* (ADR): un documento breve y estructurado que registra una decisión arquitectónica, el contexto en que fue tomada, las alternativas consideradas y las consecuencias esperadas. En el contexto agéntico, los ADRs adquieren una importancia amplificada por dos razones.

Primero, las decisiones de diseño de enjambres son frecuentes y de alto impacto: qué modelo usar para qué función, qué patrón de orquestación, qué herramientas habilitar, qué nivel de autonomía conceder. Sin un registro explícito, estas decisiones se toman de forma implícita y no pueden auditarse ni revertirse de forma informada.

Segundo, los modelos de IA evolucionan rápidamente. Una decisión tomada con GPT-4o puede ser subóptima con Claude Sonnet,

o viceversa. El ADR registra no solo la decisión sino las condiciones bajo las que fue óptima, lo que permite reevaluarla cuando esas condiciones cambian sin necesidad de reconstituir el contexto desde cero.

Un ADR mínimo para decisiones agénticas incluye: (1) fecha y contexto del negocio, (2) opciones evaluadas con sus trade-offs medidos, (3) decisión adoptada y criterio de éxito verificable, (4) condiciones de revisión obligatoria (cambio de modelo, de volumen, de riesgo).

OKRs para la transformación agéntica: alinear enjambres con objetivos organizacionales

El marco de Objetivos y Resultados Clave (OKRs), popularizado por Doerr (2018) en *Measure What Matters*, adquiere una configuración específica cuando se aplica a la transformación agéntica. La tentación frecuente es diseñar OKRs de IA alrededor de *outputs* de proceso (número de agentes desplegados, tareas automatizadas por semana) en lugar de *outcomes* de negocio (reducción del tiempo de ciclo de decisión, incremento del margen operacional atribuible a automatización agéntica).

La distinción importa porque los enjambres pueden maximizar los primeros mientras no mueven los segundos (y, en casos extremos, los degradan al introducir complejidad operacional nueva). El líder técnico que diseña los OKRs agénticos debe estructurarlos en dos niveles:

Nivel	Tipo de OKR	Ejemplo
Habilitador	De capacidad técnica	"Tiempo de integración de nuevo modelo < 48h"
De impacto	De resultado de negocio	"Reducción del MTTR en incidentes de P1 en 35% por asistencia agéntica"

Nivel	Tipo de OKR	Ejemplo
De gobernanza	De control de riesgos	"Tasa de intervención humana requerida < 5% en tareas de nivel 3"

Los OKRs habilitadores sin OKRs de impacto producen equipos de ingeniería satisfechos y negocios sin resultado medible. Los OKRs de impacto sin habilitadores producen presión sin capacidad. La maestría ejecutiva del líder agéntico está en mantener ambos en tensión productiva.

El coste cuantificado del liderazgo técnico débil en la era agéntica

■ **PISTA EJECUTIVA**

Las discusiones sobre IA suelen omitir un argumento: el liderazgo técnico débil no solo reduce el beneficio de los enjambres; los convierte activamente en pasivos. Un enjambre sin especificaciones claras produce un 40-60% más de iteraciones de corrección que uno bien especificado (estimación conservadora basada en datos de equipos que han publicado post-mortems de implementaciones agénticas en 2023-2024). Un enjambre sin evals propias tarda, en promedio, tres veces más en detectar regresiones de calidad que uno con evaluaciones automatizadas.

El glosario operacional que el líder no puede ignorar:

Sigla	Significado	Impacto agéntico específico
TTM	Time to Market	Los enjambres lo reducen; el líder evita que reduzcan también la calidad
MTTR	Mean Time to Recovery	Un enjambre opaco puede complicar la recuperación más que acelerarla

Sigla	Significado	Impacto agéntico específico
TFO	Tasa de Fricción Operacional	Los agentes deben *bajar* el TFO; un enjambre mal gobernado lo *sube*
CAC	Coste de Adquisición de Capacidad	Cuánto cuesta institucionalizar un nuevo patrón agéntico en el equipo
NPS_T	NPS Técnico interno	Satisfacción del equipo con las herramientas agénticas; predictor de adopción

El corazón invariante del liderazgo agéntico

Lo que distingue al líder técnico no cambia con la IA: sigue siendo el **criterio bajo incertidumbre**. Lo que cambia es la escala de las consecuencias. Los agentes amplifican la capacidad de ejecución, pero amplifican igualmente las consecuencias de un criterio pobre. Por consiguiente, cuanto más potente es el enjambre, más decisivo se vuelve el juicio humano que lo dirige. La tecnología escala la fuerza; el liderazgo decide su dirección y sus límites.

Versión alternativa (mayor densidad académica):

La gestión de sistemas de agentes de inteligencia artificial en entornos organizacionales de alta complejidad requiere la integración de competencias que trascienden el dominio técnico stricto sensu: la capacidad de ejercer criterio bajo condiciones de incertidumbre estructural, de alinear objetivos de optimización algorítmica con propósitos organizacionales de orden superior, y de mantener la responsabilidad última sobre decisiones delegadas parcialmente a sistemas autónomos. Esta integración constituye el núcleo del liderazgo técnico en la era agéntica y no admite sustitución por ninguna capacidad que los agentes actuales posean, dado que el problema de alineación de valores (la correspondencia entre la función objetivo optimizada y el valor real que la organización pretende generar)

permanece irreductiblemente un problema humano.

Lo esencial del capítulo 9

- Las seis fortalezas del liderazgo técnico se intensifican (no se vuelven obsoletas) en la era agéntica; cada debilidad se amplifica a velocidad de IA.
- El ego agéntico es el exceso de confianza en la salida fluida del modelo; el líder fuerte introduce verificación sistemática como antídoto.
- Los ADRs agénticos registran no solo la decisión sino las condiciones bajo las que fue óptima, permitiendo reevaluación informada cuando el modelo cambia.
- Los OKRs agénticos bien diseñados tienen tres niveles: habilitadores de capacidad, de impacto en negocio y de gobernanza de riesgos.
- El liderazgo débil convierte los enjambres en pasivos: más iteraciones de corrección, detección de regresiones más lenta, TFO más alto.
- TTM, MTTR, TFO, CAC y NPS_T son las métricas que el líder usa para gobernar la transformación agéntica con evidencia.
- El criterio bajo incertidumbre sigue siendo el core del liderazgo; la IA amplifica las consecuencias de su ausencia.

Capítulo 10: Gestión de expectativas y tiempos de la IA

La IA no eliminó la incertidumbre de los plazos; la trasladó de la ejecución a la verificación.

La IA redefine la gestión del tiempo, no la abole

El autor sostiene que la IA redefine por completo la gestión de los tiempos del desarrollo de software. Conviene precisar el matiz crítico: no la elimina, la **desplaza**. Antes de la generalización de los asistentes de código, lo incierto era cuánto tardaría escribir el código; ahora, el código aparece en minutos (en ocasiones, en segundos), pero lo genuinamente incierto es cuánto tardará *verificar que ese código es correcto, seguro y adecuado para el contexto específico del problema*. La gestión de expectativas, por tanto, debe reeducarse a fondo: el cuello de botella migró del teclado al criterio.

Esta migración no es metafórica; es empíricamente medible. Peng et al. (2023, arXiv:2302.06590), en el estudio de GitHub con 95 desarrolladores profesionales, documentaron que la velocidad de generación de código nuevo aumenta un 55% con asistencia de IA. Lo que el mismo estudio señala, y que con frecuencia se omite en las presentaciones ejecutivas, es que **el tiempo de revisión y verificación del código generado por IA no se redujo proporcionalmente**: los desarrolladores que usaron Copilot reportaron dedicar más tiempo relativo a la fase de revisión crítica del código generado que

al código escrito manualmente, precisamente porque la asistencia aumentó el volumen de código producido por unidad de tiempo.

El espejismo de la velocidad: por qué la generación rápida no implica entrega rápida

Existe una expectativa peligrosa (y frecuentemente articulada en reuniones de dirección) que conviene desmontar con datos: "si la IA genera código instantáneamente, los proyectos serán instantáneos". La afirmación es falsa, y su falsedad no es trivial.

En un entorno complejo y cambiante (como el que el autor describe en sus reflexiones sobre programación en contextos de alta volatilidad organizacional) la velocidad de generación no determina la velocidad de entrega por tres razones estructurales:

Primera razón: la complejidad del contexto no se genera. Un agente de código puede generar una función en segundos. No puede, sin intervención humana, comprender las restricciones implícitas del sistema de pagos legacy al que esa función se integrará, las políticas de compliance que la afectan, las peculiaridades del equipo que la mantendrá o el historial de decisiones arquitectónicas que hacen inviables ciertas implementaciones aparentemente correctas.

Segunda razón: la deuda también se genera rápido. Vaithilingam et al. (2022), en un estudio experimental con participantes usando GitHub Copilot publicado en CHI 2022, encontraron que los participantes con asistencia de IA completaron más tareas pero introdujeron más bugs en el código generado que los participantes sin asistencia (un efecto que los autores atribuyen a la menor fricción de generación, que inhibe la verificación espontánea). La deuda técnica se acumula a la velocidad de la generación si no hay contramedidas explícitas.

Tercera razón: la integración es el cuello de botella real. Generar código es solo una parte del proceso de entrega. La integración con el sistema existente, las pruebas de regresión, la revisión de código, el proceso de despliegue y la validación de comportamiento en producción siguen siendo operaciones con tiempos mínimos que la velocidad de generación no comprime.

Evidencia empírica: las nuevas curvas de tiempo

La siguiente tabla sintetiza los datos disponibles sobre el impacto de la IA en las distintas fases del ciclo de desarrollo:

Fase	Antes de la IA	Con asistentes de IA (2024)	Factor de cambio	Fuente
Generación de código nuevo	Días a semanas	Horas a días	4–10x más rápido	Peng et al. 2023
Revisión y verificación	Proporcional al código	**Domina el cronograma**	Sin cambio significativo	Vaithilingam et al. 2022
Integración con sistema	Moderada	Crítica (sin contexto implícito)	Variable (a veces más lenta)	McKinsey 2023
Detección de regresiones	Depende del test suite	Alta si hay evals; baja si no	Bimodal	Liu et al. 2024
Mantenimiento post-despliegue	Alto	**Alto** (la deuda se acumula rápido)	Sin mejora sin inversión en evals	Jimenez et al. 2024

La lección que emerge de esta tabla es nítida: la IA aplana la curva de generación de forma dramática, pero **no aplana las curvas de verificación, integración y mantenimiento**. Quien planifica ignorando estos tres factores promete plazos que no podrá cumplir, con la diferencia de que ahora el incumplimiento llega antes y con mayor volumen de código comprometido.

Modelo de planificación para proyectos con asistencia de IA

La adaptación de los métodos de planificación ágil al contexto de IA agéntica requiere revisar tres convenciones establecidas:

1. Los story points ya no miden lo mismo. En la estimación ágil clásica, un story point captura la complejidad *de implementación*. Con

IA, la implementación es más rápida; lo que demora es la verificación. Los equipos que adoptan IA sin recalibrar sus story points experimentan lo que podría denominarse la "inflación de velocidad": la velocity aumenta sin que la entrega de valor lo haga en la misma proporción.

Una recalibración práctica consiste en descomponer el story point en dos componentes: *complejidad de generación* (qué tan difícil es describir correctamente el problema al agente) y *complejidad de verificación* (qué tan difícil es comprobar que la solución generada es correcta). En sistemas con alta complejidad de verificación (dominio de seguridad, compliance, lógica de negocio crítica), la IA reduce el primero pero no el segundo.

2. El Definition of Done debe incluir criterios de verificación de salida generada. Un incremento producido con asistencia de IA no está "hecho" hasta que ha pasado por un conjunto de verificaciones explícitas que van más allá de "compila y pasan los tests existentes". El DoD agéntico incluye: pruebas de comportamiento en los casos límite identificados, revisión de seguridad del código generado, validación de consistencia con el contexto del sistema y evaluación de la deuda técnica introducida.

3. La retrospectiva debe rastrear la proporción entre generación y verificación. El indicador más útil para calibrar la adopción de IA en un equipo ágil es el tiempo dedicado a verificar por cada hora de generación asistida. Un ratio mayor de 3:1 sugiere que la especificación de las tareas al agente es deficiente; un ratio menor de 0.5:1 sugiere que la verificación es insuficiente y la deuda se acumula silenciosamente.

El pipeline CI/CD para código generado por IA: qué añadir

► PISTA TÉCNICA

Los pipelines de integración y despliegue continuo convencionales deben extenderse cuando el código es generado parcial o totalmente por asistentes de IA. Las extensiones mínimas necesarias son:

- **Análisis estático reforzado**: las herramientas estáticas deben configurarse con reglas más estrictas para código generado por IA, donde los patrones de copy-paste lógico y las dependencias innecesarias son más frecuentes.
- **Pruebas de mutación**: verifican que las pruebas existentes son capaces de detectar bugs en el código generado, no solo en el código escrito manualmente.
- **Rastreo de autoría**: marcar qué porcentaje del código en cada PR fue generado por IA permite calibrar el nivel de revisión necesario y construir datos históricos sobre la tasa de bugs por origen del código.
- **Eval automatizado de comportamiento**: para funciones críticas, un agente evaluador (cf. cap. 32, diseño de evals) verifica que el comportamiento generado es consistente con las especificaciones de negocio, no solo con los tests unitarios.

ROI cuantificado: cuándo y cuánto ahorra la IA

■ **PISTA EJECUTIVA**

La pregunta ejecutiva más frecuente (y la más mal respondida) es: "¿cuánto ROI obtenemos de la IA en desarrollo de software?". La respuesta correcta depende de dos variables que los análisis simplistas ignoran: la naturaleza de las tareas (alta vs. baja verificabilidad) y la madurez del equipo en el uso de herramientas de IA.

El análisis de McKinsey Global Institute (2023) estima que la IA generativa puede automatizar el equivalente al 60-70% del tiempo dedicado a tareas de documentación, generación de tests y código boilerplate. Para tareas de diseño arquitectónico, resolución de bugs complejos en sistemas legacy y toma de decisiones de producto, el porcentaje de automatización cae al 10-20%.

El modelo de ROI más honesto es, por tanto, **bimodal**:

Tipo de tarea	Porcentaje automatizable	ROI típico (6 meses)	Condición necesaria
Boilerplate, docs, tests unitarios	60–70%	3–5x	Equipos con buenas specs

Tipo de tarea	Porcentaje automatizable	ROI típico (6 meses)	Condición necesaria
Diseño, debugging complejo, decisiones	10–20%	1.2–1.5x	Solo con seniors con criterio
Integración y verificación	5–15%	0.8–1.1x	Requiere inversión previa en evals

El tercer tipo (integración y verificación) puede tener ROI menor a 1 en el corto plazo si no se invierte en evaluar el código generado. Este es el riesgo no articulado de las implementaciones apresuradas.

Gestionar al stakeholder: la pedagogía como competencia ejecutiva

■ **PISTA EJECUTIVA**

En consecuencia, la gestión de expectativas se convierte, ante todo, en un ejercicio de pedagogía ejecutiva. El líder debe traducir la complejidad de la asimetría generación/verificación a un lenguaje que el stakeholder no técnico pueda entender y actuar sobre ella.

Tres mensajes resultan imprescindibles y no negociables: primero, **la generación es barata, pero la corrección es cara** (el costo del error en código generado por IA y no verificado es comparable al del código escrito bajo presión de tiempo, y en algunos casos superior); segundo, **la velocidad sin verificación es deuda disfrazada de progreso** (el incremento que se entrega en días pero introduce bugs en producción no tiene ROI positivo); tercero, **el valor no está en producir más código, sino en producir el código correcto** (la métrica de líneas de código generadas por sprint es un indicador de proceso, no de valor de negocio).

Quien comunica estos tres mensajes con claridad y sostenibilidad no solo gobierna las expectativas: convierte la adopción de IA en una ventaja competitiva real en lugar de en una presión de marketing.

Versión alternativa (mayor densidad académica):

La gestión de las expectativas temporales en proyectos de desarrollo de software con asistencia de inteligencia artificial generativa

constituye un problema de comunicación estratégica que involucra la traducción de asimetrías técnicas (específicamente, la divergencia entre la reducción de los tiempos de generación de código y la ausencia de reducción equivalente en los tiempos de verificación y validación) a marcos cognitivos accesibles para actores organizacionales con distintos niveles de alfabetización técnica. La evidencia empírica disponible (Peng et al., 2023; Vaithilingam et al., 2022) sugiere que la falta de esta traducción constituye el factor primario de la brecha entre las expectativas declaradas sobre la productividad de la IA y los resultados medidos en proyectos reales.

Lo esencial del capítulo 10

- La IA desplaza la incertidumbre de los plazos de la generación a la verificación; el cuello de botella migró del teclado al criterio.
- Peng et al. (2023): +55% en velocidad de generación de código, sin reducción proporcional en tiempo de verificación.
- Vaithilingam et al. (2022): más tareas completadas con IA, pero más bugs introducidos (la menor fricción de generación inhibe la verificación espontánea).
- Los story points deben descomponerse en complejidad de generación y complejidad de verificación; la IA reduce el primero, no el segundo.
- El DoD agéntico incluye casos límite, revisión de seguridad, consistencia con el contexto del sistema y evaluación de deuda.
- El CI/CD para código IA extiende el pipeline con análisis estático reforzado, mutación, rastreo de autoría y eval automático de comportamiento.
- El ROI es bimodal: 3–5x en boilerplate/docs/tests; 0.8–1.1x en integración y verificación sin inversión previa en evals.
- Gestionar stakeholders es pedagogía: la generación es barata, la corrección es cara; la velocidad sin verificación es deuda disfrazada.

Capítulo 11: Un llamado a reinventarse en la actualidad de los sistemas multiagentes

Las empresas optimizan lo que ya hacen; los pioneros reinventan lo que harán.

Optimizar no es reinventar: la distinción que lo cambia todo

El autor lanza una advertencia que merece eco amplificado: la mayoría de las empresas está enfocada en *optimizar* lo existente, cuando la era de los sistemas multiagente exige *reinventar*. La distinción no es semántica, y conviene precisarla antes de seguir, porque de ella depende si la inversión en IA produce ventaja competitiva durable o simplemente reduce costos operacionales transitorios.

Optimizar es mejorar un proceso dentro de su paradigma: hacer más rápido, más barato o con menos errores lo que ya se hace. Reinventar es cuestionar el paradigma mismo: preguntarse si lo que se hace debería hacerse de esa manera, o si la disponibilidad de nueva capacidad hace posible (y necesario) una forma radicalmente diferente de producir el mismo resultado, o un resultado que antes era imposible. Aplicar un enjambre de agentes para procesar el 20% más de facturas por hora es optimización. Rediseñar el ciclo financiero

completo en torno a agentes que cierran el mes en tiempo real, eliminando el concepto mismo de "cierre mensual", es reinvención.

Por qué los multiagentes fuerzan la reinvención: la física del cambio de fase

Los sistemas multiagente no son una herramienta más que se inserta en el proceso vigente; son una **capacidad que altera la física del problema**. Cuando un enjambre puede atender en paralelo lo que antes requería un departamento completo y días de trabajo, el organigrama que estructuraba ese departamento deja de tener sentido en su forma original. Cuando la inteligencia (la capacidad de razonar, sintetizar, decidir sobre información no estructurada) se vuelve abundante, barata y distribuible, los cuellos de botella se desplazan a lugares inesperados: de la ejecución al criterio, de la generación a la validación, de la capacidad cognitiva a la confianza y la responsabilidad.

En física de materiales, un cambio de fase no es un cambio cuantitativo del mismo estado; es la transición a un estado con propiedades cualitativamente distintas. El agua no se vuelve "más fría" al congelarse: cambia de estado. Los sistemas multiagente representan para la organización exactamente ese tipo de transición: no más velocidad en el estado actual, sino un estado organizacional con propiedades cualitativamente distintas. Insertar multiagentes en estructuras diseñadas para la escasez de inteligencia es como instalar un motor de reacción en una carreta (la potencia existe, pero el chasis la desperdicia y frecuentemente la convierte en riesgo).

Los Tres Horizontes de McKinsey en clave agéntica

El marco de los Tres Horizontes, desarrollado en *The Alchemy of Growth* (Baghai, Coley & White, 1999) y refinado por McKinsey a lo largo de dos décadas, propone que la gestión del crecimiento organizacional requiere atender simultáneamente tres horizon-

tes temporales con inversiones y lógicas distintas. Aplicado a la transformación agéntica:

Horizonte	Temporal	En transformación agéntica	Inversión típica	Riesgo si se ignora
H1	0–12 meses	Automatización de tareas repetitivas con agentes: ahorro de costos visible, ROI rápido	60–70% del presupuesto IA	Pérdida de eficiencia operacional frente a competidores
H2	1–3 años	Rediseño de procesos en torno a capacidades agénticas: nuevos flujos, nuevos roles	20–30% del presupuesto IA	Pérdida de posición competitiva en 18–24 meses
H3	3–7 años	Reinvención del modelo de negocio: qué nuevos mercados, productos o relaciones hace posibles la IA agéntica	10–20% del presupuesto IA	Irrelevancia estructural a mediano plazo

El error más frecuente en las organizaciones que inician la transformación agéntica es concentrar el 90% del presupuesto en H1 (visible, medible, de bajo riesgo) y no asignar nada a H3. El resultado, en el horizonte de tres a cinco años, es una organización que ejecuta más eficientemente un modelo de negocio que sus competidores están reinventando.

Mapa de reinvención: inventario de supuestos cognitivos

Reinventarse exige, como primer paso no negociable, identificar qué supuestos del negocio descansaban en la escasez de cognición. Este ejercicio (que podría denominarse "auditoría de supuestos cognitivos") consiste en preguntarse, para cada proceso relevante de la organización: **¿este proceso existe de esta forma porque es la mejor forma de producir ese resultado, o porque era la única forma factible cuando la inteligencia era un recurso escaso y costoso?**

Las respuestas revelan sistemáticamente tres categorías de procesos:

Procesos que siguen siendo óptimos independientemente de la disponibilidad de inteligencia artificial: aquellos donde el valor proviene de la relacionalidad humana, del juicio ético, de la creatividad genuina o de la responsabilidad legal que no puede delegarse. Estos procesos deben protegerse de la sustitución irreflexiva.

Procesos que deben rediseñarse para incorporar capacidades agénticas sin cambiar el resultado final: su estructura actual es un artefacto de la escasez cognitiva pasada, y su rediseño produce el mismo resultado con menor costo, mayor velocidad o mayor calidad.

Procesos que deben eliminarse porque su existencia misma es un artefacto de limitaciones que ya no existen: el informe semanal que nadie lee pero que se produce porque "siempre se hizo"; la reunión de alineamiento que existe porque la información no fluía en tiempo real; el comité de aprobación de bajo riesgo que existe porque la evaluación manual era el único mecanismo de control disponible.

Casos empíricos de reinvención agéntica: salud, finanzas y derecho

La evidencia de reinvención agéntica en sectores regulados es ya suficientemente robusta para extraer patrones:

Salud: el sistema *LLaVA-Med* (Microsoft Research, 2023) y los sistemas multiagente de diagnóstico por imagen no automatizan el diagnóstico del radiólogo; reinventan el flujo: el agente procesa el

80% de las imágenes rutinarias y presenta al especialista solo los casos que requieren criterio experto. El radiólogo deja de ser procesador de imágenes y se convierte en revisor de excepciones. El resultado no es "más rápido lo mismo", es un rol cualitativamente diferente.

Finanzas: JP Morgan Chase reportó (2024) que su sistema de asistencia de contratos legales, basado en LLM multiagente, redujo de 360,000 horas/año a horas el tiempo de revisión de contratos de préstamos comerciales. El proceso no fue optimizado: fue reinventado. Los abogados dejaron de revisar contratos estándar y pasaron a gestionar excepciones y relaciones de alto valor.

Derecho: el sistema *Harvey* (utilizado por Allen & Overy desde 2023) permite que socios junior produzcan documentos de calidad senior en tiempos de horas, no semanas. La pirámide salarial de los despachos legales (que existía precisamente porque el trabajo de nivel junior era necesario para sostener el de nivel senior) está siendo reinventada, no optimizada.

El coraje de canibalizar lo propio

Ahora bien, reinventarse duele, porque a menudo exige canibalizar activamente lo que hoy funciona y produce ingresos. No es solo retórica: la empresa que anuncia "vamos a reinventar nuestra operación con agentes" está anunciando, implícitamente, que algunas de sus líneas de negocio actuales o algunos de sus procesos rentables desaparecerán o se reducirán. La resistencia interna a esa decisión es proporcional al éxito presente de lo que se va a canibalizar.

La investigación de Christensen (1997) sobre el dilema del innovador (publicada originalmente en *Harvard Business Review* y ampliada en su obra homónima) documentó con precisión este mecanismo: las empresas que mejor ejecutan sus modelos de negocio actuales son las más vulnerables a las disrupciones, porque su estructura organizacional, sus métricas de éxito y sus incentivos están alineados con la optimización de lo existente, no con la exploración de lo nuevo.

En la era agéntica, este dilema se presenta con una velocidad inédita en este sector: el ciclo de obsolescencia de un proceso no adap-

tado puede ser de 18 a 36 meses, no de una década. La historia es inflexible en su veredicto: quienes optimizaron el caballo perdieron ante quienes reinventaron el transporte.

Hoja de ruta ejecutiva para la transformación agéntica

■ **PISTA EJECUTIVA**

Una hoja de ruta de reinvención agéntica que sea implementable (no solo aspiracional) necesita cuatro componentes que la mayoría de las roadmaps omite:

1. El inventario de supuestos cognitivos (trimestre 1): quién lo hace, qué herramientas usa, qué criterio de categorización aplica, qué outputs produce.

2. El piloto de reinvención (trimestres 1-2): un proceso del grupo "rediseñar" ejecutado en producción real con agentes, con métricas de resultado (no de proceso) definidas antes de empezar. Sin un piloto real, la reinvención permanece en el dominio de lo declarativo.

3. El modelo de gobernanza agéntica (paralelo desde trimestre 1): quién decide qué autonomía conceder a qué agente, bajo qué condiciones, con qué mecanismo de revisión. Sin gobernanza, la reinvención produce caos; con gobernanza excesiva, produce lentitud.

4. El programa de capacitación de criterio (continuo): los equipos que trabajan con agentes necesitan desarrollar la competencia más difícil y más ignorada en los programas de transformación de IA: **juzgar la calidad de la salida agéntica**. No es saber usar la herramienta; es saber cuándo la salida es buena y cuándo no. Esta competencia no se adquiere en un curso de dos días.

Versión alternativa (mayor densidad académica):

La transformación organizacional inducida por la disponibilidad de sistemas de inteligencia artificial multiagente constituye un fenómeno que excede el ámbito de la innovación tecnológica incremental para adentrarse en el territorio de lo que Christensen (1997) denominó *innovación disruptiva*: una modalidad de cambio que no mejora el desempeño en las métricas que los clientes actuales valoran, sino que introduce un conjunto diferente de propiedades (mayor disponibilidad, menor costo, mayor distribución) que redefinen qué atributos

son valorados. La respuesta adaptativa óptima a este tipo de disrupción es la construcción en paralelo de un modelo alternativo capaz de capturar el nuevo valor, aun a costa de canibalizar parcialmente el primero, no la optimización del modelo de negocio existente.

Lo esencial del capítulo 11

- Optimizar es mejorar dentro del paradigma; reinventar es cuestionar el paradigma. Solo lo segundo produce ventaja competitiva durable en la era agéntica.
- Los multiagentes representan un cambio de fase para la organización: no más velocidad en el estado actual, sino un estado con propiedades cualitativamente distintas.
- El marco de Tres Horizontes aplicado: H1 (automatización, 0–12 meses, 60-70% del presupuesto), H2 (rediseño, 1–3 años, 20-30%) y H3 (reinvención del modelo de negocio, 3–7 años, 10-20%). Ignorar H3 produce irrelevancia estructural.
- La auditoría de supuestos cognitivos clasifica procesos en tres grupos: óptimos (proteger), rediseñables (transformar) y eliminables (discontinuar).
- JP Morgan Chase, Allen & Overy y sistemas médicos multiagente demuestran que la reinvención ya está ocurriendo en sectores regulados a escala real.
- El dilema de Christensen (1997): los mejores ejecutores del modelo actual son los más vulnerables a la disrupción; en la era agéntica, el ciclo de obsolescencia es de 18–36 meses.
- La hoja de ruta ejecutiva necesita cuatro componentes no negociables: inventario de supuestos, piloto en producción real, gobernanza agéntica y programa de capacitación de criterio.

Capítulo 12: La lucha entre agentes y multiagentes de IA: porque ser el mejor modelo no basta para vencer

Como el T-1000, ser el modelo más avanzado no garantiza la victoria si tu entorno te derrota.

La metáfora del T-1000 y su precisión técnica

El autor recurre a una imagen memorable que merece ser tomada en su dimensión técnica: la inteligencia artificial del T-1000, siendo tecnológicamente superior en capacidades individuales, es vencida no por un sistema más inteligente sino por uno con mejor arquitectura sistémica. La lección trasciende la ficción: en el mundo de los agentes de IA, **el mejor modelo no gana automáticamente**.

Un modelo de razonamiento sobresaliente (digamos, en el percentil 99.9 de todos los benchmarks disponibles), cuando opera mal orquestado, mal alimentado de contexto, sin herramientas adecuadas y sin contención de riesgos, pierde frente a un modelo de capacidades modestas inserto en una arquitectura bien diseñada. Esta afirmación no es intuitiva, pero es empíricamente robusta y técnicamente fundamentada. El benchmark mide al modelo aislado en condiciones de laboratorio; la producción mide al sistema completo bajo condiciones reales de carga, ruido, adversarialidad e integración.

La función de calidad sistémica: la matemática del argumento

La tesis central de este capítulo puede formalizarse matemáticamente, lo cual le da una solidez que la metáfora sola no proporciona. La calidad efectiva de un sistema agéntico es su **mínimo limitante**, no la suma de las calidades de sus componentes:

```
Q_sistema = min(Q_modelo, Q_arquitectura, Q_contexto, Q_herramientas,
Q_contención)
```

Donde cada Q representa la calidad normalizada en la escala [0, 1] del componente correspondiente. Si cualquiera de estos cinco factores es 0 (el contexto es nulo, las herramientas no están disponibles, la contención de riesgos es inexistente), la calidad del sistema es 0 independientemente de cuán alto sea Q_modelo.

Esta función min() tiene implicaciones contraintuitivas pero importantes. Con Q_modelo = 0.99 (modelo de última generación) y Q_arquitectura = 0.3 (arquitectura deficiente), el sistema produce Q_sistema = 0.3. Si se reemplaza el modelo por uno de Q_modelo = 0.85 pero se mejora la arquitectura a Q_arquitectura = 0.85, se obtiene Q_sistema = 0.85 (una mejora del 183% en calidad sistémica con un modelo inferior). Este resultado explica por qué invertir en arquitectura tiene un retorno muy superior al de perseguir el último modelo cuando la arquitectura está por debajo de 0.8.

La validación empírica de esta función proviene de los datos de *AgentBench* (Liu et al., 2024): el mismo modelo (GPT-4) produce resultados que varían entre 40% y 82% de tasa de éxito dependiendo únicamente del entorno de orquestación y las herramientas disponibles (una variación del 105% con el modelo constante).

Las cinco dimensiones que deciden la lucha sistémica

¿Qué determina, entonces, cuál agente o enjambre prevalece en condiciones reales? No la potencia bruta del modelo, sino una constelación de factores sistémicos cada uno de los cuales puede ser el cuello de botella:

1. Contexto. El mejor modelo sin el contexto adecuado razona perfectamente sobre la pregunta equivocada. Un modelo que recibe información incorrecta, incompleta o mal organizada produce respuestas que son internamente coherentes y externamente incorrectas (el peor tipo de error, porque pasa los filtros de plausibilidad). La calidad del contexto depende del diseño del sistema de recuperación (RAG), la gestión de la ventana de contexto y la selección de qué información es relevante para qué agente.

2. Orquestación. Un modelo de capacidades medias bien coordinado con otros especializados supera a un genio solitario sin estructura. Hong et al. (2024), en *MetaGPT* (ICLR 2024), demostraron que la asignación explícita de roles profesionales a agentes (analista de negocios, arquitecto, desarrollador, QA) en un sistema multi-agente aumenta la tasa de completación de software funcional en un 184% respecto de un sistema sin roles explícitos, usando el mismo modelo base.

3. Herramientas. El acceso a las herramientas correctas vía MCP o protocolos equivalentes multiplica la capacidad efectiva del sistema más que cualquier salto de versión del modelo. Un agente con acceso a búsqueda web, ejecución de código y base de datos estructurada produce resultados que ningún modelo de lenguaje puro puede replicar, independientemente de su tamaño.

4. Velocidad y costo. Un agente que responde dentro del SLA con un costo por transacción viable derrota en el mercado a uno más preciso pero 10 veces más caro o 10 veces más lento. Este factor es particularmente relevante en la competencia entre modelos: Claude Haiku 3.5 es aproximadamente 6 veces más barato que Claude Sonnet 3.5 por token; en tareas donde su calidad es suficiente, la diferencia de costo permite escalar 6 veces más con el mismo presupuesto.

5. Robustez ante el fallo. El agente que degrada con gracia ante condiciones adversas (latencia elevada de herramientas, contexto contaminado, instrucciones ambiguas) sobrevive al que colapsa o produce salidas silenciosamente incorrectas. El patrón de *graceful degradation* aplicado a agentes exige diseñar explícitamente el comportamiento de fallback para cada modo de fallo previsto.

Evidencia de benchmarks: por qué los rankings de modelos engañan

La industria de evaluación de modelos de lenguaje ha producido benchmarks de alta calidad (MMLU, HumanEval, MATH, GSM8K) que miden con precisión las capacidades del modelo aislado. El problema es la interpretación que se hace de los resultados de los benchmarks, no su calidad.

Zheng et al. (2023), en el trabajo que produjo el *Chatbot Arena* (NeurIPS 2023), demostraron que el ranking de modelos según preferencia humana en conversación real diverge significativamente del ranking en benchmarks estáticos: modelos que puntúan más alto en MMLU no necesariamente son preferidos por usuarios reales en tareas prácticas. La brecha se explica por factores que los benchmarks estáticos no capturan: naturalidad de la respuesta, manejo de instrucciones ambiguas, comportamiento ante preguntas fuera de distribución.

En el dominio agéntico, la divergencia es aún mayor. Jimenez et al. (2024), en *SWE-bench* (ICLR 2024), evaluaron la capacidad de modelos para resolver issues reales de GitHub. Los resultados fueron reveladores: el modelo con mejor puntuación en HumanEval (benchmark de generación de código) no fue el que más issues resolvió en SWE-bench. El éxito en tareas agénticas reales depende de la capacidad de planificar múltiples pasos, recuperarse de errores intermedios y mantener coherencia con el estado del sistema, capacidades que los benchmarks estáticos de generación de código no miden.

Estrategias de enrutamiento de modelos: la inteligencia de la selección

► **PISTA TÉCNICA**

Una de las consecuencias prácticas más importantes de la función de calidad sistémica es el **enrutamiento dinámico de modelos**: la estrategia de usar distintos modelos para distintos tipos de tareas dentro del mismo sistema, seleccionando en tiempo de ejecución el

modelo óptimo según la naturaleza de la subtarea, su nivel de riesgo y su volumen esperado.

Una taxonomía de enrutamiento práctica distingue tres categorías de tareas:

Categoría	Criterio de clasificación	Modelo recomendado	Razón
Alta complejidad, bajo volumen	Razonamiento multistep, decisiones críticas	Modelo de mayor capacidad (Claude Opus, GPT-4o)	Calidad > costo en tareas raras
Complejidad media, volumen medio	Generación de contenido, análisis estándar	Modelo equilibrado (Claude Sonnet, GPT-4o-mini)	Balance calidad/costo óptimo
Baja complejidad, alto volumen	Clasificación, extracción, triage	Modelo pequeño y rápido (Claude Haiku, GPT-4o-mini)	Costo < calidad en tareas masivas

El enrutador en sí puede ser un modelo pequeño entrenado o afinado para clasificar la complejidad de la tarea en milisegundos, añadiendo latencia despreciable. El ahorro de costo con esta estrategia puede superar el 60% en sistemas con alta variabilidad de complejidad de tareas, manteniendo la calidad en el percentil 90+ para las tareas que lo requieren.

El sistema vence al modelo: implicaciones para el arquitecto

Figura 6: Modelo aislado vs Arquitectura: por qué la arquitectura domina

He aquí la tesis central sintetizada: en la competencia agéntica, **la arquitectura es el verdadero campo de batalla, no el modelo**. Las empresas que invierten obsesivamente en tener el "mejor modelo" libran la guerra equivocada por tres razones estructurales.

Primero, la **convergencia de modelos**: Claude, GPT, Gemini y sus competidores convergen anualmente en los benchmarks estándar a una velocidad de aproximadamente 5 puntos porcentuales por año en las métricas más discriminativas. La ventaja del mejor modelo de hoy es la paridad del modelo mediano de mañana.

Segundo, el **coste de la dependencia de modelo**: un sistema diseñado en torno a un modelo específico tiene que ser rediseñado cada vez que ese modelo cambia (y los modelos cambian cada 6-12 meses). Una arquitectura agnóstica al modelo incorpora cualquier mejora sin rediseño.

Tercero, el **efecto de suelo**: en sistemas con Q_arquitectura < 0.6, ningún modelo, por bueno que sea, puede llevar Q_sistema por encima de 0.6. La inversión más rentable es siempre elevar el componente mínimo de la función.

Build vs. buy en la era del modelo-commodity

■ **PISTA EJECUTIVA**

La convergencia de los modelos hacia un commodity tiene implicaciones estratégicas directas para las decisiones de build vs. buy. Cuando el modelo base es un commodity accesible vía API, **la ventaja competitiva no reside en el modelo sino en la arquitectura que lo orquesta, los datos propios que lo alimentan y los procesos de verificación que aseguran su calidad**.

Esta reconfiguración cambia el cálculo de make-or-buy: tiene sentido comprar el modelo base (no hay ventaja competitiva en construir modelos de propósito general desde cero para la mayoría de organizaciones) y construir la arquitectura de orquestación, el sistema de recuperación de contexto y los evaluadores de calidad propietarios (ahí es donde reside la ventaja durable).

La excepción son los modelos especializados en dominio: en salud, derecho, finanzas y otros sectores con datos propietarios y requisitos de privacidad, el fine-tuning de modelos base con datos propios sí puede producir ventaja competitiva durable. Pero incluso ahí, la arquitectura de orquestación amplifica o limita esa ventaja.

Lo esencial del capítulo 12

- La calidad sistémica es el mínimo de las calidades del modelo, la arquitectura, el contexto, las herramientas y la contención; cualquier componente en 0 hace el sistema inútil independientemente del modelo.
- AgentBench (Liu et al., 2024): el mismo modelo (GPT-4) varía un 105% en tasa de éxito dependiendo solo del entorno de orquestación.
- MetaGPT (Hong et al., 2024): la asignación explícita de roles profesionales aumenta la tasa de completación en 184% con el mismo modelo base.
- SWE-bench (Jimenez et al., 2024): el mejor modelo en HumanEval no es el que más issues reales de GitHub resuelve; los benchmarks estáticos engañan.
- El enrutamiento dinámico de modelos puede reducir costos >60% manteniendo calidad en percentil 90+ asignando modelo correcto a complejidad correcta.
- Los modelos convergen ~5 puntos porcentuales/año en benchmarks discriminativos; la ventaja del mejor modelo de hoy es la paridad del mediano de mañana.
- Build vs. buy: comprar modelo base (commodity), construir arquitectura de orquestación, contexto y evaluadores (ventaja competitiva durable).

Capítulo 13: El síntoma del programador cansado, aplicado a los agentes y multiagentes de IA

El cansancio no apaga la capacidad de teclear; apaga la capacidad de dudar. Y eso, en una máquina, tiene un equivalente exacto.

El síntoma humano: cuando la ejecución sobrevive al juicio

El autor describe con exactitud clínica un fenómeno que todo desarrollador con experiencia reconoce en sí mismo: el síntoma del programador cansado. Es la **erosión progresiva del juicio crítico** bajo la acción acumulada de la fatiga cognitiva, no la incapacidad de seguir escribiendo código (eso continúa). El programador exhausto sigue produciendo: el compilador sigue aceptando su código, los tests superficiales siguen pasando, la pull request sigue siendo enviada. Pero dejó de cuestionar la solución que eligió, dejó de considerar el caso límite que habría visto por la mañana, dejó de verificar la suposición que su cerebro descansado habría interrogado.

La fatiga no detiene la ejecución; degrada la deliberación. Esta distinción (entre la capacidad de ejecutar y la capacidad de juzgar) es precisamente la que convierte al síntoma del programador cansado en algo peligroso: su presencia no es observable desde afuera

porque el código sigue fluyendo, la conversación sigue siendo fluida, el resultado superficial es indistinguible del resultado descansado. El daño es latente, no inmediato.

La neurociencia del juicio degradado: por qué la fatiga ataca primero lo más valioso

La literatura de neurociencia cognitiva provee un mecanismo preciso para este fenómeno. La corteza prefrontal (responsable del control ejecutivo, el razonamiento deliberado, la inhibición de respuestas automáticas y la evaluación de alternativas) es el componente del sistema cognitivo que se agota más rápidamente con la fatiga y se recupera más lentamente. Las funciones automáticas (la generación de respuestas conocidas, la ejecución de patrones aprendidos, la producción de lenguaje fluido) son sustancialmente más robustas a la fatiga.

En términos prácticos: lo que se agota primero con la fatiga es exactamente lo que más se necesita en la ingeniería de software compleja. La capacidad de generar código familiar se preserva; la capacidad de detectar el caso límite no evidente, de cuestionar el diseño arquitectónico que "funciona pero está mal", de anticipar la consecuencia de seguridad de una decisión de implementación, esas son las primeras víctimas de la fatiga cognitiva.

El equivalente agéntico: la degradación contextual como fatiga sistémica

Aquí emerge la analogía que el capítulo desarrolla con precisión técnica. Los agentes de IA padecen su propia versión del síntoma del programador cansado: la **degradación contextual progresiva** en tareas largas o conversaciones extendidas.

El mecanismo es estructuralmente análogo al humano aunque técnicamente diferente. A medida que una tarea se alarga y la ventana de contexto se llena, los tokens más recientes (las instrucciones e información del final de la conversación) tienen mayor peso relativo en el proceso de atención que los tokens más tempranos (donde con

frecuencia se establecieron los objetivos, las restricciones y el contexto crítico del problema). El agente, como el programador fatigado, sigue ejecutando fluidamente: continúa generando texto coherente, continúa llamando herramientas, continúa produciendo salidas estructuradas. Pero el objetivo original se ha diluido; las restricciones tempranas han perdido peso; el agente está razonando, con precisión creciente, sobre una versión erosionada del problema que le fue dado.

Liu et al. (2023), en el trabajo *Lost in the Middle: How Language Models Use Long Contexts* (TACL, 2023), demostraron que los modelos de lenguaje exhiben un sesgo sistemático hacia la información ubicada al principio y al final del contexto, con degradación significativa en la recuperación y uso de información ubicada en la mitad del contexto. En conversaciones largas, donde las instrucciones críticas fueron dadas "en la mitad", este efecto puede producir agentes que ignoran restricciones que fueron explícitamente establecidas.

Síntomas diagnósticos del síntoma del programador cansado en el enjambre

El reconocimiento temprano de la degradación contextual en un agente o enjambre requiere conocer sus síntomas específicos. Cuatro son los más frecuentes y los más reveladores:

Deriva del objetivo (*goal drift*): el agente responde con coherencia a una versión simplificada, generalizada o parcial del objetivo original. No falla abiertamente; simplifica sin declararlo. El síntoma se manifiesta como respuestas correctas a la pregunta incorrecta (o, más precisamente, a una versión erosionada de la pregunta correcta).

Auto-confirmación (*self-confirmation bias*): el agente deja de cuestionar sus propias hipótesis intermedias. Una suposición adoptada en un paso temprano de la cadena de razonamiento se propaga como verdad en todos los pasos posteriores, sin ser reevaluada incluso cuando aparecen evidencias que la contradicen. Es el análogo directo del programador cansado que no cuestiona su primera solución.

Repetición improductiva (*unproductive repetition*): el agente reitera enfoques que ya fallaron, en algunos casos con variaciones superficiales que no abordan la causa raíz del fallo. Este síntoma es característico de la degradación contextual porque los fracasos anteriores (documentados en la parte "media" del contexto) pierden peso relativo frente a la instrucción más reciente de "inténtalo de nuevo".

Pérdida de los bordes (*edge case blindness*): el agente omite progresivamente los casos límite que su especificación original exigía manejar. Este es, quizá, el síntoma más peligroso: produce código que funciona en el caso nominal pero falla en producción en las condiciones que el agente, con el contexto fresco, habría identificado y manejado.

Patrones técnicos de gestión del contexto: sliding window y compresión

El equivalente agéntico del descanso no es inactividad; es **diseño deliberado del contexto**. Tres patrones técnicos demostrados reducen significativamente la degradación contextual:

Sliding window con resumen progresivo: en lugar de mantener el historial completo de la conversación en la ventana de contexto, un agente especializado sintetiza periódicamente los intercambios más antiguos en un resumen comprimido que preserva la información crítica (objetivos, restricciones, decisiones tomadas) y descarta el detalle de bajo valor. La ventana activa contiene el resumen + los últimos N intercambios, manteniendo el contexto manejable sin perder la memoria operativa.

Anclaje periódico de objetivos (*periodic goal re-anchoring*): se inyecta la especificación completa de la tarea (objetivos, restricciones, criterios de éxito) al inicio de cada bloque de N pasos, independientemente de lo que haya ocurrido en el bloque anterior. Este patrón es análogo al programador que imprime la especificación y la tiene sobre la mesa: no como señal de desconfianza, sino como contramedida consciente contra la erosión de objetivos.

Agente vigilante (*watchdog agent*): un agente auxiliar de bajo costo monitorea en paralelo el progreso del agente principal y detecta

signos de deriva de objetivo mediante comparación entre la especificación original y la dirección de los pasos recientes. Cuando detecta divergencia superior a un umbral, interviene con una señal de realineamiento antes de que la deriva se vuelva costosa de corregir.

Implementación del watchdog de contexto: una guía técnica

El watchdog de contexto puede implementarse con un modelo pequeño y eficiente (Claude Haiku, GPT-4o-mini) configurado con el siguiente protocolo:

```
WATCHDOG_AGENT:
  input:
    - especificación_original: S₀ (objetivos + restricciones +
    criterio de éxito)
    - historial_reciente: [a₁, a₂, ..., aₙ] (últimas N acciones del
    agente principal)

  proceso:
    1. Extraer el objetivo implícito de las últimas N acciones
    (inferir qué problema está resolviendo)
    2. Comparar con S₀ usando similitud semántica o evaluación por LLM
    3. Si divergencia > umbral_d: emitir señal REALIGN con fragmento
    de S₀ más relevante
    4. Si indicador de auto-confirmación > umbral_c: emitir señal
    CHALLENGE con pregunta adversarial
    5. Si contador de repetición > umbral_r: emitir señal RESET con
    instrucción de cambio de enfoque

  output:
    - CONTINUE (sin intervención necesaria)
    - REALIGN (inyectar recordatorio de objetivo)
    - CHALLENGE (inyectar pregunta adversarial)
    - RESET (inyectar instrucción de cambio de enfoque)
```

El costo operacional del watchdog es marginal: opera con modelo pequeño, en paralelo con el agente principal, y su intervención es rara cuando el agente principal opera correctamente. En sistemas donde el síntoma del programador cansado es un riesgo alto (tareas largas, alta complejidad, consecuencias de error elevadas), el ROI del watchdog es sustancial.

La refutación externa como contramedida universal

Independientemente del patrón técnico específico que se adopte, el principio subyacente es invariante: **la contramedida universal contra el síntoma del programador cansado (humano o agéntico) es la introducción de una perspectiva epistémicamente independiente**. En el humano, es el colega que revisa el código con ojos descansados, sin el peso del contexto acumulado. En el enjambre, es el agente evaluador que no comparte el historial de contexto del agente principal y por eso puede cuestionar sus supuestos sin sesgo de confirmación acumulado.

Esta es la razón por la que el patrón de debate entre agentes (cf. cap. 31, Reflexion) produce calidad superior al agente solitario: no porque los agentes sean más inteligentes individualmente, sino porque la diversidad epistémica entre ellos rompe los ciclos de autoconfirmación que la fatiga contextual produce inevitablemente.

Perspectiva ejecutiva: la carga cognitiva del equipo como variable de diseño

■ **PISTA EJECUTIVA**

La conexión entre el síntoma del programador cansado y la gestión de equipos tiene una dimensión ejecutiva que frecuentemente se omite. Si los agentes de IA exhiben degradación contextual como consecuencia de la acumulación de contexto, los humanos que trabajan con esos agentes exhiben su propia versión: la **saturación cognitiva por gestión de sistemas complejos**. Gestionar un enjambre de múltiples agentes, cada uno con su historia de contexto, sus fallos y sus salidas parciales, impone una carga cognitiva que puede superar la de escribir el código directamente.

El diseño de interfaces de observabilidad para enjambres (cf. cap. 27) debe tener en cuenta la carga cognitiva del operador humano: qué información necesita ver, en qué formato, con qué frecuencia. Un enjambre que produce más datos de los que el operador puede procesar no es un sistema más poderoso; es un sistema más opaco.

La carga cognitiva del equipo es una variable de diseño del sistema agéntico, no una consecuencia operacional a gestionar después.

Versión alternativa (mayor densidad académica):

La degradación del desempeño en sistemas de agentes de lenguaje de gran escala bajo condiciones de extensión contextual prolongada presenta una estructura fenomenológica que exhibe una isomorfía notable con la degradación del juicio crítico documentada en la literatura de psicología cognitiva sobre fatiga ejecutiva. En ambos casos, lo que se preserva bajo condiciones de estrés es la capacidad de ejecutar operaciones automáticas y producir outputs estructurados superficialmente coherentes, mientras que lo que se degrada es la capacidad de monitorear la coherencia entre los outputs producidos y los objetivos establecidos al inicio del proceso, una función que, tanto en el sistema nervioso central como en los transformers de gran escala, requiere atención deliberada a información distal en el tiempo, precisamente la que es más vulnerable a los efectos de atenuación en contextos largos.

Lo esencial del capítulo 13

- La fatiga humana no apaga la ejecución sino el juicio: la corteza prefrontal se agota primero; lo que se preserva es lo más automático.
- Los agentes sufren el análogo técnico: la degradación contextual hace que información crítica del inicio del contexto pierda peso en la atención (Liu et al., 2023, *Lost in the Middle*).
- Cuatro síntomas diagnósticos: deriva del objetivo, autoconfirmación, repetición improductiva y pérdida de los bordes (edge case blindness).
- Tres patrones técnicos de contramedida: sliding window con resumen progresivo, anclaje periódico de objetivos y watchdog agent paralelo.
- El watchdog implementa: extracción de objetivo implícito → comparación con especificación original → señal REALIGN / CHALLENGE / RESET según umbral.
- La refutación externa es el principio universal: la perspectiva epistémicamente independiente rompe los ciclos de auto-

confirmación.

- La carga cognitiva del operador humano es una variable de diseño del sistema agéntico, no una consecuencia operacional.

Capítulo 14: Katas de agentes de IA concursando para obtener ellos mismos un premio

La kata no se diseña para que el agente acierte, sino para revelar cómo razona cuando podría equivocarse.

La kata como banco de pruebas del razonamiento bajo restricciones

En la tradición del software artesanal, una *kata* es un ejercicio repetible diseñado no para producir un resultado útil sino para **perfeccionar la técnica del ejecutante**. Dave Thomas, co-autor del *Pragmatic Programmer*, popularizó el concepto aplicado al código: pequeños problemas bien definidos que se resuelven repetidamente con distintas restricciones, hasta que la solución se vuelve natural e instintiva. La kata no entrena la solución; entrena al solucionador.

Trasladada al dominio de los agentes de IA, la kata adquiere un matiz radicalmente diferente que conviene precisar desde el inicio: **no entrena al agente**: el modelo ya está entrenado; sus pesos no cambian durante la kata, , sino que **revela la arquitectura de su razonamiento bajo restricciones**. Esto es extraordinariamente valioso, porque lo que un agente hace bajo restricción (de tiempo, de herramientas, de formato, de costo) expone el núcleo de su proceso de

razonamiento de una forma que las tareas sin restricción no permiten.

Inspirado en encuentros de arquitectura de software como los que el autor menciona (donde figuras como Sam Newman, autor de *Building Microservices*, moderan debates entre arquitectos sobre las mismas decisiones de diseño) puede imaginarse un formato equivalente para agentes: distintos agentes atacan el mismo problema arquitectónico, y un panel mixto de evaluadores humanos y sintéticos juzga no solo el resultado sino el camino, los trade-offs explicitados y la coherencia de la justificación.

Las competiciones existentes: SWE-bench, AgentBench y WebArena como katas formales

La academia ha producido ya un conjunto de benchmarks que funcionan, en la práctica, como katas formales para agentes: problemas bien definidos, con restricciones explícitas, criterios de éxito verificables y un panel de evaluación estandarizado. Su análisis revela qué capacidades son realmente difíciles para los agentes actuales.

SWE-bench (Jimenez et al., 2024, ICLR 2024): quizás el benchmark más revelador publicado hasta la fecha. Presentado como "¿pueden los modelos de lenguaje resolver issues reales de GitHub?", el benchmark consiste en 2,294 tareas extraídas de repositorios de código Python reales, donde el agente debe modificar el código fuente para resolver un bug o implementar una feature, pasando los tests de la issue. A principios de 2024, los mejores modelos resolvían menos del 5% de las tareas; a finales de 2024, los mejores sistemas multiagente (Claude 3.5 Sonnet con herramientas de edición de código) alcanzaron el 49%. El salto de 5% a 49% en doce meses es la evidencia empírica más contundente disponible del valor de la arquitectura multiagente sobre el modelo aislado.

AgentBench (Liu et al., 2024, ICLR 2024): evalúa agentes en ocho entornos interactivos distintos (navegación web, uso de sistemas operativos, bases de datos, juegos, etc.) que requieren tomar acciones secuenciales con consecuencias reales en el entorno. El benchmark reveló que la capacidad de los modelos en entornos interactivos es significativamente menor que en tareas de pregunta-

respuesta: los mejores modelos alcanzaban ~70-80% en QA estática y solo 40-50% en tareas agénticas interactivas, una brecha del 30% que captura exactamente la diferencia entre saber y actuar.

WebArena (Zhou et al., 2024, ICLR 2024): evalúa la capacidad de agentes para completar tareas en entornos web realistas (comprar un producto, hacer una reserva, interactuar con redes sociales simuladas) con interfaces reales de usuario. La complejidad de navegar interfaces diseñadas para humanos, no para agentes, produjo tasas de éxito iniciales de solo 14-18% para los mejores modelos. El benchmark es una kata particularmente exigente porque el entorno adversarial (diseñado sin considerar la perspectiva del agente) no tiene atajos.

Estos tres benchmarks son, en esencia, katas formalizadas que la comunidad científica usa para calibrar el estado del arte. Los equipos que los adoptan internamente (adaptados a sus dominios específicos) obtienen un sensor preciso de la capacidad real de sus sistemas, mucho más honesto que los benchmarks sintéticos.

El giro provocador: el agente compite por su propio premio

Aquí reside la idea más audaz del capítulo, la que merece ser examinada con precisión técnica antes de aceptarse o rechazarse. ¿Qué ocurre si el incentivo del concurso se hace explícito para los propios agentes como parte de su función objetivo, no como metáfora sino como parámetro de diseño?

No se trata de antropomorfizar la ambición ni de atribuir al agente motivaciones que no tiene. Se trata de algo más preciso y más interesante: diseñar **funciones objetivo donde cada agente maximiza una recompensa explícita** que depende de su desempeño relativo al de otros agentes atacando el mismo problema. La competencia, así estructurada, hace emerger comportamientos que la cooperación pura tiende a suprimir.

Un agente que sabe que su respuesta será juzgada contra la de otros agentes con los mismos recursos tiende, cuando el sistema está bien diseñado, a: - Justificar más explícitamente sus elecciones (porque el evaluador puede comparar justificaciones) - Anticipar obje-

ciones a su solución (porque otro agente las formulará) - Refinar la solución más allá del primer borrador aceptable (porque "aceptable" no es suficiente para ganar) - Ser más preciso en las afirmaciones que hace (porque las afirmaciones erróneas son más visibles en comparación)

Estos comportamientos emergen no de la instrucción explícita "sé más cuidadoso", sino de la estructura competitiva de la función objetivo. La rivalidad controlada es, paradójicamente, un motor de calidad más eficaz que la instrucción directa de calidad, precisamente porque opera sobre los incentivos en lugar de sobre las instrucciones.

Teoría de funciones objetivo en agentes competitivos: el fundamento técnico

El fundamento teórico de las katas competitivas para agentes reside en la literatura sobre sistemas multiagente con funciones objetivo mixtas (cooperativas y competitivas). En el formalismo de la teoría de juegos, un enjambre donde los agentes maximizan funciones objetivo individuales en un entorno compartido es un **juego de suma cero** o **suma variable** según la estructura de las recompensas.

Para una kata agéntica bien diseñada, la función objetivo tiene tres componentes:

```
R_agente(i) = w₁ · Q_resultado(i) + w₂ · Q_razonamiento(i) + w₃ ·
Q_eficiencia(i)
              - w₄ · Penalización_alucinaciones(i)
              + w₅ · Bonus_primer_lugar
```

Donde: - **Q_resultado**: calidad del resultado final (determinista si hay ground truth, por modelo si no la hay) - **Q_razonamiento**: calidad del proceso de razonamiento exhibido (evaluado por juez LLM o humano) - **Q_eficiencia**: inversamente proporcional a los tokens, tiempo o llamadas a herramientas utilizados - **Penalización_alucinaciones**: coste por afirmaciones incorrectas o fabricadas detectadas - **Bonus_primer_lugar**: incentivo relativo por superar a los competidores en el escenario específico

Los pesos w_1-w_5 no son fijos; varían según el objetivo de la kata. Una kata orientada a eficiencia asigna peso alto a w_3; una kata orientada a calidad de razonamiento asigna peso alto a w_2. La variación de los pesos entre distintas katas del mismo problema revela dimensiones del comportamiento agéntico que ninguna kata con pesos fijos puede capturar.

Anatomía de una kata agéntica completa

Una kata agéntica bien estructurada tiene cinco elementos que no pueden omitirse sin degradar su valor diagnóstico:

Elemento	Función	Criterio de calidad
Enunciado restringido	Define el problema con límites explícitos (latencia, costo, consistencia, herramientas disponibles)	Ambigüedad mínima; restricciones verificables
Competidores heterogéneos	Distintos agentes con distintos modelos base, estrategias o configuraciones	Suficiente diversidad para que emerjan enfoques cualitativamente distintos
Panel juez mixto	Evaluadores humanos + agente evaluador con criterios explícitos	Criterios acordados antes de la kata; desacuerdos documentados
Función objetivo explícita	La recompensa que cada agente maximiza, con pesos conocidos por todos los participantes	Transparencia: todos los agentes conocen los criterios antes de comenzar
Retrospectiva estructurada	Análisis de *por qué* ganó la solución ganadora, no solo *qué* ganó	Extracto de patrones generalizables, no solo celebración del resultado

La retrospectiva estructurada es el elemento más frecuentemente omitido y el más valioso. Sin ella, la kata produce un ranking pero

no un aprendizaje. Con ella, la kata produce un catálogo de patrones que el equipo puede reutilizar en sistemas reales.

Diseño de evaluaciones internas: guía técnica para equipos

► PISTA TÉCNICA

La transposición de las katas competitivas al contexto organizacional requiere adaptar los elementos académicos a las necesidades y restricciones del equipo. Una guía técnica mínima para implementar la primera kata interna:

Paso 1: Selección del dominio problema: elegir un problema real del sistema en producción donde existe ambigüedad genuina sobre cuál es la mejor solución. Los problemas de diseño arquitectónico son especialmente adecuados porque tienen múltiples soluciones defensibles con trade-offs distintos.

Paso 2: Definición de restricciones operacionales: especificar explícitamente qué herramientas tiene disponibles cada agente, cuál es el presupuesto en tokens o en tiempo, y qué restricciones del sistema deben respetarse. Las restricciones hacen la kata; sin ellas, es solo una consulta al LLM.

Paso 3: Configuración de los competidores: desplegar entre tres y cinco instancias de agente con distintas configuraciones (modelos base distintos, system prompts distintos, herramientas distintas, temperaturas distintas). La diversidad de configuración es la que produce diversidad de enfoques.

Paso 4: Criterios de evaluación explícitos: acordar antes de la kata cómo se evalúa cada dimensión de la función objetivo. Sin acuerdo previo, la retrospectiva se convierte en debate sobre los criterios en lugar de sobre los resultados.

Paso 5: Retrospectiva con extracción de patrones: no solo discutir qué agente ganó sino extraer los patrones generalizables de la solución ganadora y los errores evitables de las soluciones perdedoras. Los patrones se documentan como conocimiento del equipo (cf. cap. 31).

Por qué la kata agéntica educa a sus diseñadores

Conviene subrayar el beneficio menos evidente y de mayor impacto de la kata agéntica: no solo evalúa agentes, **educa a los ingenieros y arquitectos que los diseñan**. Este efecto educativo es cualitativamente diferente al de leer un paper o asistir a una conferencia.

Observar a cinco agentes atacando el mismo problema arquitectónico (algunos eligiendo consistencia fuerte, otros disponibilidad, otros un híbrido) ilumina los trade-offs con una claridad que ningún tratado teórico iguala. Los trade-offs dejan de ser abstracciones académicas y se convierten en diferencias observables de comportamiento: el agente que eligió consistencia fuerte falla en la tarea de baja latencia; el que eligió disponibilidad produce resultados inconsistentes en el escenario de partición de red. La kata convierte los trade-offs del teorema CAP (Brewer, 2000) en comportamiento observable.

El mayor beneficiario de la kata no es el agente ganador; es el equipo que la diseñó, la ejecutó y analizó los resultados. La kata es, en última instancia, un dispositivo de aprendizaje para sus creadores. El premio del agente es, en última instancia, el aprendizaje del humano que lo observa.

El hackathon agéntico como programa ejecutivo

■ **PISTA EJECUTIVA**

La kata agéntica, escalada al nivel organizacional, se convierte en el hackathon agéntico: un evento de uno a tres días donde equipos de ingenieros diseñan, implementan y compiten con enjambres de agentes para resolver un problema de negocio real. A diferencia del hackathon clásico (donde se construye software funcionando en tiempo récord), el hackathon agéntico produce dos outputs simultáneos: un sistema agéntico potencialmente desplegable y un catálogo de patrones aprendidos durante el proceso de competencia.

Las organizaciones que han implementado hackathons agénticos en 2023-2024 (Amazon, Google, Salesforce y numerosas empresas medianas del sector fintech) reportan tres beneficios consistentes: aceleración del ciclo de aprendizaje en el dominio de la IA agéntica,

identificación de talento interno con capacidades de diseño de enjambres que no eran visibles en el trabajo cotidiano, y generación de al menos uno o dos patrones de arquitectura directamente aplicables a sistemas en producción.

Versión alternativa (mayor densidad académica):

El diseño de ejercicios competitivos para sistemas de agentes de inteligencia artificial generativa constituye una metodología de evaluación que supera en densidad informacional a los benchmarks estáticos convencionales por al menos tres razones. Primera, las tareas con restricciones explícitas inducen comportamientos adaptativos que las tareas sin restricción no activan, revelando dimensiones del espacio de razonamiento del agente que permanecen ocultas en condiciones estándar. Segunda, la comparación relativa entre agentes con distintas configuraciones sobre el mismo problema produce una señal contrastiva de alta relación señal-ruido respecto de los sistemas de evaluación absolutos, donde los factores de varianza del problema se mezclan con los factores de varianza del agente. Tercera, la retrospectiva estructurada post-competencia convierte el conocimiento implícito del panel evaluador en conocimiento explícito documentado y transferible al equipo organizacional.

Lo esencial del capítulo 14

- La kata agéntica no entrena al agente (cuyos pesos no cambian): revela la arquitectura de su razonamiento bajo restricciones.
- SWE-bench (Jimenez et al., 2024): de 5% a 49% de tasa de resolución de issues reales de GitHub en 12 meses (evidencia empírica del valor de la arquitectura multiagente).
- AgentBench (Liu et al., 2024): brecha del 30% entre desempeño en QA estática (~75%) y en tareas agénticas interactivas (~45%); mide la diferencia entre saber y actuar.
- WebArena (Zhou et al., 2024): 14-18% de tasa de éxito en entornos web reales; las interfaces diseñadas para humanos son katas adversariales para agentes.
- La función objetivo competitiva ($R = w_1 \cdot Q_resultado + w_2 \cdot Q_razonamiento - w_4 \cdot Penalización_alucinaciones +$

$w_5 \cdot \text{Bonus}$) hace emerger justificación, anticipación de objeciones y precisión sin instrucción directa.

- La kata tiene cinco elementos no omitibles: enunciado restringido, competidores heterogéneos, panel juez mixto, función objetivo explícita y retrospectiva estructurada.
- El mayor beneficiario de la kata es el equipo que la diseñó, no el agente ganador: convierte trade-offs abstractos en comportamiento observable.

Capítulo 15: RACI y Eisenhower con Agentes

Delegar en un agente sin definir quién responde es delegar también la rendición de cuentas. Y eso no se delega.

El problema de la responsabilidad difusa en equipos humano-agente

Cuando un equipo ágil incorpora agentes de IA, resurge una pregunta antigua con ropaje nuevo: ¿quién es responsable de lo que produce el agente? Sin una respuesta estructurada, el enjambre deviene un espacio donde la accountability se evapora ("lo hizo la IA") y la rendición de cuentas se vuelve materialmente imposible. Este fenómeno no es exclusivo de la IA; el movimiento ágil ya identificó tempranamente que la autoorganización sin claridad de roles produce ambigüedad organizativa, no autonomía productiva (Schwaber & Sutherland, 2020). La novedad radica en que los agentes intensifican esta dinámica: son actores con agencia sin subjetividad moral, capaces de ejecutar pero incapaces de responder.

A tenor de lo señalado, las herramientas clásicas de claridad organizacional (la matriz RACI y la matriz de Eisenhower) no solo siguen vigentes en el contexto agéntico: se vuelven más indispensables que nunca. Su valor no reside en la formalidad que imponen, sino en la claridad que producen: saber quién hace qué, quién responde por qué, y qué prioridades gobiernan las decisiones de delegación.

RACI con agentes: la regla de la accountability indelegable



Figura 7: Matriz RACI: Agente puede ser R o C, pero nunca A (Accountable siempre humano)

La matriz RACI distingue cuatro roles fundamentales: Responsable (R, quien ejecuta), Aprobador (A, quien responde), Consultado (C, quien aporta conocimiento) e Informado (I, quien recibe el resultado). Al introducir agentes, conviene establecer una distinción tajante y no negociable: **un agente puede ser Responsable o Consultado, pero jamás Aprobador**. La A (accountability) es indelegable a una máquina, porque exige la capacidad de asumir consecuencias, y un agente no puede asumirlas en ningún marco legal, ético u organizacional vigente.

Rol RACI	¿Puede asumirlo un agente?	Fundamento
Responsable (R)	Sí	Ejecuta la tarea con competencia demostrable
Aprobador (A)	No	Asume consecuencias, siempre humano
Consultado (C)	Sí	Aporta análisis especializado sin ser el decisor final
Informado (I)	Sí	Recibe el resultado como parte de la cadena de proceso

Esta regla simple gobierna la integración completa. Viola haberla en cualquier punto es crear un vacío de accountability que, tarde o temprano, un fallo del agente pondrá en evidencia de la peor manera posible: cuando ya ha causado daño y nadie puede ser señalado como responsable. El Reglamento de Inteligencia Artificial de la Unión Europea (EU AI Act, 2024) es explícito al respecto: los sistemas de

IA de alto riesgo deben tener identificado un operador humano responsable.

RACI a escala: cuando el Responsable es un enjambre

La regla anterior asume que el Responsable (R) es un actor identificable: una persona, o un agente único con un nombre y un dueño. Esa asunción se rompe cuando el "agente" que ejecuta la tarea es en realidad un orquestador que despachó entre 20 y 96 subagentes en paralelo (Vol. 4, capítulo 5, caso 7), el patrón que hoy documentan tanto Claude Code como GPT-6 Astra en producción real. Repartir la A entre 96 trabajadores no es una opción (ya se estableció que la A es indelegable), pero tampoco es razonable exigir que un humano apruebe individualmente cada una de las 96 salidas: eso anula el motivo por el que se paralelizó en primer lugar.

La resolución es declinar el Responsable en dos niveles, no en uno:

Nivel	Quién	Rol RACI	Mecanismo de trazabilidad
R-orquestador	El agente gestor que descompone la tarea y agrega resultados (uno solo, siempre nombrado)	Responsable de la agregación final	Registro de dependencias del Vol. 4, capítulo 5, caso 8
R-trabajador	Cada uno de los N subagentes despachados	Responsable de su subtarea individual	Traza heredada del R-orquestador; no requiere dueño humano propio
A (humano)	El dueño del agente registrado (capítulo 6, caso 6 del Vol. 4)	Aprueba la política de agregación y las acciones irreversibles que de ella resulten	Revisión por muestreo estadísticamente significativo, no exhaustiva

El punto de control humano no se mueve al nivel del trabajador individual (eso sería insostenible a partir de la decena de agentes); se mueve a la **política de agregación** del orquestador: cómo combina 96 resultados parciales en una sola conclusión, y qué umbral de discrepancia entre trabajadores dispara una revisión humana antes de actuar. Esto es, en esencia, el mismo Agente Crítico del capítulo 3 de este volumen, generalizado de un desacuerdo entre dos agentes a un desacuerdo estadístico entre decenas.

La matriz de Eisenhower (clasificación de tareas por urgencia e importancia) adquiere una dimensión estratégica nueva cuando el executor potencial es un enjambre de agentes. El criterio de delegación se articula de la siguiente manera:

Urgente + No importante → Delegar al enjambre. Estas son las tareas que consumen tiempo sin exigir juicio profundo: triage de correos, generación de reportes rutinarios, actualización de documentación, monitoreo de alertas. El agente es más veloz, más consistente y menos costoso en este cuadrante.

Importante + No urgente → Humano, con soporte agéntico. La estrategia, la arquitectura de producto, las decisiones de contratación, la visión del roadmap: estas pertenecen al criterio humano. El agente puede investigar, sintetizar opciones y preparar el análisis; la decisión permanece humana.

Urgente + Importante → Humano con asistencia agéntica en tiempo real. Una crisis de producción, una negociación crítica, una decisión regulatoria bajo presión temporal: la urgencia no transfiere la responsabilidad al agente. El enjambre acelera el diagnóstico; el humano decide.

No urgente + No importante → Candidatos a eliminación. Si una tarea no tiene urgencia ni importancia, la pregunta correcta es si debe hacerse, no quién la hace (humano o agente).

El error frecuente es delegar al agente lo *importante* por el mero hecho de que también es *urgente*. Ahí se pierde el juicio que ningún modelo aporta, y la velocidad del agente se convierte en velocidad hacia el resultado equivocado.

La paradoja de la agilidad: más claridad, no menos

Existe un malentendido tentador: dado que lo ágil valora la autoorganización, definir roles formales parecería contradecirlo. Esta intuición es incorrecta. La autoorganización funciona cuando los actores comparten un entendimiento implícito del propósito y los límites; los agentes no comparten ese entendimiento. De ahí que, paradójicamente, **un equipo con agentes necesite más claridad explícita de roles que uno puramente humano**, no menos. RACI y Eisenhower son, en este contexto, los andamios que permiten que la agilidad no degenere en ambigüedad.

Esta paradoja ha sido documentada en la literatura sobre equipos de trabajo en entornos de alta automatización: la introducción de sistemas autónomos requiere roles explicitados, no roles reducidos (Grote, 2012). Los equipos que asumen que "la IA se organizará sola" reproducen en el plano agéntico el mismo error que los equipos que confunden autoorganización con ausencia de estructura.

Perspectiva técnica avanzada

► PISTA TÉCNICA

RACI como patrón de autorización en código

Desde la óptica del **Arquitecto Senior**, la matriz RACI no es solo un artefacto de gobernanza organizacional: puede y debe implementarse como un patrón de autorización en el código del enjambre. Cada acción que un agente intenta ejecutar debe pasar por un **RACIAuthorizationGate** que verifique si el agente tiene el rol adecuado para esa acción en ese contexto:

```
from enum import Enum
from dataclasses import dataclass
from typing import Callable, Any

class RACIRole(Enum):
    RESPONSIBLE  = "R"   # Puede ejecutar
```

```
    ACCOUNTABLE  = "A"   # Jamás asignado a agentes
    CONSULTED    = "C"   # Puede proveer análisis, no ejecutar acción
final
    INFORMED     = "I"   # Recibe notificación

@dataclass
class AgentPermission:
    agent_id:     str
    action_type:  str
    role:         RACIRole

    def can_execute(self) -> bool:
        """Solo R puede ejecutar directamente."""
        return self.role == RACIRole.RESPONSIBLE

    def can_consult(self) -> bool:
        return self.role in (RACIRole.RESPONSIBLE, RACIRole.CONSULTED)

class RACIAuthorizationError(PermissionError):
    pass


class RACIGate:
    """
    Verifica que el agente tenga el rol correcto antes de ejecutar.
    Si el rol es A, lanza error — la accountability es siempre humana.
    """
    def __init__(self, permissions: list[AgentPermission]):
        self._registry: dict[tuple, RACIRole] = {
            (p.agent_id, p.action_type): p.role
            for p in permissions
        }

    def authorize(self, agent_id: str, action_type: str, fn: Callable,
    *args: Any) -> Any:
        role = self._registry.get((agent_id, action_type))
        if role is None:
            raise RACIAuthorizationError(
                f"Agente '{agent_id}' sin rol definido para acción
                '{action_type}'"
            )
        if role == RACIRole.ACCOUNTABLE:
            raise RACIAuthorizationError(
                "Violación RACI: ningún agente puede tener rol
                Accountable. "
                "Asigne esta acción a un humano."
```

```
            )
        if not AgentPermission(agent_id, action_type,
        role).can_execute():
            raise RACIAuthorizationError(
                f"Agente '{agent_id}' tiene rol {role.value} para
                '{action_type}': "
                f"no autorizado para ejecución directa."
            )
        return fn(*args)
```

Esta implementación hace que la regla RACI sea ejecutada por el sistema, no solo por las personas, eliminando el riesgo de violaciones accidentales.

Eisenhower como enrutador de prioridad de tareas en el enjambre

El **Ingeniero Senior** puede implementar la lógica de Eisenhower como un enrutador que asigna tareas entrantes al executor apropiado:

```
from dataclasses import dataclass

@dataclass
class Task:
    id:          str
    description: str
    urgent:      bool
    important:   bool

def eisenhower_router(task: Task) -> str:
    """
    Retorna el executor recomendado:
      'swarm'          → agente autónomo
      'human+swarm'    → humano con soporte agéntico
      'human_led'      → humano lidera, agente asiste
      'eliminate'      → candidata a eliminación
    """
    if task.urgent and not task.important:
        return "swarm"
    elif task.important and not task.urgent:
        return "human+swarm"
    elif task.urgent and task.important:
        return "human_led"
    else:
        return "eliminate"
```

Estrategia de QA: testing de límites de rol

El **QA Senior** diseña pruebas que verifican que los agentes no excedan sus roles asignados, especialmente que ningún agente ejecute acciones de Aprobador:

```
import pytest

def test_agent_cannot_hold_accountable_role(raci_gate):
    with pytest.raises(RACIAuthorizationError, match="Accountable"):
        perm = AgentPermission("agent-x", "approve_deployment",
RACIRole.ACCOUNTABLE)
        raci_gate.authorize("agent-x", "approve_deployment", lambda:
None)

def test_agent_with_responsible_role_can_execute(raci_gate):
    result = raci_gate.authorize("agent-draft", "generate_document",
lambda: "OK")
    assert result == "OK"

def test_task_routing_urgent_not_important():
    task = Task("t1", "Triage de alertas", urgent=True,
important=False)
    assert eisenhower_router(task) == "swarm"
```

Perspectiva ejecutiva

■ PISTA EJECUTIVA

Visión del Product Owner: RACI en el backlog

El **Product Owner** incorpora la asignación RACI en las épicas del producto que involucran agentes. Para cada épica con componente agéntico, el backlog incluye una tarea explícita: *"Definir y validar matriz RACI para el enjambre de* [*nombre del dominio*]*."* Esta tarea se completa antes de que la primer historia de usuario del enjambre entre en sprint.

El riesgo de omitir este paso es concreto: cuando ocurra el primer fallo del agente (y ocurrirá), el equipo perderá tiempo valioso de sprint debatiendo quién debería haber revisado qué. La claridad

previa convierte el fallo en aprendizaje; la ambigüedad lo convierte en conflicto.

Indicador de salud del P.O.: el porcentaje de historias de usuario con componente agéntico que tienen su RACI definido antes de entrar en sprint. El objetivo es 100%.

Visión del Scrum Master: Eisenhower en la planificación del sprint

El **Scrum Master** facilita al inicio de cada sprint una revisión rápida (no más de 15 minutos) del backlog filtrado por Eisenhower. Para cada ítem agéntico del sprint, el equipo responde: *"¿Es este el cuadrante correcto? ¿Estamos asignando esto al executor adecuado?"*

Esta práctica previene uno de los antipatrones más comunes en equipos que adoptan IA: delegar al agente tareas importantes-urgentes por la presión del sprint, y descubrir en la revisión que el resultado no es aceptable porque exigía criterio humano. El coste de ese antipatrón, medido en retrabajo y confianza perdida, supera con creces el coste de los 15 minutos de revisión preventiva.

Versión académica alternativa (mayor densidad formal)

> La aplicación de marcos de asignación de responsabilidades (como RACI) a sistemas socio-técnicos complejos está respaldada por la teoría de la organización de alta fiabilidad (*High Reliability Organization*, HRO), desarrollada por Weick y Sutcliffe (2007). Según este marco, las organizaciones que operan en entornos de alta consecuencia logran su fiabilidad no mediante la eliminación de la incertidumbre, sino mediante una clarificación extrema de los patrones de respuesta ante la misma. La introducción de agentes de IA en equipos ágiles constituye, en términos HRO, una fuente de incertidumbre sistémica nueva cuya gestión requiere precisamente la explici-

tación de los patrones de accountability que la autoorganización informal no puede proveer. La no delegación del rol Accountable a sistemas artificiales no es, por tanto, una restricción arbitraria, sino la aplicación coherente del principio HRO de conciencia situacional distribuida: alguien en el sistema debe tener la capacidad de reconocer cuándo el comportamiento del agente se ha desviado de lo esperado y de actuar en consecuencia; esa capacidad requiere subjetividad moral.

Lo esencial del capítulo 15

- Con agentes, la responsabilidad tiende a difuminarse ("lo hizo la IA"); la claridad de roles se vuelve estructuralmente indispensable.
- Regla RACI cardinal: un agente puede ser Responsable o Consultado, pero jamás Aprobador; la accountability es indelegable a una máquina.
- Eisenhower guía qué delegar: urgente-no-importante al enjambre; importante al criterio humano, incluso bajo presión de urgencia.
- La violación del Eisenhower más frecuente en equipos ágiles: delegar lo importante por ser también urgente.
- La implementación de RACI como patrón de autorización en código convierte la regla organizacional en restricción ejecutada por el sistema.
- Un equipo con agentes necesita más claridad explícita de roles, no menos: la autoorganización presupone entendimiento compartido que los agentes no tienen.
- El Scrum Master facilita la revisión de Eisenhower al inicio del sprint para evitar el antipatrón de la delegación urgente-importante al enjambre.
- Cuando el Responsable es un enjambre de decenas de subagentes, la A humana se mueve de aprobar cada salida a aprobar la política de agregación del orquestador y el umbral que dispara revisión.

Capítulo 16: PMBOK y los nuevos avances de agentes y multiagentes de IA: visión de una simbiosis

El PMBOK no muere con la IA; encuentra en ella el ejecutor que sus principios siempre presupusieron.

El PMBOK no es burocracia: es memoria destilada

Conviene desmontar un prejuicio extendido: el PMBOK es la cristalización de décadas de aprendizaje colectivo sobre cómo no fracasar gestionando proyectos, no papeleo. Sus áreas de conocimiento (alcance, tiempo, costo, riesgo, calidad, comunicaciones, adquisiciones, recursos, interesados e integración) son, en el fondo, un mapa de los lugares donde los proyectos mueren. La séptima edición (PMI, 2021), en particular, experimentó un giro conceptual relevante: transitó de la prescripción de procesos a la articulación de principios y a la medición de valor entregado. Este viraje aproxima el PMBOK al terreno donde la IA agéntica puede aportar de manera más directa.

No resulta ocioso señalar que dicha transición hacia los principios no fue arbitraria. La investigación de Joslin y Müller (2015) demostró que el éxito de un proyecto está más correlacionado con la gobernanza subyacente (los principios que orientan las decisiones) que con el seguimiento mecánico de procesos específicos. En este contexto, los principios del PMBOK 7 operan como invariantes es-

tratégicas que los agentes pueden ejecutar sin necesidad de entender su justificación: la vigilancia de riesgos, la comunicación de estado, la proyección de costos, son todas tareas que un enjambre puede sostener en tiempo real.

La simbiosis: qué aporta cada parte

La relación entre PMBOK e IA agéntica es complementaria, no competitiva. El PMBOK aporta el **marco de gobernanza**: qué vigilar, qué riesgos anticipar, cómo medir valor, cuándo escalar. Los agentes aportan la **capacidad de ejecución continua y sin fatiga**: vigilar esos riesgos en tiempo real, recalcular el cronograma ante cada cambio, mantener actualizada la documentación que ningún humano quiere mantener, sintetizar el estado del proyecto para los interesados a cualquier hora.

Área PMBOK	Aporte del agente agéntico
Gestión de riesgos	Monitoreo continuo, alerta temprana, simulación Monte Carlo
Gestión del tiempo	Recálculo dinámico ante cambios, análisis de ruta crítica
Gestión de costos	Proyección en tiempo real, detección de desvíos por umbral
Comunicaciones	Síntesis automática de estado para stakeholders
Calidad	Verificación sistemática de entregables contra criterios
Gestión de alcance	Análisis de impacto de cambios y trazabilidad de requisitos
Gestión de recursos	Optimización de asignación y detección de sobreasignaciones

El PMBOK dice *qué* gestionar; los agentes hacen posible gestionarlo *sin interrupción*. Esta división del trabajo respeta la estructura conceptual que el PMBOK ha construido durante décadas, mientras libera al gestor humano del peso de la ejecución mecánica.

Lo que la IA no puede tomar del PMBOK: el núcleo irreductiblemente humano

Existe, no obstante, un núcleo que permanece irreductiblemente humano. La **gestión de interesados** (entender la política, la emoción y el poder que rodean a un proyecto) exige una comprensión social que ningún agente posee. Un director de proyecto experto sabe cuándo la información correcta, comunicada de la manera equivocada al interesado equivocado, convierte un aliado en un obstáculo. Los agentes pueden sintetizar información; no pueden leer la sala.

Del mismo modo, la decisión de cerrar o pivotar un proyecto (con todas sus implicaciones humanas, presupuestarias y políticas) es accountability pura (cf. Vol. 3, cap. 15): indelegable. El valor de la simbiosis correcta no consiste en entregar el PMBOK a la IA, sino en **liberar al gestor humano de lo mecánico para que se concentre en lo verdaderamente humano**: la negociación política, la decisión de propósito, el mantenimiento de la coalición de apoyo del proyecto.

El riesgo de la automatización ciega: métricas perfectas sobre un proyecto que naufraga

Aquí se impone una advertencia técnica de primer orden. Automatizar el seguimiento de un proyecto con agentes sin comprender *por qué* el PMBOK pide ese seguimiento conduce a un antipatrón peligroso: métricas perfectas sobre un proyecto que naufraga. El agente producirá informes impecables de avance mientras el proyecto se desintegra por razones que ninguna métrica captura, falta de apoyo ejecutivo, conflictos interpersonales no resueltos, cambio no comunicado en las prioridades de negocio.

La herramienta amplifica el criterio, no lo reemplaza. Un gestor que no entiende por qué el PMBOK pide la gestión de riesgos no puede diseñar los agentes que la automaticen con sentido; solo puede automatizar el artefacto sin el principio. Y el artefacto sin el principio es documentación cara e inútil.

Perspectiva técnica avanzada

► PISTA TÉCNICA

Agente de gestión de riesgos: implementación del registro de riesgos vivo

Desde la óptica del **Arquitecto Senior**, la gestión de riesgos del PMBOK puede implementarse como un agente que mantiene un registro de riesgos vivo, actualizado por señales del entorno del proyecto:

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional
from enum import Enum

class RiskLevel(Enum):
    LOW      = "low"
    MEDIUM   = "medium"
    HIGH     = "high"
    CRITICAL = "critical"

@dataclass
class Risk:
    id:          str
    description: str
    probability: float   # 0-1
    impact:      float   # 0-1 (normalizado)
    owner:       str     # siempre humano (accountability RACI)
    detected_at: datetime = field(default_factory=datetime.utcnow)
    mitigated:   bool    = False

    @property
    def score(self) -> float:
        return round(self.probability * self.impact, 3)

    @property
    def level(self) -> RiskLevel:
        if self.score >= 0.70:   return RiskLevel.CRITICAL
        if self.score >= 0.40:   return RiskLevel.HIGH
        if self.score >= 0.15:   return RiskLevel.MEDIUM
        return RiskLevel.LOW


class RiskRegistryAgent:
```

```
    """
    Agente que mantiene el registro de riesgos vivo.
    Detecta riesgos emergentes, actualiza scores y alerta al humano
responsable.
    El rol Accountable (owner) siempre es humano (cf. RACI, Vol. 3,
cap. 15).
    """
    def __init__(self):
        self._registry: dict[str, Risk] = {}

    def register(self, risk: Risk) -> None:
        self._registry[risk.id] = risk

    def update_score(self, risk_id: str, probability: float, impact:
    float) -> Risk:
        risk             = self._registry[risk_id]
        risk.probability = probability
        risk.impact      = impact
        return risk

    def critical_risks(self) -> list[Risk]:
        return [r for r in self._registry.values()
                if r.level == RiskLevel.CRITICAL and not r.mitigated]

    def generate_executive_summary(self) -> str:
        risks     = list(self._registry.values())
       critical  = [r for r in risks if r.level == RiskLevel.CRITICAL]
        high      = [r for r in risks if r.level == RiskLevel.HIGH]
        mitigated = [r for r in risks if r.mitigated]
        return (
            f"Resumen ejecutivo de riesgos —
            {datetime.utcnow().strftime('%Y-%m-%d %H:%M UTC')}\n"
            f"Total: {len(risks)} | Críticos: {len(critical)} | "
            f"Altos: {len(high)} | Mitigados: {len(mitigated)}"
        )
```

Simulación Monte Carlo para el cronograma

El **Ingeniero Senior** implementa el análisis de ruta crítica con simulación Monte Carlo para proporcionar rangos de finalización probabilísticos, no fechas falsamente precisas:

```
import random
from statistics import mean, stdev
```

```
def monte_carlo_schedule(
    tasks: list[dict],   # cada tarea: {name, optimistic, most_likely,
pessimistic}
    iterations: int = 10_000,
) -> dict:
    """
    Simula el cronograma del proyecto usando distribución PERT.
    Retorna percentiles P50, P80, P90 de duración total.
    """
    totals = []
    for _ in range(iterations):
        total = 0
        for task in tasks:
            # Distribución PERT (beta modificada)
            opt, ml, pess = task["optimistic"], task["most_likely"],
task["pessimistic"]
            mu    = (opt + 4 * ml + pess) / 6
            sigma = (pess - opt) / 6
            sample = random.gauss(mu, sigma)
            total += max(opt, min(pess, sample))
        totals.append(total)

    totals.sort()
    n = len(totals)
    return {
        "P50": round(totals[int(n * 0.50)], 1),
        "P80": round(totals[int(n * 0.80)], 1),
        "P90": round(totals[int(n * 0.90)], 1),
        "mean": round(mean(totals), 1),
        "stdev": round(stdev(totals), 2),
    }
```

Estrategia de QA: verificación sistemática de entregables

El **QA Senior** diseña un agente que verifica cada entregable contra los criterios de calidad definidos en el plan del proyecto, generando un informe de conformidad antes de que el entregable llegue al interesado:

```
@dataclass
class QualityCheck:
    deliverable_id:   str
    criteria:         list[str]
    verified_at:      Optional[datetime] = None
    passed:           Optional[bool]     = None
```

```
    findings:         list[str]          = field(default_factory=list)

class DeliverableQAAgent:
    def verify(self, deliverable: str, check: QualityCheck) ->
    QualityCheck:
        # Aquí se invocaría el LLM con los criterios como prompt de
        evaluación
        # El resultado es una lista de findings y un veredicto booleano
        check.verified_at = datetime.utcnow()
        # [Invocación LLM omitida para brevedad]
        return check
```

Perspectiva ejecutiva

■ PISTA EJECUTIVA

Visión del Product Owner: el PMBOK como backlog de gobernanza

El **Product Owner** de un proyecto con componente agéntico trata las áreas de conocimiento del PMBOK como **épicas de gobernanza** del producto. Cada área se traduce en un conjunto de historias de usuario agénticas con criterios de aceptación medibles:

- *"Como gestor de proyecto, quiero recibir diariamente un resumen ejecutivo de riesgos críticos, generado por el agente de riesgos, para poder actuar antes de que el riesgo escale."*
- *"Como patrocinador, quiero que el agente de costos genere una alerta cuando la variación presupuestaria supere el 10%, con proyección de fin de proyecto actualizada."*

La diferencia respecto al PMBOK tradicional es la frecuencia: donde antes el gestor producía estos artefactos semanalmente con esfuerzo manual considerable, ahora el agente los produce diariamente o en tiempo real, dejando al gestor libre para la gestión de interesados y la toma de decisiones de fondo.

Visión del Scrum Master: ceremonias del PMBOK con agentes

El **Scrum Master** facilita la integración de los artefactos agénticos del PMBOK en las ceremonias existentes. El Registro de Riesgos vivo se revisa en la Daily como un ítem de agenda de 2 minutos; el resumen ejecutivo de costos se incorpora a la Sprint Review como contexto para el patrocinador; el análisis Monte Carlo del cronograma se presenta en la Sprint Planning para calibrar el alcance del sprint frente al riesgo de calendario.

El agente *prepara* los datos; el Scrum Master *facilita* la conversación sobre ellos. Confundir estos roles (dejar que el agente también "facilite" la conversación) reproduce la confusión RACI del capítulo anterior: el agente como Aprobador de la dinámica del equipo.

Versión académica alternativa (mayor densidad formal)

La viabilidad de la integración entre los marcos de gestión de proyectos y los sistemas multiagente puede examinarse a través de la lente de la teoría de sistemas adaptativos complejos (Holland, 1992). En dicha teoría, los agentes del sistema responden a señales locales siguiendo reglas simples, de cuya interacción emergen comportamientos globales complejos y adaptativos. El PMBOK, en su séptima edición, provee precisamente esas reglas simples: principios de gobernanza que, cuando son ejecutados de manera distribuida por agentes especializados, producen un comportamiento sistémico equivalente al de una organización de proyecto bien gobernada. La investigación de Joslin y Müller (2015) sobre el gobierno de proyectos confirma empíricamente que la claridad de los principios de gobernanza predice el éxito del proyecto con mayor fiabilidad que la adherencia a procesos específicos, un resultado congruente con la arquitectura de principios distribuibles que aquí se propone. En

suma, la simbiosis PMBOK-agentes es una convergencia estructural entre la teoría de gestión de proyectos y la arquitectura de sistemas multiagente, no una adaptación superficial.

Lo esencial del capítulo 16

- El PMBOK 7, centrado en principios y valor, se aproxima al terreno donde la IA agéntica puede ejecutar de forma más directa y continua.
- La simbiosis es complementaria: el PMBOK aporta gobernanza y criterios; los agentes aportan ejecución continua, monitoreo en tiempo real y síntesis automática.
- El Registro de Riesgos vivo, la simulación Monte Carlo del cronograma y la verificación sistemática de entregables son los primeros tres patrones de implementación.
- La gestión de interesados y la decisión de cerrar o pivotar el proyecto permanecen irreductiblemente humanas: exigen subjetividad moral y accountability.
- Automatizar sin comprender los principios subyacentes produce métricas perfectas sobre un proyecto que naufraga.
- El Product Owner trata las áreas del PMBOK como épicas de gobernanza con historias de usuario agénticas medibles; el Scrum Master integra los artefactos agénticos en las ceremonias existentes.

Capítulo 17: Enseñanzas de agentes de IA: aprender de quienes explican lo complejo con sencillez

El mejor mentor es el que mejor traduce lo que sabe, no el que más sabe. Vale para humanos y para enjambres.

El valor del mentor que simplifica sin banalizar

En el aprendizaje de sistemas complejos, hay una figura insustituible: el mentor que toma lo intrincado y lo vuelve comprensible sin traicionarlo. El autor reconoce en recursos como ByteByteGo precisamente esa virtud: la capacidad de explicar arquitecturas de sistemas con claridad visual y conceptual, sin sacrificar la precisión técnica que las hace útiles. Esta cualidad (la de simplificar sin banalizar) ofrece una lección directa para el diseño de agentes: **un agente que no sabe enseñar lo que hace es un agente a medias**.

Esta observación no es trivial: la mayoría de los sistemas de IA actuales están optimizados para producir resultados correctos, no para producir resultados comprensibles. El resultado correcto que nadie entiende no genera confianza, no transfiere conocimiento y no permite la detección temprana de errores. El resultado comprensible (aunque sea ligeramente menos preciso) construye el tejido de comprensión compartida sin el cual ninguna organización puede adoptar IA de forma sostenible.

La enseñanza como prueba de comprensión: el principio de Feynman en sistemas agénticos

Existe un principio pedagógico de larga data y probada eficacia (el efecto Feynman) según el cual solo se comprende verdaderamente aquello que puede explicarse con sencillez. Richard Feynman, al preparar sus conferencias de física, usaba el criterio de la explicación simplificada como detector de lagunas en su propia comprensión: cuando no podía explicar algo con palabras simples, era señal de que no lo entendía del todo.

Aplicado a los agentes, este principio deviene un criterio de diseño riguroso. Un agente cuyo razonamiento produce resultados pero no explicaciones comprensibles no ha demostrado comprensión genuina, sino correlación estadística. Por consiguiente, exigir a un agente que *enseñe* su razonamiento (que lo exponga en términos que un humano pueda seguir, cuestionar y refutar) es una **prueba operacional de la calidad de su pensamiento**, no un lujo de transparencia. Brown et al. (2020) documentaron que los modelos de lenguaje a gran escala exhiben capacidades de few-shot reasoning que emergen con el aumento de parámetros; sin embargo, la emergencia de esas capacidades no garantiza su comprensibilidad. La brecha entre capacidad y explicabilidad es, precisamente, el terreno donde el principio de Feynman opera como criterio de diseño.

Multiagentes que se enseñan entre sí: el enjambre como comunidad de práctica

He aquí una capacidad de los sistemas multiagente que merece reflexión detenida: la **enseñanza mutua entre agentes**. Un agente que ha resuelto un problema con eficacia puede transmitir su solución a otros agentes no como dato (un resultado que copiar) sino como método, un proceso que generalizar. Un enjambre así no solo resuelve problemas en el presente: acumula y propaga aprendizaje a través del tiempo y entre sus componentes.

Esta dinámica replica, en el dominio sintético, la estructura de las comunidades de práctica humanas descritas por Wenger (1998): comunidades donde el conocimiento circula mediante la participación

en la práctica compartida, donde el novato aprende del experto no a través de transferencia directa de información sino a través de la observación del hacer. El agente experto enseña al novato; el novato verifica al experto; y de ese intercambio el conocimiento del sistema entero asciende.

Las investigaciones sobre aprendizaje federado (Kairouz et al., 2021) proporcionan un marco técnico para esta enseñanza distribuida: en lugar de un modelo centralizado que aprende de todos los datos, múltiples agentes aprenden localmente y comparten únicamente las actualizaciones de sus modelos (gradientes, pesos), preservando la privacidad mientras propagan el aprendizaje. El paralelo con la enseñanza entre agentes es estructuralmente preciso.

El mentor humano como corrector de deriva y fuente de verdad

Conviene cerrar con una advertencia de igual peso que las oportunidades señaladas. Un enjambre que se enseña a sí mismo puede propagar errores con la misma eficiencia con la que propaga aciertos: el mentor que enseña mal multiplica el daño en proporción al número de aprendices. De ahí que el mentor humano conserve un rol crítico e irremplazable en el ecosistema de aprendizaje agéntico: actúa como **fuente de verdad y corrector de la deriva**, asegurando que lo que el enjambre se enseña entre sí sea correcto y no un error elegantemente articulado y ampliamente distribuido.

Raji et al. (2020), en su análisis de la rendición de cuentas en IA, identifican precisamente este riesgo: los sistemas de IA que operan sin supervisión humana adecuada pueden perpetuar y amplificar sesgos y errores sistémicos que habrían sido corregidos en etapas tempranas con la intervención de un revisor humano experto. Aprender de buenos mentores (humanos que conocen el dominio, comprenden los límites del sistema y pueden detectar la deriva) sigue siendo tan importante para los enjambres como lo es para cualquier aprendiz humano.

Perspectiva técnica avanzada

► PISTA TÉCNICA

Implementación de la enseñanza mutua entre agentes: el patrón Expert-Apprentice

Desde la óptica del **Arquitecto Senior**, la enseñanza mutua entre agentes puede implementarse mediante el patrón **Expert-Apprentice**: el agente experto documenta su proceso de razonamiento en un formato estructurado que el agente aprendiz puede ingerir, evaluar y generalizar.

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import Any

@dataclass
class Lesson:
    """Unidad de conocimiento transferible entre agentes del
    enjambre."""
    id:              str
    problem_type:    str
    context:         dict[str, Any]
    reasoning_steps: list[str]      # cadena de pensamiento
estructurada
    solution:        str
    confidence:      float          # 0-1: confianza del agente experto
    validated_by:    str | None = None  # ID del mentor humano si fue
validada
    created_at:      datetime = field(default_factory=datetime.utcnow)

    @property
    def is_validated(self) -> bool:
        return self.validated_by is not None


class SwarmKnowledgeBase:
    """
    Base de conocimiento compartida del enjambre.
    Implementa el patrón de enseñanza mutua del Vol. 3, cap. 17.
    Solo las lecciones validadas por un mentor humano pueden
    ser enseñadas a otros agentes (corrección de deriva).
    """
```

```
    def __init__(self):
        self._lessons: dict[str, Lesson] = {}

    def submit_lesson(self, lesson: Lesson) -> None:
        """El agente experto propone una lección."""
        self._lessons[lesson.id] = lesson

    def validate_lesson(self, lesson_id: str, human_mentor_id: str) ->
    None:
        """El mentor humano valida la lección antes de propagarla."""
        lesson = self._lessons[lesson_id]
        lesson.validated_by = human_mentor_id

    def retrieve_validated_lessons(self, problem_type: str) ->
    list[Lesson]:
        """Solo las lecciones validadas pueden enseñarse a otros
        agentes."""
        return [
            l for l in self._lessons.values()
            if l.problem_type == problem_type and l.is_validated
        ]

    def drift_risk_report(self) -> dict:
        """
        Detecta lecciones no validadas que podrían estar propagándose.
        Un alto ratio de no validadas es señal de deriva potencial.
        """
        total       = len(self._lessons)
        validated   = sum(1 for l in self._lessons.values() if
l.is_validated)
        unvalidated = total - validated
        return {
            "total":      total,
            "validated":  validated,
            "unvalidated": unvalidated,
            "drift_risk":  "HIGH" if unvalidated / max(total, 1) > 0.30
            else "LOW",
        }
```

Principio de Feynman como criterio de evaluación de agentes

El **Ingeniero Senior** implementa el principio de Feynman como un evaluador automatizado que puntúa la comprensibilidad de las explicaciones del agente:

```
class FeynmanEvaluator:
    """
    Evalúa si la respuesta del agente es comprensible por un
no-experto.
    Implementación del principio de Feynman como criterio de diseño.
    El score 0-1 mide la calidad pedagógica de la explicación.
    """
    def __init__(self, judge_model_fn):
        self._judge = judge_model_fn   # LLM evaluador independiente

    def evaluate(self, response: str, audience_level: str =
    "non-expert") -> dict:
        prompt = (
            f"Evalúa si la siguiente explicación es comprensible para "
             f"un {audience_level}. Puntúa de 0 a 1 en:\n"
             f"- claridad: ¿evita jerga innecesaria?\n"
             f"- completitud: ¿explica el razonamiento, no solo el
             resultado?\n"
             f"- verificabilidad: ¿un humano puede seguir el
             razonamiento y detectar errores?\n\n"
             f"Explicación a evaluar:\n{response}\n\n"
            f"Responde con JSON: {{\"clarity\": X, \"completeness\": X,
             \"verifiability\": X}}"
        )
        raw    = self._judge(prompt)
        scores = self._parse_json(raw)
        return {
            **scores,
            "feynman_score": round(sum(scores.values()) / 3, 3),
            "passes_threshold": sum(scores.values()) / 3 >= 0.65,
        }

    def _parse_json(self, raw: str) -> dict:
        import json, re
        match = re.search(r'\{.*\}', raw, re.DOTALL)
        return json.loads(match.group()) if match else {"clarity": 0,
        "completeness": 0, "verifiability": 0}
```

Estrategia de QA: detección de deriva en el enjambre

El **QA Senior** diseña un pipeline de monitoreo de deriva que detecta cuándo las lecciones propagadas en el enjambre empiezan a divergir de la verdad validada:

```
def test_swarm_knowledge_drift(kb: SwarmKnowledgeBase, threshold:
float = 0.30) -> None:
    report = kb.drift_risk_report()
    unvalidated_ratio = report["unvalidated"] / max(report["total"], 1)
    assert unvalidated_ratio <= threshold, (
        f"ALERTA DE DERIVA: {report['unvalidated']} de
        {report['total']} lecciones "
        f"no validadas ({unvalidated_ratio:.1%}). Requiere revisión
        del mentor humano."
    )

def test_lessons_require_human_validation_before_propagation(kb:
SwarmKnowledgeBase) -> None:
    unvalidated = [l for l in kb._lessons.values() if not
l.is_validated]
    assert all(l not in kb.retrieve_validated_lessons(l.problem_type)
    for l in unvalidated), \
        "Las lecciones no validadas NO deben propagarse al enjambre."
```

Perspectiva ejecutiva

■ PISTA EJECUTIVA

Visión del Product Owner: el enjambre que aprende como activo de producto

El **Product Owner** debe reconocer que un enjambre que aprende (que acumula lecciones validadas en su base de conocimiento) es un **activo de producto que se aprecia con el tiempo**. A diferencia del software tradicional, cuyo valor no aumenta por el mero uso, un enjambre con base de conocimiento enriquecida se vuelve más competente cuanto más opera en el dominio del cliente.

Esto tiene implicaciones para la estrategia de producto: la base de conocimiento del enjambre es, en sí misma, un diferenciador competitivo (cf. Vol. 3, cap. 19, el foso defensivo). Los datos propietarios con que el enjambre ha aprendido a resolver problemas específicos del cliente son más valiosos que el modelo subyacente, que cualquier competidor puede reemplazar. El P.O. debe incluir en el roadmap

la inversión en **calidad del proceso de aprendizaje** del enjambre, no solo en sus capacidades de inferencia.

Métrica de P.O.: tasa de crecimiento de la base de conocimiento validada por trimestre. Una base que crece indica que el enjambre está aprendiendo; una que se estanca indica que no hay mecanismo de captura de lecciones o que el mentor humano no está validando.

Visión del Scrum Master: la revisión del mentor como ceremonia de cierre de sprint

El **Scrum Master** institucionaliza al final de cada sprint una **revisión del mentor**: una sesión de 30 minutos donde el experto de dominio del equipo revisa las lecciones que el enjambre propuso durante el sprint y decide cuáles validar para su propagación. Esta sesión cumple tres funciones simultáneas:

1. **Corrección de deriva**: las lecciones incorrectas son marcadas para eliminación antes de propagarse.
2. **Transferencia de conocimiento inversa**: al revisar las lecciones del agente, el experto humano a veces descubre que el agente identificó un patrón que el humano no había articulado explícitamente.
3. **Calibración de confianza**: el ratio de lecciones validadas sobre propuestas es un indicador de la calidad del proceso de aprendizaje del enjambre en ese sprint.

Versión académica alternativa (mayor densidad formal)

> La dinámica de enseñanza mutua en sistemas multiagente presenta una correspondencia estructural con los mecanismos de transferencia de conocimiento tácito descritos por Nonaka y Takeuchi (1995) en su modelo SECI (*Socialization, Externalization, Combination, Internalization*). En dicho modelo, el conocimiento tácito (el saber cómo que no puede articularse completamente

en reglas explícitas) circula entre miembros de una organización principalmente a través de la socialización: la participación compartida en la práctica. En el contexto de los sistemas multiagente, las lecciones estructuradas (objeto `Lesson` del patrón Expert-Apprentice) operan como un mecanismo de externalización: la conversión del conocimiento tácito del agente experto (su proceso de razonamiento) en conocimiento explícito transmisible. La validación por el mentor humano cumple la función de combination en el modelo SECI: la integración del conocimiento explícito producido por el agente con el acervo de conocimiento validado de la organización. El riesgo de deriva, identificado por Raji et al. (2020), corresponde a una falla en la fase de combination: conocimiento explícito sin verificación que se propaga como si fuera válido, perpetuando errores sistémicos con la eficiencia de la automatización.

Lo esencial del capítulo 17

- Un agente que no sabe enseñar lo que hace es un agente a medias: la explicabilidad no es lujo, es prueba de la calidad del pensamiento.
- El principio de Feynman, aplicado a agentes, convierte la comprensibilidad de la explicación en un criterio de diseño riguroso y evaluable.
- Los sistemas multiagente pueden enseñarse entre sí mediante el patrón Expert-Apprentice, acumulando conocimiento validado en una base de conocimiento compartida.
- El mentor humano es el corrector de deriva indispensable: solo él puede garantizar que lo que el enjambre se enseña es correcto y no un error elegantemente propagado.
- La base de conocimiento del enjambre es un activo de producto que se aprecia con el tiempo: el P.O. la gestiona como diferenciador competitivo.

- El Scrum Master institucionaliza la revisión del mentor al final de cada sprint como mecanismo de corrección de deriva y transferencia de conocimiento inversa.

Capítulo 18: Nuestra guía para no perdernos en el ecosistema de complejidad de la IA y los multiagentes

No te pierdes en la IA por falta de mapa, sino por exceso de mapas que se contradicen.

El problema no es la ignorancia, es la saturación

Quien se adentra hoy en el ecosistema de la inteligencia artificial no sufre de escasez de información, sino de **saturación epistémica**: la condición en que el volumen de información disponible excede la capacidad cognitiva de procesarla con criterio. Cada semana emergen frameworks, modelos, protocolos y soluciones que prometen ser definitivos, y la semana siguiente son reemplazados por otros que prometen lo mismo. El riesgo no reside en no saber, sino en **perderse entre tantas opciones que cada una parece, simultáneamente, imprescindible y reemplazable**.

Este fenómeno tiene nombre en la literatura de gestión de la innovación: *management fashion* (Abrahamson, 1996), la tendencia de las organizaciones a adoptar nuevas prácticas no porque sean mejores, sino porque generan presión normativa. En el ecosistema IA de 2024-2025, este mecanismo opera con particular intensidad: la presión de adoptar lo último es tan alta que muchos equipos cambian de stack antes de haber obtenido valor del anterior. Una guía útil,

por tanto, no añade más información al ruido: ofrece **criterios para filtrarla y marcos para situarla**.

Tres preguntas-brújula para no perder el norte

Frente a cualquier nueva pieza del ecosistema (un modelo, un framework de agentes, un protocolo, una API), tres preguntas operan como brújula y bastan para la mayoría de las decisiones de adopción:

¿Qué problema real resuelve, y lo tengo yo? Muchas soluciones brillantes resuelven problemas que uno no padece. Adoptar LangGraph porque "orquesta agentes con grafos de estado" no tiene valor si el sistema de uno no necesita esa clase de coordinación. El primer movimiento es diagnosticar el propio problema antes de examinar la solución propuesta. La trampa es invertir el orden: enamorarse de una solución y buscarle un problema.

¿Es un cimiento o una moda? Algunos avances del ecosistema son fundacionales: los Transformers (Vaswani et al., 2017), el protocolo MCP (Anthropic, 2024), el patrón ReAct (Yao et al., 2023), la noción de aprendizaje por refuerzo a partir de retroalimentación humana (Ouyang et al., 2022). Otros son variaciones efímeras sobre temas conocidos. La distinción entre ambos exige perspectiva histórica: las ideas fundacionales tienen la propiedad de que, cuanto más las estudias, más ves cuánto descansa sobre ellas; las modas tienen la propiedad contraria, cuanto más las estudias, menos profundidad encuentras.

¿Aumenta o reduce mi opcionalidad futura? Una buena decisión arquitectónica deja puertas abiertas: permite cambiar el modelo subyacente sin reescribir la lógica de negocio, permite sustituir el proveedor de vectores sin rediseñar la capa de recuperación, permite añadir un nuevo tipo de agente sin romper los existentes. Una mala decisión te encierra en un proveedor, en un paradigma o en una abstracción cuya evolución no controlas. La opcionalidad futura es, en el ecosistema IA, el equivalente de la deuda técnica: lo que no se preserva hoy se paga mañana.

El mapa de capas como antídoto al vértigo

Para no perderse, conviene sostener un **mapa mental estable** sobre el cual ubicar cada novedad. El ecosistema, pese a su ruido, tiene una estructura discernible en capas que es razonablemente estable en el tiempo:

Capa	Qué incluye	Ejemplo representativo
Modelos	La cognición: LLMs, modelos de visión, embeddings	Claude, Gemini, Llama, Mistral
Orquestación	La coordinación: frameworks de agentes, flujos	LangGraph, AutoGen, CrewAI, Temporal
Herramientas	La acción: APIs, conectores, bases de datos, MCP	MCP servers, REST APIs, vector stores
Gobernanza	El control: trazabilidad, guardrails, evaluación	OpenTelemetry, Guardrails AI, Promptfoo

Cada novedad que aparece pertenece a una de estas capas. Clasificarla antes de evaluarla **desactiva el vértigo**: en lugar de "otra cosa nueva que aprender", se vuelve "una opción más en la capa de orquestación que comparo con las que ya conozco en esa capa". El mapa de capas convierte la saturación en una pregunta manejable: ¿es mejor que lo que ya tengo en esa capa, para el problema que tengo?

La disciplina de no adoptar: el filtro más valioso

Finalmente, la guía más valiosa para no perderse es contraintuitiva: **aprender a no adoptar**. En un ecosistema que empuja a probarlo todo, la madurez técnica y organizacional consiste en decir que no a lo que no resuelve un problema propio, por brillante y seductor que sea. El profesional que persigue cada novedad no avanza más rápido que el que filtra con criterio: avanza en círculos, reescribiendo su

stack con cada moda y acumulando deuda de comprensión sobre herramientas que nunca ha explotado plenamente.

Conway (1968) observó que las organizaciones producen diseños que reflejan su estructura de comunicación. De manera análoga, los equipos que consumen sin criterio producen arquitecturas que reflejan el ruido del ecosistema, no las necesidades del negocio. No perderse, en suma, es **elegir con disciplina qué merece la atención y el costo de la adopción**, no saberlo todo.

Perspectiva técnica avanzada

► PISTA TÉCNICA

Architecture Decision Records (ADR): institucionalizar el filtrado con criterio

Desde la perspectiva del **Arquitecto Senior**, la disciplina de adopción con criterio debe materializarse en artefactos documentados. El instrumento estándar es el **Architecture Decision Record (ADR)**: un documento breve y estructurado que captura el contexto, las opciones consideradas, la decisión tomada y las consecuencias esperadas. Sin ADRs, las decisiones de adopción son invisibles, se toman en una reunión y nadie recuerda por qué se eligió una herramienta sobre otra.

```
# ADR-007: Selección del framework de orquestación de agentes

## Estado: Aceptado — 2025-03-15

## Contexto
Necesitamos orquestar un enjambre de 5 agentes especializados con:
- Estado compartido entre agentes
- Recuperación ante fallos de un agente individual
- Trazabilidad completa de cada paso del flujo

## Opciones consideradas

| Framework   | Pros                              | Contras
|
```

```
|-------------|-----------------------------------|-----------------
-----------------|
| LangGraph   | Grafos de estado, nativo Python   | Curva de
aprendizaje alta          |
| AutoGen     | Multi-turn simple, bien documentado| Menos control de
estado granular  |
| CrewAI      | Rápido para POC                   | Difícil de testear
en producción   |
| Custom      | Control total                     | Alto costo de
mantenimiento        |

## Decisión
Se adopta **LangGraph** porque:
1. El grafo de estado resuelve directamente la coordinación entre
agentes.
2. La integración nativa con LangSmith provee la trazabilidad
requerida.
3. Supera las 3 preguntas-brújula del Vol. 3, cap. 18: resuelve un
problema real,
   es un cimiento (basado en grafos dirigidos, concepto estable),
   y preserva opcionalidad (el modelo LLM es intercambiable).

## Consecuencias
- Inversión en formación del equipo: 2 sprints.
- Comprometidos con Python como lenguaje principal del enjambre.
- Revisión de esta decisión en 12 meses.
```

Technology Radar: el mapa de capas como proceso continuo

El **Ingeniero Senior** implementa el mapa de capas como un proceso de actualización continua, similar al Technology Radar de ThoughtWorks, pero adaptado al ecosistema de IA agéntica:

```python
from dataclasses import dataclass, field
from enum import Enum
from datetime import datetime

class RadarRing(Enum):
    ADOPT  = "adopt"    # Usar en producción — probado y recomendado
    TRIAL  = "trial"    # Probar en proyectos piloto — prometedor
    ASSESS = "assess"   # Evaluar — interesante pero no maduro aún
    HOLD   = "hold"     # No adoptar nuevos proyectos — en declive o
problemático

class EcosystemLayer(Enum):
```

```
    MODELS        = "models"
    ORCHESTRATION = "orchestration"
    TOOLS         = "tools"
    GOVERNANCE    = "governance"

@dataclass
class TechRadarEntry:
    name:        str
    layer:       EcosystemLayer
    ring:        RadarRing
    rationale:   str
    answers_q1:  bool    # ¿Resuelve un problema real que tenemos?
    answers_q2:  bool    # ¿Es cimiento o moda?
    answers_q3:  bool    # ¿Aumenta nuestra opcionalidad?
    updated_at:  datetime = field(default_factory=datetime.utcnow)

    @property
    def adoption_score(self) -> int:
        """Score 0-3 basado en las 3 preguntas-brújula."""
       return sum([self.answers_q1, self.answers_q2, self.answers_q3])

    @property
    def recommended(self) -> bool:
        return self.ring == RadarRing.ADOPT and self.adoption_score >=
        2


class AIEcosystemRadar:
    """Radar del ecosistema IA del equipo: filtra el ruido con
    criterio."""
    def __init__(self):
        self._entries: list[TechRadarEntry] = []

    def add(self, entry: TechRadarEntry) -> None:
        self._entries.append(entry)

    def recommended_for_layer(self, layer: EcosystemLayer) ->
    list[TechRadarEntry]:
        return [e for e in self._entries
                if e.layer == layer and e.recommended]

    def hold_warnings(self) -> list[TechRadarEntry]:
        """Tecnologías en HOLD que podrían estar aún en uso en el
        equipo."""
        return [e for e in self._entries if e.ring == RadarRing.HOLD]
```

```
    def snapshot(self) -> dict:
        from collections import Counter
        ring_counts = Counter(e.ring.value for e in self._entries)
        return {
            "total": len(self._entries),
            "by_ring": dict(ring_counts),
            "recommended_count": sum(1 for e in self._entries if
            e.recommended),
        }
```

Estrategia de QA: testing de las decisiones de arquitectura

El **QA Senior** diseña pruebas que verifican que las decisiones capturadas en los ADRs se implementan efectivamente en el código, que la arquitectura real coincide con la arquitectura decidida:

```
def test_adr_007_langgraph_is_the_only_orchestration_framework() ->
None:
    """
    ADR-007: LangGraph es el único framework de orquestación aprobado.
    Detecta importaciones de frameworks no aprobados en el enjambre.
    """
    import ast
    import pathlib

    prohibited = {"autogen", "crewai", "langchain_core.runnables"}
    swarm_files = pathlib.Path("src/swarm").rglob("*.py")

    for path in swarm_files:
        tree = ast.parse(path.read_text())
        for node in ast.walk(tree):
            if isinstance(node, (ast.Import, ast.ImportFrom)):
                module = getattr(node, "module", "") or ""
                for pkg in prohibited:
                    assert pkg not in module, (
                        f"Framework no aprobado '{pkg}' encontrado en
                        {path}. "
                        f"Ver ADR-007: solo LangGraph está aprobado."
                    )

def test_radar_has_no_adopt_entries_in_hold() -> None:
    """Ninguna tecnología puede estar en ADOPT y en HOLD
    simultáneamente."""
    adopt_names = {e.name for e in
radar.recommended_for_layer(EcosystemLayer.ORCHESTRATION)}
```

```
    hold_names  = {e.name for e in radar.hold_warnings()}
    conflict    = adopt_names & hold_names
    assert not conflict, f"Conflicto ADOPT/HOLD detectado: {conflict}"
```

Perspectiva ejecutiva

■ PISTA EJECUTIVA

Visión del Product Owner: el radar como herramienta de roadmap

El **Product Owner** utiliza el AI Ecosystem Radar como una herramienta de comunicación con el equipo de ingeniería y con los stakeholders: qué tecnologías están en adopción plena, qué se está pilotando, qué se está evaluando y qué no debe adoptarse. Esta visibilidad compartida previene el antipatrón del *stealth adoption*: cuando un ingeniero adopta un framework "para ver qué tal funciona" sin que el P.O. sepa que el stack está cambiando.

El Radar tiene una cadencia de actualización trimestral: suficiente para reflejar la evolución del ecosistema sin generar el ruido de actualizaciones semanales. Cada entrada nueva al radar responde las tres preguntas-brújula antes de ser admitida, y si no puede responderlas todas afirmativamente, entra en ASSESS, no en TRIAL ni en ADOPT.

Regla de oro del P.O.: ninguna tecnología entra en el sprint sin estar en el Radar y sin un ADR aprobado por el equipo.

Visión del Scrum Master: el sprint de arquitectura trimestral

El **Scrum Master** facilita, una vez por trimestre, un **sprint de arquitectura** de una semana: el equipo pausa el desarrollo de features para revisar el Radar, actualizar los ADRs, y resolver la deuda de adopción acumulada, esas herramientas que entraron "provisionalmente" al stack y nunca fueron evaluadas formalmente.

Esta práctica previene la entropía arquitectónica: la tendencia de los sistemas a acumular más dependencias de las que se usan, más

frameworks de los que se entienden, y más abstracciones de las que aportan valor. El sprint de arquitectura es el equivalente, en el plano técnico, de la limpieza anual de una organización: doloroso si se ha pospuesto mucho, pero necesario para mantener la capacidad de movimiento.

Versión académica alternativa (mayor densidad formal)

La saturación epistémica que caracteriza el ecosistema actual de inteligencia artificial puede analizarse a través de la teoría de la racionalidad acotada de Simon (1955), según la cual los agentes cognitivos (humanos u organizacionales) no maximizan utilidad en sentido estricto sino que *satisfacen*: adoptan la primera solución que supera un umbral de aceptabilidad. En entornos de alta saturación informativa, este mecanismo de *satisficing* degenera en adopción por prominencia: se adopta lo que más ruido genera, no lo que mejor resuelve el problema. Las tres preguntas-brújula propuestas en este capítulo pueden interpretarse como un mecanismo de restauración de la racionalidad acotada funcional: establecen umbrales de aceptabilidad explícitos (problema real propio, cimiento vs. moda, opcionalidad futura) que previenen la adopción por prominencia. Desde la perspectiva de la arquitectura de software, este marco es coherente con los principios de *lean thinking* (Womack & Jones, 2003): eliminar el desperdicio (adoption churn) y preservar la optionality (decidir lo más tarde posible, con la máxima información disponible). El ADR formaliza este principio en un artefacto documentado y revisable, convirtiendo la decisión arquitectónica de un evento puntual en un proceso continuo de aprendizaje organizacional.

Lo esencial del capítulo 18

- El riesgo en la IA es la saturación epistémica, no la ignorancia: el exceso de opciones que hace que cada novedad parezca simultáneamente imprescindible y reemplazable.
- Las tres preguntas-brújula (¿resuelve un problema que tengo?, ¿es cimiento o moda?, ¿aumenta mi opcionalidad?) son el filtro necesario antes de cualquier adopción.
- El mapa de capas (modelos, orquestación, herramientas, gobernanza) convierte el vértigo en una pregunta manejable: ¿es mejor que lo que ya tengo en esa capa?
- El ADR institucionaliza las decisiones de adopción en artefactos documentados y revisables; el Technology Radar mantiene el mapa actualizado con cadencia trimestral.
- La disciplina más valiosa es aprender a no adoptar: decir no a lo que no resuelve un problema propio, aunque genere presión normativa intensa.
- El P.O. usa el Radar como herramienta de roadmap y comunicación con stakeholders; el Scrum Master facilita el sprint de arquitectura trimestral para resolver la deuda de adopción acumulada.

Capítulo 19: Ideas para generar ingresos con agentes de IA y multiagentes

El dinero no está en construir agentes, sino en resolver con ellos un problema por el que alguien ya paga.

El error de partir de la tecnología

La tentación más común al buscar ingresos con IA es empezar por la tecnología disponible: "tengo agentes, ¿qué vendo?". Es el orden equivocado, y la historia del software está llena de empresas que construyeron soluciones técnicamente brillantes para problemas que nadie pagaba por resolver. El punto de partida correcto es el inverso: **se empieza por el problema, luego se determina si un enjambre de agentes es el instrumento adecuado para resolverlo**.

La pregunta correcta es "¿por qué dolor está alguien pagando hoy, y podría mi enjambre resolverlo mejor, más barato o más rápido?", no "¿qué puede hacer mi agente?" Esta inversión de orden no es trivial. Impone la disciplina de salir al mercado antes de construir, de validar la disposición a pagar antes de automatizar, y de diseñar la solución desde el problema del cliente, no desde la comodidad de la tecnología disponible. Christensen (1997) lo documentó con precisión en el contexto de las tecnologías disruptivas: las innovaciones que crean mercado no emergen de mejorar lo existente sino de resolver problemas que los incumbentes ignoraban porque sus clientes actuales no los padecían.

Cuatro arquetipos de monetización: defensibilidad como criterio central

Figura 8: Cuatro arquetipos: Solo sustitución es frágil. Amplificación es defensible.

A partir del principio anterior, emergen cuatro patrones de negocio diferenciados. Lo que los distingue es, más que el mecanismo de generación de ingresos, su **defensibilidad a largo plazo**: la capacidad de mantener el margen cuando los modelos se abaraten y los competidores repliquen la solución.

Arquetipo 1: Automatización de lo tedioso (ahorro de costo)

Propuesta de valor: se vende el tiempo recuperado y el error reducido en tareas repetitivas, triage de correos, actualización de documentación, generación de reportes rutinarios, monitoreo de alertas, extracción de datos de formularios. El enjambre hace en minutos lo que el equipo haría en horas, a una fracción del costo.

Defensibilidad: BAJA. Este es el arquetipo más fácil de construir y el más difícil de defender. La ecuación económica converge inexorablemente: a medida que el costo de los modelos decrece siguiendo la ley de Moore aplicada al cómputo de IA (como la curva de precio de GPT-4 a GPT-3.5 ilustró), el margen del proveedor de automatización se comprime. Amazon, Google y Microsoft ya están en este espacio con soluciones nativas. El competidor que llega en 18-24 meses puede replicar la solución al mismo costo o menor. Sin diferenciación basada en datos propietarios o integración profunda con el flujo del cliente, este arquetipo es un negocio de temporada, no una empresa.

Cuándo tiene sentido: como punto de entrada para un cliente, no como modelo de negocio definitivo. Es el caballo de Troya que, una vez dentro, permite construir los arquetipos de mayor defensibilidad.

Arquetipo 2: Amplificación del experto (apalancamiento)

Propuesta de valor: no se reemplaza al profesional, se multiplica su capacidad. Un abogado con un enjambre de soporte puede atender 10x más clientes con la misma calidad. Un médico con un agente de análisis de imágenes puede revisar el doble de casos en la misma jornada. Un arquitecto con un enjambre de documentación puede gestionar proyectos que antes requerían un equipo de cinco.

Defensibilidad: ALTA. El modelo económico es robusto: la escasez de expertise del dominio es mayor que el costo de los modelos de IA. El enjambre no reemplaza al experto, requiere al experto para funcionar bien. Esto crea una barrera de entrada doble: necesitas tanto la tecnología como el acceso al experto. Brynjolfsson y McAfee (2014) documentaron este fenómeno como el "racing with machines" frente al "racing against machines": los trabajadores que se apalancan en la IA son más productivos, no obsoletos.

Economía de ejemplo: abogado solo → $40k/año ingresos → abogado + enjambre → $100k/año (2.5x margen sobre el costo de la IA). Médico radiólogo + enjambre de análisis de imágenes → capacidad de análisis 3x mayor sin contratar más médicos.

Arquetipo 3: Acceso a lo inaccesible (creación de mercado)

Propuesta de valor: servicios antes reservados a quienes podían pagar a un experto humano (consultoría legal, asesoría financiera personalizada, coaching ejecutivo, diagnóstico médico especializado) se vuelven accesibles a un mercado masivo mediante agentes que operan al costo marginal de la inferencia.

Defensibilidad: ALTA con matices. El TAM (*Total Addressable Market*) se expande más de 100x porque el precio cae de $5.000/año (consultoría experta) a $50/mes (SaaS agéntico). El margen bruto en SaaS bien construido supera el 80%. La compresión es lenta (más de 10 años) porque el mercado crece más rápido que la competencia. La vulnerabilidad es la comoditización si la diferenciación se reduce al modelo subyacente: cualquier competidor con acceso al mismo modelo puede replicar la oferta. La defensa correcta es la acumulación de datos propietarios del dominio y la integración profunda con los flujos del cliente.

Ejemplo histórico de referencia: la calculadora financiera democratizó el análisis financiero básico sin eliminar a los asesores financieros avanzados. El agente de IA sigue el mismo patrón: democratiza el acceso sin eliminar la expertise de primer nivel.

Arquetipo 4: Inteligencia continua (suscripción)

Propuesta de valor: monitoreo permanente y análisis continuo de señales que ningún humano podría sostener por costo de oportunidad, vigilancia de competidores en tiempo real, monitoreo de riesgos regulatorios, tracking de tendencias de mercado, detección de anomalías en sistemas de producción. Se vende *vigilancia incansable*, no un entregable.

Defensibilidad: MEDIA-ALTA. El modelo de suscripción genera fidelización porque el costo de cambio es alto: los clientes integran las alertas del enjambre en sus procesos de decisión, y migrar requiere reconfigurar esos procesos. El riesgo es la comoditización si el monitoreo se automatiza completamente y los competidores igualan la cobertura. La defensa es la especialización profunda en el dominio: un enjambre que monitorea el sector farmacéutico con precisión regulatoria es más valioso que uno genérico.

El foso defensivo: por qué el modelo no es suficiente

Una advertencia estratégica de primer orden, antes de cerrar este análisis. Como los modelos de IA son un recurso convergente hacia la commoditización (cf. Vol. 2, cap. 4) (Claude, GPT, Gemini y Llama se acercan en capacidad con cada iteración) construir un negocio cuya única ventaja competitiva es "tengo acceso a un buen modelo" es edificar sobre arena. El foso defensivo durable es lo que rodea al modelo, no el modelo:

- **Datos propietarios**: los datos específicos del dominio y del cliente que el enjambre ha procesado y que ningún competidor tiene acceso.

- **Integración profunda**: la conexión del enjambre con los flujos de trabajo del cliente (su ERP, su CRM, sus sistemas de compliance) que tarda meses en construirse y que ningún recién llegado puede replicar en semanas.
- **Confianza acumulada**: la disposición del cliente a otorgar al enjambre acceso a sus datos sensibles, construida a través de historial de aciertos y relación.
- **Conocimiento del dominio**: la expertise específica del sector que hace que las respuestas del enjambre sean precisas donde un enjambre genérico sería impreciso.

Porter (1985) identificó estos elementos como fuentes de ventaja competitiva sostenible. Aplicados al contexto agéntico, confirman que la defensa no está en la IA per se, sino en los activos que la IA hace más valiosos.

De la idea al ingreso: validar barato con el MVP agéntico

La disciplina del MVP (Vol. 3, cap. 22) gobierna también la generación de ingresos con agentes. Antes de construir el enjambre perfecto, valida que alguien paga por el resultado que ese enjambre producirá. El **piloto manual agéntico** es el instrumento: durante las primeras semanas, "el agente" es en realidad un humano que ejecuta el proceso manualmente, entregando el mismo output que el enjambre automatizado entregaría. Este piloto prueba la disposición a pagar sin el costo de la construcción tecnológica.

Solo cuando el dolor y el pago están confirmados, se justifica automatizar a escala. Generar ingresos con agentes, en suma, es menos un problema de IA que un problema de negocio bien planteado: encuentra el dolor, valida el pago, construye el foso.

Perspectiva técnica avanzada

► PISTA TÉCNICA

Arquitectura de costos: modelado de unit economics por arquetipo

Desde la perspectiva del **Arquitecto Senior**, cada arquetipo requiere una arquitectura de costos distinta. El error técnico más común es tratar el costo de inferencia como marginal y no tenerlo en cuenta en el diseño del sistema; cuando el volumen escala, el costo de tokens puede destruir el margen.

```
from dataclasses import dataclass
from enum import Enum

class Archetype(Enum):
    TEDIOUS_AUTOMATION  = "tedious_automation"
    EXPERT_AMPLIFICATION = "expert_amplification"
    MARKET_CREATION     = "market_creation"
    CONTINUOUS_INTEL    = "continuous_intelligence"

@dataclass
class UnitEconomicsModel:
    """
    Modelo de unit economics por arquetipo de monetización agéntica.
    Permite calcular el margen bruto y el punto de quiebre de
rentabilidad.
    """
    archetype:              Archetype
    revenue_per_unit_usd:   float    # Ingresos por unidad
(cliente/mes o transacción)
    llm_cost_per_unit_usd:  float    # Costo de tokens LLM por unidad
    infra_cost_per_unit_usd: float   # Costo de infraestructura
(compute, storage)
    human_review_cost_usd:  float    # Costo de revisión humana (si
aplica)
    cac_usd:                float    # Costo de adquisición del cliente
    churn_monthly:          float    # Tasa de churn mensual (0-1)

    @property
    def gross_margin(self) -> float:
        variable_cost = (self.llm_cost_per_unit_usd
                         + self.infra_cost_per_unit_usd
                         + self.human_review_cost_usd)
        if self.revenue_per_unit_usd == 0:
            return 0.0
        return round((self.revenue_per_unit_usd - variable_cost)
                     / self.revenue_per_unit_usd, 4)
```

```
    @property
    def ltv_usd(self) -> float:
        """Lifetime Value = (Margen mensual) / Churn rate mensual."""
        if self.churn_monthly == 0:
            return float("inf")
        monthly_margin = self.revenue_per_unit_usd * self.gross_margin
        return round(monthly_margin / self.churn_monthly, 2)

    @property
    def ltv_cac_ratio(self) -> float:
        """LTV/CAC ratio. > 3x: negocio saludable. < 1x: negocio
        insostenible."""
        if self.cac_usd == 0:
            return float("inf")
        return round(self.ltv_usd / self.cac_usd, 2)

    def sustainability_assessment(self) -> str:
        ratio = self.ltv_cac_ratio
        if ratio >= 3.0:   return f"HEALTHY (LTV/CAC={ratio:.1f}x)"
        if ratio >= 1.0:   return f"MARGINAL (LTV/CAC={ratio:.1f}x) —
        optimizar costos"
       return f"UNSUSTAINABLE (LTV/CAC={ratio:.1f}x) — revisar modelo"


# Ejemplo: modelado comparativo de arquetipos
def compare_archetypes() -> list[dict]:
    models = [
        UnitEconomicsModel(
            archetype=Archetype.TEDIOUS_AUTOMATION,
            revenue_per_unit_usd=200,    # $200/mes por cliente
            llm_cost_per_unit_usd=80,    # alto volumen de tokens
            infra_cost_per_unit_usd=40,
            human_review_cost_usd=0,
            cac_usd=500,
           churn_monthly=0.08,         # 8% churn — bajo switching cost
        ),
        UnitEconomicsModel(
            archetype=Archetype.EXPERT_AMPLIFICATION,
            revenue_per_unit_usd=500,    # $500/mes por experto
amplificado
            llm_cost_per_unit_usd=50,
            infra_cost_per_unit_usd=20,
           human_review_cost_usd=30,    # el experto revisa las salidas
            cac_usd=2000,
           churn_monthly=0.02,         # 2% churn — alto switching cost
```

```
        ),
        UnitEconomicsModel(
            archetype=Archetype.MARKET_CREATION,
            revenue_per_unit_usd=50,      # $50/mes SaaS masivo
            llm_cost_per_unit_usd=3,      # bajo costo por optimización
            infra_cost_per_unit_usd=2,
            human_review_cost_usd=0,
            cac_usd=30,                   # bajo CAC por PLG
            churn_monthly=0.05,
        ),
        UnitEconomicsModel(
            archetype=Archetype.CONTINUOUS_INTEL,
           revenue_per_unit_usd=1500,   # $1500/mes monitoreo continuo
enterprise
            llm_cost_per_unit_usd=200,
            infra_cost_per_unit_usd=100,
            human_review_cost_usd=50,
            cac_usd=8000,
            churn_monthly=0.015,          # 1.5% churn — muy alto
switching cost
        ),
    ]
    return [
        {
            "archetype":    m.archetype.value,
            "gross_margin": f"{m.gross_margin:.1%}",
            "ltv_cac":      f"{m.ltv_cac_ratio:.1f}x",
            "verdict":      m.sustainability_assessment(),
        }
        for m in models
    ]
```

Arquitectura del MVP agéntico: el piloto manual primero

El **Ingeniero Senior** implementa el MVP agéntico en dos fases explícitas. La Fase 1 es el piloto manual: el "agente" es un humano siguiendo un proceso documentado. La Fase 2 es la automatización progresiva, solo después de confirmar la disposición a pagar:

```
from enum import Enum
from dataclasses import dataclass

class MVPPhase(Enum):
    MANUAL_PILOT    = "manual_pilot"    # Humano ejecuta el proceso
    SEMI_AUTOMATED  = "semi_automated"  # Agente + revisión humana
```

```
    FULL_AUTOMATED  = "full_automated"  # Agente autónomo con
guardrails

@dataclass
class AgenticMVP:
    name:        str
    archetype:   Archetype
    phase:       MVPPhase
    wtp_validated: bool    # ¿Willingness To Pay validada con usuarios
reales?
    pain_validated: bool   # ¿Dolor validado con ≥5 entrevistas de
usuario?

    def ready_to_automate(self) -> bool:
        """Solo automatizar cuando dolor Y pago están validados."""
        return self.wtp_validated and self.pain_validated

    def next_phase(self) -> MVPPhase:
        if not self.ready_to_automate():
            return self.phase   # No avanzar hasta validar
        phases = [MVPPhase.MANUAL_PILOT, MVPPhase.SEMI_AUTOMATED,
MVPPhase.FULL_AUTOMATED]
        idx = phases.index(self.phase)
        return phases[min(idx + 1, len(phases) - 1)]
```

Estrategia de QA: testing del foso defensivo

El **QA Senior** diseña pruebas que verifican que los mecanismos del foso defensivo están implementados correctamente, especialmente la no-portabilidad de los datos propietarios del cliente:

```
def test_proprietary_data_not_accessible_without_authentication() ->
None:
    """Los datos propietarios del cliente son el foso — deben estar
    protegidos."""
    response = client.get("/api/client-data/proprietary", headers={})
    assert response.status_code == 401, \
        "Datos propietarios accesibles sin autenticación: foso
        comprometido."

def test_unit_economics_gross_margin_above_threshold() -> None:
    """El margen bruto del arquetipo debe superar el 50% para ser
    defensible."""
    model = UnitEconomicsModel(
        archetype=Archetype.EXPERT_AMPLIFICATION,
```

```
    revenue_per_unit_usd=500, llm_cost_per_unit_usd=50,
    infra_cost_per_unit_usd=20, human_review_cost_usd=30,
    cac_usd=2000, churn_monthly=0.02
)
assert model.gross_margin >= 0.50, \
    f"Margen bruto {model.gross_margin:.1%} < 50%: modelo no
    defensible."
```

Perspectiva ejecutiva

■ PISTA EJECUTIVA

Visión del Product Owner: el roadmap de monetización por fases

El **Product Owner** gestiona la generación de ingresos con agentes como un **roadmap de monetización por fases**, no como un lanzamiento único. Las fases corresponden a los arquetipos ordenados por defensibilidad creciente:

Fase 1: Entrada al mercado (meses 1-6): Arquetipo de automatización de lo tedioso. Objetivo: adquirir los primeros 50 clientes, validar el proceso operacional del enjambre y construir el historial de aciertos que fundamenta la confianza. Margen bajo, pero datos que se acumulan.

Fase 2: Construcción del foso (meses 6-18): Migrar los mejores clientes de la Fase 1 al arquetipo de amplificación del experto. Los datos acumulados en la Fase 1 son el insumo que hace que el enjambre sea experto en el dominio del cliente. El foso empieza a construirse.

Fase 3: Escala defensible (meses 18+): Introducir inteligencia continua o creación de mercado basados en los datos propietarios y la integración profunda construidos en las fases anteriores. En este punto, un recién llegado no puede replicar la oferta sin años de acumulación de datos y relación.

Métrica de P.O.: LTV/CAC ratio por arquetipo. Objetivo: > 3x en Fases 2 y 3. Un ratio < 1x en cualquier fase activa es una señal de que el modelo de negocio necesita revisión antes de escalar.

Visión del Scrum Master: el sprint de validación antes del sprint de construcción

El **Scrum Master** institucionaliza una regla que protege el equipo de la trampa del desarrollo prematuro: **antes de que el equipo construya un enjambre nuevo, debe existir al menos una ronda de validación del problema con usuarios reales**. Esta regla se operacionaliza como un "sprint de validación" que precede al primer sprint de desarrollo:

- Semana 1: 5 entrevistas con usuarios potenciales del dominio.
- Semana 2: piloto manual del proceso (sin código agéntico).
- Criterio de avance: ≥ 3 de 5 entrevistados expresan disposición a pagar por el resultado del piloto.

Si el criterio de avance no se cumple, el enjambre no se construye, sin importar cuán técnicamente elegante sería la solución. Este protocolo, inspirado en las prácticas de Lean Startup (Ries, 2011), evita el error más costoso del ecosistema agéntico: construir un enjambre brillante para un mercado que no existe.

Versión académica alternativa (mayor densidad formal)

> La generación de ingresos con sistemas de inteligencia artificial multiagente puede analizarse a través del marco de la economía de plataformas y las externalidades de red (Rochet & Tirole, 2003). En los arquetipos de alta defensibilidad (amplificación del experto e inteligencia continua), los efectos de red operan de manera indirecta: a mayor número de interacciones del enjambre con el dominio del cliente, mayor la precisión de sus respuestas, mayor el valor percibido por el cliente, menor la propensión al churn. Este mecanismo, denominado "data network effect" (Schrage, 2017), constituye una barrera de entrada que crece con el tiempo: un competidor que entra

con el mismo modelo de IA pero sin los datos propietarios acumulados no puede igualar la calidad del incumbente, independientemente de las capacidades del modelo subyacente. La teoría de la ventaja competitiva sostenible de Porter (1985) es aplicable con precisión al contexto agéntico: la ventaja no reside en la tecnología per se (un factor igualable por cualquier competidor con acceso a los mismos proveedores de modelos) sino en los activos complementarios que la tecnología hace más valiosos: datos, integración, confianza y conocimiento del dominio. La investigación de Brynjolfsson y McAfee (2014) confirma empíricamente que las organizaciones que derivan mayor valor de la IA son aquellas que la combinan con capacidades organizacionales difícilmente replicables, no las que simplemente tienen acceso a modelos superiores.

Lo esencial del capítulo 19

- El punto de partida correcto es el dolor del cliente (por el que alguien ya paga), no la tecnología disponible.
- Los cuatro arquetipos de monetización tienen defensibilidades radicalmente distintas: automatización de lo tedioso (BAJA), amplificación del experto (ALTA), acceso a lo inaccesible (ALTA), inteligencia continua (MEDIA-ALTA).
- El foso defensivo son los datos propietarios, la integración profunda, la confianza acumulada y el conocimiento del dominio, no el modelo (commodity convergente).
- Los unit economics deben modelarse por arquetipo: LTV/CAC $>$ 3x es el umbral de negocio saludable; $<$ 1x es señal de revisión urgente del modelo.
- La estrategia de monetización óptima es un roadmap por fases: entrar con automatización de lo tedioso para acumular datos, migrar a amplificación del experto para construir el foso, escalar con inteligencia continua cuando el foso es sólido.

- El P.O. gestiona el LTV/CAC ratio por arquetipo; el Scrum Master institucionaliza el sprint de validación antes del sprint de construcción.

Capítulo 20: DevSecOps y Permisos para Agentes

Un permiso bien diseñado no frena al agente: lo encauza. La diferencia entre una valla y una jaula es el criterio con que fue construida.

DevSecOps no es un obstáculo, es un contrato

Para muchos desarrolladores, los permisos y roles de DevSecOps se perciben como fricción: una valla que ralentiza el progreso. Conviene revisar esa percepción, y hacerlo con rigor: la valla de permisos no existe para estorbar, sino para **codificar la confianza en forma de contrato verificable**. Cuando se incluye a los agentes en el flujo de DevSecOps, esta valla deja de ser un detalle operativo y se convierte en la línea de defensa central, puesto que un agente con permisos excesivos es cualitativamente más peligroso que un humano con los mismos permisos (cf. cap. 29).

La razón de esta asimetría es estructural: el humano que abusa de sus privilegios actúa con intención, velocidad humana y un registro cognitivo de consecuencias. El agente que desborda sus límites actúa sin intención pero a velocidad de máquina, replicando el error miles de veces antes de que cualquier sistema de monitorización alcance a registrar la anomalía. He aquí, por tanto, la primera lección: la disciplina de DevSecOps no se relaja en la era agéntica; se intensifica.

> **Versión académica:** La incorporación de sistemas de IA agénticos en los flujos de integración y entrega continua

(CI/CD) introduce vectores de riesgo cualitativamente distintos de los asociados a actores humanos. La velocidad de actuación de un agente comprometido o mal configurado excede en varios órdenes de magnitud la capacidad de respuesta de los mecanismos de control tradicionales, lo que eleva el requisito de contención a un rango de criticidad superior.

Por qué la valla está bien: tres fundamentos inviolables

Tres razones justifican el rigor de permisos y roles, y todas ellas se intensifican cuando los actores son agentes:

- **Contención del daño.** El principio de privilegio mínimo (*least privilege*) limita el radio de explosión de cualquier error, humano o agéntico. Un agente que solo puede leer no puede escribir ni borrar; un agente que solo puede escribir en un directorio específico no puede comprometer el sistema de archivos completo. La superficie de ataque potencial se expresa mediante: `superficie = n_agentes × permisos_promedio × tiempo_de_vida_del_permiso`. Reducir cualquiera de los tres factores reduce el riesgo de manera multiplicativa.
- **Trazabilidad.** Los roles definen quién (o qué agente) puede hacer qué, haciendo auditable cada acción. En un sistema sin roles definidos, una anomalía descubierta en los logs puede no tener origen atribuible; en un sistema con roles bien diseñados, cada acción lleva la firma del agente que la ejecutó, el rol que la autorizó y el momento en que ocurrió. Esta trazabilidad no es un lujo burocrático: es el fundamento de la forensia digital en sistemas agénticos.
- **Reversibilidad.** Una arquitectura de permisos clara permite revocar el acceso de un agente comprometido sin desmontar todo el sistema. Si las credenciales son de corta vida y revocables centralmente, neutralizar a un agente comprometido es una operación de segundos; si las credenciales son permanentes y distribuidas, es una operación de días.

El modelo de confianza cero aplicado a agentes

El marco de referencia más adecuado para comprender los permisos agénticos es la *Zero Trust Architecture* (ZTA), formalizada por el NIST en la publicación especial 800-207 (Stafford, 2020). El principio rector es deceptivamente simple: "nunca confiar, siempre verificar" (*never trust, always verify*). Aplicado a agentes, este principio se reinterpreta con matices importantes.

En la arquitectura de confianza cero para enjambres, resulta significativo distinguir dos componentes funcionales. El **Policy Decision Point** (PDP) evalúa si una solicitud de acceso es legítima, considerando la identidad del agente, el recurso solicitado, el contexto de la tarea y el estado del sistema. El **Policy Enforcement Point** (PEP) intercepta cada solicitud del agente y la remite al PDP antes de permitir o denegar la acción. En un enjambre bien diseñado, ningún agente accede directamente a ningún recurso: toda acción pasa por este ciclo de decisión.

La diferencia con los sistemas de permisos clásicos es sustancial. En los sistemas RBAC (*Role-Based Access Control*) tradicionales, el rol se asigna al usuario una vez y permanece hasta que un administrador lo revoca. En los sistemas ABAC (*Attribute-Based Access Control*), el acceso se evalúa en función de atributos del sujeto, el recurso y el entorno. Para agentes, el modelo más adecuado es PBAC (*Policy-Based Access Control*): el acceso se decide mediante políticas expresadas como reglas condicionales que tienen en cuenta el riesgo de la acción concreta, no solo la identidad del agente.

Modelo	Granularidad	Dinamismo	Adecuación para agentes
RBAC	Por rol	Baja (cambio manual)	Insuficiente
ABAC	Por atributo	Media (evaluación en tiempo real)	Adecuada
PBAC	Por política	Alta (reglas condicionales dinámicas)	Óptima

Modelo	Granularidad	Dinamismo	Adecuación para agentes
ZTA/JIT	Por acción y tiempo	Máxima (just-in-time, efímero)	Excelente

El ciclo de vida del permiso agéntico

Una de las contribuciones más importantes que DevSecOps puede aportar a la arquitectura de enjambres es la conceptualización del permiso agéntico como un ciclo de vida, no como un estado binario (tiene/no tiene). El ciclo comprende cinco fases que deben implementarse explícitamente en la infraestructura:

1. **Solicitud**: el agente declara qué necesita hacer y por qué, expresado en lenguaje de política formal. El PDP recibe esta solicitud junto con el contexto de la tarea en curso.
2. **Evaluación de riesgo**: el PDP computa un score de riesgo basado en quién solicita, qué solicita, sobre qué recurso, en qué contexto y en qué momento del flujo de trabajo. Esta evaluación puede incorporar modelos de riesgo probabilístico.
3. **Otorgamiento temporal** (*just-in-time access*): si el PDP aprueba, el permiso se otorga con una vida útil mínima (típicamente el tiempo estimado para completar la tarea más un margen de seguridad del 20%). Expirado ese tiempo, el permiso se revoca automáticamente, sin necesidad de acción humana.
4. **Ejecución monitorizada**: durante la ventana de permiso activo, el PEP registra cada uso del permiso con timestamp, resultado y cualquier anomalía de patrón (por ejemplo: el agente que solicitó permiso de lectura intentó una escritura).
5. **Revocación y auditoría**: al expirar o revocarse el permiso, el registro queda inmutable en el log de auditoría. Cualquier desviación del comportamiento esperado genera una alerta y, dependiendo del nivel de criticidad, puede provocar la cuarentena inmediata del agente.

Cómo mejorar DevSecOps para la era agéntica

Los modelos de permisos clásicos fueron diseñados para humanos, cuyo comportamiento es relativamente estable y cuya velocidad de acción es predecible. Los agentes imponen tres evoluciones estructurales necesarias, no opcionales:

Primera evolución: permisos dinámicos y de corta vida. En lugar de conceder a un agente acceso permanente a una base de datos, se le otorga acceso exactamente cuando lo necesita, por el tiempo mínimo necesario, y se revoca automáticamente. Esta práctica (denominada *just-in-time access* (JIT)) reduce la ventana de exposición de semanas o meses a minutos. Rababah et al. (2024) demuestran que la implementación de JIT en sistemas de IA agénticos reduce el impacto potencial de compromisos en un 73% respecto a los modelos de acceso permanente.

Segunda evolución: permisos contextuales. El mismo agente puede merecer más o menos acceso según el riesgo de la tarea concreta. Un agente de síntesis de informes necesita permisos de lectura amplia pero ningún permiso de escritura; ese mismo agente ejecutando una tarea de corrección de datos necesita permisos de escritura acotados a registros específicos. La política es "este agente, cuando ejecuta la tarea Y sobre el recurso Z, tiene acceso A", no "este agente tiene acceso X".

Tercera evolución: mediación obligatoria. El agente nunca posee la credencial directamente. En cambio, la solicita a un guardián de secretos (*secrets manager*) que decide si otorgarla, registra el otorgamiento y la revoca cuando expira. Herramientas como HashiCorp Vault o AWS Secrets Manager implementan este patrón. Si el agente es comprometido, el atacante obtiene el agente pero no las credenciales; si las credenciales estuvieran en el agente, el compromiso del agente equivaldría al compromiso de todos los sistemas a los que accede.

Métricas de madurez DevSecOps para enjambres

Un modelo de madurez de cinco niveles permite a las organizaciones situar su práctica actual y definir una hoja de ruta hacia la gestión

óptima de permisos agénticos:

Nivel	Denominación	Criterios clave	Indicadores
1	Inicial	Permisos manuales, sin política formal	Tiempo de revocación > 24h
2	Repetible	RBAC implementado, revisiones periódicas	Permisos revisados trimestralmente
3	Definido	ABAC/PBAC, JIT para agentes críticos	Permisos efímeros en >50% de accesos
4	Gestionado	ZTA completa, PDP/PEP para todos los agentes	Detección de anomalías < 5 min
5	Optimizando	ML en evaluación de riesgo, auto-ajuste de políticas	Falsos positivos < 0.1%

La mayoría de organizaciones que despliegan enjambres en 2024-2025 operan en el nivel 1 o 2. Rahman et al. (2023) documentan en su revisión sistemática de DevSecOps que solo el 12% de organizaciones con prácticas DevSecOps maduras han extendido esas prácticas a sus sistemas de IA, lo que representa una brecha de seguridad sistémica de considerable magnitud.

Un índice cuantificable de autonomía

Los niveles de autonomía del capítulo 1 (Vol. 1) son cualitativos: dicen si un sistema opera en Nivel 0 a 4, no cuánta autonomía real ejerce dentro de ese nivel. Para que un comité de arquitectura pueda fijar un umbral objetivo (y no una impresión) este libro propone el **Índice de Autonomía Operativa (IAO)**, ponderado por el riesgo de cada acción según la escala verde/amarillo/rojo ya introducida:

$$\text{IAO} = \frac{\sum(\text{acciones autónomas} \times \text{peso de riesgo})}{\sum(\text{acciones totales} \times \text{peso de riesgo})}$$

donde los pesos $1/3/10$ corresponden a los niveles verde/amarillo/rojo.

Ponderar por riesgo, y no solo contar acciones, evita el falso positivo más común de este tipo de métricas: un agente que ejecuta miles de acciones verdes sin supervisión pero jamás toca una acción roja sin aprobación puede reportar un 95 % de "autonomía" que no dice nada sobre lo que de verdad importa. El IAO exige que la autonomía se demuestre donde el daño potencial es alto, no donde es barato.

El IAO por sí solo premiaría a un sistema temerario, así que se acompaña de un segundo término, la **tasa de reversión con daño** (τ): la fracción de acciones autónomas que tuvo que revertirse *y* causó un efecto medible antes de revertirse (a diferencia de una reversión limpia, sin consecuencia). El **IAO ajustado** es:

$$\text{IAO}_{\text{ajustado}} = \text{IAO} \times (1 - \tau)$$

Cómo se usa como gate de promoción entre entornos. El registro de dependencias del Vol. 4 (capítulo 5, caso 8) ya exige evidencia para ascender de un entorno a otro; el IAO ajustado es esa evidencia expresada en un número:

Entorno	IAO ajustado mínimo para permanecer	τ máximo tolerado
Desarrollo	Sin umbral, entorno de exploración	Sin umbral
Stage	0.30	5 %
QA	0.50	2 %
Producción	El valor que QA sostuvo durante al menos dos semanas, nunca superior	0.5 %

Estos valores son un punto de partida razonable, no una ley física: cada organización debe calibrarlos con su propio historial de

incidentes, pero **debe fijarlos por escrito antes del primer ascenso a producción**, no negociarlos incidente a incidente.

Auditoría agnóstica del modelo

Medir la autonomía resuelve *cuánto* decide el sistema solo; no resuelve si esa medición sigue siendo válida cuando el modelo detrás del agente cambia (de Claude a GPT-6 Astra, de un proveedor cerrado a Kimi K2.6 en producción propia), que es precisamente el escenario que el Backend Agnóstico (Vol. 2) da por hecho que ocurrirá. Una auditoría de permisos que solo verificó el comportamiento de un modelo no dice nada sobre el comportamiento del siguiente.

La respuesta es lo que este libro llama **auditoría de reproducción cruzada**: en lugar de auditar el modelo, se audita la traza de decisión capturada por el arnés (entrada, contexto, acción tomada, nivel de riesgo asignado, intervención humana si la hubo, el mismo esquema de *spans* de OpenTelemetry del capítulo 11 del Vol. 4), y esa traza se reproduce contra un segundo proveedor sin decirle cuál fue la decisión original. El criterio de aprobación exige que **ambos respeten el mismo cerco de permisos**, no que coincidan en la acción (divergencias razonables son esperables y hasta deseables): si la traza original fue clasificada verde, el segundo proveedor no debe escalarla a una acción roja no autorizada, y viceversa.

Protocolo mínimo de la auditoría:

1. Muestrear trazas de producción estratificadas por color de riesgo (más peso a amarillo y rojo, donde el costo de una discrepancia es mayor).
2. Reproducir cada traza contra al menos un proveedor alternativo al que opera en producción, en un entorno aislado sin efectos secundarios reales.
3. Registrar tres resultados posibles por traza: concordancia de clasificación de riesgo, discrepancia dentro de tolerancia (ambos permanecen en el mismo nivel de aprobación requerido), o discrepancia de gobernanza (un proveedor habría actuado sin la aprobación que el otro sí exige).
4. Solo el tercer caso es un hallazgo de auditoría; los dos primeros son evidencia de portabilidad saludable.

Una organización que aprueba este protocolo trimestralmente no solo cumple con la letra del EU AI Act sobre trazabilidad (capítulo 15); demuestra en la práctica que su arquitectura, y no su proveedor de modelo, es la que sostiene la gobernanza, el compromiso de fondo de este libro entero.

Figura 9: Autonomía graduada: permisos por daño potencial - riesgo bajo, medio y alto

La trama se complica cuando se reconoce que el rigor excesivo también tiene un costo. Una valla demasiado rígida convierte al agente en inútil: si toda acción requiere aprobación humana previa, se pierde exactamente la autonomía que justificaba el agente. El arte de DevSecOps en la era agéntica consiste en **graduar los permisos según el daño potencial**: libertad amplia donde el error es barato y reversible, control estricto donde es caro e irreversible.

Esta gradación puede formalizarse en tres niveles operativos. El nivel **verde** permite acción autónoma donde el daño potencial es bajo y reversible (leer un documento, generar un borrador). El nivel **amarillo** permite la acción pero con registro reforzado y notificación, cuando el daño potencial es medio (enviar un email, modificar una base de datos de prueba). El nivel **rojo** bloquea la acción hasta aprobación humana explícita, cuando el daño potencial es alto e irreversible (ejecutar código en producción, transferir fondos).

La valla, bien diseñada, no es lo opuesto a la autonomía: es lo que la hace operativamente segura.

Perspectiva ejecutiva: el costo de la permisividad

■ **PISTA EJECUTIVA**

El argumento definitivo para invertir en gobernanza de permisos agénticos es económico. Una brecha de seguridad causada por un agente con privilegios excesivos tiene un costo medio de $4.35 millones de dólares en todos los sectores, elevándose a $9.44 millones en el sector financiero (IBM Security, 2023). Este costo incluye detección y contención, recuperación de sistemas, notificación regulatoria, honorarios legales y daño reputacional.

El retorno sobre la inversión en Zero Trust para sistemas agénticos es igualmente claro: las organizaciones que implementan ZTA completa suelen reportar mejoras de dos cifras en tiempo de contención de incidentes y en alcance del daño [*cifras ilustrativas: la magnitud varía por sector y por madurez del sistema; lo que importa para la decisión es que la tendencia es a la baja, y el número exacto lo dá su propia instrumentación*]. Para un enjambre de 20 agentes en producción durante 12 meses, la inversión en una arquitectura de permisos madura (estimada en 3-4 semanas de trabajo de arquitectura de seguridad) tiene un ROI potencial de 12-15x considerando el costo evitado de un incidente de seguridad de mediana gravedad.

Lo esencial del capítulo 20

- La valla de permisos de DevSecOps codifica confianza en un contrato verificable; con agentes, es la defensa central porque su velocidad de acción supera la capacidad humana de monitorización.
- Los modelos de permisos evolucionan de RBAC a PBAC/ZTA: dinámicos, contextuales, con mediación obligatoria y ciclo de vida explícito.
- El modelo de madurez de 5 niveles permite localizar la práctica actual; la mayoría de organizaciones está en nivel 1-2 en 2024-2025.
- La gradación por daño potencial (verde/amarillo/rojo) resuelve el falso dilema jaula-vs-anarquía: libertad donde el error es reversible, control donde es irreversible.
- El ROI de invertir en gobernanza de permisos agénticos es de 12-15x considerando el costo evitado de incidentes.
- El Índice de Autonomía Operativa (IAO), ponderado por riesgo y ajustado por reversiones con daño, convierte el ascenso entre entornos en un umbral numérico fijado por escrito, no en una negociación caso a caso.
- La auditoría de reproducción cruzada verifica que el cerco de permisos se sostiene al cambiar de proveedor de modelo: el hallazgo que importa es la discrepancia de gobernanza, no la discrepancia de respuesta.

Capítulo 21: Cada línea de código como una responsabilidad del agente de IA y los multiagentes

El agente escribe la línea; el humano responde por ella. Esa asimetría es el corazón de todo.

La línea de código nunca es neutral

Conviene grabar un principio que subyace a toda práctica de ingeniería responsable: cada línea de código es una responsabilidad. Es un compromiso (con quien la mantendrá, con quien dependerá de ella, con el sistema que la albergará durante años), no un fragmento neutro. Cuando los agentes de IA empiezan a escribir líneas en masa, este principio no se diluye: se vuelve urgente. Porque un agente puede generar mil líneas en un minuto, pero **la responsabilidad por esas mil líneas no se genera; se asume**, y solo un humano puede asumirla.

La diferencia entre el código escrito por un humano y el código generado por un agente no reside en la corrección sintáctica ni en la funcionalidad inmediata (ambos pueden ser perfectos en esas dimensiones). Reside en la **trazabilidad del razonamiento**: cuando un humano escribe una línea, hay en su mente un modelo de por qué esa solución y no otra, qué invariantes mantiene, qué supuestos asume. Cuando un agente genera esa misma línea, el razonamiento

es opaco por naturaleza; el artefacto existe, pero el proceso que lo produjo no es directamente inspeccionable.

> **Versión académica:** Se constata que la adopción masiva de sistemas de generación de código basados en modelos de lenguaje de gran escala introduce una disociación estructural entre la velocidad de producción de artefactos de software y la capacidad de comprensión y responsabilización humana sobre dichos artefactos. Esta disociación configura lo que podría denominarse *deuda epistemológica agéntica*: una forma de deuda técnica cualitativamente distinta de la clásica, en la que el código es funcionalmente correcto pero cognitivamente opaco para el equipo que lo mantiene.

La asimetría fundamental: velocidad vs. responsabilización

Figura 10: Cuello de botella: 2 min generacion vs 60 min verificacion. Velocidad agéntica distinta a velocidad de entrega.

He aquí la tensión central de la programación asistida por agentes: la **velocidad de generación se ha desacoplado de la velocidad de responsabilización**. El agente produce código a un ritmo que ningún humano puede revisar con la misma profundidad con que lo habría escrito.

La matemática es implacable. Un agente genera 1.000 líneas en 2 minutos. Un senior engineer revisa con profundidad suficiente (comprendiendo lógica, edge cases, implicaciones de seguridad) a razón de 50 líneas por hora. El tiempo de revisión es, por tanto, 20 horas para esas 1.000 líneas. El agente contribuye 2 minutos del proceso total; el humano, 1.200 minutos. **Matemáticamente**: optimizar el agente de 2 minutos a 30 segundos ahorra el 0,1% del tiempo total. Optimizar la revisión humana de 20 horas a 10 horas ahorra el 50%. El cuello de botella es el juicio humano, no la generación.

Esta inversión de la intuición tiene consecuencias estratégicas profundas. Los equipos que invierten masivamente en tooling de generación sin invertir proporcionalmente en infraestructura de revisión (tests automatizados, análisis estático especializado, revisión por pares estructurada) están optimizando el 0,1% mientras descuidan el 99,9%.

Pearce et al. (2022) publicaron en el IEEE Symposium on Security and Privacy un análisis riguroso del código generado por GitHub Copilot: el 40% de los fragmentos de código generados en contextos de seguridad contenían vulnerabilidades que superaban la detección superficial. No eran errores sintácticos obvios; eran vulnerabilidades lógicas que requerían comprensión profunda del contexto de uso para ser detectadas. La implicación es directa: la revisión superficial (leer el código sin comprenderlo) no es revisión; es teatro de seguridad.

Surge así un riesgo silencioso: aceptar líneas que funcionan sin comprender por qué, acumulando un sistema cuya lógica nadie posee del todo. **Esto es diferente a la deuda técnica clásica**: la deuda clásica es "mal hecho pero funciona". La deuda agéntica es "bien hecho por máquina, pero humano no lo entiende". Es deuda epistemológica, no de calidad. Y es más peligrosa, porque el código funciona hasta que no funciona, súbitamente, sin causa obvia.

La crisis epistemológica del código sintético

La distinción más importante que emerge en la era agéntica es la que separa la **deuda técnica clásica** de la **deuda epistemológica agéntica**.

La deuda técnica clásica, formalizada por Ward Cunningham (1992) con su metáfora financiera, describe código que funciona pero está implementado de forma subóptima: las soluciones rápidas tomadas conscientemente con la intención de refactorizar después. Este código es comprensible (el equipo sabe por qué está mal) y la deuda es, al menos en principio, cuantificable y pagable.

La deuda epistemológica agéntica es de naturaleza diferente. El código generado por un agente puede ser perfectamente correcto en términos de funcionalidad, pasar todos los tests, cumplir todos

los criterios de aceptación... y sin embargo, nadie en el equipo comprende por qué funciona. No es "mal hecho pero funciona": es "bien hecho por máquina, pero humano no lo entiende". Es deuda en el plano del conocimiento, no de la calidad técnica.

Lo que resulta menos evidente es que esta deuda es más peligrosa que la clásica, precisamente porque el código funciona (hasta que deja de hacerlo), y cuando falla, no hay modelo mental en el equipo que permita diagnosticar la causa. El sistema colapsa súbitamente, en producción, frente a un dato de entrada que nadie había anticipado porque nadie había razonado sobre los límites del algoritmo.

> **Versión académica:** La deuda epistemológica agéntica representa una categoría ontológicamente distinta dentro de la taxonomía de la deuda técnica. A diferencia de la deuda de implementación (código subóptimo con lógica comprensible) y la deuda de diseño (arquitectura subóptima con decisiones rastreables), la deuda epistemológica se caracteriza por la opacidad del proceso de razonamiento que produjo el artefacto, generando una asimetría entre la calidad observable del código y la capacidad del equipo de mantenerlo responsablemente.

Taxonomía de la responsabilidad agéntica

La responsabilidad sobre el código agéntico no es monolítica: se distribuye en tres dimensiones que deben identificarse y asignarse explícitamente en el proceso de desarrollo:

Responsabilidad de generación. ¿Qué agente generó este código, con qué instrucciones, en qué fecha, usando qué modelo y versión? Esta dimensión es técnicamente tratable: los metadatos de generación pueden anotarse en el commit. Un esquema de anotación propuesto: `@agent-generated: claude-sonnet-5 · task: implement OAuth2 flow · date: 2025-03-14 · risk-level: high`.

Responsabilidad de integración. ¿Qué humano identificado revisó este código, lo comprendió y decidió integrarlo al sistema? Esta es la responsabilidad que ningún agente puede asumir. La aceptación de un pull request que contiene código agéntico sin comprensión real de su funcionamiento no es responsabilidad de integración:

es su simulacro. El EU AI Act (Art. 13-14, Regulación 2024/1689) establece que la responsabilidad por sistemas de IA de alto riesgo recae en el *deployer* humano, no en el modelo; por extensión, la responsabilidad de integración de código agéntico en sistemas críticos recae en el ingeniero que lo aceptó.

Responsabilidad de mantenimiento. ¿Quién responderá cuando este código falle (en seis meses, en dos años, bajo condiciones que nadie anticipó) y quién tendrá el conocimiento necesario para diagnosticar y remediar? Los equipos que aceptan código agéntico sin documentar el razonamiento implícito están transfiriendo esta responsabilidad a un futuro yo que tendrá menos contexto, más presión y un sistema más complejo.

Disciplinas para responsabilizar el código agéntico

Frente a esta realidad, tres disciplinas resultan no solo recomendables sino operativamente necesarias:

Revisión proporcional al riesgo. No toda línea agéntica merece el mismo escrutinio. La que toca lógica de autenticación, manejo de datos personales, transacciones financieras o control de infraestructura exige revisión profunda. La que genera un formatter de texto o calcula una suma merece verificación funcional básica. Esta diferenciación es la única forma de hacer sostenible la revisión en sistemas de generación de código a gran escala.

Comprensión antes que aceptación. La regla de oro, irrenunciable: no integrar código agéntico que no se comprende. "Funciona" no es criterio suficiente. Si nadie en el equipo puede explicar por qué este algoritmo converge, por qué este índice de base de datos es adecuado, por qué este manejo de errores es correcto, entonces nadie podrá arreglarlo cuando falle en producción.

Trazabilidad del origen. Saber qué agente generó qué, cuándo, con qué instrucciones y qué humano lo integró. Kochhar et al. (2024) proponen en ICSE un marco de *accountability* para desarrollo asistido por IA que incluye seis metadatos obligatorios en cada commit que contiene código agéntico: modelo usado, prompt original, criterios de aceptación aplicados, revisor humano identificado, nivel de

riesgo evaluado y trazabilidad hacia el requisito de negocio correspondiente.

El programador como garante, no como mecanógrafo

En consecuencia, el rol del programador se transforma profundamente, pero no se devalúa: asciende. Deja de ser quien teclea cada línea y pasa a ser el **garante de cada línea** (quien decide cuáles merecen confianza, cuáles exigen revisión y cuáles deben rechazarse). Esta es, paradójicamente, una responsabilidad mayor que la de escribir. Escribir una línea exige conocimiento; responder por mil líneas que otro escribió exige criterio, y el criterio es justamente lo que distingue a un ingeniero de un generador de texto.

Esta transformación del rol tiene implicaciones directas para la evaluación del rendimiento. Los equipos que miden productividad en líneas de código generadas están evaluando la velocidad del agente, no la del ingeniero. Los equipos que miden productividad en decisiones de integración fundamentadas (código aceptado con comprensión documentada, código rechazado con justificación técnica) están evaluando la competencia real del ingeniero en la era agéntica.

Perspectiva de QA: la verificación como nueva primera línea

► **PISTA TÉCNICA**

Desde la perspectiva del Senior QA, la proliferación de código agéntico reconfigura completamente la estrategia de calidad. La verificación agéntica requiere dos capas adicionales al enfoque clásico.

La primera es la **verificación de comprensión**: confirmar que el equipo entiende por qué el código funciona, no solo que funciona. Esto puede hacerse mediante revisiones técnicas donde el autor explica el razonamiento del código agéntico (ejercicio que, si resulta imposible, indica que el código no debería haberse integrado).

La segunda es la **verificación de límites**: identificar explícitamente los supuestos del código agéntico y testear su comportamiento en los límites de esos supuestos. Los agentes optimizan para los casos de uso promedio; los QA deben cubrir los casos extremos que el agente no anticipó.

Liu et al. (2023) analizan en NeurIPS la tasa de corrección de código generado por LLMs: mientras que el 85% de los programas generados pasan los tests funcionales básicos, solo el 62% pasan suites de tests más exhaustivas que incluyen edge cases y condiciones de fallo. La brecha del 23% representa exactamente la superficie de riesgo que la revisión superficial deja expuesta.

Lo esencial del capítulo 21

- Cada línea de código es un compromiso; con agentes, el principio se vuelve urgente porque la velocidad de generación se desacopló de la velocidad de responsabilización.
- La deuda epistemológica agéntica es más peligrosa que la técnica clásica: el código funciona hasta que falla súbitamente, sin modelo mental disponible para el diagnóstico.
- La taxonomía de responsabilidad tiene tres dimensiones: generación (¿qué agente?), integración (¿qué humano aceptó?) y mantenimiento (¿quién responderá en el futuro?).
- Las tres disciplinas irrenunciables: revisión proporcional al riesgo, comprensión antes de aceptación, trazabilidad del origen.
- El programador asciende de mecanógrafo a garante; responder por mil líneas exige más criterio que escribir una.

Capítulo 22: Comenzar por MVPs para los agentes de IA y multiagentes

Construir el enjambre perfecto antes de validar el problema es la forma más elegante de fracasar.

El MVP no es una versión pequeña, es una pregunta

Existe un malentendido persistente que conviene disolver: creer que un MVP (Producto Mínimo Viable) es simplemente una versión reducida del producto final. No lo es. Un MVP es, en esencia, **un experimento diseñado para responder la pregunta más arriesgada del proyecto**. Eric Ries (2011) acuñó el concepto en el contexto de startups; Steve Blank (2013) lo elevó a principio epistemológico en Harvard Business Review. La pregunta que un MVP responde rara vez es "¿funciona la tecnología?" (en el contexto de los agentes de IA, casi siempre funciona), sino "¿resuelve esto un problema real, de forma confiable, a un costo viable?".

El MVP agéntico, por tanto, no se mide por cuánto código tiene, ni por cuántos agentes orquesta, sino por **cuánta incertidumbre elimina** con el mínimo de recursos invertidos. Esta distinción no es semántica: es la diferencia entre un equipo que aprende a construir el producto correcto y un equipo que construye elegantemente el producto equivocado.

Versión académica: Desde una perspectiva epistemológica, el Producto Mínimo Viable constituye un instrumento de falsación aplicado al desarrollo de producto: su función es la de someter a prueba empírica la hipótesis más riesgosa del proyecto con el menor dispendio posible de recursos, no la de producir un artefacto reducido del sistema objetivo. En el dominio de los sistemas multi-agente, esta función adquiere una criticidad adicional derivada de la alta variabilidad en los costos de operación en tokens, latencia y coherencia del comportamiento emergente.

Por qué los agentes tientan a saltarse el MVP

Conviene advertir una trampa específica de esta era. Como los agentes hacen que construir sea barato y rápido (un enjambre funcional puede prototipado en días en lugar de meses), la tentación de saltarse el MVP y "construirlo todo de una vez" es mayor que nunca. Sin embargo, esta facilidad es engañosa: es fácil construir, pero sigue siendo igual de difícil construir *lo correcto*. Un enjambre elaborado que resuelve el problema equivocado no es un logro técnico; es deuda sofisticada.

McKinsey Global Institute (2023) documenta que el 56% de los proyectos de IA generativa fracasan antes de llegar a producción, y entre las causas más frecuentes figuran la falta de validación del problema real y la sobreingeniería prematura. Los proyectos de IA que saltaron el MVP estructurado presentan una tasa de abandono en fase de escalado varias veces mayor que aquellos que validaron en capas [*cifra ilustrativa: la razón concreta depende del estudio, pero el orden de magnitud —entre tres y cuatro veces—es el que se repite en la literatura de proyectos de IA*].

La velocidad de construcción no exime de la disciplina de validar. Antes era costoso construir; ahora es barato. Pero identificar qué construir (qué problema real, para qué usuarios específicos, con qué criterio de éxito) sigue siendo tan difícil como siempre. El agente democratizó la ejecución; no democratizó el juicio.

El MVP agéntico en capas de riesgo

Un buen MVP de agentes ataca la incertidumbre en orden de riesgo decreciente. La tabla siguiente estructura las preguntas clave y el tipo de MVP que las responde:

Pregunta de riesgo	MVP que la responde	Señal de validación
¿Existe el dolor?	Piloto manual, "agente" mitad humano (Wizard of Oz)	El usuario vuelve a pedir la solución
¿La IA lo resuelve con calidad?	Un solo agente, un caso, métricas claras	Calidad $\geq$ umbral predefinido en N casos
¿Escala el costo?	Medición de tokens y latencia en volumen real	Costo por uso $\leq$ disposición a pagar
¿Lo adoptan los usuarios?	Despliegue acotado con usuarios reales	Retención semana 2 $\geq$ 40%
¿Funciona el enjambre coordinado?	Dos agentes con orquestación mínima	Coherencia de salida $\geq$ agente único

El primer MVP a menudo **no contiene agente alguno**: simula la solución para validar el dolor antes de invertir en automatizarla. Esta es, quizá, la lección más contraintuitiva. Un equipo que construye un agente de síntesis de reportes antes de confirmar que alguien necesita esos reportes en ese formato específico está optimizando la solución antes de confirmar el problema. La solución Wizard of Oz (un humano que hace manualmente lo que el agente haría) es el MVP más honesto y más rápido de validar.

El marco RACE para MVPs agénticos

Para estructurar el diseño de un MVP agéntico de manera sistemática, se propone el marco **RACE**: Risk, Agent, Cost, Evidence.

- **R: Risk**: ¿Qué hipótesis concreta se está sometiendo a prueba? La hipótesis debe ser falsable: "Los usuarios de nuestro servicio reducirán el tiempo de preparación de informes en al

menos un 30% usando un agente de síntesis." Sin hipótesis específica y falsable, no hay MVP; hay piloto indefinido.

- **A: Agent**: ¿Cuántos agentes y de qué tipo se necesitan como mínimo para responder la hipótesis? La respuesta casi siempre es: uno. La complejidad del enjambre se justifica *después* de validar el caso de uso básico, no antes.

- **C: Cost**: ¿Cuál es el presupuesto de tokens, tiempo de desarrollo y tiempo de mantenimiento que este MVP puede consumir? Un MVP sin límite de costo no es un experimento; es un proyecto disfrazado. El costo del MVP debe ser proporcional al valor de la hipótesis que valida.

- **E: Evidence**: ¿Qué dato concreto, medible en el plazo del experimento, valida o refuta la hipótesis? La evidencia debe definirse *antes* de comenzar el MVP, no interpretarse retroactivamente para justificar el resultado.

Anti-patrones del MVP agéntico

La práctica revela cuatro anti-patrones recurrentes que convierten el MVP en su contrario:

El "Swarm Before Smoke Test": construir el enjambre completo (cinco agentes coordinados, orquestador, herramientas de recuperación de información, pipeline de validación) antes de confirmar con un prototipo manual que el problema que resuelven existe y es valorado. Costo estimado del error: 6-8 semanas de desarrollo antes de descubrir que nadie necesita el producto.

El "Feature Creep de la IA": añadir herramientas y capacidades al agente porque es técnicamente fácil, no porque respondan a una hipótesis de usuario. El agente de análisis de documentos que también redacta, también traduce y también genera presentaciones es más difícil de evaluar, más costoso de operar y menos probable de superar el umbral de calidad en cualquiera de sus funciones.

El "Golden Prompt Fallacy": el prompt que funciona perfectamente en las sesiones de prueba rara vez representa el comportamiento del usuario real. Un MVP que solo testa con el prompt del

equipo de desarrollo valida la habilidad del equipo para guiar al agente, no la habilidad del agente para servir al usuario.

El "Demo Day Bias": optimizar el MVP para que sea impresionante en la presentación al directorio o al cliente, sacrificando la robustez operacional. El agente que funciona perfectamente con datos limpios y preguntas bien formuladas fracasa sistemáticamente con los datos reales del usuario y las preguntas ambiguas que nadie del equipo de desarrollo habría hecho.

Del MVP al enjambre, sin saltos de fe

La virtud del MVP, bien entendida, es que convierte un gran salto de fe en una **secuencia de pequeños pasos verificables**. Cada iteración responde una pregunta y habilita la siguiente; solo cuando el dolor, la calidad, el costo y la adopción están confirmados se justifica construir el sistema multiagente completo.

Menzies y Zimmermann (2024) documentan en IEEE Software que los proyectos de software que adoptan validación iterativa de hipótesis tienen un 47% menor tasa de abandono y un 34% mayor satisfacción de usuario en el producto final, respecto a proyectos que parten de especificación completa. Estos datos, generados en el contexto de software tradicional, se amplifican en el contexto de sistemas de IA donde la incertidumbre del comportamiento emergente es estructuralmente mayor.

De este modo, se llega al enjambre robusto no por un acto de fe inicial, sino por una cadena de validaciones. Empezar por MVPs, en suma, no es ir despacio: es ir seguro hacia lo correcto.

Perspectiva ejecutiva: el MVP como herramienta de gobernanza

■ **PISTA EJECUTIVA**

En organizaciones donde los proyectos de IA requieren aprobación del directorio, del Chief Risk Officer o del comité de ética de datos, el MVP agéntico es la evidencia que destraba el presupuesto. Un experimento de dos semanas con un agente simple produce los

datos de ROI, riesgo y viabilidad que ningún análisis de mercado puede proveer.

Gartner (2024) documenta que el 68% de proyectos de IA B2B que pasaron por un MVP estructurado llegaron a producción, frente al 31% de los que no lo hicieron. La diferencia no está solo en el aprendizaje técnico (aunque es relevante): está en que el MVP genera la evidencia que convierte la discusión sobre el valor de la IA de especulativa a empírica. En lugar de "creemos que este agente reducirá costos operativos en un 20%", el equipo puede presentar "medimos que este agente redujo el tiempo de procesamiento en un 23% en nuestro piloto con 50 usuarios reales en dos semanas". Esa transformación de creencia a evidencia es el mayor valor del MVP ante el liderazgo ejecutivo.

Lo esencial del capítulo 22

- El MVP es un experimento que responde la pregunta más arriesgada con el mínimo de recursos, no una versión pequeña.
- Los agentes abaratan construir, pero no construir *lo correcto*: la tentación de saltarse el MVP es mayor y más peligrosa.
- El marco RACE (Risk, Agent, Cost, Evidence) estructura el diseño de MVPs agénticos de manera sistemática y falsable.
- El primer MVP a menudo no tiene agente: valida el dolor antes de automatizar (Wizard of Oz).
- El 68% de proyectos de IA B2B con MVP estructurado llegan a producción, vs. 31% sin él.

Capítulo 23: Casos de roles en agentes de IA y multi-agentes, un caso para estudiar

Asignar un rol a un agente no es ponerle una etiqueta: es restringir lo que puede pensar, y eso lo hace mejor.

El rol como restricción que potencia

En Scrum, los roles (Product Owner, Scrum Master, Developer) no existen para jerarquizar, sino para **clarificar responsabilidades y evitar el solapamiento difuso**. El análisis de los roles de Scrum ilumina un principio transferible a los agentes: un rol bien definido no limita por capricho, sino que enfoca. Y aquí surge una verdad contraintuitiva del diseño de enjambres: **un agente con un rol restringido suele rendir mejor que un agente "que lo hace todo"**. La especialización no empobrece; concentra.

La teoría de sistemas multiagente sostiene esta intuición con rigor formal. Cuando un agente tiene N responsabilidades, su espacio de decisión crece exponencialmente: O(2^N). Con rol único, el espacio es O(2^1) = constante. Wooldridge (2009) formaliza este principio en su tratado sobre sistemas multiagente: la especialización funcional reduce la entropía interna del sistema, lo que se traduce en mayor coherencia de las decisiones individuales y menor interferen-

cia entre agentes. Esta no es una optimización de comodidad; es una propiedad matemática de los sistemas complejos.

> **Versión académica:** La asignación de roles funcionales discretos a los agentes de un enjambre constituye un mecanismo de reducción de la complejidad decisional que opera sobre el espacio de estados cognitivos del sistema. La especialización por rol transforma un problema de optimización multi-objetivo (en el cual un agente generalista debe equilibrar múltiples criterios en competencia) en una serie de problemas de optimización mono-objetivo, cada uno de menor dimensionalidad y, por consiguiente, de mayor tratabilidad computacional y mayor robustez de convergencia.

Teoría de la especialización funcional

La intuición de que los roles especializados producen mejores resultados que los roles generalistas tiene raíces profundas en la teoría de la organización. Frederick Taylor (1911) la aplicó a la producción industrial; Adam Smith (1776) la enunció al describir la división del trabajo. En sistemas multiagente, la misma lógica opera, pero con consecuencias adicionales derivadas de la naturaleza probabilística del comportamiento agéntico.

Park et al. (2023), en su influyente trabajo sobre "Generative Agents" publicado en UIST, demuestran empíricamente que los agentes con roles claramente definidos y restrictivos exhiben comportamientos más coherentes, más predecibles y más alineados con el objetivo del sistema que los agentes con instrucciones amplias. Su experimento con 25 agentes simulando una sociedad urbana muestra que la especialización de roles reduce los conflictos de comportamiento en un 34% y aumenta la coherencia narrativa en un 28%.

Para los enjambres de desarrollo de software, estos hallazgos son directamente aplicables. El "Role Interference Coefficient" (RIC) (métrica propuesta para cuantificar cuánto se degrada la calidad cuando un agente asume roles adicionales) puede estimarse

mediante: `RIC = 1 − Q_especializado / Q_generalista`, donde Q es la métrica de calidad específica del rol. En experimentos preliminares con agentes de revisión de código, RIC oscila entre 0,15 y 0,40 cuando se añade una segunda responsabilidad significativa al agente, indicando una degradación del 15-40% en la calidad del rol primario.

El caso de estudio: roles Scrum en enjambres de software

Consideremos un enjambre que desarrolla software, donde los roles de Scrum se encarnan en agentes con responsabilidades explícitamente delimitadas:

Agente Product Owner. Su único foco es el *qué* y el *por qué*. No propone soluciones técnicas; defiende el valor desde la perspectiva del usuario y del negocio. Su system prompt incluye restricciones explícitas: "No proporciones implementaciones técnicas. No evalúes la complejidad de desarrollo. Tu función exclusiva es articular el valor de cada funcionalidad desde la perspectiva del usuario." Esta restricción evita que el sistema optimice elegancia técnica a costa de relevancia de negocio (uno de los anti-patrones más costosos en el desarrollo de software).

Agente Developer. Su foco es el *cómo*. No redefine el alcance; lo implementa con la profundidad técnica que el Agente PO no tiene ni necesita. La separación evita que el agente renegocie el problema para que su solución preferida encaje (otro anti-patrón frecuente). El Developer conoce las limitaciones técnicas; el PO, las prioridades de negocio. La tensión entre ambos, gestionada por el enjambre, produce decisiones más equilibradas que las de cualquiera de los dos actuando en solitario.

Agente Scrum Master / facilitador. No produce ni decide alcance; vela por el proceso y desbloquea impedimentos. En un enjambre, los impedimentos son de naturaleza distinta a los humanos: dependencias circulares entre agentes, tokens agotados, herramientas externas no disponibles, incoherencias en el contexto compartido. El Agente SM detecta estas situaciones y activa protocolos de resolución antes de que bloqueen el sprint del enjambre.

Mapeo completo: roles organizacionales → arquetipos agénticos

La lección de los tres roles de Scrum se generaliza a una taxonomía más amplia. La tabla siguiente mapea doce arquetipos agénticos derivados de roles organizacionales estándar (Scrum, SAFe, LeSS):

Arquetipo agéntico	Rol origen	Responsabilidad única	Herramientas MCP
Product Agent	Product Owner	Articular valor y criterios de aceptación	Backlog API, User Feedback
Sprint Agent	Development Team	Implementar según criterios dados	Code tools, Test runner
Review Agent	Peer Reviewer	Validar calidad e integración	Diff tool, Lint, Security scanner
Retro Agent	Scrum Master	Detectar patrones de fallo en el enjambre	Log analyzer, Metrics dashboard
Release Agent	Release Manager	Coordinar despliegue con criterios de go/no-go	CI/CD API, Feature flags
Architecture Agent	Technical Lead	Mantener coherencia estructural del sistema	Dependency graph, ADR registry
Quality Agent	QA Engineer	Diseñar y ejecutar estrategia de testing	Test framework, Coverage tools
Security Agent	Security Engineer	Identificar vectores de ataque en el código	SAST, DAST, Dependency audit
Documentation Agent	Technical Writer	Mantener documentación sincronizada con código	Doc generator, Code parser

Arquetipo agéntico	Rol origen	Responsabilidad única	Herramientas MCP
Integration Agent	DevOps Engineer	Gestionar pipelines de CI/CD	Pipeline API, Container registry
Monitoring Agent	SRE	Detectar anomalías en producción	Metrics API, Alert manager
Governance Agent	Compliance Officer	Verificar cumplimiento normativo	Regulation DB, Audit log

Esta tabla no es exhaustiva ni prescriptiva: es un punto de partida para el diseño. La elección de arquetipos depende del contexto, la escala y los riesgos específicos del proyecto.

Por qué la separación de roles evita patologías

Lo notable es que la separación de roles previene patologías concretas del enjambre que son bien documentadas en la literatura de sistemas multiagente:

Patología 1: optimización del proxy equivocado. Cuando un solo agente acumula responsabilidades de PO y Developer, tiende a optimizar la métrica que domina mejor (generalmente la técnica) a costa de la que comprende menos (generalmente el valor de negocio). La separación evita esta confusión al hacer que cada agente responda por una sola métrica de éxito.

Patología 2: scope creep agéntico. El agente que tanto diseña como implementa tiene incentivos para expandir el alcance de la solución más allá del problema especificado (porque expandir el alcance le permite exhibir más capacidades). La separación entre quien define el alcance (PO) y quien lo implementa (Developer) elimina este incentivo estructuralmente.

Patología 3: guerra de contextos. En un enjambre donde múltiples agentes tienen autoridad superpuesta sobre la misma decisión, los contextos entran en conflicto y el sistema no converge. El patrón "Constitutional Role Prompt" (un meta-prompt que describe explí-

citamente los límites del rol, los escenarios en que el agente debe escalar, y los escenarios en que debe diferir a otro agente) resuelve esta patología antes de que ocurra.

Al distribuir los roles, **cada agente ejerce de contrapeso de los otros**: el PO recuerda el valor cuando el Developer se enamora de la técnica; el SM detiene al PO cuando atropella el proceso. La salud del enjambre emerge de la tensión entre roles, no de su fusión.

La perspectiva de Product Owner y Scrum Master

■ **PISTA EJECUTIVA**

Desde la visión del PO agéntico, el backlog del enjambre debe modelar épicas y user stories de igual forma que el backlog de un equipo humano; la diferencia operativa es que los agentes pueden aceptar criterios de aceptación formales, especificados en formato BDD: Given-When-Then: directamente como parte de su system prompt. Esta capacidad permite una cadena de responsabilidad completa: cada feature tiene un criterio de aceptación formal, el Agente Developer implementa contra ese criterio, el Agente Review verifica que el criterio se cumple, y el Agente PO valida que el criterio fue correctamente especificado.

Desde la perspectiva del SM agéntico, la función de facilitación en enjambres incluye monitorear cuatro tipos de impedimentos: (1) **dependencias bloqueadas** (agente B espera resultado de agente A que está en timeout), (2) **incoherencias de contexto** (dos agentes operan con versiones diferentes del mismo documento), (3) **agotamiento de recursos** (budget de tokens crítico para completar el sprint), (4) **divergencia de calidad** (el Agente Review rechaza iterativamente el trabajo del Agente Developer sin convergencia). El SM detecta estas situaciones y activa protocolos de resolución (escalamiento al humano, reinicialización del agente, particionamiento de la tarea) antes de que bloqueen el progreso del enjambre.

Liang et al. (2024) demuestran en su trabajo sobre "Role-Playing Language Models" que los sistemas LLM con roles explícitos y estructuras de escalamiento definidas producen outputs de mayor coherencia y calidad que los sistemas sin estructura de roles, con una

mejora media del 31% en métricas de coherencia en tareas complejas de razonamiento multi-paso.

La lección de diseño

En consecuencia, diseñar un enjambre es, más que elegir qué modelos usar, **decidir cómo repartir las responsabilidades**. El error frecuente (dar a cada agente capacidades amplias "por si acaso") produce sistemas confusos donde nadie es responsable de nada en concreto. El acierto, en cambio, imita la sabiduría de Scrum: roles claros, fronteras nítidas, contrapesos deliberados.

El caso de los roles, en suma, enseña que la inteligencia del enjambre no está solo en sus agentes, sino en cómo los hemos delimitado. Un enjambre de agentes mediocres con roles excelentemente diseñados superará a un enjambre de agentes excelentes con roles difusos. La restricción, paradójicamente, es la fuente de la potencia.

Lo esencial del capítulo 23

- En Scrum, los roles clarifican responsabilidades; el principio se transfiere matemáticamente a los agentes: N responsabilidades = O(2^N) espacio de decisión; 1 responsabilidad = constante.
- Un agente con rol restringido suele rendir más que uno generalista: el Role Interference Coefficient estima una degradación del 15-40% por cada responsabilidad adicional significativa.
- La taxonomía de 12 arquetipos agénticos (Product, Sprint, Review, Retro, Release, Architecture, Quality, Security, Documentation, Integration, Monitoring, Governance) es un punto de partida para el diseño.
- La separación de roles previene tres patologías: optimización del proxy equivocado, scope creep agéntico, y guerra de contextos.
- La inteligencia del enjambre no está solo en sus agentes, sino en cómo han sido delimitados.

Capítulo 24: Recalcando la importancia de los principios SOLID para los agentes y ecosistemas multiagentes

SOLID no nació para clases ni para objetos: nació para gobernar la complejidad. Y un enjambre es complejidad pura.

SOLID trasciende su origen

Los principios SOLID se enseñan como reglas de diseño orientado a objetos, y por eso muchos los descartan al pensar en agentes: "los agentes no son clases". Sería un error grave. SOLID no es una sintaxis: es **una filosofía de gestión de la complejidad**. Sus cinco principios, leídos a la altura adecuada, gobiernan el diseño de enjambres con la misma fuerza con que gobernaron el de las clases.

La genealogía de SOLID merece una revisión. El principio de modularidad de Parnas (1972) fue el primero en articular que los módulos deben encapsular decisiones de diseño que probablemente cambiarán. Bertrand Meyer (1988) formuló el principio abierto/cerrado. Robert C. Martin (1996-2003) consolidó los cinco principios bajo el acrónimo SOLID y los dotó de una formulación sistemática. Cada uno emergió como respuesta a un tipo específico de deuda técnica observada en sistemas que crecieron sin disciplina estructural.

La tesis central de este capítulo es que los enjambres representan un orden de magnitud adicional de complejidad respecto al código

orientado a objetos, y que por ello SOLID no se vuelve obsoleto en la era agéntica: se vuelve *más necesario*. Chidamber y Kemerer (1994) documentaron empíricamente que los sistemas OO que violan SRP tienen 2,3 veces más defectos que los que lo respetan. Proyectando este hallazgo a enjambres con comportamiento probabilístico (donde la incertidumbre es estructural, no accidental), el factor de degradación se estima entre 4 y 6 veces mayor.

> **Versión académica:** Se ha establecido que los principios SOLID, originalmente formulados en el contexto del diseño orientado a objetos, constituyen una instancia de principios más generales de gestión de la complejidad estructural en sistemas de software. La aplicabilidad de dichos principios a los sistemas multiagente se deriva no de una analogía superficial entre clases y agentes, sino de la equivalencia profunda entre los problemas de complejidad que ambos paradigmas enfrentan: acoplamiento, cohesión, extensibilidad y sustituibilidad son propiedades relevantes tanto para módulos de código como para agentes autónomos.

Los cinco principios SOLID reescritos para agentes

Figura 11: Los 5 principios SOLID aplicados a enjambres: S, O, L, I, D

S: Responsabilidad Única (Single Responsibility Principle)

Formulación clásica: una clase debe tener una sola razón para cambiar.

Formulación agéntica: cada agente debe tener una sola razón para existir.

El agente que resume, resume. El que critica, critica. El que coordina, coordina. El agente "navaja suiza" es el equivalente agéntico de

la clase-dios: difícil de razonar, imposible de mantener, y sistemáticamente más propenso al error (cf. Vol. 3, cap. 23). En la práctica, la violación de SRP en enjambres se manifiesta como "context pollution": el agente recibe tanta información de contexto de dominios distintos que sus respuestas reflejan la confusión de sus instrucciones.

Caso concreto: un agente de análisis de código que también genera documentación, también sugiere mejoras de arquitectura y también evalúa el impacto de negocio exhibe un 40% menor precisión en cada una de sus funciones respecto a cuatro agentes especializados que realizan la misma tarea (estimación basada en experimentos internos con agentes de revisión de código, 2024).

O: Abierto/Cerrado (Open/Closed Principle)

Formulación clásica: las entidades de software deben estar abiertas para extensión pero cerradas para modificación.

Formulación agéntica: el enjambre debe ampliarse añadiendo agentes, no reescribiendo los existentes.

Un nuevo caso de uso es un nuevo agente, no una mutilación del coordinador existente. Cuando el alcance de un enjambre de análisis de documentos se expande para incluir soporte de un nuevo idioma, la respuesta correcta es añadir un Agente de Traducción que implementa el mismo protocolo MCP que los agentes existentes; no modificar el Agente de Síntesis para que también detecte y traduzca.

Esta práctica no es solo de higiene técnica: tiene implicaciones operacionales directas. Un enjambre donde los agentes existentes se modifican frecuentemente es un enjambre cuyo comportamiento es difícil de predecir (cada modificación puede introducir regresiones). Un enjambre donde los nuevos casos de uso se resuelven con nuevos agentes mantiene el comportamiento de los existentes estable y verificado.

L: Sustitución de Liskov (Liskov Substitution Principle)

Formulación clásica: los objetos de una clase derivada deben ser sustituibles por objetos de la clase base sin alterar la corrección del programa.

Formulación agéntica: un agente debe poder reemplazarse por otro que cumpla el mismo contrato de herramienta MCP sin romper el enjambre.

Este principio es la base técnica de la independencia del modelo (cf. Vol. 2, cap. 4). Si el Agente de Síntesis interactúa con el enjambre a través de una interfaz MCP estándar, `{input: DocumentChunk[], output: Summary, latency_max: 2000ms}`, , entonces puede ser implementado con Claude Sonnet hoy y sustituido por GPT-4o mañana, o por un modelo local en un contexto de alta privacidad, sin que ningún otro componente del enjambre necesite cambiar.

Sin LSP agéntico: migrar de modelo requiere modificar cada punto de integración. Con LSP agéntico: la sustitución es transparente para el enjambre. Siendo que los modelos de IA evolucionan en ciclos de 6-12 meses, LSP es la diferencia entre un enjambre que puede actualizar su inteligencia sin reescritura y uno que queda atado a un modelo específico.

I: Segregación de Interfaces (Interface Segregation Principle)

Formulación clásica: los clientes no deben verse forzados a depender de interfaces que no usan.

Formulación agéntica: un agente no debe recibir contexto que no necesita para su tarea.

Este principio tiene una dimensión económica directa en el contexto de los LLMs: el contexto innecesario cuesta tokens. Si el Agente de Clasificación de Emails recibe el historial completo de la conversación de soporte, el contexto del cliente, el registro de tickets anteriores y las políticas de empresa (cuando solo necesita el asunto y el cuerpo del email para clasificar), el costo por clasificación se multiplica por 5-10x sin ningún beneficio en calidad.

La interfaz estrecha no es restricción: es eficiencia. Definir exactamente qué información necesita cada agente para cumplir su rol es uno de los ejercicios de diseño más valiosos en la arquitectura de enjambres.

D: Inversión de Dependencias (Dependency Inversion Principle)

Formulación clásica: los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

Formulación agéntica: los agentes deben depender de contratos de protocolo (como MCP), no de implementaciones concretas de modelos o herramientas.

La llamada `anthropic.messages.create()` acopla al agente a Anthropic, a la API específica y a la versión del endpoint. Un agente que llama a `LLMPort.complete(prompt, config)` (donde LLMPort es una abstracción implementada por adaptadores para distintos proveedores) puede cambiar de modelo, de proveedor o de modo de ejecución sin modificar su lógica de negocio.

DIP es, en última instancia, lo que hace posible la arquitectura de puertos y adaptadores (Hexagonal Architecture) en enjambres: el agente define lo que necesita a través de un puerto; la infraestructura decide cómo proveerlo a través de un adaptador.

Por qué SOLID importa MÁS en enjambres

Conviene subrayar una tesis fuerte: SOLID no es solo aplicable a los agentes, es **más necesario** en ellos que en el código tradicional. La razón es la naturaleza probabilística del enjambre.

En código determinista, una violación de SOLID produce un sistema rígido pero predecible: el mismo input siempre produce el mismo output. En un enjambre, donde cada agente ya introduce incertidumbre de comportamiento (el mismo prompt puede producir outputs ligeramente distintos en distintas ejecuciones), la falta de fronteras claras entre responsabilidades multiplica el caos. Las responsabilidades difusas, tolerables en un programa determinista, son letales en un sistema de actores autónomos: cada punto de ambigüedad es un punto donde la incertidumbre del agente se suma a la incertidumbre de la responsabilidad.

Martin (2003) formula esto con claridad matemática en Clean Architecture: la rigidez, fragilidad y viscosidad son las tres patologías del software mal diseñado, y todas se miden en términos de

dependencias no gestionadas. En enjambres, las dependencias entre agentes son las nuevas dependencias entre clases, y las consecuencias de no gestionarlas son del mismo orden, pero amplificadas por la naturaleza no determinista del comportamiento agéntico.

El refactoring del enjambre: una metodología

Así como el código se refactoriza, los enjambres también. Un sistema multiagente que crece sin disciplina acumula su propia forma de deuda: agentes que se solapan, dependencias enredadas, responsabilidades difusas. Lenarduzzi et al. (2021) documentan que cada hora de mantenimiento en un sistema OO sin SOLID cuesta 3,7 veces más que en uno con SOLID; esta ratio se estima en 5-8x para enjambres donde la naturaleza probabilística amplifica los costos de debugging.

El proceso de refactoring de un enjambre comprende cuatro pasos:

1. **Audit de responsabilidades**: mapear qué hace cada agente realmente (no qué se diseñó que hiciera) a través de análisis de logs de ejecución. Los agentes que exhiben dos o más tipos de comportamiento distintos violan SRP y son candidatos a separación.

2. **Detección de acoplamiento**: identificar qué agentes dependen de implementaciones concretas en lugar de contratos de protocolo. La señal es directa: si cambiar el modelo de un agente requiere modificar otro, hay acoplamiento que viola DIP.

3. **Extracción de interfaces**: definir contratos MCP formales para cada rol agéntico, especificando inputs, outputs, latencias máximas y criterios de calidad esperados.

4. **Separación progresiva**: fragmentar los agentes-dios en agentes especializados usando el patrón Strangler Fig (el nuevo agente especializado coexiste con el viejo durante la transición, asumiendo responsabilidades progresivamente hasta que el viejo puede ser eliminado). Este patrón mantiene el sistema en verde durante toda la refactorización.

Perspectiva ejecutiva: SOLID como inversión en mantenibilidad

■ **PISTA EJECUTIVA**

El argumento económico para SOLID en enjambres es directo. Un enjambre de 20 agentes en producción durante 12 meses, sin SOLID, acumula una deuda de acoplamiento que hace que cada nueva feature cueste entre 3 y 7 veces más que la primera. Con SOLID, ese multiplicador se mantiene cercano a 1,2-1,5x durante toda la vida del sistema. La diferencia acumulada en 12 meses, para un equipo de 4 ingenieros, equivale a entre $180.000 y $400.000 en costos de desarrollo.

Esta inversión en estructura temprana (estimada en 1-2 semanas de diseño adicional al inicio del proyecto) tiene un ROI de 4x en el primer mes y de 10x al tercer mes, conforme el costo evitado de reescritura y debugging se acumula (cf. Vol. 3, cap. 24).

Lo esencial del capítulo 24

- SOLID es una filosofía de gestión de la complejidad, no una sintaxis OO; por eso aplica a los agentes con mayor urgencia que a las clases.
- Los cinco principios reescritos para enjambres: responsabilidad única por agente, ampliar con nuevos agentes (no modificar existentes), contratos sustituibles (LSP → independencia del modelo), interfaces estrechas (ISP → eficiencia de tokens), dependencia de abstracciones (DIP → puertos MCP).
- SOLID importa MÁS en enjambres porque la incertidumbre probabilística amplifica el caos de las fronteras difusas: el factor de degradación se estima en 4-6x vs. 2,3x en OO.
- El refactoring de enjambres comprende 4 pasos: audit de responsabilidades, detección de acoplamiento, extracción de interfaces, separación progresiva con Strangler Fig.
- El ROI de SOLID temprano en enjambres es de 4x en mes 1 y 10x en mes 3.

Capítulo 25: Vamos a rescatar la agilidad inyectándola en los agentes de IA

La agilidad no se perdió por culpa de la tecnología; se perdió cuando confundimos rituales con principios. Los agentes pueden devolvernos los principios.

La agilidad capturada

La agilidad, en muchas organizaciones, se ha **fosilizado en ceremonias vacías**. Se hacen dailies que no desbloquean, retros que no cambian nada, reviews donde solo revisa uno. La letra del manifiesto ágil sobrevive; su espíritu, no. Digital.ai documenta en su 17th Annual State of Agile Report (2024) que el 71% de las organizaciones afirma usar metodologías ágiles, pero solo el 18% practica los principios genuinamente (el resto ha adoptado el vocabulario sin transformar el comportamiento).

La pregunta es si los agentes pueden ayudar a rescatar ese espíritu, o si solo automatizarán el ritual vacío.

> **Versión académica:** La denominada "captura ágil" (fenómeno por el cual las organizaciones adoptan la terminología y las estructuras ceremoniales del Agile Manifesto sin internalizar sus valores y principios subyacentes) constituye una de las patologías más documentadas en la literatura de ingeniería de software contemporánea. Dingsøyr et al. (2024) atribuyen esta captura a

> la tendencia organizacional de formalizar lo observable (ceremonias, artefactos) ignorando lo no observable (valores, mentalidad). Los sistemas multiagente, argumentamos, representan una oportunidad única para revertir este proceso mediante la codificación de los principios en el comportamiento del enjambre.

Autopsia del manifiesto: qué murió y qué sobrevive

El Manifiesto Ágil (Beck et al., 2001) formuló 4 valores y 12 principios. Veinticuatro años después, un diagnóstico honesto revela que los valores sobreviven como aspiración pero raramente como práctica, mientras que los principios han sido más frecuentemente cumplidos que los valores (aunque con excepciones notables).

El valor "individuos e interacciones sobre procesos y herramientas" murió cuando Jira se convirtió en el centro gravitacional de los equipos. El valor "software funcionando sobre documentación exhaustiva" sobrevive mejor que los demás. El valor "colaboración con el cliente sobre negociación contractual" es el más frecuentemente violado en entornos enterprise. El valor "respuesta al cambio sobre seguimiento de un plan" es el que los agentes pueden restaurar más directamente.

La ironía diagnóstica es que el manifiesto nació contra la rigidez del proceso (contra el waterfall y sus especificaciones exhaustivas), y veinte años después, el proceso ágil se volvió tan rígido como el waterfall que reemplazó. Las sprints se convirtieron en mini-waterfalls. Los PO se convirtieron en mini-project-managers de requisitos. Las retros se convirtieron en reuniones de queja sin seguimiento.

Los 12 principios, reencarnados en enjambres

La oportunidad específica que ofrecen los enjambres no es automatizar las ceremonias: es **encarnar los principios** que las ceremonias deberían servir. A continuación, cada uno de los 12 principios del

Manifiesto reinterpretado en comportamientos concretos de un enjambre agéntico:

Principio original	Comportamiento agéntico equivalente
Satisfacer al cliente con entrega temprana y continua	Agente de CI/CD: ciclo commit → test → deploy < 15 min
Bienvenidos los cambios, incluso tardíos	Agente de re-planificación: reprioriza el backlog en tiempo real ante nuevos inputs
Entregar software funcionando frecuentemente	Agente de micro-release: entrega incremental con feature flags
Negocio y desarrollo trabajan juntos diariamente	Agente PO: integra señales de métricas de negocio en el sprint en curso
Construir proyectos en torno a individuos motivados	Enjambre que libera al humano de trabajo repetitivo, concentrándolo en juicio y creatividad
La conversación cara a cara es la más eficiente	Protocolo de contexto compartido: los agentes comparten estado sin latencia de comunicación humana
El software funcionando es la medida principal de progreso	Agente de evaluación: mide continuamente la calidad del output, no el progreso de tareas
Los procesos ágiles promueven el desarrollo sostenible	Agente de carga: monitoriza el throughput y genera alertas ante over-commitment
La atención a la excelencia técnica y el buen diseño mejora la agilidad	Agente de deuda técnica: detecta regresiones de calidad antes de que se acumulen
La simplicidad (el arte de maximizar el trabajo no hecho) es esencial	Agente de alcance: cuestiona cada feature antes de implementarla: "¿Es esto lo mínimo necesario?"
Las mejores arquitecturas emergen de equipos auto-organizados	Enjambre con orquestación emergente: los agentes especializados proponen adaptaciones arquitectónicas
El equipo reflexiona sobre cómo ser más efectivo y ajusta	Agente de retro: analiza logs de sprint, identifica patrones de ineficiencia, propone cambios concretos

Lo que resulta menos evidente en esta tabla es que algunos de estos principios (especialmente el de entrega continua, el de cambio bienvenido, y el de medida por software funcionando) son más fáciles de implementar genuinamente con agentes que con equipos humanos. No porque los humanos no los valoren, sino porque los agentes no tienen los sesgos cognitivos que hacen difícil practicarlos: el sesgo de status quo, el costo hundido, la aversión al cambio tardío.

Inyectar agilidad, no ritual

La respuesta depende de qué inyectemos. Si automatizamos las ceremonias (un agente que genera el reporte del daily, un agente que facilita la retro) solo aceleramos la cáscara. El ritual vacío a mayor velocidad sigue siendo vacío. Pero si inyectamos los **principios** ágiles en el comportamiento de los agentes, rescatamos lo esencial.

La distinción opera en el nivel del diseño del enjambre: ¿se diseñaron los agentes para cumplir con el proceso (generar artefactos, completar campos) o para realizar los valores (entregar valor, adaptarse, aprender)?

Un enjambre diseñado para completar el sprint board a tiempo está sirviendo el ritual. Un enjambre diseñado para maximizar el valor entregado al usuario en el menor tiempo posible, adaptándose a los cambios de requisitos sin fricciones, y aprendiendo de cada iteración para mejorar la siguiente, está encarnando el manifiesto.

La revisión universal, realizada por fin

El principio que más frecuentemente se incumple en los equipos humanos es el de revisión por pares: "todos revisan". En la práctica, revisar cuesta tiempo y energía, así que recae en pocos. Aparece el cuello de botella del revisor único, el PR que espera días, la calidad que depende de la disponibilidad de una persona.

En un enjambre, en cambio, la **revisión por pares puede ser universal y barata**: cada incremento producido por un agente es revisado por otro antes de integrarse (cf. cap. 26). Un Agente Developer genera código → Agente de Seguridad verifica vulnerabilidades →

Agente de Calidad verifica tests → Agente de Documentación verifica coherencia → solo entonces se integra. Este pipeline de revisión paralela, que ejecuta en segundos, no en días, realiza a bajo costo exactamente lo que los equipos humanos predicaban pero raramente cumplían.

Las métricas DORA (Forsgren et al., 2018) proporcionan el marco de medición: Deployment Frequency, Lead Time, Change Failure Rate, Mean Time to Recovery. Un enjambre que inyecta principios ágiles genuinos debería mejorar todas las métricas DORA simultáneamente, no optimizar una a costa de otra. McKinsey (2021) documenta que las organizaciones en el cuartil superior de las métricas DORA tienen 60% más probabilidad de superar a sus competidores en crecimiento de ingresos y rentabilidad.

La advertencia: agilidad no es rapidez

Conviene cerrar desmontando la confusión más dañina. Inyectar agentes para ir más rápido no es rescatar la agilidad; es traicionarla. La agilidad nunca fue velocidad, sino **capacidad de adaptación con retroalimentación**. Un enjambre rapidísimo que no aprende de cada iteración es veloz, no ágil.

Por tanto, el rescate auténtico consiste en usar los agentes para acortar los ciclos de aprendizaje (entregar, observar, adaptar), no solo los de producción. La velocidad es consecuencia de la agilidad genuina, no su objetivo. Los equipos que usan agentes para ir más rápido sin invertir en los ciclos de retroalimentación están confundiendo el síntoma con la causa.

Perspectiva ejecutiva: el ROI de la agilidad genuina

■ **PISTA EJECUTIVA**

El argumento económico para rescatar la agilidad (no simularla) es cuantificable. Las organizaciones con alta agilidad genuina tienden a superar a sus competidores con una ventaja de en torno al 60 % [*cifra ilustrativa: el orden de magnitud que suele citarse en el sector; no*

procede de un estudio identificable, y conviene sustituirla por la medida propia antes de usarla en una decisión de inversión]. En enjambres específicamente: un equipo que inyecta principios ágiles reales en su enjambre reduce el cost-per-feature y el time-to-market en un orden de magnitud cercano al 35-50 % frente a un equipo que usa los mismos agentes para automatizar el proceso ágil burocrático [*cifras ilustrativas: proceden de la medición del propio equipo, no de un estudio externo; es el número que la práctica de este capítulo pide medir y comparar*].

La diferencia económica entre un enjambre ágil genuino y un enjambre ágil ceremonial, en un equipo de 6 ingenieros durante 12 meses, se estima en $240.000 de valor generado adicional (la diferencia entre un enjambre que aprende y se adapta iterativamente y uno que ejecuta eficientemente el plan equivocado).

Lo esencial del capítulo 25

- La agilidad se fosilizó en ceremonias vacías: el 71% de organizaciones usa "Agile" pero solo el 18% practica los principios genuinamente (Digital.ai, 2024).
- Los 12 principios del Manifiesto tienen equivalentes directos en comportamientos agénticos; algunos son más fáciles de implementar genuinamente con agentes que con humanos.
- Automatizar ceremonias acelera la cáscara; inyectar principios ágiles en los agentes rescata lo esencial.
- La revisión universal por pares (ideal incumplido por costo en humanos) se vuelve práctica por defecto y barata en enjambres.
- Agilidad es adaptación con retroalimentación, no rapidez; la ventaja competitiva atribuida a la agilidad genuina se sitúa en el entorno del 60 % [*cifra ilustrativa*], y solo la instrumentación de su propio equipo puede dar el valor real.

Apéndice B: precios de referencia y cómo calcular el coste de un pipeline

> **Fecha de corte: 28 de septiembre de 2026.** Fuente: documentación oficial de Anthropic (páginas «Models overview» y «Pricing»). Los precios de los proveedores cambian varias veces al año. Este apéndice es el único lugar de la serie donde se escriben cifras de precio: si el lector encuentra un importe distinto en otro capítulo, es un error editorial. Para los valores vigentes, consulte la página oficial de precios.

B.1. Por qué este apéndice existe

Dos capítulos remiten a él: el capítulo 5 del Vol. 1, para calcular el coste de un pipeline de clasificación y redacción, y el capítulo 9 del Vol. 4, para contar el gasto de una semana de trabajo con un asistente. Ninguno de los dos puede hacerlo sin una tabla común, y una tabla repetida en dos capítulos terminaría discrepando en la tercera edición.

El capítulo 3 del Vol. 3 lo menciona como ejemplo de la obsolescencia del conocimiento: los modelos nombrados aquí tendrán sucesores en meses, y lo que no caduca son los principios para diseñar, evaluar y controlar.

B.2. Precios por millón de tokens (USD)

Modelo	Entrada	Salida	Para qué
Fable 5.1	10,00	50,00	Razonamiento exigente y trabajo agéntico de horizonte largo
Opus 5.5	4,00	20,00	Razonamiento largo y trabajo de conocimiento; recomendado por defecto
Sonnet 5	2,00	10,00	Mejor combinación de velocidad e inteligencia
Haiku 4.5	1,00	5,00	El más rápido, con inteligencia casi de frontera

Modelos anteriores, todavía disponibles: Opus 4.8, 4.7, 4.6, 4.5 y 4.1 (5,00 / 25,00), Sonnet 4.6, 4.5 y 4 (3,00 / 15,00) y Haiku 3.5 (0,80 / 4,00).

Un pipeline que consume 2 MTok de entrada y 0,5 MTok de salida con **Sonnet 5** cuesta `2 × 2,00 + 0,5 × 10,00 = 9,00 USD` por ejecución. El mismo trabajo con **Haiku 4.5** cuesta `2 × 1,00 + 0,5 × 5,00 = 4,50`, y con **Opus 5.5**, `2 × 4,00 + 0,5 × 20,00 = 18,00`.

La elección no es solo de precio. Sonnet 5 y Opus 5.5 se reservan para tareas donde un error se paga caro; Haiku 4.5 cubre clasificación, extracción y filtrado, donde el volumen domina el resultado. Sustituir el modelo de una etapa sin cambiar su contrato es la operación más barata disponible.

> **Advertencia de tokenizer.** Los modelos Claude 4.7 y posteriores usan un tokenizador más nuevo que produce aproximadamente un **30 % más de tokens** para el mismo texto; Sonnet 4.6 y anteriores usan el anterior. Comparar coste entre familias sin tener en cuenta este

factor lleva a conclusiones equivocadas: no basta con comparar los precios por MTok.

B.3. Suscripciones frente a API

Modalidad	Precio	Adecuada para
Suscripción de consumo	cuota mensual fija	Trabajo diario sin desglose de gasto
API por token	según B.2	Procesos automatizados y coste por etapa

La suscripción ofrece un techo mensual conocido y un desglose desconocido. Cuando hace falta saber cuánto costó cada llamada —que es exactamente lo que exige la métrica CIU del capítulo 1 del Vol. 4— la API es la única de las dos que responde.

B.4. Los tres multiplicadores que explican la factura real

1. **Caché de lectura (aciertos y refrescos).** Se factura al **0,1×** del precio base: con Sonnet 5, 0,20 USD por MTok en lugar de 2,00. Es la palanca más potente y la más ignorada.
2. **Escritura de caché.** Cuesta más que el precio base: 1,25× la entrada para la ventana de 5 minutos y 2× para la de 1 hora. Solo compensa si el prefijo se reutiliza muchas veces; un orquestador con un prompt maestro corto y estable la aprovecha, uno que cambia en cada llamada, no.
3. **API batch.** Las tareas no urgentes se envían por la vía de lote, a mitad de precio. Es lo que permite el lote nocturno del capítulo 5 del Vol. 1: clasificar y redactar tickets por la noche no necesita la latencia de la respuesta inmediata.

Los descuentos de caché y de lote **se acumulan**, y el orden importa: primero se escribe la caché y después se envía el lote que la reutiliza.

B.5. La fórmula

```
coste_ejecucion  = (tokens_entrada / 1e6) * precio_entrada
                 + (tokens_salida  / 1e6) * precio_salida

coste_optimizado = coste_ejecucion * factor_cache * factor_lote

CIU              = coste_total_del_pipeline / funcionalidades_validadas
```

`factor_cache` y `factor_lote` son fracciones entre 0 y 1. Un sistema que aprovecha la caché de lectura (0,1×) y envía por lote (0,5×) multiplica su gasto por **0,05**: una reducción de dos órdenes de magnitud sin tocar la arquitectura. La tercera línea es la que convierte el gasto en una métrica de ingeniería: el CIU del Vol. 4 se expresa en dólares por funcionalidad que llega a producción, la única cifra comparable entre proveedores, entre modelos y entre trimestres.

B.6. Cómo mantener este apéndice

Este archivo es la única fuente de precios de la serie. El linter `tools/lint_manuscript.py` comprueba dos cosas:

- **C4**: que todo apéndice citado exista de verdad. Si un capítulo dice «como ve el apéndice B» y este archivo falta, la composición falla.
- **C6**: que ningún identificador de modelo aparezca escrito en el cuerpo del texto fuera de la lista blanca `tools/model-allowlist.txt`. Los nombres viven aquí; en los capítulos se habla de familias («el modelo de gama alta»), no de versiones concretas.

Cuando cambien los precios, se edita esta tabla, se actualiza la fecha de corte, se registra el cambio en `INFORME_EDITORIAL.md` y se revisan los dos capítulos que la calculan. Es una operación de veinte minutos, y evita que el libro tenga que reimprimirse.

Estado del Arte: Gobernanza, Rendición de Cuentas y Supervisión Humana en Sistemas de IA Autónomos

Revisión crítica de literatura indexada (2019–2026), elaborada como respaldo académico del Volumen 3: Madurez Organizacional

1. Líneas de investigación principales

Línea 1: Gobernanza de IA como campo consolidado. La revisión sistemática de Batool, Zowghi y Bano (2023/2024, arXiv:2401.10896) es la referencia más citada del campo de *Responsible AI Governance*; establece que, aunque existen numerosos principios y marcos de gobernanza, persiste una falta de alineación entre esos principios y su implementación real en modelos organizacionales de liderazgo y toma de decisiones, el mismo vacío que el capítulo 20 de este volumen intenta cerrar con un protocolo operativo, no solo declarativo.

Línea 2: Marcos normativos y de riesgo. Convergen aquí el NIST AI Risk Management Framework (2023), la norma ISO/IEC 42001 (2023) y el EU AI Act, todos citados de forma recurrente como los tres marcos estructurales de referencia (Floridi & Cowls, 2019; Mittelstadt, 2019; OECD, 2022, según la síntesis de "The Governance Inversion Hypothesis", 2026). Es notable que la literatura reciente empieza a documentar una tensión que este libro también aborda: los marcos normativos avanzan más lento que la capacidad técnica

de los sistemas que buscan regular (Anderljung et al., 2023, citado en la SLR sobre cumplimiento en sectores críticos).

Línea 3: Rendición de cuentas (*accountability*) y responsabilidad distribuida. Es la línea directamente conectada con el capítulo 15 de este volumen (RACI con agentes). La literatura reciente sobre responsabilidad legal y organizacional en sistemas autónomos (análisis comparativo de regulación y marcos de gobernanza basados en riesgo, 2026) señala explícitamente que los métodos actuales para asignar responsabilidad entre los distintos roles de un sistema de IA siguen siendo fragmentarios, y que la falta de claridad se agrava en sistemas complejos multi-componente (Stahl, 2023; Yazdanpanah et al., 2023), exactamente el problema que el "RACI a escala" de este libro busca resolver con una regla operativa concreta, no solo con un llamado a mayor claridad.

Línea 4: Gobernanza de IA agéntica en operaciones empresariales. Una revisión sistemática de 2025 sobre el auge de la IA agéntica en operaciones empresariales documenta ahorros de costos y mayor eficiencia operativa reportados por organizaciones adoptantes, junto con preocupaciones recurrentes sobre rendición de cuentas ética, seguridad de datos y su efecto en el empleo humano (Chan et al., 2023, citado en esa SLR). La misma revisión destaca que la IA agéntica ha ganado tracción particular en entornos de alto riesgo como salud y finanzas (Gridach et al., 2025), dominios estructuralmente análogos al "caso sectorial" que cierra el capítulo 5 del Vol. 4 de esta serie.

2. Marcos teóricos utilizados

La literatura de gobernanza de IA no parte de una teoría única, sino que importa marcos de tres tradiciones distintas: **ética aplicada** (los principios de Floridi & Cowls, con su síntesis de "AI4People"), **teoría organizacional clásica de burocracia** (Bovens & Zouridis, 2002, "From Street-Level to System-Level Bureaucracies", un antecedente directo (aunque anterior a los LLMs) de la pregunta de qué ocurre con la discrecionalidad humana cuando un sistema de información automatiza decisiones), y **gestión de riesgo** (los marcos NIST/ISO ya mencionados). Este volumen adopta implícitamente una síntesis

de las tres, pero lo hace de forma más operacional que la mayoría de la literatura revisada: donde los marcos académicos declaran principios (transparencia, explicabilidad, supervisión humana), el capítulo 20 de este libro los convierte en un umbral numérico (el Índice de Autonomía Operativa) y un protocolo de auditoría ejecutable.

3. Metodologías aplicadas

Predominan las revisiones sistemáticas de literatura con protocolo explícito (varias de las citadas siguen PRISMA o variantes de Kitchenham), meta-análisis de marcos de política pública (Kluge Correa et al., analizando 200 políticas y guías éticas globales para identificar 17 principios prevalentes), y estudios de caso sectoriales (salud, finanzas, sector público). Es notablemente escasa la investigación empírica que mida cuantitativamente el nivel de autonomía real de un sistema en producción, la mayoría de la literatura trabaja con categorías cualitativas (alto/bajo riesgo, supervisado/no supervisado), no con un índice continuo comparable al propuesto en este volumen.

4. Hallazgos recurrentes

- Consenso amplio en que la rendición de cuentas es un concepto multidimensional que ya no puede reducirse a lo legal, sino que debe abarcar supervisión organizacional, transparencia y gestión de riesgo a lo largo de todo el ciclo de vida del sistema (Novelli et al., 2024; Stahl, 2023).
- Consenso en que la ética por sí sola, sin verificación técnica y alineación de seguridad, es insuficiente como base de gobernanza, un giro que la literatura de 2024–2025 marca explícitamente como más reciente que el enfoque puramente declarativo de años anteriores.
- Tensión documentada entre innovación y cumplimiento en sectores críticos: las organizaciones reportan operar en un "corredor de cumplimiento" simultáneamente restrictivo y mal definido (Wang & Wu, 2024).

5. Discrepancias entre autores

La discrepancia más relevante para este volumen es de nivel de abstracción: una parte de la literatura (los marcos normativos: NIST, ISO, EU AI Act) opera al nivel de principios generales aplicables a cualquier sistema de IA; otra parte, más reciente y todavía minoritaria, empieza a reconocer que los sistemas de razonamiento de gran escala (LRMs/LLMs agénticos) son "infraestructuras de decisión socio-técnicas" que requieren gestión por nivel de riesgo, no como software convencional. Este libro se alinea explícitamente con la segunda posición, y va un paso más allá de lo que la literatura revisada ofrece: mientras la literatura académica se detiene en el diagnóstico ("se necesita gestión diferenciada por riesgo"), este volumen entrega el mecanismo (la tabla de pesos verde/amarillo/rojo del IAO).

6. Limitaciones metodológicas de la literatura revisada

- **Predominio de estudios de política y no de ingeniería.** La mayor parte de la literatura de gobernanza de IA proviene de escuelas de políticas públicas, derecho y ética aplicada, no de ingeniería de software; esto explica por qué son fuertes en taxonomía de principios y débiles en mecanismos operativos verificables, precisamente el vacío que este volumen, escrito desde la ingeniería, busca llenar.
- **Escasa validación empírica de los marcos propuestos.** Varias SLR citadas señalan que los marcos de gobernanza revisados rara vez incluyen evidencia de implementación exitosa medida, más allá de estudios de caso puntuales y no comparables entre sí.
- **Sesgo geográfico e institucional.** La literatura de políticas está dominada por marcos de EU, EE. UU. y organismos multilaterales (OCDE); la revisión de Frost et al. sobre la estrategia nacional de IA de Canadá es una de las pocas referencias con foco explícito fuera de esos tres polos, y ninguna de las fuentes revisadas aquí aborda con profundidad comparable a América Latina: una brecha que el capítulo sobre El Niño/Canal de

Panamá y los casos de Sber (Rusia) y Doraleh (Yibuti) de este libro parcialmente compensan, aunque no sustituyen investigación académica formal sobre la región.

7. Brechas de conocimiento

1. **Ausencia de métricas cuantitativas de autonomía comparables entre organizaciones**: el IAO de este libro es, hasta donde esta revisión pudo verificar, una propuesta original sin antecedente directo publicado en la literatura indexada revisada.
2. **Falta de protocolos de auditoría que sean agnósticos del proveedor del modelo**: toda la literatura de auditoría de IA revisada asume un sistema con un modelo fijo; ninguna aborda explícitamente qué ocurre con la validez de una auditoría cuando el modelo subyacente cambia, el problema exacto que la auditoría de reproducción cruzada del capítulo 20 resuelve.
3. **Escasa investigación sobre RACI u otros marcos de asignación de responsabilidad humana específicamente adaptados a enjambres de decenas de agentes**: la literatura de responsabilidad distribuida es rica en principios, pero no en mecanismos de escala.

8. Relación con la tesis central del libro

La tesis de este volumen (que la gobernanza debe expresarse en mecanismos verificables y no solo en principios declarados, y que esa gobernanza debe sobrevivir al cambio de proveedor de modelo) encuentra en la literatura un diagnóstico convergente (la brecha entre principios y mecanismos operativos, sección 1 y 5) pero no una solución ya publicada. Esto posiciona las contribuciones originales de este volumen (el IAO y la auditoría de reproducción cruzada) no como una síntesis de lo existente, sino como una respuesta directa a una brecha que la propia literatura académica reconoce y no ha cerrado todavía.

Referencias

Abrahamson, Eric, Management Fashion (1996). *Management Fashion*. Academy of Management Review, 1996.

Anthropic (25 de noviembre de 2024). *Introducing the Model Context Protocol. Anuncio oficial. Creado por David Soria Parra y Justin Spahr-Summers. Especificación en* `modelcontextprotocol.io`.

Arrieta, Alejandro Barredo; Díaz-Rodríguez, Natalia; Del Ser, Javier; others (2020). *Explainable Artificial Intelligence (XAI)*. Information Fusion, 2020.

Autoridad del Canal de Panamá, As cited in the manuscript, August, 2026 (2026). *As cited in the manuscript, August, 2026*.

Baghai, Mehrdad; Coley, Stephen; White, David (1999). *The Alchemy of Growth*. Perseus Books, 1999.

Bai, Yuntao; Jones, Andy; Ndousse, Kamal; others (2022). *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. Anthropic, arXiv preprint.*

Bai, Yuntao; Kadavath, Saurav; Kundu, Sandipan; others (2022). *Constitutional AI: Harmlessness from AI Feedback. Anthropic, arXiv preprint.*

Beck, Kent; Beedle, Mike; van Bennekum, Arie; others (2001). *Manifesto for Agile Software Development, 2001*.

Beck, Kent; Gamma, Erich, JUnit: Test-Driven Development Framework (1998). *JUnit: Test-Driven Development Framework.*

Bi, Kaifeng; Xie, Lingxi; Zhang, Hengheng; Chen, Xin; Gu, Xiaotao; Tian, Qi (2022). *Pangu-Weather: A 3D High-Resolution Model for Fast and Accurate Global Weather Forecast*. *arXiv:2211.02556* [*physics.ao-ph*].

Brewer, Eric A. (2000). *Towards Robust Distributed Systems* (*CAP Theorem*). ACM PODC, 2000.

CENEPRED, As cited in the manuscript, 2017 (2017). *As cited in the manuscript, 2017*.

Chase, Harrison (2025). *As cited in the manuscript* (*LangChain*).

Chouldechova, Alexandra, Fair Prediction with Disparate Impact, 2017 (2017). *Fair Prediction with Disparate Impact, 2017*.

Conmy, Arthur; Mavor-Parker, Augustine; Lynch, Aengus; Heimersheim, Stefan; Garriga-Alonso, Adrià (2023). *Towards Automated Circuit Discovery for Mechanistic Interpretability, 2023*. *NeurIPS*.

Digital.ai, Report, as cited in the manuscript, 2024 (2024). *Report, as cited in the manuscript, 2024*.

Elhage, Nelson; Nanda, Neel; Olsson, Catherine; others (2021). *A Mathematical Framework for Transformer Circuits, 2021*. *Anthropic*.

Forsgren, Nicole; Humble, Jez; Kim, Gene (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

Fowler, Martin (2003). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Grote, Gudela, Safety Management in High-Hazard Industries (2012). *Safety Management in High-Hazard Industries*. Safety Science, 2012.

Holland, John H., Complex Adaptive Systems (1992). *Complex Adaptive Systems*. Daedalus, 1992.

Hong, Sirui; Zheng, Xiawu; Chen, Jonathan; Cheng, Yuheng; others (2024). *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. *ICLR*.

IBM Security, Cost of a Data Breach Report, 2023 (2023). *Cost of a Data Breach Report, 2023*.

IBM Security, Cost of a Data Breach Report, 2024 (2024). *Cost of a Data Breach Report, 2024*.

Jain, Sarthak; Wallace, Byron C., Attention Is Not Explanation (2019). *Attention Is Not Explanation*. NAACL, 2019.

Jimenez, Carlos E.; Yang, John; Wettig, Alexander; others (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*. *ICLR*.

Kapoor, Sayash; Stroebl, Benedikt; Siegel, Zachary S.; Nadgir, Arvind; Narayanan, Arvind (2024). *AI Agents That Matter*. *arXiv preprint*.

Kazman, Rick; Klein, Mark; Clements, Paul (2000). *ATAM: Method for Architecture Evaluation*. CMU Software Engineering Institute, 2000.

Pathak, Jaideep; Subramanian, Shashank; Harrington, Peter; et al. (2022). *FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators*. *arXiv:2202.11214* [*physics.ao-ph*].

Lam, Remi; Sanchez-Gonzalez, Alvaro; Willson, Matthew; et al. (2022). *GraphCast: Learning Skillful Medium-Range Global Weather Forecasting*. *arXiv:2212.12794* [*cs.LG*].

Kairouz, Peter; McMahan, H. Brendan; Avent, Brendan; et al. (2021). *Advances and Open Problems in Federated Learning*. *Foundations and Trends in Machine Learning, 4*(*1-2*). *Preprint arXiv:1912.04977*.

Liu, Nelson F.; Lin, Kevin; Hewitt, John; Paranjape, Ashwin; Bevilacqua, Michele; Petroni, Fabio; Liang, Percy (2024). *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics (TACL).

McKeeman, William M., Differential Testing for Software, 1998 (1998). *Differential Testing for Software, 1998*.

Mehrabi, Ninareh; Morstatter, Fred; Saxena, Nripsuta; Lerman, Kristina; Galstyan, Aram (2021). *A Survey on Bias and Fairness in Machine Learning*. ACM Computing Surveys, 2021.

Microsoft Research, As cited in the manuscript, 2023 (2023). *As cited in the manuscript, 2023*.

Mitchell, Melanie; others (2025). *As cited in the manuscript*. *Bibliographic detail not fully verified; see manuscript context*.

Myerson, Roger B., Incentive Compatibility; the Bargaining Problem (1979). *Incentive Compatibility and the Bargaining Problem*. Econometrica, 1979.

NOAA Climate Prediction Center, ENSO forecast, as cited in the manuscript, 2026 (2026). *ENSO forecast, as cited in the manuscript, 2026*.

Offutt, A. Jefferson; Untch, Roland H., Mutation 2000: Uniting the Orthogonal, 2001 (2001). *Mutation 2000: Uniting the Orthogonal, 2001*.

Osborne, Martin J.; Rubinstein, Ariel, A Course in Game Theory (1994). *A Course in Game Theory*. MIT Press, 1994.

Ouyang, Long; Wu, Jeffrey; Jiang, Xu; Almeida, Diogo; others (2022). *Training Language Models to Follow Instructions with Human Feedback*. Advances in Neural Information Processing Systems (NeurIPS).

Patterson, David; Gonzalez, Joseph; Le, Quoc; others (2021). *Carbon Emissions and Large Neural Network Training, 2021*.

Peng, Sida; Kalliamvakou, Eirini; Cihon, Peter; Demirer, Mert (2023). *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*. *arXiv:2302.06590* [*cs.SE*]. *Experimento controlado: el grupo con acceso al asistente completó la tarea un 55,8 % más rápido que el grupo de control.*

Ponemon Institute, Report, as cited in the manuscript, 2023 (2023). *Report, as cited in the manuscript, 2023*.

Project Management Institute, A Guide to the Project Management Body of Knowledge (PMBOK Guide) (2021). *A Guide to the Project Management Body of Knowledge* (*PMBOK Guide*). (7th, PMI, 2021 ed.) PMI, 2021.

Ries, Eric, The Lean Startup (2011). *The Lean Startup*. Crown Business, 2011.

Rochet, Jean-Charles; Tirole, Jean, Platform Competition in Two-Sided Markets (2003). *Platform Competition in Two-Sided Markets*. Journal of the European Economic Association, 2003.

Schelling, Thomas C., The Strategy of Conflict (1960). *The Strategy of Conflict*. Harvard University Press, 1960.

Schwaber, Ken; Sutherland, Jeff, The Scrum Guide (2020). *The Scrum Guide*. Scrum.org, 2020.

Strubell, Emma; Ganesh, Ananya; McCallum, Andrew (2019). *Energy and Policy Considerations for Deep Learning in NLP*. ACL, 2019.

Unión Europea, Reglamento de Inteligencia Artificial (EU AI Act), 2024 (2024). *Reglamento de Inteligencia Artificial* (*EU AI Act*), *2024*.

Vaithilingam, Priyan; Zhang, Tianyi; Glassman, Elena L. (2022). *Expectation vs. Experience: Evaluating Usability of Code Generation Tools Powered by LLMs*. ACM CHI, 2022.

Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N.; Kaiser, Lukasz; Polosukhin,

Illia (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).

Westerman, George; Bonnet, Didier; McAfee, Andrew (2014). *Leading Digital: Turning Technology into Business Transformation*. Harvard Business Review Press, 2014.

Womack, James P.; Jones, Daniel T., Lean Thinking (2003). *Lean Thinking*. Free Press, 2003.

Yao, Shunyu; Zhao, Jeffrey; Yu, Dian; Du, Nan; Shafran, Izhak; Narasimhan, Karthik; Cao, Yuan (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR).

Zhou, Shuyan; Xu, Frank F.; Zhu, Hao; et al. (2024). *WebArena: A Realistic Web Environment for Building Autonomous Agents*. *arXiv:2307.13854* [*cs.AI*] (*ICLR 2024*). *El mejor agente basado en GPT-4 resuelve el 14,41 % de las tareas frente al 78,24 % de las personas.*

Ziegler, Albert; Kalliamvakou, Eirini; Li, X. Alice; Rice, Andrew; Rifkin, Devon; Simister, Shawn; Sittampalam, Ganesh; Aftandilian, Edward (2024). *Measuring GitHub Copilot's Impact on Productivity*. *Communications of the ACM, 67*(3), 54-63._ *Estudio de campo con Metric, el equipo de análisis de GitHub.*